

—— // ZSHRS // ——

zshrs – Shell Reference

v0.12.10

SHELL REFERENCE

every builtin · keyword · option · special var

2026-07-13

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Pipeline Architecture	2
Coverage Gate	3
Control Flow	5
Arrays & Associative Arrays	8
Parameter Expansion	11
Zsh Parameter Flags	15
Redirection & Pipelines	18
POSIX / Zsh Builtins	21
Job Control & Background	29
Anti-Fork Coreutils	31
Parallel Primitives	34
AOP Intercept	35
Diagnostics & Profiling	36
Completion System	37
ZLE Line Editor	39
PCRE & Regex	40
Recently Closed Compat Gaps (eighty-eighth-pass)	41
Compat Builtin Index	47
Keyword Index	100
Option Index	107
Special Variable Index	132
Compsys Function Index	161
Extension Builtin Index	179
Operator / Punctuation Index	190

Pipeline Architecture

Two execution pipelines exist; the new one is the default as of session 2026-04-27:

- **New (default):** `crate::ported::lex` (port of `src/zsh/5rc/lex.c`, in `src/ported/lex.rs`) → `crate::ported::parse` (port of `src/zsh/5rc/parse.c`, in `src/ported/parse.rs`) → `ZshCompiler` (original Rust, `src/extensions/compile_zsh.rs`) → fusevm bytecode. Lex + parse are strict-port code; the bytecode compiler lives in `src/extensions/` because it has no C ancestor. Faster + more faithful to zsh's grammar than the legacy tree-walker.
- **Old (escape hatch):** set `ZSHRS_OLD_PIPELINE=1` to route through the legacy hand-rolled `ShellLexer/ShellParser/ShellCompiler`. Slated for deletion once corpus parity is proven across all dog-food scenarios.

A residual bridge inside `compile_word_str` still re-parses param-modifier forms (`${x:-default}`, `${x#prefix}`, `${x/pat/rep}`) through the legacy `ShellParser` via `untokenize_preserve_quotes` round-trip. Each iteration of the migration replaces one runtime-fallback call site with native ops; tracked as Phase 1 / G8 in `ROADMAP.md`.

Coverage Gate

Each construct in this reference is pinned by at least one assertion in `tests/zsh_construct_corpus.rs`. The corpus runs the real `zshrs -f -c <script>` binary and asserts exact stdout + exit status. Categories covered:

- Simple commands, multi-word arguments, assignments (chained, empty, quoted, single-quoted)
- List operators: `;`, `&&`, `||`, `&`, short-circuit chains
- Control flow: `if/elif/else`, `while`, `until`, `for in`, `for ((;))`, `case` (literal/glob/alternation), `select`, `coproc`
- `break` / `continue` / `return` with status
- Functions: `name()` `{}`, `function` keyword, `local` scope, `$1...$#` + `"$@"` splice
- Pipelines: 2/3-stage, function-in-first-stage, negation
- Redirects: `>`, `>>`, `<`, `<<EOF`, `<<<str`, `2>&1`, block redir, `<&fd` with var-expanded fd, close-fd
- Quoting: single, double, `'...'` ANSI-C, backtick, escaped `\$` in dq
- Parameter expansion: every modifier from `zshexpn(1)` — default, assign, error, alternate, length, substring, strip prefix/suffix (short/long, with glob), replace first/all, upper/lower postfix
- ZshFlags: L, U, j:s:, s:s:, f, o, O, P, @, k, v, #, q (level 1-4), q+, q-, q*, q!, g, n, i, t, %, e, p, A, ~, plus stacking
- Special parameters: `$?`, `$$`, `$@`, `$*`, `$#`, `$N`, `$!`, `$_`, `$-`
- Command substitution: `$(...)`, backticks, nested, in array literal (with word-split)
- Arithmetic: `$(...)`, `((...))`, `let` — addition/precedence/paren/div/mod/pow/bitwise/shift/inc/dec/compare/logical/ternary, with var sync to `executor.variables`
- Tilde, brace ranges (numeric/letter/zero-pad), brace alternation (nested + prefix/suffix), brace in for-loop
- Glob: `*`, `?`, `[...]`, qualifiers `(.)`, `(/)`, `(@)`, `(N)`, `(x/r/w)`, sort modifiers `o/O/L/OL/om/Om`
- Process substitution: `<(cmd)`, `>(cmd)`
- Here doc + herestring
- Indexed arrays: literal, append `+=( )`, index 1-based + negative, splice `${arr[@]}`, length, iteration, empty, quoted-element preservation
- Associative arrays: `typeset -A, m[k]=v, m[k]+=tail`, lookup, keys, values, missing-key empty
- `[[ ... ]]`: `==/!=/=/</>`, glob, `-z/-n`, numeric `-eq/-lt/-le/-gt/-ge`, file tests `-e/-f/-d/-r`, logical `&&/||` `!/!`, regex `==` with anchors + capture groups
- POSIX builtins: `cd`, `pwd`, `echo`, `print`, `printf`, `read` (single/multi-var/array), `eval`, `exec`, `set`, `shift`, `unset`, `export`, `alias/unalias`, `type`, `true/false/:`, `test/`
- Coreutils builtins (anti-fork): `seq`, `basename`, `dirname`, `tr` (with range expansion), `cat`, `head/tail`, `wc`, `sort`, `uniq`, `find`, `cut`, `rev`, `tee`, `sleep`, `mktemp`, `whoami`, `id`, `hostname`, `uname`, `date`, `touch`, `realpath`
- Background `&`, `async/await`, coproc round-trip via `/dev/fd/N`
- Eval with deferred expansion (single-quoted args), multi-command eval
- Bang (!) — literal in non-interactive (with surrounding chars), command negation when standalone
- Aliases — regular + with args
- Subshells `(...)` with full state isolation (variables/arrays/assoc/positional/cwd restored)
- `(exit N)` in a subshell exits the subshell only, not the parent; status propagates through `SubshellEnd`
- Brace groups `{...}` (no isolation, command-scope only) — and *compound forms with trailing redirects*: `{ ... } 2>&1`, `(...)>file`, `if ...; fi >file` via `ZshCommand::Redirected`
- Anonymous functions `() { body } args...` — auto-call form executes once with the trailing words as `$1...$N`
- Trap EXIT (fires on script exit through both pipelines)
- Repeat N cmd
- Case fall-through `&f` (run next arm without re-testing) and test-next `;&|` (re-test next arm's pattern)
- `|&` pipe with stderr merge (`cmd1 |& cmd2` $\equiv$ `cmd1 2>&1 | cmd2`)
- Here doc variants: `<<EOF` (var-expand), `<<-EOF` (strip leading tabs), `<<'EOF' / <<"EOF"` (quoted-terminator $\rightarrow$ verbatim, no expansion)
- Special parameters: `$?`, `$$`, `$@`, `$*`, `$#`, `$N`, `$!`, `$_`, `$-`, `$RANDOM`, `$SECONDS`, `$EPOCHSECONDS`, `$LINENO`, `$REPLY`, `PIPESTATUS` / `pipestatus`
- `[[ -v var ]]` existence test (scalar/array/assoc/env)
- `${a[$i]}` variable subscript (`$i`, `${expr}`, `$(arith)`) all resolve before subscript parsing)
- `${v: -3}` negative substring offset (space disambiguates from `${v:-default}`)
- `${var:=default}`, `${var:?msg}`, `${var:+alt}`, `${v/#prefix/repl}` / `${v/%suffix/repl}` (anchor variants of replace)

- Arithmetic with `$NAME` / `${NAME}` / `$N` inside `((...))`, `$(...)`, `for` `((...))` — pre-loaded via `BUILTIN_GET_VAR`
- `set -e`, `getopts`, `read` into multiple vars / default `$REPLY`
- `printf` format-string repetition (POSIX: re-applies format while args remain)

Out-of-scope (separate Phase work): zsh modules (zsh/curses, zsh/net/socket, zsh/zftp, zsh/zselect, zsh/sched), interactive ZLE widget firing on Tab + bindkey'd user widgets, async `compinit` fpath pre-warm, full setopt option matrix (~200 options × 2 toggles), some long-tail glob qualifiers (`L+N` size, `mh-N` mtime, `om[1,N]` "newest N"), and the dogfood gate (zpw load + `.zshrc` as login shell for 14 days). The `tests/ztst_runner.rs` wrapper has been removed from CI — it was a fake-pass per-block runner; replacement is the PTY harness designed in `docs/ZTST_PTY_HARNESS.md`, not yet built.

Control Flow

14 topics · compiled to fusevm bytecode with deferred jump patches; no tree-walker dispatch. Includes anonymous-fn auto-call and `case` fall-through (`;f`) / test-next (`;|`).

`if ... then ... fi`

Standard POSIX conditional. `elif` branches and an optional `else` are supported. Status semantics: a `Status(0)` is truthy in zshrs, so `if cmd; then ...; fi` runs the *then* arm when `cmd` exits 0. The compile path emits `JumpIfFalse` at the end of each conditional probe.

```
if [[ -d "$dir" ]]; then
  cd "$dir"
elif [[ -f "$dir" ]]; then
  echo "regular file"
else
  echo "no such path"
fi
```

`while ... do ... done`

Loops while the condition exits 0. Loop-body status is preserved across iterations via a dedicated slot — the loop's exit status is the body's last-command status (or 0 if the body never ran), matching POSIX.

```
i=0
while (( i < 5 )); do
  echo "iter $i"
  (( i++ ))
done
```

`until ... do ... done`

Inverse of `while` — loops while the condition is non-zero. Compiles to the same bytecode shape with the condition jump inverted.

```
until ping -c1 -W1 example.com >/dev/null; do
  sleep 1
done
echo "online"
```

`for var in words; do ... done`

Iterates `var` over the word list. Words are *compiled at runtime* — `${arr[@]}` splices into N iterations via `BUILTIN_ARRAY_FLATTEN`. The for-loop preserves break/continue patches as deferred jump sites so nested loops work correctly.

```
for f in *.rs; do
  echo "compiling $f"
done

arr=(alpha beta gamma)
for x in ${arr[@]}; do
  echo "got $x"
done
```

for ((init; cond; step)); do ... done

C-style arithmetic for loop. The arithmetic expressions compile through the inline arithmetic compiler (no `$(())` wrapping needed inside the parens).

```
for ((i = 0; i < 10; i++)); do
    echo "i=$i"
done
```

case ... esac

Pattern-matched dispatch. Patterns are compiled via `compile_case_pattern` which preserves `$`-expansion but does *not* glob-expand the pattern itself (otherwise `case x in *)` ... would expand `*` to the cwd listing). Match check uses `Op::StrMatch` (host-routed glob match).

```
case "$1" in
    start) echo starting ;;
    stop)  echo stopping ;;
    *)     echo "usage: $0 {start|stop}"; exit 1 ;;
esac
```

select var in words; do ... done

Interactive numbered-menu loop. Prints `1) word1\n2) word2\n...` to stderr, prompts via `$PROMPT3` (default `?#`), reads stdin, sets `var` to the chosen word, runs the body. `$REPLY` contains the raw input. EOF on stdin exits the loop. The real `break` keyword exits early via the cross-VM `LoopSignal` mechanism; the legacy `BREAK_SELECT=1` sentinel still works for back-compat.

```
select choice in build test deploy quit; do
    case $choice in
        build) cargo build ;;
        test)  cargo test  ;;
        deploy) ./deploy.sh ;;
        quit)  break ;;
    esac
done
```

() { body } args...

Anonymous function. Defined and immediately invoked with the trailing words as `$1...$N`. Parser routes `Inoutpar` (the `()` token) to `parse_anon_funcdef`, which generates a unique `_zshrs_anon_N` name; `compile_funcdef` registers and calls when `auto_call_args` is set. `()` with no body falls back to an empty subshell (POSIX no-op).

```
() { echo "hi $1, $2 args"; } world 1
# → hi world, 1 args
```

case; & fall-through, ;| test-next

Per-arm separators after a case body. `;;` exits (default), `&` falls through to the next arm *without* re-testing its pattern, `;;|` falls through and *does* re-test. `compile_case` tracks a `pending_fall` patch site so `&` emits a forward jump that the next arm's body-start patches.

```
case "$x" in
    a) print A & # falls into b's body
    b) print B ;;
    *) print other ;;
esac
```

coproc [name] { body }

Forks the body with bidirectional pipes. Two pipes are created (parent → child stdin, child → parent stdout); child `setsids` and runs the body, parent stores `[read_fd, write_fd]` in `$COPROC` (or the given name as an array). Read child output via

`/dev/fd/${COPROC[1]}`, write to its stdin via `/dev/fd/${COPROC[2]}`. Job-table integration is Phase G6.

```
coproc { while read line; do echo "ECHO: $line"; done }
echo hello > /dev/fd/${COPROC[2]}
read reply < /dev/fd/${COPROC[1]}
echo "$reply" # ECHO: hello
```

break

Exits the innermost `for/while/until`. Compiled as a deferred forward `Jump(0)` patched to land just past the loop's exit point. Inside `select` use `BREAK_SELECT=1` instead until cross-construct loop control lands in Phase G6.

```
for f in *.log; do
  [[ -s $f ]] || break # stop on first empty
  process "$f"
done
```

continue

Skip to the next iteration. Same patch mechanism as `break` — patches into the loop's `continue target` which is the iteration step (e.g. `PreIncSlotVoid` in the for-loop).

```
for n in 1 2 3 4 5; do
  (( n % 2 == 0 )) || continue
  echo "even: $n"
done
```

return [n]

Return from a function with the given status (default last-status). Compile emits a forward `Op::Jump(0)` patched to past the chunk's end; standalone-chunk VMs interpret this as halt. `Op::Return` alone restarts the body from `ip=0`, which is wrong for top-level scripts.

```
greet() {
  [[ -z "$1" ]] && return 1
  echo "hello, $1"
}
```

; && || &

Sequential, conditional, and background separators. `cmd1 && cmd2` runs `cmd2` only if `cmd1` succeeded; `||` is the inverse. `&` backgrounds the preceding command via `BUILTIN_RUN_BG` — fork + `setid`, parent returns `Status(0)` immediately. Compiled with deferred short-circuit jumps to avoid double-compilation of the next command.

```
git pull && cargo test || echo "test failed"
sleep 30 &
echo "going to bg"
```

Arrays & Associative Arrays

13 topics · indexed arrays in `executor.arrays`, assoc in `executor.assoc_arrays`; argv splice via fusevm 0.12.2+.

`arr=(a b c)`

Indexed-array literal assignment. Compile emits N element pushes + name push, then `BUILTIN_SET_ARRAY` (id 287) which writes a `Vec<String>` into `executor.arrays` and clears any prior scalar binding for the same name.

```
arr=(alpha beta "two words" gamma)
echo ${arr[@]}      # 4
```

`arr+=(d e)`

Append elements to an existing array (or create if missing). Routes through `BUILTIN_APPEND_ARRAY` (id 295) which extends `executor.arrays[name]`.

```
arr=(a b)
arr+=(c d)
echo ${arr[@]}      # a b c d
```

`${arr[N]}`

Indexed access. zsh is 1-based for positive indices; negative indices count from the end (`${arr[-1]}` = last). Routes through `BUILTIN_ARRAY_INDEX` (id 289). For assoc arrays the same form does string-key lookup.

```
arr=(alpha beta gamma)
echo ${arr[1]}      # alpha
echo ${arr[-1]}     # gamma
```

`${arr[@]}`

Splice all elements as separate argv slots. Pushes a `Value::Array` which fusevm's `Op::Exec/Op::ExecBg/Op::CallFunction` and `pop_args` flatten into individual args. Without the splice, the array would collapse to one space-joined scalar.

```
arr=(--verbose --color=always /etc /var)
ls "${arr[@]}"      # ls --verbose --color=always /etc /var
```

`${#arr[@]}`

Number of elements. Routes through `BUILTIN_ARRAY_LENGTH` (id 291).

```
arr=(a b c d)
echo ${#arr[@]}     # 4
```

`for x in ${arr[@]}`

Iterate over array elements. The for-loop word-list compile path calls `BUILTIN_ARRAY_FLATTEN` (id 293) which flattens nested arrays one level — so `for x in start ${arr[@]} end` produces N+2 iterations, not 3.

```
arr=(red green blue)
for c in ${arr[@]}; do
    echo "color: $c"
done
```

arr=()

Empty-array literal. `${#arr[@]}` returns 0; iteration runs zero times.

```
arr=()
for x in ${arr[@]}; do echo "never"; done
echo "len=${#arr[@]}" # len=0
```

typeset -A name / declare -A name

Declare an associative array. The executor's `builtin_typeset` handles `-A` and pre-creates the `HashMap<String, String>` entry in `executor.assoc_arrays`. Optional `— name[key]=val` creates the assoc on demand too.

```
typeset -A user
user[name]=Jacob
user[role]=eng
echo "${user[name]}" # Jacob
```

name[key]=value

Assoc-array key set. Compile detects `name[key]` shape on the LHS of an assignment and routes to `BUILTIN_SET_ASSOC` (id 288) which stores into `executor.assoc_arrays[name][key]`.

```
db[host]=localhost
db[port]=5432
db[user]=admin
echo "${db[host]}:${db[port]}"
```

\${name[key]}

Assoc-array lookup. `BUILTIN_ARRAY_INDEX` checks `assoc_arrays` first; if the name has an assoc binding the string key is used directly. Missing keys return empty string.

```
typeset -A m
m[a]=1; m[b]=2
echo "a=${m[a]} b=${m[b]}"
echo "missing=${m[nope]}" # missing=
```

\${(k)name}

List the keys of an associative array. Order is `HashMap` iteration order (implementation-defined). See the [Zsh Parameter Flags](#) chapter for the full flag set.

```
typeset -A m
m[apple]=1; m[banana]=2
for k in "${(k)m}"; do echo $k; done | sort
```

\${(v)name}

List the values of an associative array. `${name[@]}` on an assoc returns the same value list.

```
typeset -A m
m[apple]=1; m[banana]=2
for v in "${(v)m}"; do echo $v; done | sort
```

`m[k]=new` **overwrites**

Re-assigning a key replaces the prior value. The set-builtin always inserts; use `+=` for append-on-existing-key semantics.

```
m[k]=first
m[k]=second
echo "${m[k]}" # second
```

`m[k]+=tail`

Append onto an existing assoc value (string concat). If the key is missing, behaves like a plain set. Routes through `BUILTIN_APPEND_ASSOC` (id 298).

```
m[host]=localhost
m[host]+=":5432"
echo "${m[host]}" # localhost:5432
```

Parameter Expansion

21 topics · 15 of 19 VarModifier variants lower to native fusevm ops via `Op::ExpandParam`; `${var:-default}` + `${#var}` have been native-lowered in `compile_zsh.rs` (Phase 1 steps 1–2); 4 remaining variants + stacked ZshFlags + `${cmd}/${(expr)}/concat ${a}${b}` still hit the runtime fallback or the `untokenize_preserve_quotes` bridge to the legacy `ShellParser`.

`${var:-default}`

Use `default` if `var` is unset or empty. Lowers to `Op::ExpandParam(DEFAULT)`. The default expression is itself compiled (variables, command substitution, etc. all expand at use time).

```
name=${1:-anonymous}
log_level=${LOG_LEVEL:-info}
target=${1:-$(date +%Y%m%d)}
```

`${var:=default}`

Assign `default` to `var` if unset/empty, then expand. Lowers to `Op::ExpandParam(ASSIGN)`. The variable is set in `executor.variables` as a side effect.

```
: ${CACHE_DIR:=$HOME/.cache}
echo "$CACHE_DIR" # CACHE_DIR is now set
```

`${var:?msg}`

Print error and exit if `var` is unset/empty. Lowers to `Op::ExpandParam(ERROR)`. Common in script preamble for required env vars.

```
: ${API_TOKEN:?must be set}
: ${DEPLOY_TARGET:?usage: deploy <target>}
```

`${var:+alt}`

Expand to `alt` if `var` is set and non-empty, else empty. Lowers to `Op::ExpandParam(ALTERNATE)`.

```
flag=${VERBOSE:++--verbose}
cargo build $flag
```

`${#var}`

String length of scalar (or via `${(#)arr}` the element count of an array — see Zsh Flags). Lowers to `Op::ExpandParam(LENGTH)`.

```
name=Jacob
echo ${#name} # 5
```

`${var:offset:length}`

Substring extraction. Lowers to `Op::ExpandParam(SLICE)`. Negative offsets count from the end. Length is optional (omitted = to end).

```
s=abcdefgh
echo ${s:2:3}      # cde
echo ${s:-3}       # fgh    (last 3 chars)
echo ${s:2}        # cdefgh (offset 2 to end)
```

`${var#pat} / ${var##pat}`

Strip shortest / longest matching prefix. `#` = shortest, `##` = longest. Lowers to `STRIP_SHORT` / `STRIP_LONG`.

```
path=/usr/local/bin/zshrs
echo ${path##*/}    # zshrs    (basename via longest-prefix-strip)
echo ${path#*/}     # usr/local/bin/zshrs
```

`${var%pat} / ${var%%pat}`

Strip shortest / longest matching suffix. `%` = shortest, `%%` = longest. Lowers to `RSTRIP_SHORT` / `RSTRIP_LONG`.

```
file=archive.tar.gz
echo ${file%.gz}    # archive.tar
echo ${file%%.*}    # archive    (dirname via longest-suffix-strip)
```

`${var/pat/repl}`

Replace first match of `pat` with `repl`. Lowers to `SUBST_FIRST`. `repl` may use `%` for the matched text in some shells; `zshrs` follows `zsh`'s literal-replace semantics.

```
s="foo bar foo"
echo ${s/foo/baz}   # baz bar foo
```

`${var//pat/repl}`

Replace *all* matches. Lowers to `SUBST_ALL`. Heavily used by `zpw`-style idioms (path normalization, sigil replacement).

```
s="hello world hello"
echo ${s//hello/HI} # HI world HI
path="/a/b/c"
echo ${path//\//.}  # .a.b.c
```

`${var:u} / ${(U)var}`

Uppercase. The `:u` postfix lowers to `Op::ExpandParam(UPPER)`; the `(U)` flag form goes through `BUILTIN_PARAM_FLAG` and supports stacking with other flags.

```
x=hello
echo ${x:u}         # HELLO
echo ${(U)x}        # HELLO
```

`${var:l} / ${(L)var}`

Lowercase. Same dual-form pattern as upper.

```
x=HELLO
echo ${x:l}         # hello
echo ${(L)x}        # hello
```

`$?`

Exit status of the last command. Routed through `BUILTIN_GET_VAR("?")` which reads `vm.last_status` synced into the executor.

```
cargo test
if (( $? != 0 )); then echo "tests failed"; fi
```

\$\$

Current shell PID (`std::process::id()`).

```
tmpdir=/tmp/build_$$
mkdir -p "$tmpdir"
```

\$1 \$2 ... \$@ \$* \$#

Positional parameters. `$@` / `$*` expand to the full list (joined by IFS), `$#` is the count, numeric `$N` is the Nth positional. Native fast-path: "`$@`" and `${arr[@]}` reach `BUILTIN_GET_VAR` / `BUILTIN_ARRAY_ALL` as `Value::Array` and spread through `BUILTIN_ARRAY_FLATTEN`.

```
greet() {
  echo "got $# args"
  for arg in "$@"; do echo "- $arg"; done
}
greet alpha "two words" gamma
```

\$! \$_ \$-

Auxiliary special params. `$!` = PID of the most recent backgrounded job. `$_` = last argument of the previous command (or empty at script start). `$-` = current shell flag set as a single string. All routed through `BUILTIN_GET_VAR`.

\$RANDOM \$SECONDS \$EPOCHSECONDS \$LINENO

Dynamic special parameters resolved on each read in `get_variable`:

- `$RANDOM` — 15-bit pseudorandom int. Mixes `nanos+pid` via Knuth's hash, masked to 15 bits. No seedable state today (independent draws).
- `$SECONDS` — seconds since shell start, derived from `__zshrs_start_secs` baseline.
- `$EPOCHSECONDS` — Unix time, `SystemTime::now()`.
- `$LINENO` — current line in the script (falls back to 1 today; full line tracking is Phase G work).

```
tmp="/tmp/build_$$_$RANDOM"
echo "elapsed: $SECONDS s"
echo "now: $EPOCHSECONDS"
```

\$PIPESTATUS / \$pipestatus

Per-stage exit codes of the most recent pipeline. `BUILTIN_RUN_PIPELINE` collects each stage's status and writes both `pipestatus` (zsh convention) and `PIPESTATUS` (bash convention). Useful for `set -e`-style scripts that need to differentiate which stage failed.

```
seq 100 | grep '^5' | wc -l
echo "${pipestatus[@]}" # 0 0 0
false | true
echo "${pipestatus[1]}" # 1 (zsh: 1-indexed)
```

[[-v var]] — existence test

`BUILTIN_VAR_EXISTS` (id 306) checks every binding table — scalar `variables`, `arrays`, `assoc_arrays`, plus the process env — and returns true if any matches. Useful for "set but empty" vs "unset" disambiguation that `-z` conflates.

```
x=""
[[ -v x ]] && echo "set"      # set (even though empty)
[[ -z $x ]] && echo "empty"   # empty
unset x
[[ -v x ]] || echo "unset"   # unset
```

`$((expr))`

Arithmetic substitution. The expression compiles through the inline arithmetic compiler — no runtime parser invocation. Integers are `i64`; supports `+` `-` `*` `/` `%` `**` `&` `|` `^` `<<` `>>` `&&` `||` and pre/post inc/dec.

```
(( total = 1 + 2 * 3 ))
echo $((2 ** 16))      # 65536
echo $((0xff & 0x0f))
```

`$(cmd) / ` cmd ``

Command substitution. `zshrs` runs the inner command on a nested VM and captures stdout via `os_pipe::pipe()` + `dup2` — *no fork*. Trailing newlines are stripped per POSIX.

```
now=$(date +%s)
files=$(ls *.rs | wc -l)
echo "$files files at $now"
```

Zsh Parameter Flags

12 topics · `${(flags)name}` form. Flags apply left-to-right; `(jL)` joins-then-lowercases, `(s:, :U)` splits-then-uppercases. Routes through `BUILTIN_PARAM_FLAG` (id 297).

(L) – lowercase

Lowercases the value. For arrays, lowercases each element.

```
x=Hello
echo ${L)x}    # hello

arr=(One Two Three)
echo ${L}arr    # one two three
```

(U) – uppercase

Uppercases.

```
echo ${U)x}    # HELLO
```

(j:sep:) – join

Join array elements with `sep`. The delimiter char following `j` sets the bracket — `:`, `.`, `,`, `|` are common. `j` with no delimiter joins with a single space (IFS-default).

```
arr=(one two three)
echo ${j:-:}arr    # one-two-three
echo ${j:|}arr    # one|two|three
echo ${j}arr    # one two three
```

(s:sep:) – split

Split a scalar on `sep` into an array. The result splices into `argv` via the standard array-flatten path.

```
csv="alpha,beta,gamma"
for x in "${(s:, :)csv}"; do
  echo "$x"
done
```

(f) – split on newlines

Shorthand for `(s:\n:)` — split on newlines. Common with command substitution.

```
for line in "${(f)$(ls)}"; do
  echo "FILE: $line"
done
```

(o) – sort ascending

Lexicographic sort.

```
arr=(charlie alpha bravo)
echo ${o}arr    # alpha bravo charlie
```

(0) – sort descending

Reverse lexicographic sort.

```
echo ${0}arr # charlie bravo alpha
```

(P) – indirect

Use the value of the variable as *another* variable name and look that up. Useful for dynamic dispatch tables.

```
real=42
ref=real
echo ${P}ref # 42
```

(@) – force array

Coerce a scalar to a single-element array (so subsequent flags treat it as array-shape).

```
x=hello
echo ${#x} # 5 (string length)
echo ${#@}x # 1 (array length after @-coerce)
```

(k) – keys of assoc

Returns the keys of an associative array as an array. Order is HashMap iteration order.

```
typeset -A m
m[apple]=1; m[banana]=2
for k in "${(k)m}"; do echo "$k"; done
```

(v) – values of assoc

Returns the values of an associative array as an array.

```
echo ${v)m} # 1 2
```

(#) – count

Element count for arrays, character count for scalars. Different from `${#var}` in that it can be combined with other flags via stacking.

```
arr=(a b c d)
echo ${#}arr # 4

echo ${#L}x # length after lowercasing
```

(q) / (qq) / (qqq) – quote

Wrap each value with shell-safe quoting. Consecutive `qs` raise the quoting level: `q` = POSIX single-quote (escaping inner `'`), `qq` = double-quote (escaping `$/`/"\`), qqq = ANSI-C $'...' form (escaping control chars + backslashes).`

```
x="hi 'world'"
echo ${q}x # 'hi \'world'\''
echo ${qq}x # "hi 'world'"
s=${printf 'a\tb'}
echo ${qqq}s # $'a\tb'
```

(g) – backslash-escape unwrap

Process backslash escapes (`\n`, `\t`, `\r`, `\\`, `\xNN`). Useful when reading lines from a config that stored real newlines as literal `\n`.

```
s='hello\nworld'
echo "${(g)s}"
# hello
# world
```

(n) – natural-numeric sort

Compare digit runs as integers and other runs lexicographically. `file2` sorts before `file10` (vs lex order which would put `file10` first). Combine with `(o)/(O)` for ascending/descending.

```
arr=(file10 file2 file1 file20)
echo "${(on)arr}" # file1 file2 file10 file20
```

(i) – case-insensitive sort

Sort comparing lowercase forms while preserving original case in the output.

```
arr=(Banana apple Cherry)
echo "${(i)arr}" # apple Banana Cherry
```

(t) – type query

Returns the variable's typeset shape: `scalar`, `array`, `association`, or empty (unset). Useful in scripts that branch on whether a name is an indexed vs assoc array.

```
arr=(a b)
typeset -A m; m[k]=v
sc=str
echo "${(t)arr}|${(t)m}|${(t)sc}"
# array|association|scalar
```

(%) – prompt expansion

Process `%F/%B/%f/%/` etc. via `expand_prompt_string`. Useful for storing prompt fragments in variables and rendering them at use time.

```
fragment='%F{cyan}>>%f '
echo "${(% )fragment}"
```

(e) – re-evaluate

Run the value as a shell command and return its captured stdout. Equivalent to `$(eval "$value")` but uses the in-process pipe-capture path (no fork). Late-bound config strings.

```
cmd='echo "user is ${whoami}"'
echo "${(e)cmd}" # user is jacob
```

(p) – print-style escapes

Process backslash escapes the same way `print -e` does. Same set as `(g)` with `\e` / `\E` mapped to `ESC`.

```
fmt='\e[36mcolor\e[0m'
echo "${(p)fmt}" # cyan "color"
```

Redirection & Pipelines

14 topics · pipelines fork-per-stage via `BUILTIN_RUN_PIPELINE` (id 285); redirects scoped via `WithRedirectsBegin/End`.

`cmd > file --write`

Truncates and writes stdout to file. Bracketed in a redirect scope so subsequent commands see the original fd 1.

```
echo hello > out.txt
```

`cmd >> file --append`

Appends stdout. Creates the file if missing.

```
date >> access.log
```

`cmd < file --read`

Connects file to stdin.

```
sort < data.txt
```

`cmd 2> file --stderr`

Redirect stderr (fd 2). Use `2>&1` to merge into stdout.

```
cargo build 2> build.err  
make 2>&1 | tee build.log
```

`cmd &> file --write both`

Redirect both stdout and stderr (zsh/bash extension). `&>>` appends.

```
./run.sh &> combined.log
```

`cmd <<EOF --heredoc (with variants)`

Multi-line stdin literal. Three flavors:

- `<<EOF` — variables, command sub, arithmetic all expand inside the body. Body trailing newline trimmed before `HereString` emit so stdin is byte-identical to source.
- `<<-EOF` — leading tabs stripped from each body line *and* from the closing-terminator-line check. Lets you indent the heredoc body inside a function body.
- `<<'EOF' / <<"EOF"` — quoted terminator → body is verbatim, **no** expansion. Detected by `SNULL/DNULL` markers in the lexer's terminator string; `HereDocInfo.quoted` drives compile-side behavior.

Body capture happens in a parser post-pass (`fill_heredoc_bodies`) — the lexer collects bodies into `self.heredocs[]` at `process_heredocs` (called on `Newlin/Endinput`); the parser records a `heredoc_idx` per redir during `parse_redirection`; the post-pass walks the AST resolving indices into `ZshRedir.heredoc`.

```
cat <<EOF
hello $USER
host: $(hostname)
EOF

cat <<-EOF
    indented
    body
EOF

cat <<'EOF'
$USER stays literal
EOF
```

cmd <<< "string" — herestring

Pass a single string (with trailing newline) as stdin.

```
tr a-z A-Z <<< "hello" # HELLO
```

cmd1 | cmd2 — pipeline

Connect cmd1's stdout to cmd2's stdin via a pipe. zshrs's pipeline is bytecode-native: each stage compiles to a sub-chunk; `BUILTIN_RUN_PIPELINE` creates N-1 pipes, forks N children, wires fds, runs each stage's bytecode on a fresh VM. `SIGPIPE` works correctly.

```
seq 100 | sort | uniq | wc -l
```

cmd1 |& cmd2 — pipe stdout+stderr

zsh extension: pipes both fd 1 and fd 2 of the LHS to RHS.

```
cargo test |& head -40
```

! cmd — invert status

Negate the pipeline's exit status. `! cmd` exits 0 if cmd failed, 1 if cmd succeeded.

```
! grep -q ERROR log # status 0 means no ERROR found
```

<(cmd) — process sub (input)

Spawn cmd, present its stdout as a path readable by the consumer. zshrs uses worker pool threads instead of fork — the FIFO/temp file is wired to a thread that runs the bytecode for cmd.

```
diff <(sort a.txt) <(sort b.txt)
```

>(cmd) — process sub (output)

Spawn cmd, present its stdin as a path writable by the producer.

```
tar c . | tee >(gzip > backup.tgz) > backup.tar
```

cmd N>&M — duplicate fd

Duplicate fd M to fd N. Common idiom: `2>&1` merges stderr into stdout. `<&fd` defaults the destination to fd 0; `>&fd` defaults to fd 1. Variable-expanded fd targets work: `read line <&${COPROC[1]}`.

```
find / 2>&1 | grep '\.zsh$' | head

coproc { echo from-child; }
read reply <&${COPROC[1]}
echo "$reply" # from-child
```

cmd N<&- / cmd N>&- — close fd

Close fd N. POSIX form for explicitly tearing down a previously-opened fd (typically after `exec FD< file`).

```
exec 5< data.txt
read line <&5
exec 5<&- # close fd 5
```

{ cmds; } > file — block redirect

Apply a redirect to a compound command. `WithRedirectsBegin/End` bracket the inner ops so the parent shell's fds are restored after.

```
{
  echo header
  date
  uname -a
} > report.txt
```

POSIX / Zsh Builtins

52 topics · the full POSIX-required set plus zsh's interactive-shell builtins. Every entry routes through fusevm `CallBuiltin(id, argc)`; arrays splice via `pop_args`.

`cd [dir]`

Change directory. With no argument, goes to `$HOME`. `cd -` jumps to `$OLDPWD`. Updates `$PWD`.

```
cd ~/repos/zshrs
cd -      # back to previous
```

`pwd`

Print working directory.

```
echo "now in $(pwd)"
```

`echo [args]`

Print arguments space-separated with trailing newline. `-n` suppresses newline; `-e` enables backslash escapes.

```
echo hello world
echo -n "no newline"
echo -e "tab:\there"
```

`print`

zsh's enhanced echo. `-r` raw, `-n` no newline, `-l` one arg per line, `-z` push to history.

```
print -l alpha beta gamma    # one per line
print -P "%F{cyan}cyan%f"    # prompt expansion
```

`printf format args...`

C-style formatted output. Format directives: `%s %d %f %x %o %b`; backslash escapes `\n \t` etc.

```
printf "%-10s %5d\n" "items" 42
```

`export VAR=value`

Mark a variable for export to subprocesses (writes to `std::env`).

```
export PATH="/opt/bin:$PATH"
export RUST_LOG=debug
```

`unset VAR...`

Remove a variable. Removes from `variables`, `arrays`, `assoc_arrays`, and exported env.

```
unset DEBUG_FLAG
unset -f myFunc    # remove function
```

source file / . file

Execute a script in the current shell. zshrs's source path uses bytecode caching: first run compiles + stores to SQLite; subsequent runs deserialize cached chunks. Plugin delta cache replays state changes (functions, aliases, vars, hooks, zstyles, options) in microseconds.

```
source ~/.zshrc
. /etc/profile.d/lang.sh
```

exit [n]

Exit the shell with status *n* (default last status). Aliases: **bye**, **logout**. Compile emits a forward jump patched past chunk-end (halts the VM). EXIT trap fires before halt.

```
[[ -z $1 ]] 66 { echo "missing arg"; exit 2; }
```

true / false / :

Status helpers. **true** and **:** exit 0; **false** exits 1. **:** is also the no-op for argument expansion side effects.

```
: ${VAR:=default} # set if unset; expansion side effect, no command
while true; do ...; done
```

test EXPR / [EXPR]

POSIX conditional test. Use **[[...]]** for the zsh extended form (in-shell, no fork). Operators: **-f -d -e -r -w -x -s -z -n = != < > -eq -ne -lt -le -gt -ge**.

```
[ -f /etc/passwd ] 66 echo "exists"
[[ $count -gt 10 66 -n $name ]]
```

local VAR[=value]

Function-local scope. zshrs's local-scope semantics: declared vars are pushed onto a scope stack at function entry, popped at return. Assignment-without-declare-at-top-of-function is allowed (zsh idiom) but has caveats — see CLAUDE.md "Never use 'local' inside loops".

```
my_func() {
  local count=0
  local result
  ...
}
```

typeset / declare

Declare a variable with attributes. **-a** indexed array, **-A** assoc array, **-i** integer, **-r** readonly, **-x** exported, **-l** lowercase, **-u** uppercase, **-g** global (in func scope), **-f** function-scoped.

```
typeset -i count=0
typeset -A config
typeset -r APP_VERSION=1.0
declare -a names=(alice bob carol)
```

readonly VAR=val

Mark a variable read-only. Subsequent assignments error.

```
readonly RELEASE_BRANCH=main
```

integer VAR=expr

Declare an integer-typed variable. Subsequent assignments evaluate as arithmetic.

```
integer i=10
i=i+5
echo $i      # 15
```

float VAR=expr

Declare a float-typed variable.

```
float pi=3.14159
echo ${pi * 2}
```

read VAR...

Read a line from stdin into one or more variables. Flags: **-r** raw (no backslash escapes), **-n** N N chars, **-t** SEC timeout, **-s** silent, **-p** prompt, **-A** arr read into array.

```
echo -n "name? "
read name
read -r line < /etc/hostname
read -A words <<< "alpha beta gamma"
```

mapfile / readarray

Read each line of a file (or stdin) into successive elements of an array. Bash compat alias.

```
mapfile -t lines < /etc/hosts
echo "${#lines[@]} hosts"
```

shift [n]

Remove the first *n* positional parameters (default 1).

```
process() {
  cmd=$1; shift
  do_thing "$cmd" "$@"
}
```

eval string

Re-parse and execute the joined-args string as shell code. Single-quoted args defer expansion correctly (the lexer's `\0`-sentinel for single-quoted specials is honored by the compile path's trigger detection).

```
x=10
eval 'echo $x'      # 10 (expansion deferred to eval-time)
eval "${starship init zsh}"
```

exec [cmd args]

Replace the current shell with *cmd*. With no command, applies redirections to the current shell (e.g. `exec > log` redirects subsequent output).

```
exec > build.log 2>&1
exec /usr/local/bin/newshell
```

command [-pvV] cmd

Bypass functions/aliases and run the underlying command. **-v** prints how the name would resolve; **-V** verbose form.

```
command rm /tmp/junk      # bypass user's `rm` function
command -v cargo          # /Users/wizard/.cargo/bin/cargo
```

builtin cmd

Force builtin dispatch (skip functions/aliases).

```
builtin cd /tmp          # zshrs's cd, never a user override
```

let "expr"

Evaluate arithmetic expression(s); exit 0 if last result is non-zero.

```
let "i = 1 + 2"
let "i++"
```

set [-eux] [args...]

Set shell options or positional parameters. **-e** exit on error, **-u** nounset, **-x** trace, **-o pipefail** propagate pipeline status.

```
set -euo pipefail
set -- alpha beta gamma  # set $1 $2 $3
```

setopt / unsetopt

Toggle zsh-style options. Options live in `executor.options`; **setopt foo** turns on, **unsetopt foo** turns off.

```
setopt extended_glob
setopt null_glob
unsetopt nomatch
```

shopt

Bash-compatible option toggling. Maps onto the same options table as **setopt**.

```
shopt -s globstar
```

emulate [-LR] [shell]

Switch emulation mode (**zsh**, **sh**, **ksh**, **csh**). **-L** local-to-function, **-R** reset all options to that shell's defaults.

```
emulate -L zsh          # function-local zsh defaults
```

getopts spec var

POSIX option parsing. Iterates flags from `$@`, sets `$var` to each option, `$OPTARG` to its value.

```
while getopts "vh:" opt; do
  case $opt in
    v) verbose=1 ;;
    h) host=$OPTARG ;;
  esac
done
```

zparseopts

zsh's richer option parser. **-A arr** stores into an assoc array; supports long options.

```
zparseopts -A opts -- v h: c::
echo "${opts[-h]}"
```

autoload [name...]

Mark a function for lazy loading from `$fpath`. zshrs's autoload path uses SQLite-cached bytecodes: first invocation deserializes a chunk; subsequent invocations skip lex/parse/compile entirely.

```
autoload -Uz compinit
compinit
```

functions [name]

List defined functions, or print a function's body. `+f` prints names only.

```
functions          # all functions
functions my_func  # body of my_func
```

unfunction name

Remove a function definition. Same as `unset -f`.

```
unfunction old_helper
```

trap 'cmd' SIG

Install a signal handler. Special pseudo-signals: `EXIT` (run at shell exit), `DEBUG` (before each command), `ERR` (on non-zero status), `ZERR` (zsh-specific).

```
trap 'rm -rf $tmpdir' EXIT
trap 'echo got SIGINT' INT
```

pushd / popd / dirs

Directory stack manipulation. `pushd dir` changes to `dir` and pushes onto the stack; `popd` pops and changes back. `dirs -v` lists.

```
pushd ~/work/proj
...
popd
```

alias name='cmd'

Define a command alias. With no args lists all aliases. `alias -g` creates a global alias (substitutes anywhere on the command line).

```
alias ll='ls -lAh'
alias -g G='| grep'
```

unalias [-m pattern] name

Remove an alias. `-m` matches by glob pattern.

```
unalias ll
unalias -m 'g*'  # remove all aliases starting with g
```

type / whence / where / which

Resolve a name to its source: alias, function, builtin, or external. `whence -p` shows path only; `whence -a` shows all matches.

```
type ls
whence -a cargo
which gcc
```

hash / rehash / unhash

Command-name → path cache. **rehash** rebuilds the cache from **\$PATH** (parallel scan across worker pool). **unhash -dm 'pat'** removes named-directory entries by glob.

```
rehash          # after installing new binaries
hash            # show cache
unhash -dm 'tmp*' # remove tmp* hashed dir refs
```

ulimit / limit / unlimit

Resource limit query/set. **-n** open files, **-s** stack size, **-u** max processes, **-c** core dump.

```
ulimit -n 65536
ulimit -a      # show all
```

umask [mask]

Set/show file-creation mask.

```
umask 077      # owner-only by default
```

times

Print user + system times for the shell and its children.

```
times
```

caller

Print the line + filename of the caller of the current function. Useful in error reporters and debuggers.

```
my_assert() {
  [[ $1 -eq $2 ]] || { echo "assertion failed at ${caller}"; exit 1; }
}
```

help [name]

Print help for a builtin (or list builtins).

```
help echo
help
```

enable / disable

Enable/disable a builtin by name. Useful for testing external-vs-builtin behavior.

```
disable cat      # falls back to /bin/cat
enable cat       # restore zshrs's cat builtin
```

noglob cmd

Run cmd with glob expansion disabled for its arguments. Equivalent to **setopt noglob; cmd; unsetopt noglob** as a one-liner.

```
noglob find . -name '*.Is'  # don't pre-glob *.Is
```

ttctl -f / ttctl -u

Freeze / unfreeze the tty state. Used by completion code that reads stdin.

```
ttctl -f      # freeze
ttctl -u      # unfreeze
```

sync

Force the OS to flush its buffer cache to disk. Direct `sync(2)` syscall.

```
cp big.iso /mnt/usb/
sync
```

zmodload [-i] mod

Load a zsh module. zsh's modules are statically linked, so `zmodload` is a no-op-and-succeed for compat (`-i` idempotent flag).

```
zmodload zsh/datetime
zmodload zsh/regex
```

zsleep n

Subsecond-precision sleep. `zsleep 0.5` sleeps 500ms. Different from coreutils `sleep` only in supporting fractional seconds reliably across implementations.

```
zsleep 0.25      # 250ms
```

zsystem subcmd

zsh/system module operations. `zsystem flock file` for file locking; `zsystem getflags` for flag dumps.

```
zsystem flock /tmp/lockfile
```

strftime fmt [time]

Format a unix timestamp using `strftime(3)` directives. Default time is *now*.

```
strftime "%Y-%m-%d %H:%M:%S"
```

mkdir [-p] dirs...

Create directories. `-p` creates parents and doesn't fail if exists.

```
mkdir -p ~/.config/myapp/{cache,logs}
```

vared VAR

Edit a variable's value via the line editor. Handy for interactive config edits.

```
vared PATH
```

zformat -f var fmt args...

Sprintf with `%X:value` argument bindings; common in completion display formatting.

```
zformat -f line "%name (%size)" name:foo size:42
```

zregexparse

State-machine regex parser used by completion and filename rewriting.

```
zregexparse vname tname regex
```

zcompile file

Pre-compile a zsh script to `.zwc`. zshrs treats `.zwc` as one of its bytecode-cache inputs and deserializes the chunk directly when sourcing the original file.

```
zcompile ~/.zshrc
```

Job Control & Background

11 topics · `cmd &` compiles to a sub-chunk + `BUILTIN_RUN_BG` (id 290) fork dispatch; full job-table integration is Phase G6.

`cmd & -- background`

Run `cmd` in the background. The compile path detects `ListOp::Amp`, compiles `cmd` into a sub-chunk, emits `CallBuiltin(BUILTIN_RUN_BG, 1)`. The handler forks; child `setsids` and runs the chunk on a fresh VM; parent returns `Status(0)` immediately.

```
sleep 30 &
make all & make docs & wait
```

`jobs`

List background jobs known to the shell. Note: pids spawned via `BUILTIN_RUN_BG` aren't yet registered in the `JobTable` (Phase G6); externally-launched bg cmds via `std::process::Command::spawn()` are.

```
jobs          # active jobs
jobs -l       # with pids
```

`fg [%jobspec]`

Bring a background/stopped job to the foreground. `%1, %2` select by job number; `%name` by command-name prefix.

```
fg %1
fg %vim
```

`bg [%jobspec]`

Resume a stopped job in the background.

```
bg %1
```

`kill [-SIG] pid|%job`

Send a signal. Default is `SIGTERM`. Builtin form supports both PID and jobspec.

```
kill -INT 12345
kill %1
kill -l          # list signals
```

`wait [pid|%job]`

Wait for a child to exit; with no args, waits for all bg children. Status is the wait-on'd child's exit code.

```
worker1.sh & pid1=$!
worker2.sh & pid2=$!
wait $pid1 $pid2
```

`disown [%job]`

Remove a job from the job table without killing it. The process keeps running but is no longer tracked.

```
long_task &  
disown %1
```

suspend

Stop the current shell (sends `SIGTSTP` to itself). Common idiom in nested shells.

```
suspend
```

async 'cmd'

Ship work to the worker pool and return an opaque ID. Unlike `&`, this runs on a thread within the same process — no fork, no `setsid`.

```
id=$(async 'sleep 5; curl https://api.example.com')
```

await ID

Wait for an async task by ID and capture its stdout.

```
id=$(async 'expensive-job')  
result=$(await $id)
```

\$! — last bg pid

PID of the most-recently-backgrounded command. Used to wait/signal it later.

```
long_task &  
echo "spawned $!"  
wait $!
```

Anti-Fork Coreutils

23 topics · **2000-5000x faster** than forking to /bin — every invocation runs in-process via direct syscalls.

cat [files...]

Concatenate files to stdout. With no args, copies stdin. **-n** numbers lines; **-A** shows non-printable. No fork.

```
cat /etc/hosts
cat *.log | grep ERROR
```

head [-n N] [files...]

First N lines (default 10). **-c N** first N bytes.

```
head -20 /var/log/system.log
```

tail [-n N] [-f] [files...]

Last N lines. **-f** follow (in-process).

```
tail -f access.log
```

wc [-lwc] [files...]

Line / word / char count.

```
wc -l *.rs          # total Rust LOC
```

sort [-n -r -u -k] [files...]

Sort lines. **-n** numeric, **-r** reverse, **-u** unique, **-k N** sort on key.

```
du -h * | sort -hr
```

find path [predicates]

Walk directories applying predicates. **-name**, **-type**, **-size**, **-mtime**, **-exec**. In-process walk.

```
find . -name '*.rs' -size +1k
```

uniq [-c -d -u] [files...]

Filter adjacent duplicates. **-c** prefix counts, **-d** show duplicates only.

```
cat history.log | sort | uniq -c | sort -nr
```

cut [-d delim] -f fields

Extract fields from each line.

```
cut -d: -f1 /etc/passwd    # all usernames
```

tr SET1 SET2

Translate or delete characters.

```
echo HELLO | tr A-Z a-z  
tr -d '\r' < crlf.txt > lf.txt
```

seq [start] [step] end

Print a sequence of numbers.

```
for i in $(seq 1 10); do ...; done  
seq 0 0.5 5
```

rev

Reverse each line character-wise.

```
echo hello | rev # olleh
```

tee [-a] file...

Copy stdin to stdout AND to one or more files. **-a** appends.

```
build.sh 2>&1 | tee build.log
```

basename path [suffix]

Strip directory and optionally a trailing suffix.

```
basename /tmp/file.txt # file.txt  
basename /tmp/file.txt .txt # file
```

dirname path

Strip the trailing component.

```
dirname /tmp/file.txt # /tmp
```

touch [-a -m -t time] file...

Update file timestamps; create empty file if missing.

```
touch deploy.lock
```

realpath path

Resolve symlinks and relative paths to a canonical absolute path.

```
realpath ./src/exec.rs
```

sleep seconds

Pause for *seconds* (fractional supported).

```
sleep 0.5
```

whoami

Effective username (direct `geteuid` + `getpwuid` syscalls).

```
echo "running as $(whoami)"
```

id [-u -g -n] [user]

User/group ID info.

```
id -u          # numeric uid
id -un         # username
```

hostname [-s -f]

System hostname. Direct syscall.

```
hostname       # short
hostname -f    # FQDN
```

uname [-a -s -r -m]

Kernel name / release / arch. Direct `uname(3)`.

```
uname -sim
```

date [+fmt]

Print current date/time. `+fmt` uses `strftime` directives. Direct syscall.

```
date
date +%s
date "+%Y-%m-%d %H:%M:%S"
```

mktemp [-d] [template]

Atomic temp file/dir creation.

```
tmp=$(mktemp)
tmpdir=$(mktemp -d)
```

Parallel Primitives

5 topics · all run on the persistent worker pool, not via fork. Bytecode-VM execution under [rayon](#) task scheduler.

`pmap 'cmd {}' args...`

Parallel map across worker pool. Output preserves input order. The `{}` placeholder gets each arg.

```
pmap 'gzip {}' *.log
pmap 'sha256sum {}' **/*.iso
```

`pgrep 'pred {}' args...`

Parallel filter. Predicate runs concurrently; matching args are collected in input order.

```
pgrep 'grep -q TODO {}' **/*.rs    # files containing TODO
```

`peach 'cmd {}' args...`

Parallel for-each, unordered. Fire each task and don't preserve order — useful when output order doesn't matter.

```
peach 'convert {} {}.png' *.svg
```

`barrier 'cmd1' ::: 'cmd2' ::: 'cmd3'`

Run all commands in parallel; wait for all to complete. Final status is the worst (highest) of all child statuses.

```
barrier 'cargo test' ::: 'cargo doc' ::: 'cargo audit'
```

`barrier --fail-fast ...`

Cancel remaining tasks as soon as one fails. Useful for guarded fan-out (*compile*, *test*, *lint*) where one failure makes the rest moot.

```
barrier --fail-fast 'rustfmt --check src/' ::: 'cargo clippy'
```

AOP Intercept

3 topics · the first shell with aspect-oriented programming. `before/after/around` advice. Routes through `host_exec_external` → `run_intercepts`.

`intercept before pat { code }`

Run `code` before any command matching `pat`. `$INTERCEPT_NAME`, `$INTERCEPT_ARGS`, `$INTERCEPT_CMD` are bound. Pattern is glob: `git`, `git*`, `_*`.

```
intercept before git { echo "[$(date)] $INTERCEPT_CMD" >> ~/git.log }
```

`intercept after pat { code }`

Run after the matched command finishes. Adds `$INTERCEPT_MS` (elapsed) and `$INTERCEPT_STATUS`.

```
intercept after '*' {  
  echo "$INTERCEPT_NAME took ${INTERCEPT_MS}ms"  
}
```

`intercept around pat { code }+intercept_proceed`

Wrap the command. Call `intercept_proceed` from inside to invoke the original; skip it to suppress the original entirely. Replaces `defer`, `profile`, `memo`, `retry`, `timeout` with one primitive.

```
intercept around expensive_func {  
  local cache=/tmp/cache_${INTERCEPT_ARGS// /_}  
  if [[ -f $cache ]]; then  
    cat $cache  
  else  
    intercept_proceed | tee $cache  
  fi  
}
```

Diagnostics & Profiling

5 topics · zshrs-exclusive observability builtins.

doctor

Full diagnostic dump: worker-pool metrics, SQLite cache stats, bytecode coverage, registered intercepts, history db size, options table.

```
doctor
```

dbview [table] [filter]

Browse SQLite caches without writing SQL. Lists tables + row counts; with arguments shows specific entries.

```
dbview                                # tables + row counts
dbview autoloads _git                 # one autoload entry
dbview comps git                      # completion search
dbview history docker                # history search
```

profile cmd

In-process profiling with nanosecond accuracy. No fork, no `strace`. Reports CPU time, syscalls, allocations.

```
profile cargo build
```

zprof

Function-level profile (zsh module compat). Enable with `zmodload zsh/zprof`; print accumulated stats with `zprof`.

```
zmodload zsh/zprof
source ~/.zshrc
zprof | head -20
```

history [-n N] [pattern]

Browse command history. Backed by SQLite + FTS5 — full-text search, frequency-ranked, with timestamps and exit status per command.

```
history -10                          # last 10
history docker                        # FTS search
```

Completion System

10 topics · compsys backed by rkyv-mmap'd shards (zero-copy hot path). Read-only SQLite mirrors sit beside them for [dbview](#) / SQL inspection only — no role in Tab cache hit/miss. [compinit](#) pre-warms the autoload index across the worker pool.

compinit

Initialize the completion system. zshrs's [compinit](#) is async: fpath is scanned + bytecode-compiled in the background while the prompt drops.

```
autoload -Uz compinit && compinit
```

compdef func cmd

Bind a completion function to a command name.

```
compdef _docker docker  
compdef _kubectl kubectl k
```

compadd [-J group] words...

Add candidate completions. [-J](#) groups them; [-d desc](#) attaches descriptions.

```
compadd -J commands start stop status restart
```

compset

Manipulate the completion state — peel off prefixes, set the position, etc.

```
compset -P 'http:/'  
compset -S '*.com'
```

compopt

Toggle completion options for the current invocation.

```
compopt -o nospace
```

compgen [opts] word

Bash-compatible completion candidate generator.

```
compgen -c git          # all commands starting with "git"
```

complete

Bash-compatible completion definition.

```
complete -W "start stop status" myservice
```

zstyle

Style configuration for completion + prompt. zsh's universal config DSL.

```
zstyle ':completion:*' menu select
zstyle ':completion:*' matcher-list 'm:{a-zA-Z}={A-Za-z}'
```

cdreplay

Replay deferred `compdef` calls accumulated during async `compinit`.

```
compinit; cdreplay -q
```

promptinit + prompt

Theme system for prompts.

```
autoload -Uz promptinit && promptinit
prompt redhat
```

ZLE Line Editor

4 topics · 26 ZLE widgets, syntax highlighting, autosuggestions, abbreviations.

`zle widget [-N name handler]`

Manipulate the line editor. `-N name handler` defines a user widget; `zle name` invokes one.

```
my-widget() { LBUFFER+=" --verbose"; }  
zle -N my-widget  
bindkey '^Xv' my-widget
```

`bindkey [seq widget]`

Bind a key sequence to a widget. `-M map` targets a specific keymap.

```
bindkey '^R' history-incremental-search-backward  
bindkey -M vicmd 'k' up-line-or-beginning-search
```

`vared VAR (line-edit form)`

Use ZLE to edit a variable's value interactively.

```
vared LS_COLORS
```

Abbreviations

Fish-style abbreviations: type the abbrev + space, the full command expands inline. Configured via the `fish/abbrs.rs` backend.

```
abbr gco='git checkout'  
abbr -g G='| grep'
```

PCRE & Regex

4 topics · zsh/pcre module support; the `=~` operator inside `[[ ... ]]` uses PCRE2.

`pcre_compile` pattern

Compile a PCRE pattern; bound to `$ZPCRE_OP`.

```
pcre_compile '^(\d+)\s+(\w+)$'
```

`pcre_match` string

Match against the most-recently compiled pattern. Captures land in `$match`, `$mbegin`, `$mend`.

```
pcre_compile '(\w+)@(\w+\.\w+)'
if pcre_match "alice@example.com"; then
  echo "user: ${match[1]}, domain: ${match[2]}"
fi
```

`pcre_study`

Optimize the compiled pattern for repeated use.

```
pcre_compile '\b\d{4}-\d{2}-\d{2}\b'
pcre_study
```

`[[ string =~ pattern ]]`

POSIX/PCRE regex match inside the conditional. Routed through `Op::RegexMatch` (host method `regex_match`).

```
if [[ $version =~ ^([0-9]+)\.([0-9]+)\.([0-9]+) ]]; then
  major=${match[1]}; minor=${match[2]}; patch=${match[3]}
fi
```

Recently Closed Compat Gaps (eighty-eighth-pass)

46 constructs · all verified against `~/forkedRepos/zsh/5rc/` via direct C-source ports. Each entry is pinned by at least one assertion in `tests/zshrs_shell.rs`; workspace-wide test hooks today: 14,552 (see header strip). These were the highest-impact gaps in `man zshall` coverage that remained after the seventy-seventh pass — most are param expansion, pattern matching, function scoping, and trap behavior.

Glob & Pattern

`extendedglob` ~ exclusion at PATH level

Direct port of `pattern.c` `P_EXCLUDE`: matches RHS as a pattern against each LHS candidate's basename + full path, not as a separate CWD glob.

```
setopt extendedglob
echo /tmp/dir/*.txt~*README* # all .txt except README.txt
```

`/^pat` at any path component

Negation operator now triggers in any path component, not just the leading word.

```
setopt extendedglob
echo /tmp/dir/^skipme # everything in /tmp/dir except files matching skipme
```

`$D/*`, `$D/(a|b)` – glob meta after var ref

New `BUILTIN_GLOB_EXPAND` (id 343): pops a scalar pattern, runs `expand_glob`, pushes `Value::Array`. Compile path detects glob meta in literal segments only (so `$?/$#`/etc don't trigger).

```
D=/tmp/build
echo $D/* # globs after $D substitution
echo $D/(a|b) # alternation after $D substitution
```

`**/*` recursive sort by full path

Recursive globs sort by full-path lex order (zsh's depth-first walk). Non-recursive globs keep basename-sorted output.

```
echo /tmp/dir/**/* # f, sub, sub/g – not f, g, sub
```

Parameter Expansion

`${${a%.txt}#hel}` – outer strip on inner result

Direct port of `subst.c` `getarg` machinery — same dispatch for inner and outer.

```
a=hello.txt
echo "${${a%.txt}#hel}" # lo
```

`${(s. .)}${(j. .)a}` – outer flag on inner expansion

Flag handler now recurses on leading `${` in `rest`; applies `U/L/C/Split/Join/SplitWords/SplitLines` to inner result.

```
a=(a b c)
echo "${(s. .)}${(j. .)a}" # a b c (split-then-rejoin)
```

`${(flags)}$(...)` – cmd-subst as flag operand

Branch added for cmd-subst operands. `(P)` indirect honored: captured output is treated as a variable name and looked up.

```
a=hi; echo "${(P)}$(echo a)" # hi (NOT "a")
echo "${(U)}$(echo hello)" # HELLO
echo "${(z)}$(echo a b c)" # a b c
```

`${(U)}${(s. .)s}[N]}` – INI subscript after inner expansion

Splits the flag-applied joined-scalar back to parts, indexes, re-applies case-transform flags.

```
s="x y z"
echo "${(U)}${(s. .)s}[1]}" # X
```

`${(l:N::s2:)val}` – pad with empty s1 + s2 fallback

Pad parser collects both strings; when s1 empty and s2 given, s2 acts as fill char.

```
echo "${(l:5::0:)42}" # 00000 (zsh-faithful)
```

`${(j[+])a}`, `${(s[|])s}` – bracket-pair flag delimiters

zsh subst.c `get_strarg` accepts matched bracket pairs as flag delimiters: `[/]`, `{/}`, `(/)`, `</>`. Both flag-parser sites updated.

```
a=(a b c)
echo "${(j[+])a}" # a+b+c
echo "${(j<X>)a}" # aXbXc
echo "${(s[|])\"x|y|z\"}" # x y z
```

`:Q` modifier – backslash escape removal

`hist.c` `remquote`: strips paired `'/'` AND `\X` backslash escapes. Both `:Q` paths fixed.

```
a="a\\ b"
echo ${a:Q} # a b
```

`${a//\X/rep1}` – backslash unescape for non-meta chars

Pattern pre-pass strips `\X` when X is not a glob meta. `\?/\*[/\]/\(/)/\|/\` still escape.

```
a="x:y:z"
echo "${a//\:/-}" # x-y-z
echo "${a//\./X}" # works on dots too
```

`"${(o)a[@]}"`, `"${(O)a[@]}"`, `"${(n)a[@]}"` – sort flags survive DQ when `!@` given

Compile path encodes the at-subscript context through a new `\u{03}` sentinel in the flags string so the runtime DQ-stripper preserves array-only flags.

```
a=(c a b)
echo "${(o)a[@]}" # a b c
echo "${(O)a[@]}" # c b a
a=(10 2 1 22)
echo "${(n)a[@]}" # 1 2 10 22
```

`${SECONDS-default}` – zsh-special params always-set

Whitelist treats `SECONDS`, `EPOCHSECONDS`, `EPOCHREALTIME`, `RANDOM`, `LINENO`, `HISTCMD`, `PPID`, `UID`, `EUID`, `GID`, `EGID`, `SHLVL` as set even when not in `self.variables`. `HISTCMD` getter also added.

```
echo "${SECONDS-default}" # 0 (or current value)
echo "${UID-default}"    # 501
echo "${HISTCMD-default}" # 0
```

Arrays & Subscripts

Associative-array key insertion order

Storage switched from `HashMap` to `indexmap::IndexMap` so `${(k)h}` / `${(kv)h}` iterate in insertion order matching zsh's `HashTable` `hnodes`.

```
typeset -A h
h=(a 1 b 2 c 3)
echo ${(k)h}      # a b c
echo ${(kv)h}     # a 1 b 2 c 3
```

for k v in arr — multi-name for loop

Direct port of `parse.c par_for`: parser collects all leading identifiers; compiler emits N-stride iteration with empty-string fill on short tail.

```
arr=(a 1 b 2 c 3)
for k v in $arr; do echo "$k=$v"; done

typeset -A h=(a 1 b 2)
for k v in ${(kv)h}; do echo "$k=$v"; done
```

b=("\${a[@]}") — array-to-array splice preserves boundaries

New `scalar_assign_depth` separate from `assign_context_depth` — only scalar RHS forces `JOIN_STAR`. Array init keeps splice.

```
a=("1 2" "3 4")
b=("${a[@]}")
echo $#b      # 2 (preserves both elements)
```

\$a[@], \$a[*] — bare (no-braces) splice

`array_splice_ref` extended to accept the no-braces form, identical semantics to `${a[@]}`.

```
a=(x y z)
printf "%s\n" $a[@]      # x, y, z (3 lines)
f() { echo $#; }; f $a[@] # 3
```

b="\${a[@]}" — scalar assignment joins via JOIN_STAR

Compile path forces `BUILTIN_ARRAY_JOIN_STAR` when `scalar_assign_depth > 0` (zsh `subst.c` forces single-string for scalar RHS).

```
a=(1 2 3)
b="${a[@]}"
echo $b    # 1 2 3
```

a[\$n]=() — variable subscript element-remove

Compile path now routes the subscript through `compile_word_str` when key has `$` or backtick.

```

a=(1 2 3 4)
n=3
a[$n]=() # remove 3rd element
echo "${a[@]}" # 1 2 4
a=(1 2 3 4)
a[$#a]=() # remove last
echo "${a[@]}" # 1 2 3

```

`${h[(I)pat]}`, `${h[(R)pat]}` on `assoc` – return ALL matches

`(I)/(R)` return all matching keys/values space-joined; `(i)/(r)` return first. `(i)/(I)` search keys; `(r)/(R)` search values.

```

typeset -A h=(a 1 b 2)
echo "${h[(I)*]}" # a b
echo "${h[(i)*]}" # a
echo "${h[(I)a]}" # a (single match)
typeset -A h=(a 1 b 1 c 2)
echo "${h[(R)1]}" # 1 1

```

`typeset -a a` preserves existing array at top scope

Bare-declaration path now guards with `in_function || !exists`. `typeset -aU` dedupes in place.

```

a=(1 2 3); typeset -a a; echo $a # 1 2 3 (was empty)
a=(a b a c b); typeset -aU a; echo $a # a b c (was empty)

```

Arithmetic

`((i=a[2]))` – RHS array subscript pre-resolve

[added to `needs_eval` trigger so `MathEval` (which runs `pre_resolve_array_subscripts`) handles it.

```

a=(10 20 30)
((i=a[2])); echo $i # 20
((sum=a[1]+a[2]+a[3])); echo $sum # 60

```

`((h[a]++))`, `((h[a]+=v))` – compound ops on `assoc`

Direct port of `zsh's math.c LVAL_NUM_SUBSC`: subscript receiver retains lvalue identity through compound operators.

```

typeset -A h
h[a]=5
((h[a]++)) # h[a] = 6
((h[a]+=10)) # h[a] = 16

```

`((++a[i]))`, `((++h[k]))` – pre-increment on subscript

New `parse_subscript_arith_pre_inc` + compile-side `subscripted_arith_compound_check` accepts the pre-op shape. Pre-op returns NEW value, post-op OLD.

```

a=(10 20 30)
((++a[2])); echo $a # 10 21 30
typeset -A h
h[a]=5
((++h[a])); echo $h[a] # 6

```

`((a = cond ? T : F))` – ternary in assignment

? added to `needs_eval` triggers so `MathEval` handles ternary fully. `ArithCompiler` can't write back through `?:`.

```

((a = 5 > 3 ? 99 : 0)); echo $a # 99

```

abs/min/max/int/floor/ceil/trunc – int-preserving

Int-input return int (not 5.). Float input still returns float.

```
echo ${abs(-5)}      # 5   (was 5.)
echo ${max(3,5,7)}   # 7
echo ${abs(-5.5)}    # 5.5 (still float)
```

Conditionals & Parser

`[[ a == a && (b == b || c == c) ]]` – cond grouping parens

Lexer's `incondpat` resets on `&&`, `||`, `(`, `)`, `!`, `]` per `cond.c par_cond_3`.

```
[[ a == a && (b == b || c == c) ]] && echo y
```

`case W in (P|Q)) BODY ;;` – wrapped pattern with alternation

Parser accepts both bare `(P) BODY` and wrapped `(P)) BODY` (the `(...)` is the pattern wrapper, the second `)` is the arm-close).

```
case foo in
  (foo|bar)) echo y;;
  (*)      echo n;;
esac
```

Functions & Scoping

`. file.sh ARG1 ARG2` – source passes positionals

Save outer `positional_params`, install `args[1..]` as new positionals, restore on exit.

```
# file.sh: echo "$1=$2"
. ./file.sh hi bye      # hi=bye

set -- a b c
. ./file.sh inner       # inside file.sh: $1=inner
echo "$@"               # outside: a b c (preserved)
```

`typeset -A h=(...)` in function shadows outer

New `local_assoc_save_stack` mirrors `local_array_save_stack` lifecycle. Both `call_function` paths (legacy + byte-code) updated.

```
typeset -A h=(a 1)
f() { typeset -A h=(b 2); echo $h[b]; }
f
echo $h[a]      # 1 (outer preserved)
echo "${h[b]-empty}" # [empty] (inner didn't leak)
```

Subshell umask snapshot+restore

`SubshellSnapshot` snapshots `libc::umask`; `subshell_end` restores. `zsh` forks for `(...)` so `umask` dies with child; we run in-process so we manually reset.

```
umask 022
(umask 077) # subshell
umask      # 022 (parent unchanged)
```

Subshell EXIT trap fires at subshell end

Was firing at process exit. Now fires before parent continues. `SubshellSnapshot` snapshots+restores parent traps; inner trap fires with `traps.remove("EXIT")` so the inner `execute_script` doesn't recurse.

```
(trap "echo trapped" EXIT; true)
echo done # output: trapped\ndone (was: done\ntrapped)
```

Brace & Word Expansion

{one,\${a},three} – brace expansion after var substitution

Segment-concat path now emits `BUILTIN_BRACE_EXPAND` after concat when any literal segment contains { or }.

```
a=hi
echo {one,${a},three} # one hi three
echo pre{1,${a},2}post # pre1post prehipost pre2post
```

Builtins & I/O

print -C N – column-padded output

Per-column width padding with 2-space separator (zsh format). Trailing partial rows skip padding after the last present item.

```
print -C 2 a b c d
# a c
# b d
print -C 2 alpha beta gamma delta
# alpha gamma
# beta delta
```

read -e / read -E – echo line

zsh's `bin_read` calls `fputs(buf, stdout)` under both. `-e` echoes only (no assign); `-E` echoes AND assigns.

```
echo "abc" | { read -E v; echo "[$v]"; }
# abc
# [abc]
echo "abc" | { read -e v; echo "[$v]"; }
# abc
# []
```

trap "... ZERR / trap "... ERR – fires on non-zero status

Wired into `BUILTIN_ERREXIT_CHECK`. Runs the trap body before the `errexit` decision; `last_status` saved/restored to prevent recursion on trap-body failures.

```
trap "echo zerr" ZERR
false
echo done
# zerr
# done
```

Compat Builtin Index

156 entries · zsh-faithful ports from `ported::builtin::BUILTINS`. Each mirrors an upstream `Src/Builtins/*.c` entry 1:1, with the hover body extracted from `man zshall` yodl. See also: [Extension Builtin Index](#) for zshrs-only additions.

- 𐀀

The command is executed with a "-" prepended to its `argv[0]` string.

. 𐀀

Read commands from *file* and execute them in the current shell environment.

If *file* does not contain a slash, or if `PATH_DIRS` is set, the shell looks in the components of `$path` to find the directory containing *file*. Files in the current directory are not read unless "." appears somewhere in `$path`. If a file named `_file_.zwc` is found, is newer than `_file_`, and is the compiled form (created with the `zcompile` builtin) of `_file_`, then commands are read from that file instead of `_file_`.

If any arguments *arg* are given, they become the positional parameters; the old positional parameters are restored when the *file* is done executing. However, if no arguments are given, the positional parameters remain those of the calling context, and no restoring is done.

If *file* was not found the return status is 127; if *file* was found but contained a syntax error the return status is 126; else the return status is the exit status of the last command executed.

: 𐀀

Null command. Returns true. Side-effects of argument expansion still happen.

[𐀀

POSIX `test` command (also spelled `test`). Same conditional semantics as POSIX `test`. Prefer `[[ ... ]]` in zsh — it's safer (no word splitting) and supports more operators.

alias 𐀀

For each *name* with a corresponding *value*, define an alias with that value. A trailing space in *value* causes the next word to be checked for alias expansion. If the `-g` flag is present, define a global alias; global aliases are expanded even if they do not occur in command position:

```
% perldoc --help 2>&1 | grep 'built-in functions' -f Search Perl built-in functions % alias -g HG='--help 2>&1 | grep' % perldoc HG 'built-in functions' -f Search Perl built-in functions
```

If the `-s` flag is present, define a suffix alias: if the command word on a command line is in the form `_text_.name`, where *text* is any non-empty string, it is replaced by the text `_value_ _text_.name`. Note that *name* is treated as a literal string, not a pattern. A trailing space in *value* is not special in this case. For example,

```
alias -s ps='gv --'
```

will cause the command `"*.ps"` to be expanded to `"gv -- *.ps"`. As alias expansion is carried out earlier than globbing, the `"*.ps"` will then be expanded. Suffix aliases constitute a different name space from other aliases (so in the above example it is still possible to create an alias for the command `ps`) and the two sets are never listed together.

For each *name* with no *value*, print the value of *name*, if any. With no arguments, print all currently defined aliases other than suffix aliases. If the `-m` flag is given the arguments are taken as patterns (they should be quoted to preserve them from being interpreted as glob patterns), and the aliases matching these patterns are printed. When printing aliases and one of the `-g`, `-r` or `-s` flags is present, restrict the printing to global, regular or suffix aliases, respectively; a regular alias is one which is neither

a global nor a suffix alias. Using "+" instead of "-", or ending the option list with a single "+", prevents the values of the aliases from being printed.

If the `-L` flag is present, then print each alias in a manner suitable for putting in a startup script. The exit status is nonzero if a *name* (with no *value*) is given for which no alias has been defined.

For more on aliases, including common problems, see *Aliasing* (zshmisc).

autoload ¶

`vindex(fpath, searching)` See *Autoloading Functions* (zshmisc) for full details. The `fpath` parameter will be searched to find the function definition when the function is first referenced.

If *name* consists of an absolute path, the function is defined to load from the file given (searching as usual for dump files in the given location). The name of the function is the basename (non-directory part) of the file. It is normally an error if the function is not found in the given location; however, if the option `-d` is given, searching for the function defaults to `$fpath`. If a function is loaded by absolute path, any functions loaded from it that are marked for `autoload` without an absolute path have the load path of the parent function temporarily prepended to `$fpath`.

If the option `-r` or `-R` is given, the function is searched for immediately and the location is recorded internally for use when the function is executed; a relative path is expanded using the value of `$PWD`. This protects against a change to `$fpath` after the call to `autoload`. With `-r`, if the function is not found, it is silently left unresolved until execution; with `-R`, an error message is printed and command processing aborted immediately the search fails, i.e. at the `autoload` command rather than at function execution..

The flag `-X` may be used only inside a shell function. It causes the calling function to be marked for autoloading and then immediately loaded and executed, with the current array of positional parameters as arguments. This replaces the previous definition of the function. If no function definition is found, an error is printed and the function remains undefined and marked for autoloading. If an argument is given, it is used as a directory (i.e. it does not include the name of the function) in which the function is to be found; this may be combined with the `-d` option to allow the function search to default to `$fpath` if it is not in the given location.

The flag `+X` attempts to load each *name* as an autoloaded function, but does *not* execute it. The exit status is zero (success) if the function was not previously defined *and* a definition for it was found. This does *not* replace any existing definition of the function. The exit status is nonzero (failure) if the function was already defined or when no definition was found. In the latter case the function remains undefined and marked for autoloading. If ksh-style autoloading is enabled, the function created will contain the contents of the file plus a call to the function itself appended to it, thus giving normal ksh autoloading behaviour on the first call to the function. If the `-m` flag is also given each *name* is treated as a pattern and all functions already marked for autoload that match the pattern are loaded.

With the `-t` flag, turn on execution tracing; with `-T`, turn on execution tracing only for the current function, turning it off on entry to any called functions that do not also have tracing enabled.

With the `-U` flag, alias expansion is suppressed when the function is loaded.

With the `-w` flag, the `_name_s` are taken as names of files compiled with the `zcompile` builtin, and all functions defined in them are marked for autoloading.

The flags `-z` and `-k` mark the function to be autoloaded using the zsh or ksh style, as if the option `KSH_AUTOLOAD` were unset or were set, respectively. The flags override the setting of the option at the time the function is loaded.

Note that the `autoload` command makes no attempt to ensure the shell options set during the loading or execution of the file have any particular value. For this, the `emulate` command can be used:

```
emulate zsh -c 'autoload -Uz func'
```

arranges that when *func* is loaded the shell is in native `zsh` emulation, and this emulation is also applied when *func* is run.

Some of the functions of `autoload` are also provided by `functions -u` or `functions -U`, but `autoload` is a more comprehensive interface.

bg ¶

Put each specified *job* in the background, or the current job if none is specified.

bindkey ¶

bindkey's options can be divided into three categories: keymap selection for the current command, operation selection, and others. The keymap selection options are:

- **-e** — Selects keymap "emacs" for any operations by the current command, and also links "emacs" to "main" so that it is selected by default the next time the editor starts.
- **-v** — Selects keymap "viins" for any operations by the current command, and also links "viins" to "main" so that it is selected by default the next time the editor starts.
- **-a** — Selects keymap "vicmd" for any operations by the current command.
- **-M keymap** — The *keymap* specifies a keymap name that is selected for any operations by the current command.

If a keymap selection is required and none of the options above are used, the "main" keymap is used. Some operations do not permit a keymap to be selected, namely:

- **-l** — List all existing keymap names; if any arguments are given, list just those keymaps.

If the **-l** option is also used, list in the form of **bindkey** commands to create or link the keymaps. **bindkey -lL main'** shows which keymap is linked to "main", if any, and hence if the standard emacs or vi emulation is in effect. This option does not show the .safe keymap because it cannot be created in that fashion; however, neither is "bindkey -ll .safe" reported as an error, it simply outputs nothing.

- **-d** — Delete all existing keymaps and reset to the default state.
- **-D keymap ...** — Delete the named *_keymap_s*.
- **-A old-keymap new-keymap** — Make the *new-keymap* name an alias for *old-keymap*, so that both names refer to the same keymap. The names have equal standing; if either is deleted, the other remains. If there is already a keymap with the *new-keymap* name, it is deleted.
- **-N new-keymap [old-keymap]** — Create a new keymap, named *new-keymap*. If a keymap already has that name, it is deleted. If an *old-keymap* name is given, the new keymap is initialized to be a duplicate of it, otherwise the new keymap will be empty.

To use a newly created keymap, it should be linked to **main**. Hence the sequence of commands to create and use a new keymap "mymap" initialized from the **emacs** keymap (which remains unchanged) is:

```
bindkey -N mymap emacs bindkey -A mymap main
```

Note that while **bindkey -A _newmap_ main'** will work when *_newmap_* is emacs or viins, it will not work for vicmd, as switching from vi insert to command mode becomes impossible.

The following operations act on the "main" keymap if no keymap selection option was given:

- **-m** — Add the built-in set of meta-key bindings to the selected keymap. Only keys that are unbound or bound to **self-insert** are affected.
- **-r in-string ...** — Unbind the specified *_in-string_s* in the selected keymap. This is exactly equivalent to binding the strings to **undefined-key**.

When **-R** is also used, interpret the *_in-string_s* as ranges.

When **-p** is also used, the *_in-string_s* specify prefixes. Any binding that has the given *in-string* as a prefix, not including the binding for the *in-string* itself, if any, will be removed. For example,

```
bindkey -rpM viins '^['
```

will remove all bindings in the vi-insert keymap beginning with an escape character (probably cursor keys), but leave the binding for the escape character itself (probably **vi-cmd-mode**). This is incompatible with the option **-R**.

- **-s in-string out-string ...** — Bind each *in-string* to each *out-string*. When *in-string* is typed, *out-string* will be pushed back and treated as input to the line editor. When **-R** is also used, interpret the *_in-string_s* as ranges.

Note that both *in-string* and *out-string* are subject to the same form of interpretation, as described below.

- **-in-string command ...** — Bind each *in-string* to each *command*. When **-R** is used, interpret the *_in-string_s* as ranges.
- **[in-string]** — List key bindings. If an *in-string* is specified, the binding of that string in the selected keymap is displayed. Otherwise, all key bindings in the selected keymap are displayed. (As a special case, if the **-e** or **-v** option is used alone, the keymap is *not* displayed - the implicit linking of keymaps is the only thing that happens.)

When the option `-p` is used, the *in-string* must be present. The listing shows all bindings which have the given key sequence as a prefix, not including any bindings for the key sequence itself.

When the `-L` option is used, the list is in the form of `bindkey` commands to create the key bindings.

When the `-R` option is used as noted above, a valid range consists of two characters, with an optional "-" between them. All characters between the two specified, inclusive, are bound as specified.

For either *in-string* or *out-string*, the following escape sequences are recognised:

- `\a` — bell character - `\b` — backspace - `\e`, `\E` — escape - `\f` — form feed
- `\n` — linefeed (newline)
- `\r` — carriage return - `\t` — horizontal tab - `\v` — vertical tab - `\NNN` — character code in octal - `\xNN` — character code in hexadecimal - `\uNNNN` — unicode character code in hexadecimal - `\UNNNNNNNNN` — unicode character code in hexadecimal
- `\M[-]X` — character with meta bit set - `\C[-]X` — control character - `^X` — control character

In all other cases, `"\"` escapes the following character. Delete is written as `"^?"`. Note that `"\M^?"` and `"^\M?"` are not the same, and that (unlike emacs), the bindings `\M-"X"` and `\e"X"` are entirely distinct, although they are initialized to the same bindings by "bindkey -m".

break ¶

Exit from an enclosing `for`, `while`, `until`, `select` or `repeat` loop. If an arithmetic expression *n* is specified, then break *n* levels instead of just one.

builtin ¶

The command word is taken to be the name of a builtin command, rather than a shell function or external command.

bye ¶

Alias for `exit`. Exit the shell with the given status.

cap ¶

Change the shell's process capability sets to the specified *capabilities*, otherwise display the shell's current capabilities.

cd ¶

Change the current directory. In the first form, change the current directory to *arg*, or to the value of `$HOME` if *arg* is not specified. If *arg* is `"-"`, change to the previous directory.

Otherwise, if *arg* begins with a slash, attempt to change to the directory given by *arg*.

If *arg* does not begin with a slash, the behaviour depends on whether the current directory `"."` occurs in the list of directories contained in the shell parameter `cdpath`. If it does not, first attempt to change to the directory *arg* under the current directory, and if that fails but `cdpath` is set and contains at least one element attempt to change to the directory *arg* under each component of `cdpath` in turn until successful. If `"."` occurs in `cdpath`, then `cdpath` is searched strictly in order so that `"."` is only tried at the appropriate point.

The order of testing `cdpath` is modified if the option `POSIX_CD` is set, as described in the documentation for the option.

If no directory is found, the option `CDABLE_VARS` is set, and a parameter named *arg* exists whose value begins with a slash, treat its value as the directory. In that case, the parameter is added to the named directory hash table.

The second form of `cd` substitutes the string *new* for the string *old* in the name of the current directory, and tries to change to this new directory.

The third form of `cd` extracts an entry from the directory stack, and changes to that directory. An argument of the form `+"n"` identifies a stack entry by counting from the left of the list shown by the `dirs` command, starting with zero. An argument of the form `-"n"` counts from the right. If the `PUSHD_MINUS` option is set, the meanings of `+"` and `-"` in this context are swapped. If the `POSIX_CD` option is set, this form of `cd` is not recognised and will be interpreted as the first form.

If the `-q` (quiet) option is specified, the hook function `chpwd` and the functions in the array `chpwd_functions` are not called. This is useful for calls to `cd` that do not change the environment seen by an interactive user.

If the `-s` option is specified, `cd` refuses to change the current directory if the given pathname contains symlinks. If the `-P` option is given or the `CHASE_LINKS` option is set, symbolic links are resolved to their true values. If the `-L` option is given symbolic links are retained in the directory (and not resolved) regardless of the state of the `CHASE_LINKS` option.

`chdir` ¶

Alias for `cd`. Change the working directory.

`chgrp` ¶

Changes group of files specified. This is equivalent to `chown` with a *user-spec* argument of `:"group"`.

`chmod` ¶

Changes mode of files specified.

The specified *mode* must be in octal.

The `-R` option causes `chmod` to recursively descend into directories, changing the mode of all files in the directory after changing the mode of the directory itself.

The `-s` option is a zsh extension to `chmod` functionality. It enables paranoid behaviour, intended to avoid security problems involving a `chmod` being tricked into affecting files other than the ones intended. It will refuse to follow symbolic links, so that (for example) `""chmod 600 /tmp/foo/passwd""` can't accidentally `chmod /etc/passwd` if `/tmp/foo` happens to be a link to `/etc`. It will also check where it is after leaving directories, so that a recursive `chmod` of a deep directory tree can't end up recursively `chmoding /usr` as a result of directories being moved up the tree.

`chown` ¶

Changes ownership and group of files specified.

The *user-spec* can be in four forms:

- *user* — change owner to *user*; do not change group - *user::* — change owner to *user*; do not change group - *user:"*
- change owner to *_user_*; change group to *_user_'s* primary group - *_user_:group_* - change owner
to *_user_*; change group to *_group_* - *:_group_* - do not change owner; change group to *_group_*

In each case, the `":"` may instead be a `."`. The rule is that if there is a `":"` then the separator is `":"`, otherwise if there is a `."` then the separator is `."`, otherwise there is no separator.

Each of *user* and *group* may be either a username (or group name, as appropriate) or a decimal user ID (group ID). Interpretation as a name takes precedence, if there is an all-numeric username (or group name).

If the target is a symbolic link, the `-h` option causes `chown` to set the ownership of the link instead of its target.

The `-R` option causes `chown` to recursively descend into directories, changing the ownership of all files in the directory after changing the ownership of the directory itself.

The `-s` option is a zsh extension to `chown` functionality. It enables paranoid behaviour, intended to avoid security problems involving a `chown` being tricked into affecting files other than the ones intended. It will refuse to follow symbolic links, so that (for example) `""chown luser /tmp/foo/passwd""` can't accidentally `chown /etc/passwd` if `/tmp/foo` happens to be a link to `/etc`. It will also check where it is after leaving directories, so that a recursive `chown` of a deep directory tree can't end up recursively `chowning /usr` as a result of directories being moved up the tree.

`clone` ¶

Creates a forked instance of the current shell, attached to the specified *tty*. In the new shell, the `PID`, `PPID` and `TTY` special parameters are changed appropriately. `$_` is set to zero in the new shell, and to the new shell's PID in the original shell.

The return status of the builtin is zero in both shells if successful, and non-zero on error.

The target of `clone` should be an unused terminal, such as an unused virtual console or a virtual terminal created by

```
xterm -e sh -c 'trap : INT QUIT TSTP; tty; while ;; do sleep 100000000; done'
```

Some words of explanation are warranted about this long xterm command line: when doing clone on a pseudo-terminal, some other session ("session" meant as a unix session group, or SID) is already owning the terminal. Hence the cloned zsh cannot acquire the pseudo-terminal as a controlling tty. That means two things:

startitemize() itemiz(the job control signals will go to the sh-started-by-xterm process group (that's why we disable INT QUIT and TSTP with trap; otherwise the while loop could get suspended or killed)) itemiz(the cloned shell will have job control disabled, and the job control keys (control-C, control-\ and control-Z) will not work.) enditemize()

This does not apply when cloning to an *unused* vc.

Cloning to a used (and unprepared) terminal will result in two processes reading simultaneously from the same terminal, with input bytes going randomly to either process.

`clone` is mostly useful as a shell built-in replacement for `openvt`.

command ¶

The command word is taken to be the name of an external command, rather than a shell function or builtin. If the `POSIX_BUILTINS` option is set, builtins will also be executed but certain special properties of them are suppressed. The `-p` flag causes a default path to be searched instead of that in `$path`. With the `-v` flag, `command` is similar to `whence` and with `-V`, it is equivalent to `whence -v`.

compadd ¶

This builtin command can be used to add matches directly and control all the information the completion code stores with each possible completion. The return status is zero if at least one match was added and non-zero if no matches were added.

The completion code breaks each match into seven fields in the order:

```
<ipre><apre><hpre><body><hsuf><asuf><isuf>
```

The first field is an ignored prefix taken from the command line, the contents of the `IPREFIX` parameter plus the string given with the `-i` option. With the `-U` option, only the string from the `-i` option is used. The field `<apre>` is an optional prefix string given with the `-P` option. The `<hpre>` field is a string that is considered part of the match but that should not be shown when listing completions, given with the `-p` option; for example, functions that do filename generation might specify a common path prefix this way. `<body>` is the part of the match that should appear in the list of matches shown to the user. The suffixes `<hsuf>`, `<asuf>` and `<isuf>` correspond to the prefixes `<hpre>`, `<apre>` and `<ipre>` and are given by the options `-s`, `-S` and `-I`, respectively.

The supported flags are:

- `-P prefix` — This gives a string to be inserted before each match. The string given is not considered as part of the match and any shell metacharacters in it will not be quoted when the string is inserted.
- `-S suffix` — Like `-P`, but gives a string to be inserted after each match.
- `-p hidden-prefix` — This gives a string that should be inserted before each match but that should not appear in the list of matches. Unless the `-U` option is given, this string must be matched as part of the string on the command line.
- `-s hidden-suffix` — Like `-p`, but gives a string to insert after each match.
- `-i ignored-prefix` — This gives a string to insert just before any string given with the `-P` option. Without `-P` the string is inserted before the string given with `-p` or directly before each match.
- `-I ignored-suffix` — Like `-i`, but gives an ignored suffix.
- `-a` — With this flag the *completions* are taken as names of arrays and the actual completions are their values. If only some elements of the arrays are needed, the *completions* may also contain subscripts, as in `"foo[2,-1]"`.
- `-k` — With this flag the *completions* are taken as names of associative arrays and the actual completions are their keys. As for `-a`, the *words* may also contain subscripts, as in `"tt(foo[(R)*bar*])"`.
- `-d array` — This adds per-completion display strings. The *array* should contain one element per *completion* given. The completion code will then display the first element instead of the first *completion*, and so on. The *array* may be given as the name of an array parameter or directly as a space-separated list of words in parentheses.

If there are fewer display strings than *completions*, the leftover *completions* will be displayed unchanged and if there are more display strings than *completions*, the leftover display strings will be silently ignored.

- **-l** — This option only has an effect if used together with the **-d** option. If it is given, the display strings are listed one per line, not arrayed in columns.
- **-o [order]** — This controls the order in which matches are sorted. *order* is a comma-separated list comprising the following possible values. These values can be abbreviated to their initial two or three characters. Note that the order forms part of the group name space so matches with different orderings will not be in the same group; additionally, since ordering flags are not respected on the default group, a group name must be given explicitly. See **-J** and **-V** below.
- **match** — If given, the order of the output is determined by the match strings; otherwise it is determined by the display strings (i.e. the strings given by the **-d** option). This is the default if **-o** is specified but the *order* argument is omitted.
- **nosort** — This specifies that the *completions* are pre-sorted and their order should be preserved. This value only makes sense alone and cannot be combined with any others.
- **numeric** — If the matches include numbers, sort them numerically rather than lexicographically.
- **reverse** — Arrange the matches backwards by reversing the sort ordering.
- **-J group-name** — Gives the name of the group that the matches should be stored in.
- **-V group-name** — Like **-J** but naming an unsorted group. This option is identical to the combination of **-J** and **-o nosort**.
- **-1** — If given together with the **-V** option, makes only consecutive duplicates in the group be removed. If combined with the **-J** option, this has no visible effect. Note that groups with and without this flag are in different name spaces.
- **-2** — If given together with the **-J** or **-V** option, makes all duplicates be kept. Again, groups with and without this flag are in different name spaces.

- **-X explanation** — The *explanation* string will be printed with the list of matches, above the group currently selected.

Within the *explanation*, the following sequences may be used to specify output attributes as described in *Expansion of Prompt Sequences* (zshmisc): "%B", "%S", "%U", "%F", "%K" and their lower case counterparts, as well as "%H" and "%{...%}"'. "%F", "%K", "%H" and "%{...%}" take arguments in the same form as prompt expansion. (Note that the sequence "%G" is not available; an argument to "%{" should be used instead.) The sequence "%%" produces a literal "%".

These sequences are most often employed by users when customising the **format** style (see subref(Standard Styles)(zsh-compsys)), but they must also be taken into account when writing completion functions, as passing descriptions with unescaped "%" characters to utility functions such as **_arguments** and **_message** may produce unexpected results. If arbitrary text is to be passed in a description, it can be escaped using e.g. `${my_str//\%/%%}`.

- **-x message** — Like **-X**, but the *message* will be printed even if there are no matches in the group.

- **-q** — The suffix given with **-S** will be automatically removed if the next character typed is a blank or does not insert anything, or if the suffix consists of only one character and the next character typed is the same character.

- **-r remove-chars** — This is a more versatile form of the **-q** option. The suffix given with **-S** or the slash automatically added after completing directories will be automatically removed if the next character typed inserts one of the characters given in the *remove-chars*. This string is parsed as a characters class and understands the backslash sequences used by the **print** command. For example, `-r "a-z\t"` removes the suffix if the next character typed inserts a lower case character or a TAB, and `-r "^0-9"` removes the suffix if the next character typed inserts anything but a digit. One extra backslash sequence is understood in this string: `"\"` stands for all characters that insert nothing. Thus `"-S "="-q"` is the same as `"-S "="-r "\t\n\"`.

This option may also be used without the **-S** option; then any automatically added space will be removed when one of the characters in the list is typed.

- **-R remove-func** — This is another form of the **-r** option. When a match has been accepted and a suffix has been inserted, the function *remove-func* will be called after the next character typed. It is passed the length of the suffix as an argument and can use the special parameters available in ordinary (non-completion) zle widgets (see *User-Defined Widgets* (zshzle)) to analyse and modify the command line.

- **-f** — If this flag is given, all of the matches built from the *completions* are marked as being the names of files. They are not required to be actual filenames, but if they are, and the option **LIST_TYPES** is set, the characters describing the types of the files in the completion lists will be shown. This also forces a slash to be added when the name of a directory is completed.

- **-e** — This flag can be used to tell the completion code that the matches added are parameter names for a parameter expansion. This will make the **AUTO_PARAM_SLASH** and **AUTO_PARAM_KEYS** options be used for the matches.

- **-W file-prefix** — This string is a pathname that will be prepended to each match together with any prefix specified by the **-p** option to form a complete filename for testing. Hence it is only useful if combined with the **-f** flag, as the tests will not otherwise be performed.

- **-F array** — Specifies an array containing patterns. *completions* that match one of these patterns are ignored, that is, not considered to be matches.

The *array* may be the name of an array parameter or a list of literal patterns enclosed in parentheses and quoted, as in `tt(-F "(*?.o *?.h)")'`. If the name of an array is given, the elements of the array are taken as the patterns.)

- **-Q** — This flag instructs the completion code not to quote any metacharacters in the matches when inserting them into the command line.

- **-M match-spec** — This gives local match specifications as described below in (Completion Matching Control). This option may be given more than once. In this case all *_match-spec_s* given are concatenated with spaces between them to form the specification string to use. Note that they will only be used if the **-U** option is not given.

- **-n** — Specifies that matching *completions* are to be added to the set of matches, but are not to be listed to the user.

- **-U** — If this flag is given, all *completions* are added to the set of matches and no matching will be done by the completion code. Normally this is used in functions that do the matching themselves.

- **-O array** — If this option is given, the *completions* are *not* added to the set of matches. Instead, matching is done as usual and all of the *completions* that match will be stored in the array parameter whose name is given as *array*.

- **-A array** — As the **-O** option, except that instead of those of the *completions* which match being stored in *array*, the strings generated internally by the completion code are stored. For example, with a match specification of `"-M "L:|no=""`, a current word of "nof" and *completions* of "foo", this option stores the string "nofoo" in the array, whereas the **-O** option stores the "foo" originally given.

- **-D array** — As with **-O**, the *completions* are not added to the set of matches. Instead, whenever the *_n_th* *completion* does not match, the *_n_th* element of the *array* is removed. Elements for which the corresponding *completion* matches are retained. This option can be used more than once to remove elements from multiple arrays.

- **-C** — This option adds a special match which expands to all other matches when inserted into the line, even those that are added after this option is used. Together with the **-d** option it is possible to specify a string that should be displayed in the list for this special match. If no string is given, it will be shown as a string containing the strings that would be inserted for the other matches, truncated to the width of the screen.

- **-E number** — This option adds *number* empty matches after matching *completions* have been added. An empty match takes up space in completion listings but will never be inserted in the line and can't be selected with menu completion or menu selection. This makes empty matches only useful to format completion lists and to make explanatory string be shown in completion lists (since empty matches can be given display strings with the **-d** option). And because all but one empty string would otherwise be removed, this option implies the **-V** and **-2** options (even if an explicit **-J** option is given). This can be important to note as it affects the name space into which matches are added.

- -

- **--** — This flag ends the list of flags and options. All arguments after it will be taken as the *completions* even if they begin with hyphens.

Except for the **-M** flag, if any of these flags is given more than once, the first one (and its argument) will be used.

comparguments ¶

This is used by the *_arguments* function to do the argument and command line parsing. Like *compdescribe* it has an option **-i** to do the parsing and initialize some internal state and various options to access the state information to decide what should be completed.

compcall ¶

This allows the use of completions defined with the *compctl* builtin from within completion widgets. The list of matches will be generated as if one of the non-widget completion functions (*complete-word*, etc.) had been called, except that only *compctls* given for specific commands are used. To force the code to try completions defined with the **-T** option of *compctl* and/or the default completion (whether defined by *compctl -D* or the builtin default) in the appropriate places, the **-T** and/or **-D** flags can be passed to *compcall*.

The return status can be used to test if a matching *compctl* definition was found. It is non-zero if a *compctl* was found and zero otherwise.

Note that this builtin is defined by the *zsh/compctl* module.

compctl ¶

Old completion control (compctl mechanism). Largely superseded by `compdef` / `compsys`.

compdescribe ¶

This is used by the `_describe` function to build the displays for the matches and to get the strings to add as matches with their options. On the first call one of the options `-i` or `-I` should be supplied as the first argument. In the first case, display strings without the descriptions will be generated, in the second case, the string used to separate the matches from their descriptions must be given as the second argument and the descriptions (if any) will be shown. All other arguments are like the definition arguments to `_describe` itself.

Once `compdescribe` has been called with either the `-i` or the `-I` option, it can be repeatedly called with the `-g` option and the names of four parameters as its arguments. This will step through the different sets of matches and store the value of `compstate[list]` in the first scalar, the options for `compadd` in the second array, the matches in the third array, and the strings to be displayed in the completion listing in the fourth array. The arrays may then be directly given to `compadd` to register the matches with the completion code.

compfiles ¶

Used by the `_path_files` function to optimize complex recursive filename generation (globbing). It does three things. With the `-p` and `-P` options it builds the glob patterns to use, including the paths already handled and trying to optimize the patterns with respect to the prefix and suffix from the line and the match specification currently used. The `-i` option does the directory tests for the `ignore-parents` style and the `-r` option tests if a component for some of the matches are equal to the string on the line and removes all other matches if that is true.

compgroups ¶

Used by the `_tags` function to implement the internals of the `group-order` style. This only takes its arguments as names of completion groups and creates the groups for it (all six types: sorted and unsorted, both without removing duplicates, with removing all duplicates and with removing consecutive duplicates).

compquote ¶

There may be reasons to write completion functions that have to add the matches using the `-Q` option to `compadd` and perform quoting themselves. Instead of interpreting the first character of the `all_quotes` key of the `compstate` special association and using the `q` flag for parameter expansions, one can use this builtin command. The arguments are the names of scalar or array parameters and the values of these parameters are quoted as needed for the innermost quoting level. If the `-p` option is given, quoting is done as if there is some prefix before the values of the parameters, so that a leading equal sign will not be quoted.

The return status is non-zero in case of an error and zero otherwise.

compset ¶

This command simplifies modification of the special parameters, while its return status allows tests on them to be carried out.

The options are:

- `-p number` — If the value of the `PREFIX` parameter is at least *number* characters long, the first *number* characters are removed from it and appended to the contents of the `IPREFIX` parameter.
- `-P [ number ] pattern` — If the value of the `PREFIX` parameter begins with anything that matches the *pattern*, the matched portion is removed from `PREFIX` and appended to `IPREFIX`.

Without the optional *number*, the longest match is taken, but if *number* is given, anything up to the `_number_th` match is moved. If the *number* is negative, the `_number_th` longest match is moved. For example, if `PREFIX` contains the string "a=b=c", then `"compset -P '*\=' will move the string "a=b=" into the IPREFIX"` parameter, but `"compset -P 1 '*\=' will move only the string "a="`.

- `-s number` — As `-p`, but transfer the last *number* characters from the value of `SUFFIX` to the front of the value of `ISUFFIX`.
- `-S [ number ] pattern` — As `-P`, but match the last portion of `SUFFIX` and transfer the matched portion to the front of the value of `ISUFFIX`.

- **-n begin [end]** — If the current word position as specified by the parameter **CURRENT** is greater than or equal to *begin*, anything up to the *_begin_th* word is removed from the **words** array and the value of the parameter **CURRENT** is decremented by *begin*.

If the optional *end* is given, the modification is done only if the current word position is also less than or equal to *end*. In this case, the words from position *end* onwards are also removed from the **words** array.

Both *begin* and *end* may be negative to count backwards from the last element of the **words** array.

- **-N beg-pat [end-pat]** — If one of the elements of the **words** array before the one at the index given by the value of the parameter **CURRENT** matches the pattern *beg-pat*, all elements up to and including the matching one are removed from the **words** array and the value of **CURRENT** is changed to point to the same word in the changed array.

If the optional pattern *end-pat* is also given, and there is an element in the **words** array matching this pattern, the parameters are modified only if the index of this word is higher than the one given by the **CURRENT** parameter (so that the matching word has to be after the cursor). In this case, the words starting with the one matching *end-pat* are also removed from the **words** array. If **words** contains no word matching *end-pat*, the testing and modification is performed as if it were not given.

- **-q** — The word currently being completed is split on spaces into separate words, respecting the usual shell quoting conventions. The resulting words are stored in the **words** array, and **CURRENT**, **PREFIX**, **SUFFIX**, **QIPREFIX**, and **QISUFFIX** are modified to reflect the word part that is completed.

In all the above cases the return status is zero if the test succeeded and the parameters were modified and non-zero otherwise. This allows one to use this builtin in tests such as:

```
if compset -P '*\='; then ...
```

This forces anything up to and including the last equal sign to be ignored by the completion code.

comptags ¶

These implement the internals of the tags mechanism.

comptry ¶

These implement the internals of the tags mechanism.

compvalues ¶

Like **compargments**, but for the *_values* function.

continue ¶

Resume the next iteration of the enclosing **for**, **while**, **until**, **select** or **repeat** loop. If an arithmetic expression *n* is specified, break out of *n*-1 loops and resume at the *_n_th* enclosing loop.

declare ¶

Alias for **typeset**. Set variable attributes. **-a** array, **-A** assoc, **-i** integer, **-r** readonly.

dirs ¶

With no arguments, print the contents of the directory stack. Directories are added to this stack with the **pushd** command, and removed with the **cd** or **popd** commands. If arguments are specified, load them onto the directory stack, replacing anything that was there, and push the current directory onto the stack.

- **-c** — clear the directory stack.

- **-l** — print directory names in full instead of using *~* expressions (see *Filename Expansion* (zshexprn)).

- **-p** — print directory entries one per line.

- **-v** — number the directories in the stack when printing.

disable ¶

Temporarily disable the `_name_d` hash table elements or patterns. The default is to disable builtin commands. This allows you to use an external command with the same name as a builtin command. The `-a` option causes `disable` to act on regular or global aliases. The `-s` option causes `disable` to act on suffix aliases. The `-f` option causes `disable` to act on shell functions. The `-r` options causes `disable` to act on reserved words. Without arguments all disabled hash table elements from the corresponding hash table are printed. With the `-m` flag the arguments are taken as patterns (which should be quoted to prevent them from undergoing filename expansion), and all hash table elements from the corresponding hash table matching these patterns are disabled. Disabled objects can be enabled with the `enable` command.

With the option `-p, name ...` refer to elements of the shell's pattern syntax as described in *Filename Generation* (zshexpn). Certain elements can be disabled separately, as given below.

Note that patterns not allowed by the current settings for the options `EXTENDED_GLOB`, `KSH_GLOB` and `SH_GLOB` are never enabled, regardless of the setting here. For example, if `EXTENDED_GLOB` is not active, the pattern `^` is ineffective even if `"disable -p '^'"` has not been issued. The list below indicates any option settings that restrict the use of the pattern. It should be noted that setting `SH_GLOB` has a wider effect than merely disabling patterns as certain expressions, in particular those involving parentheses, are parsed differently.

The following patterns may be disabled; all the strings need quoting on the command line to prevent them from being interpreted immediately as patterns and the patterns are shown below in single quotes as a reminder.

- `'?'` — The pattern character `?` wherever it occurs, including when preceding a parenthesis with `KSH_GLOB`.
- `'*'` — The pattern character `*` wherever it occurs, including recursive globbing and when preceding a parenthesis with `KSH_GLOB`.
- `[]` — Character classes.
- `'<'` (`NO_SH_GLOB`) — Numeric ranges.
- `'|'` (`NO_SH_GLOB`) — Alternation in grouped patterns, case statements, or `KSH_GLOB` parenthesised expressions.
- `'(` (`NO_SH_GLOB`) — Grouping using single parentheses. Disabling this does not disable the use of parentheses for `KSH_GLOB` where they are introduced by a special character, nor for glob qualifiers (use `setopt NO_BARE_GLOB_QUAL 'to disable glob qualifiers that use parentheses only'`).
- `'~'` (`EXTENDED_GLOB`) — Exclusion in the form `A~B`.
- `'^'` (`EXTENDED_GLOB`) — Exclusion in the form `A^B`.
- `'#'` (`EXTENDED_GLOB`) — The pattern character `#` wherever it occurs, both for repetition of a previous pattern and for indicating globbing flags.
- `'?(` (`KSH_GLOB`) — The grouping form `?(...)`. Note this is also disabled if `'?'` is disabled.
- `'*(` (`KSH_GLOB`) — The grouping form `*(...)`. Note this is also disabled if `'*'` is disabled.
- `'+(` (`KSH_GLOB`) — The grouping form `+(...)`.
- `'!(` (`KSH_GLOB`) — The grouping form `!(...)`.
- `'@('` (`KSH_GLOB`) — The grouping form `@( ... )`.

disown ¶

Remove the specified `_job_s` from the job table; the shell will no longer report their status, and will not complain if you try to exit an interactive shell with them running or stopped. If no `job` is specified, disown the current job.

If the `_job_s` are currently stopped and the `AUTO_CONTINUE` option is not set, a warning is printed containing information about how to make them running after they have been disowned. If one of the latter two forms is used, the `_job_s` will automatically be made running, independent of the setting of the `AUTO_CONTINUE` option.

echo ¶

Write each `arg` on the standard output, with a space separating each one. If the `-n` flag is not present, print a newline at the end. `echo` recognizes the following escape sequences:

- `\a` — bell character - `\b` — backspace - `\c` — suppress subsequent characters and final newline - `\e` — escape - `\f` — form feed

- `\n` — linefeed (newline)

- `\r` — carriage return - `\t` — horizontal tab - `\v` — vertical tab - `\\` — backslash - `\0NNN` — character code in octal - `\xNN` — character code in hexadecimal - `\uNNNN` — unicode character code in hexadecimal - `\UNNNNNNNN` — unicode character code in hexadecimal

index(BSD_ECHO, use of) The `-E` flag, or the `BSD_ECHO` option, can be used to disable these escape sequences. In the latter case, `-e` flag can be used to enable them.

Note that for standards compliance a double dash does not terminate option processing; instead, it is printed directly. However, a single dash does terminate option processing, so the first dash, possibly following options, is not printed, but everything following it is printed as an argument. The single dash behaviour is different from other shells. For a more portable way of printing text, see `printf`, and for a more controllable way of printing text within zsh, see `print`.

echotc ¶

Output the termcap value corresponding to the capability *cap*, with optional arguments.

echoti ¶

Output the terminfo value corresponding to the capability *cap*, instantiated with *arg* if applicable.

emulate ¶

Without any argument print current emulation mode.

With single argument set up zsh options to emulate the specified shell as much as possible. `cs`h will never be fully emulated. If the argument is not one of the shells listed above, `zsh` will be used as a default; more precisely, the tests performed on the argument are the same as those used to determine the emulation at startup based on the shell name, see *Compatibility* (zsh). In addition to setting shell options, the command also restores the pristine state of pattern enables, as if all patterns had been enabled using `enable -p`.

If the `emulate` command occurs inside a function that has been marked for execution tracing with `functions -t` then the `xtrace` option will be turned on regardless of emulation mode or other options. Note that code executed inside the function by the `.`, `source`, or `eval` commands is not considered to be running directly from the function, hence does not provoke this behaviour.

If the `-R` switch is given, all settable options are reset to their default value corresponding to the specified emulation mode, except for certain options describing the interactive environment; otherwise, only those options likely to cause portability problems in scripts and functions are altered. If the `-L` switch is given, the options `LOCAL_OPTIONS`, `LOCAL_PATTERNS` and `LOCAL_TRAPS` will be set as well, causing the effects of the `emulate` command and any `setopt`, `disable -p` or `enable -p`, and `trap` commands to be local to the immediately surrounding shell function, if any; normally these options are turned off in all emulation modes except `ksh`. The `-L` switch is mutually exclusive with the use of `-c` in *flags*.

If there is a single argument and the `-l` switch is given, the options that would be set or unset (the latter indicated with the prefix "no") are listed. `-l` can be combined with `-L` or `-R` and the list will be modified in the appropriate way. Note the list does not depend on the current setting of options, i.e. it includes all options that may in principle change, not just those that would actually change.

The *flags* may be any of the invocation-time flags described in *Invocation* (zsh), except that `"-o EMACS"` and `"-o VI"` may not be used.

If `-c arg` appears in *flags*, *arg* is evaluated while the requested emulation is temporarily in effect. In this case the emulation mode and all options are restored to their previous values before `emulate` returns. The `-R` switch may precede the name of the shell to emulate; note this has a meaning distinct from including `-R` in *flags*.

Use of `-c` enables "sticky" emulation mode for functions defined within the evaluated expression: the emulation mode is associated thereafter with the function so that whenever the function is executed the emulation (respecting the `-R` switch, if present) and all options are set (and pattern disables cleared) before entry to the function, and the state is restored after exit. If the function is called when the sticky emulation is already in effect, either within an `emulate _shell_ -c 'expression` or within another function with the same sticky emulation, entry and exit from the function do not cause options to be altered (except due to standard processing such as the `LOCAL_OPTIONS` option). This also applies to functions marked for autoload within the sticky emulation; the appropriate set of options will be applied at the point the function is loaded as well as when it is run.

For example:

```
example(emulate sh -c 'fni+() { setopt cshnullglob; } fno+() { fni; }' fno)
```

The two functions `fni` and `fno` are defined with sticky `sh` emulation. `fno` is then executed, causing options associated with emulations to be set to their values in `sh`. `fno` then calls `fni`; because `fni` is also marked for sticky `sh` emulation, no option changes take place on entry to or exit from it. Hence the option `cshnullglob`, turned off by `sh` emulation, will be turned on within `fni` and remain on return to `fno`. On exit from `fno`, the emulation mode and all options will be restored to the state they were in before entry to the temporary emulation.

The documentation above is typically sufficient for the intended purpose of executing code designed for other shells in a suitable environment. More detailed rules follow.

- 1. — The sticky emulation environment provided by emulate `_shell_ -c` is identical to that provided by entry to a function marked for sticky emulation as a consequence of being defined in such an environment. Hence, for example, the sticky emulation is inherited by subfunctions defined within functions with sticky emulation.
- 2. — No change of options takes place on entry to or exit from functions that are not marked for sticky emulation, other than those that would normally take place, even if those functions are called within sticky emulation.
- 3. — No special handling is provided for functions marked for autoload nor for functions present in wordcode created by the `zcompile` command.
- 4. — The presence or absence of the `-R` switch to emulate corresponds to different sticky emulation modes, so for example "emulate `sh -c`", "emulate `-R sh -c`" and "emulate `csh -c`" are treated as three distinct sticky emulations.
- 5. — Difference in shell options supplied in addition to the basic emulation also mean the sticky emulations are different, so for example "emulate `zsh -c`" and "emulate `zsh -o cbases -c`" are treated as distinct sticky emulations.

enable ¶

Enable the `_name_d` hash table elements, presumably disabled earlier with `disable`. The default is to enable builtin commands. The `-a` option causes `enable` to act on regular or global aliases. The `-s` option causes `enable` to act on suffix aliases. The `-f` option causes `enable` to act on shell functions. The `-r` option causes `enable` to act on reserved words. Without arguments all enabled hash table elements from the corresponding hash table are printed. With the `-m` flag the arguments are taken as patterns (should be quoted) and all hash table elements from the corresponding hash table matching these patterns are enabled. Enabled objects can be disabled with the `disable` builtin command.

`enable -p` reenables patterns disabled with `disable -p`. Note that it does not override globbing options; for example, enable `-p '~'` does not cause the pattern character `~` to be active unless the `EXTENDED_GLOB` option is also set. To enable all possible patterns (so that they may be individually disabled with `disable -p`), use "setopt `EXTENDED_GLOB KSH_GLOB NO_SH_GLOB`".

eval ¶

Read the arguments as input to the shell and execute the resulting command+(s+) in the current shell process. The return status is the same as if the commands had been executed directly by the shell; if there are no *args* or they contain no commands (i.e. are an empty string or whitespace) the return status is zero.

example ¶

Displays the flags and arguments it is invoked with.

exec ¶

The following command together with any arguments is run in place of the current process, rather than as a sub-process. The shell does not fork and is replaced. The shell does not invoke `TRAPEXIT`, nor does it source `zlogout` files. The options are provided for compatibility with other shells.

The `-c` option clears the environment.

The `-l` option is equivalent to the `-` precommand modifier, to treat the replacement command as a login shell; the command is executed with a `-` prepended to its `argv[0]` string. This flag has no effect if used together with the `-a` option.

The `-a` option is used to specify explicitly the `argv[0]` string (the name of the command as seen by the process itself) to be used by the replacement command and is directly equivalent to setting a value for the `ARGV0` environment variable.

exit ¶

Exit the shell with the exit status specified by an arithmetic expression *n*; if none is specified, use the exit status from the last command executed. `pindex(IGNORE_EOF, use of)` An EOF condition will also cause the shell to exit, unless the `IGNORE_EOF` option is set.

See notes at the end of *Signals* (zshmisc) for some possibly unexpected interactions of the `exit` command with jobs.

export ¶

The specified `_name_s` are marked for automatic export to the environment of subsequently executed commands. Equivalent to `typeset -gx`. If a parameter specified does not already exist, it is created in the global scope.

false ¶

Do nothing and return an exit status of 1.

fc ¶

The `fc` command controls the interactive history mechanism. Note that reading and writing of history options is only performed if the shell is interactive. Usually this is detected automatically, but it can be forced by setting the `interactive` option when starting the shell.

The first two forms of this command select a range of events from *first* to *last* from the history list. The arguments *first* and *last* may be specified as a number or as a string. A negative number is used as an offset to the current history event number. A string specifies the most recent event beginning with the given string. All substitutions *old=new*, if any, are then performed on the text of the events.

The range of events selected by numbers can be narrowed further by the following flags.

- `-I` — restricts to only internal events (not from `$HISTFILE`)
- `-L` — restricts to only local events (not from other shells, see `SHARE_HISTORY` in `nmref(Description of Options)(zshoptions)` -- note that `$HISTFILE` is considered local when read at startup)
- `-m` — takes the first argument as a pattern (which should be quoted) and only the history events matching this pattern are considered

If *first* is not specified, it will be set to -1 (the most recent event), or to -16 if the `-l` flag is given. If *last* is not specified, it will be set to *first*, or to -1 if the `-l` flag is given. However, if the current event has added entries to the history with "print -s" or "fc -R", then the default *last* for `-l` includes all new history entries since the current event began.

When the `-l` flag is given, the resulting events are listed on standard output. Otherwise the editor program specified by `-e` *ename* is invoked on a file containing these history events. If `-e` is not given, the value of the parameter `FCEDIT` is used; if that is not set the value of the parameter `EDITOR` is used; if that is not set a builtin default, usually "vi" is used. If *ename* is "-", no editor is invoked. When editing is complete, the edited command is executed.

The flag "-s" is equivalent to "-e -". The flag `-r` reverses the order of the events and the flag `-n` suppresses event numbers when listing.

Also when listing,

- `-d` — prints timestamps for each event - `-f` — prints full time-date stamps in the US `_MM_/_DD_/_YY_ _hh_:"mm"` format - `-E` — prints full time-date stamps in the European `_dd_._mm_._yyyy_ _hh_:"mm"` format - `-i` — prints full time-date stamps in ISO8601 `_yyyy_._mm_._dd_ _hh_:"mm"` format
- `-t fmt` — prints time and date stamps in the given format; *fmt* is formatted with the `strftime` function with the zsh extensions described for the `%D{string}` prompt format in *Simple Prompt Escapes* (zshmisc). The resulting formatted string must be no more than 256 characters or will not be printed
- `-D` — prints elapsed times; may be combined with one of the options above

`cindex(history, stack)` `cindex(stack, history)` "fc -p" pushes the current history list onto a stack and switches to a new history list. If the `-a` option is also specified, this history list will be automatically popped when the current function scope is exited, which is a much better solution than creating a trap function to call "fc -P" manually. If no arguments are specified, the history list is left empty, `$HISTFILE` is unset, and `$HISTSIZE` & `$SAVEHIST` are set to their default values. If one argument is given, `$HISTFILE` is set to that filename, `$HISTSIZE` & `$SAVEHIST` are left unchanged, and the history file is read in (if it exists)

to initialize the new list. If a second argument is specified, `$HISTSIZE` & `$SAVEHIST` are instead set to the single specified numeric value. Finally, if a third argument is specified, `$SAVEHIST` is set to a separate value from `$HISTSIZE`. You are free to change these environment values for the new history list however you desire in order to manipulate the new history list.

"fc -P" pops the history list back to an older list saved by "fc -p". The current list is saved to its `$HISTFILE` before it is destroyed (assuming that `$HISTFILE` and `$SAVEHIST` are set appropriately, of course). The values of `$HISTFILE`, `$HISTSIZE`, and `$SAVEHIST` are restored to the values they had when "fc -p" was called. Note that this restoration can conflict with making these variables "local", so your best bet is to avoid local declarations for these variables in functions that use "fc -p". The one other guaranteed-safe combination is declaring these variables to be local at the top of your function and using the automatic option (`-a`) with "fc -p". Finally, note that it is legal to manually pop a push marked for automatic popping if you need to do so before the function exits.

`cindex(history, file)` `cindex(file, history)` "fc -R" reads the history from the given file, "fc -W" writes the history out to the given file, and "fc -A" appends the history out to the given file. If no filename is specified, the `$HISTFILE` is assumed. If the `-I` option is added to `-R`, only those events that are not already contained within the internal history list are added. If the `-I` option is added to `-A` or `-W`, only those events that are new since last incremental append/write to the history file are appended/written. In any case, the created file will have no more than `$SAVEHIST` entries.

fg ¶

Bring each specified *job* in turn to the foreground. If no *job* is specified, resume the current job.

float ¶

Equivalent to `typeset -E`, except that options irrelevant to floating point numbers are not permitted.

functions ¶

Equivalent to `typeset -f`, with the exception of the `-c`, `-x`, `-M` and `-W` options. For `functions -u` and `functions -U`, see `autoload`, which provides additional options. For `functions -t` and `functions -T`, see `typeset -f`.

The `-x` option indicates that any functions output will have each leading tab for indentation, added by the shell to show syntactic structure, expanded to the given number *num* of spaces. *num* can also be 0 to suppress all indentation.

The `-W` option turns on the option `WARN_NESTED_VAR` for the named function or functions only. The option is turned off at the start of nested functions (apart from anonymous functions) unless the called function also has the `-W` attribute.

The `-c` option causes *oldfn* to be copied to *newfn*. The copy is efficiently handled internally by reference counting. If *oldfn* was marked for autoload it is first loaded and if this fails the copy fails. Either function may subsequently be redefined without affecting the other. A typical idiom is that *oldfn* is the name of a library shell function which is then redefined to call *newfn*, thereby installing a modified version of the function.

The `-M` and `+M` flags `_cindex(defining mathematical functions)` `cindex(functions, defining mathematical)`

Use of the `-M` option may not be combined with any of the options handled by `typeset -f`.

`functions -M mathfn` defines *mathfn* as the name of a mathematical function recognised in all forms of arithmetical expressions; see *Arithmetic Evaluation* (`zshmisc`). By default *mathfn* may take any number of comma-separated arguments. If *min* is given, it must have exactly *min* args; if *min* and *max* are both given, it must have at least *min* and at most *max* args. *max* may be -1 to indicate that there is no upper limit.

By default the function is implemented by a shell function of the same name; if *shellfn* is specified it gives the name of the corresponding shell function while *mathfn* remains the name used in arithmetical expressions. The name of the function in `$0` is *mathfn* (not *shellfn* as would usually be the case), provided the option `FUNCTION_ARGZERO` is in effect. The positional parameters in the shell function correspond to the arguments of the mathematical function call.

The result of the last arithmetical expression evaluated inside the shell function gives the result of the mathematical function. This is not limited to arithmetic substitutions of the form `tt($+(+)...++)`, but also includes arithmetical expressions evaluated in any other way, including by the `let` builtin, by `tt(+(+)...++)` statements, and even by the `return` builtin and by array subscripts. Therefore, care must be taken not to use syntactical constructs that perform arithmetic evaluation after evaluating what is to be the result of the function. For example:

```
findex(zmath_cube) findex(cube) example(# WRONG zmath_cube+(+) { (( $1 * $1 * $1 )) return 0 } functions -M cube 1 1
zmath_cube print $(( cube+(3+) )))
```

This will print "0" because of the `return`.

Commenting the `return` out would lead to a different problem: the `tt(+()...++)` statement would become the last statement in the function, so the *return status* (`$?`) of the function would be non-zero (indicating failure) whenever the *arithmetic result* of the function would happen to be zero (numerically):

```
example(# WRONG zmath_cube+(+) { (( $1 * $1 * $1 )) } functions -M cube 1 1 zmath_cube print $(( cube+(0+) )))
```

Instead, the `true` builtin can be used:

```
example(# RIGHT zmath_cube+(+) { (( $1 * $1 * $1 )) true } functions -M cube 1 1 zmath_cube print $(( cube+(3+) )))
```

If the additional option `-s` is given to `functions -M`, the argument to the function is a single string: anything between the opening and matching closing parenthesis is passed to the function as a single argument, even if it includes commas or white space. The minimum and maximum argument specifiers must therefore be 1 if given. An empty argument list is passed as a zero-length string. Thus, the following string function takes a single argument, including the commas, and prints 11:

```
example(stringfn+(+) { (( $#1 )); true } functions -Ms stringfn print $(( stringfn+(foo,bar,rod+) )))
```

`functions -M` with no arguments lists all such user-defined functions in the same form as a definition. With the additional option `-m` and a list of arguments, all functions whose *mathfn* matches one of the pattern arguments are listed.

`function +M` removes the list of mathematical functions; with the additional option `-m` the arguments are treated as patterns and all functions whose *mathfn* matches the pattern are removed. Note that the shell function implementing the behaviour is not removed (regardless of whether its name coincides with *mathfn*).

getcap ¶

This is a built-in implementation of the POSIX standard utility. It displays the capability sets on each specified *filename*.

getln ¶

Read the top value from the buffer stack and put it in the shell parameter *name*. Equivalent to `read -zr`.

getopts ¶

Checks the `_arg_s` for legal options. If the `_arg_s` are omitted, use the positional parameters. A valid option argument begins with a "+" or a "-". An argument not beginning with a "+" or a "-", or the argument `-"`, ends the options. Note that a single `-` is not considered a valid option argument. *optstring* contains the letters that `getopts` recognizes. If a letter is followed by a ":", that option requires an argument. The options can be separated from the argument by blanks.

Each time it is invoked, `getopts` places the option letter it finds in the shell parameter *name*, prepended with a "+" when *arg* begins with a "+". The index of the next *arg* is stored in `OPTIND`. The option argument, if any, is stored in `OPTARG`. `vindex(OPTIND, use of)` `vindex(OPTARG, use of)`

The first option to be examined may be changed by explicitly assigning to `OPTIND`. `OPTIND` has an initial value of 1, and is normally set to 1 upon entry to a shell function and restored upon exit. (The `POSIX_BUILTINS` option disables this, and also changes the way the value is calculated to match other shells.) `OPTARG` is not reset and retains its value from the most recent call to `getopts`. If either of `OPTIND` or `OPTARG` is explicitly unset, it remains unset, and the index or option argument is not stored. The option itself is still stored in *name* in this case.

A leading ":" in *optstring* causes `getopts` to store the letter of any invalid option in `OPTARG`, and to set *name* to "?" for an unknown option and to ":" when a required argument is missing. Otherwise, `getopts` sets *name* to "?" and prints an error message when an option is invalid. The exit status is nonzero when there are no more options.

hash ¶

`hash` can be used to directly modify the contents of the command hash table, and the named directory hash table. Normally one would modify these tables by modifying one's `PATH` (for the command hash table) or by creating appropriate shell parameters (for the named directory hash table). The choice of hash table to work on is determined by the `-d` option; without the option the command hash table is used, and with the option the named directory hash table is used.

A command *name* starting with a / or with a relative path starting with ./ or ../ is never executed by lookup in the command hash table, and these can only be added to the table by explicit use of the `hash` command. Such a command is always found by direct look up in the file system.

Given no arguments, and neither the `-r` or `-f` options, the selected hash table will be listed in full.

The `-r` option causes the selected hash table to be emptied. It will be subsequently rebuilt in the normal fashion. The `-f` option causes the selected hash table to be fully rebuilt immediately. For the command hash table this hashes all (and *only*) the absolute directories in the `PATH`, and for the named directory hash table this adds all users' home directories. These two options cannot be used with any arguments. Both options remove any explicitly-added elements.

The `-m` option causes the arguments to be taken as patterns (which should be quoted) and the elements of the hash table matching those patterns are printed. This is the only way to display a limited selection of hash table elements.

For each *name* with a corresponding *value*, put "*name*" in the selected hash table, associating it with the pathname "*value*". In the command hash table, this means that whenever "*name*" is used as a command argument, the shell will try to execute the file given by "*value*". In the named directory hash table, this means that "*value*" may be referred to as `~"name"`.

For each *name* with no corresponding *value*, attempt to add *name* to the hash table, checking what the appropriate *value* is in the normal manner for that hash table. If an appropriate *value* "can't be found, then the hash table will be unchanged".

The `-v` option causes hash table entries to be listed as they are added by explicit specification. It has no effect if used with `-f`.

If the `-L` flag is present, then each hash table entry is printed in the form of a call to `hash`.

hashinfo ¶

Print internal hash-table statistics. Debug builtin in `zsh/parameter`-adjacent code.

history ¶

Search backward in the history for a line beginning with the current line up to the cursor. This leaves the cursor in its original position.

integer ¶

Equivalent to `typeset -i`, except that options irrelevant to integers are not permitted.

jobs ¶

Lists information about each given job, or all jobs if *job* is omitted. The `-l` flag lists process IDs, and the `-p` flag lists process groups. If the `-r` flag is specified only running jobs will be listed and if the `-s` flag is given only stopped jobs are shown. If the `-d` flag is given, the directory from which the job was started (which may not be the current directory of the job) will also be shown.

The `-Z` option replaces the shell's argument and environment space with the given string, truncated if necessary to fit. This will normally be visible in `ps (manref(ps)(1))` listings. This feature is typically used by daemons, to indicate their state.

Full job control is only available in the top-level interactive shell, not in commands run in the left hand side of pipelines or within the `tt((...))` construct. However, a snapshot of the job state at that point is taken, so it is still possible to use the `jobs` builtin, or any parameter providing job information. This gives information about the state of jobs at the point the subshell was created. If background processes are created within the subshell, then instead information about those processes is provided.

For example,

```
example(sleep 10 & # Job in background ( # Shell forks jobs # Shows information about "sleep 10 &" sleep 5 & # Process in
background (no job control) jobs # Shows information about "sleep 5 &" ))
```

kill ¶

Sends either `SIGTERM` or the specified signal to the given jobs or processes. Signals are given by number or by names, with or without the "SIG" prefix. If the signal being sent is not "KILL" or "CONT", then the job will be sent a "CONT" signal if it is stopped. The argument *job* can be the process ID of a job not in the job list. In the second form, `kill -l`, if *sig* is not specified the signal names are listed. Otherwise, for each *sig* that is a name, the corresponding signal number is listed. For each *sig* that is a signal number or a number representing the exit status of a process which was terminated or stopped by a signal the name of the signal is printed. The final form with `-L` lists each signal name with its corresponding number.

On some systems, alternative signal names are allowed for a few signals. Typical examples are `SIGCHLD` and `SIGCLD` or `SIGPOLL` and `SIGIO`, assuming they correspond to the same signal number. `kill -l` will only list the preferred form, however `kill -l alt` will show if the alternative form corresponds to a signal number. For example, under Linux `kill -l IO` and `kill -l POLL` both output 29, hence `kill -IO` and `kill -POLL` have the same effect.

Many systems will allow process IDs to be negative to kill a process group or zero to kill the current process group.

The `-q` option allows an integer value to be sent with the signal on systems that support `tt(sigqueue+())`.

let ¶

Evaluate each *arg* as an arithmetic expression. See *Arithmetic Evaluation* (zshmisc) for a description of arithmetic expressions. The exit status is 0 if the value of the last expression is nonzero, 1 if it is zero, and 2 if an error occurred.

limit ¶

Set or display resource limits. Unless the `-s` flag is given, the limit applies only the children of the shell. If `-s` is given without other arguments, the resource limits of the current shell is set to the previously set resource limits of the children.

If *limit* is not specified, print the current limit placed on *resource*, otherwise set the limit to the specified value. If the `-h` flag is given, use hard limits instead of soft limits. If no *resource* is given, print all limits.

When looping over multiple resources, the shell will abort immediately if it detects a badly formed argument. However, if it fails to set a limit for some other reason it will continue trying to set the remaining limits.

resource can be one of:

- `addressspace` — Maximum amount of address space used. - `aiomemorylocked` — Maximum amount of memory locked in RAM for AIO operations. - `aiooperations` — Maximum number of AIO operations. - `cachedthreads` — Maximum number of cached threads. - `coredumpsize` — Maximum size of a core dump. - `cputime` — Maximum CPU seconds per process.
- `datasize` — Maximum data size (including stack) for each process.
- `descriptors` — Maximum value for a file descriptor. - `filesize` — Largest single file allowed. - `kqueues` — Maximum number of kqueues allocated. - `maxfilelocks` — Maximum number of file locks. - `maxproc` — Maximum number of processes. - `maxpthreads` — Maximum number of threads per process. - `memorylocked` — Maximum amount of memory locked in RAM. - `memoryuse` — Maximum resident set size. - `msgqueue` — Maximum number of bytes in POSIX message queues. - `nice` — Maximum nice value. - `pipebuf` — Maximum size of buffers for pipes/fifos. - `posixlocks` — Maximum number of POSIX locks per user. - `pseudoterminals` — Maximum number of pseudo-terminals. - `resident` — Maximum resident set size. - `rt_priority` — Maximum real-time priority. - `rt_time` — Maximum CPU time without a blocking system call. - `sigpending` — Maximum number of pending signals. - `sockbufsize` — Maximum size of all socket buffers. - `stacksize` — Maximum stack size for each process. - `swapspace` — Maximum amount of swap used. - `umtxp` — Maximum number of POSIX thread library objects. - `vmemorysize` — Maximum amount of virtual memory. - `vnodemonitors` — Maximum number of open vnode monitors.

Which of these resource limits are available depends on the system. *resource* can be abbreviated to any unambiguous prefix. It can also be an integer, which corresponds to the integer defined for the resource by the operating system.

If argument corresponds to a number which is out of the range of the resources configured into the shell, the shell will try to read or write the limit anyway, and will report an error if this fails. As the shell does not store such resources internally, an attempt to set the limit will fail unless the `-s` option is present.

limit is a number, with an optional scaling factor, as follows:

- `nh` — hours
- `nk` — kilobytes (default)
- `nm` — megabytes or minutes - `ng` — gigabytes - `[mm:]ss` — minutes and seconds

The `limit` command is not made available by default when the shell starts in a mode emulating another shell. It can be made available with the command "zmodload -F zsh/rlimits b:limit".

ln ¶

for symbolic links. If this has the special value `target`, symbolic links are dereferenced and the target file used to determine the display format.

local ㉿

Same as `typeset`, except that the options `-g`, and `-f` are not permitted. In this case the `-x` option does not force the use of `-g`, i.e. exported variables will be local to functions.

log ㉿

List all users currently logged in who are affected by the current setting of the `watch` parameter.

logout ㉿

Same as `exit`, except that it only works in a login shell.

mem ㉿

Print zsh memory-allocator statistics. Debug builtin compiled only with `--enable-zsh-mem`.

mkdir ㉿

Creates directories. With the `-p` option, non-existing parent directories are first created if necessary, and there will be no complaint if the directory already exists. The `-m` option can be used to specify (in octal) a set of file permissions for the created directories, otherwise mode 777 modified by the current `umask` (see `manref(umask(2))`) is used.

mv ㉿

Moves files. In the first form, the specified *filename* is moved to the specified `_dest_ination`. In the second form, each of the `_filename_s` is taken in turn, and moved to a pathname in the specified `_dir_ectory` that has the same last pathname component.

By default, the user will be queried before replacing any file that the user cannot write to, but writable files will be silently removed. The `-i` option causes the user to be queried about replacing any existing files. The `-f` option causes any existing files to be silently deleted, without querying. `-f` takes precedence.

Note that this `mv` will not move files across devices. Historical versions of `mv`, when actual renaming is impossible, fall back on copying and removing files; if this behaviour is desired, use `cp` and `rm` manually. This may change in a future version.

nameref ㉿

Equivalent to `typeset -n pname=rname`

However, `nameref` is a builtin command rather than a reserved word, so when *rname* uses subscript syntax it must be quoted against globbing. Subscripts in referenced parameters are not supported in ksh93, so this is not a significant compatibility issue.

noglob ㉿

Filename generation (globbing) is not performed on any of the words.

patdebug ㉿

Print pattern-matcher internals for a glob/regex. Debug builtin from `zsh/pattern`.

pcre_compile ㉿

Compiles a perl-compatible regular expression.

Option `-a` will force the pattern to be anchored. Option `-i` will compile a case-insensitive pattern. Option `-m` will compile a multi-line pattern; that is, `^` and `$` will match newlines within the pattern. Option `-x` will compile an extended pattern, wherein whitespace and `#` comments are ignored. Option `-s` makes the dot metacharacter match all characters, including those that indicate newline.

pcre_match ¶

Returns successfully if `string` matches the previously-compiled PCRE.

Upon successful match, if the expression captures substrings within parentheses, `pcre_match` will set the array `match` to those substrings, unless the `-a` option is given, in which case it will set the array `arr`. Similarly, the variable `MATCH` will be set to the entire matched portion of the string, unless the `-v` option is given, in which case the variable `var` will be set. Furthermore, any named captures will be stored in the associative array `.pcre.match` unless an alternative is given with `-A`. No variables are altered if there is no successful match. A `-n` option starts searching for a match from the byte *offset* position in *string*. If the `-b` option is given, the variable `ZPCRE_OP` will be set to an offset pair string, representing the byte offset positions of the entire matched portion within the *string*. For example, a `ZPCRE_OP` set to "32 45" indicates that the matched portion began on byte offset 32 and ended on byte offset 44. Here, byte offset position 45 is the position directly after the matched portion. Keep in mind that the byte position isn't necessarily the same as the character position when UTF-8 characters are involved. Consequently, the byte offset positions are only to be relied on in the context of using them for subsequent searches on *string*, using an offset position as an argument to the `-n` option. This is mostly used to implement the "find all non-overlapping matches" functionality.

A simple example of "find all non-overlapping matches":

```
example(string="The following zip codes: 78884 90210 99513" pcre_compile -m "\d{5}" accum=() pcre_match -b -- $string
while [[ $? -eq 0 ]] do b=$(ZPCRE_OP) accum+=${MATCH} pcre_match -b -n ${b[2]} -- $string done print -l $accum)
```

pcre_study ¶

Requests JIT compilation for the previously-compiled PCRE which may result in faster matching.

popd ¶

Remove an entry from the directory stack, and perform a `cd` to the new top directory. With no argument, the current top entry is removed. An argument of the form `+n` identifies a stack entry by counting from the left of the list shown by the `dirs` command, starting with zero. An argument of the form `-n` counts from the right. `pindex(PUSHD_MINUS, use of)` If the `PUSHD_MINUS` option is set, the meanings of `+` and `-` in this context are swapped.

If the `-q` (quiet) option is specified, the hook function `chpwd` and the functions in the array `$chpwd_functions` are not called, and the new directory stack is not printed. This is useful for calls to `popd` that do not change the environment seen by an interactive user.

print ¶

With the `-f` option the arguments are printed as described by `printf`. With no flags or with the flag `-`, the arguments are printed on the standard output as described by `echo`, with the following differences: the escape sequence `\M-"x"` (or `\M"x"`) metafiles the character *x* (sets the highest bit), `\C-"x"` (or `\C"x"`) produces a control character ("`C-@`" and "`C-?`" give the characters NULL and delete), a character code in octal is represented by `\"NNN"` (instead of `\0"NNN"`), and `"\E"` is a synonym for `"\e"`. Finally, if not in an escape sequence, `"\"` escapes the following character and is not printed.

- `-a` — Print arguments with the column incrementing first. Only useful with the `-c` and `-C` options.
- `-b` — Recognize all the escape sequences defined for the `bindkey` command, see *Zle Builtins* (zshzle).
- `-c` — Print the arguments in columns. Unless `-a` is also given, arguments are printed with the row incrementing first.
- `-C cols` — Print the arguments in *cols* columns. Unless `-a` is also given, arguments are printed with the row incrementing first.
- `-D` — Treat the arguments as paths, replacing directory prefixes with `~` expressions corresponding to directory names, as appropriate.
- `-i` — If given together with `-o` or `-O`, sorting is performed case-independently.
- `-l` — Print the arguments separated by newlines instead of spaces. Note: if the list of arguments is empty, `print -l` will still output one empty line. To print a possibly-empty list of arguments one per line, use `print -C1 -- "$list[@]"`.
- `-m` — Take the first argument as a pattern (should be quoted), and remove it from the argument list together with subsequent arguments that do not match this pattern.
- `-n` — Do not add a newline to the output.

- **-N** — Print the arguments separated and terminated by nulls. Again, `print -rNC1 -- "$list[@]"` is a canonical way to print an arbitrary list as null-delimited records.
 - **-o** — Print the arguments sorted in ascending order.
 - **-O** — Print the arguments sorted in descending order.
 - **-p** — Print the arguments to the input of the coprocess.
 - **-P** — Perform prompt expansion (see *Expansion of Prompt Sequences* (zshmisc)). In combination with `"-f"`, prompt escape sequences are parsed only within interpolated arguments, not within the format string.
 - **-r** — Ignore the escape conventions of `echo`.
 - **-R** — Emulate the BSD `echo` command, which does not process escape sequences unless the `-e` flag is given. The `-n` flag suppresses the trailing newline. Only the `-e` and `-n` flags are recognized after `-R`; all other arguments and options are printed.
 - **-s** — Place the results in the history list instead of on the standard output. Each argument to the `print` command is treated as a single word in the history, regardless of its content.
 - **-S** — Place the results in the history list instead of on the standard output. In this case only a single argument is allowed; it will be split into words as if it were a full shell command line. The effect is similar to reading the line from a history file with the `HIST_LEX_WORDS` option active.
 - **-u *n*** — Print the arguments to file descriptor *n*.
 - **-v *name*** — Store the printed arguments as the value of the parameter *name*.
 - **-x *tab-stop*** — Expand leading tabs on each line of output in the printed string assuming a tab stop every *tab-stop* characters. This is appropriate for formatting code that may be indented with tabs. Note that leading tabs of any argument to print, not just the first, are expanded, even if `print` is using spaces to separate arguments (the column count is maintained across arguments but may be incorrect on output owing to previous unexpanded tabs).
- The start of the output of each print command is assumed to be aligned with a tab stop. Widths of multibyte characters are handled if the option `MULTIBYTE` is in effect. This option is ignored if other formatting options are in effect, namely column alignment or `printf` style, or if output is to a special location such as shell history or the command line editor.
- **-X *tab-stop*** — This is similar to `-x`, except that all tabs in the printed string are expanded. This is appropriate if tabs in the arguments are being used to produce a table format.
 - **-z** — Push the arguments onto the editing buffer stack, separated by spaces.

If any of `"-m"`, `"-o"` or `"-O"` are used in combination with `"-f"` and there are no arguments (after the removal process in the case of `"-m"`) then nothing is printed.

printf ¶

Print the arguments according to the format specification. Formatting rules are the same as used in C. The same escape sequences as for `echo` are recognised in the format. All C conversion specifications ending in one of `csdiouxXeEfgGn` are handled. In addition to this, `"%b"` can be used instead of `"%s"` to cause escape sequences in the argument to be recognised and `"%q"` can be used to quote the argument in such a way that allows it to be reused as shell input. With the numeric format specifiers, if the corresponding argument starts with a quote character, the numeric value of the following character is used as the number to print; otherwise the argument is evaluated as an arithmetic expression. See *Arithmetic Evaluation* (zshmisc) for a description of arithmetic expressions. With `"%n"`, the corresponding argument is taken as an identifier which is created as an integer parameter.

Normally, conversion specifications are applied to each argument in order but they can explicitly specify the `_n`th argument is to be used by replacing `"%"` by `%"_n_$"` and `"*"` by `*n$`. It is recommended that you do not mix references of this explicit style with the normal style and the handling of such mixed styles may be subject to future change.

If arguments remain unused after formatting, the format string is reused until all arguments have been consumed. With the `print` builtin, this can be suppressed by using the `-r` option. If more arguments are required by the format than have been specified, the behaviour is as if zero or an empty string had been specified as the argument.

The `-v` option causes the output to be stored as the value of the parameter *name*, instead of printed. If *name* is an array and the format string is reused when consuming arguments then one array element will be used for each use of the format string.

private ¶

The **private** builtin accepts all the same options and arguments as **local** (nmref(Shell Builtin Commands)(zshbuiltins)) except for the **-T** option. Tied parameters may not be made private.

The **-p** option is presently disabled because the state of private parameters cannot reliably be reloaded. When **typeset -p** outputs a private parameter, it is treated as a local with the **-h** (hide) option enabled.

If used at the top level (outside a function scope), **private** creates a normal parameter in the same manner as **declare** or **typeset**. A warning about this is printed if **WARN_CREATE_GLOBAL** is set (nmref(Options)(zshoptions)). Used inside a function scope, **private** creates a local parameter similar to one declared with **local**, except having special properties noted below.

Special parameters which expose or manipulate internal shell state, such as **ARGC**, **argv**, **COLUMNS**, **LINES**, **UID**, **EUID**, **IFS**, **PROMPT**, **RANDOM**, **SECONDS**, etc., cannot be made private unless the **-h** option is used to hide the special meaning of the parameter. This may change in the future.

pushd ¶

Change the current directory, and push the old current directory onto the directory stack. In the first form, change the current directory to *arg*. If *arg* is not specified, change to the second directory on the stack (that is, exchange the top two entries), or change to **\$HOME** if the **PUSHD_TO_HOME** option is set or if there is only one entry on the stack. Otherwise, *arg* is interpreted as it would be by **cd**. The meaning of *old* and *new* in the second form is also the same as for **cd**.

The third form of **pushd** changes directory by rotating the directory list. An argument of the form **+"n"** identifies a stack entry by counting from the left of the list shown by the **dirs** command, starting with zero. An argument of the form **-"n"** counts from the right. If the **PUSHD_MINUS** option is set, the meanings of **+"n"** and **-"n"** in this context are swapped.

If the **-q** (quiet) option is specified, the hook function **chpwd** and the functions in the array **\$chpwd_functions** are not called, and the new directory stack is not printed. This is useful for calls to **pushd** that do not change the environment seen by an interactive user.

If the option **-q** is not specified and the shell option **PUSHD_SILENT** is not set, the directory stack will be printed after a **pushd** is performed.

The options **-s**, **-L** and **-P** have the same meanings as for the **cd** builtin.

pushln ¶

Equivalent to **print -nz**.

pwd ¶

Print the absolute pathname of the current working directory. If the **-r** or the **-P** flag is specified, or the **CHASE_LINKS** option is set and the **-L** flag is not given, the printed path will not contain symbolic links.

r ¶

Re-execute the previous command. Shorthand for **fc -e -**.

read ¶

vindex(REPLY, use of) **vindex(reply, use of)** Read one line and break it into fields using the characters in **\$IFS** as separators, except as noted below. The first field is assigned to the first *name*, the second field to the second *name*, etc., with leftover fields assigned to the last *name*. If *name* is omitted then **REPLY** is used for scalars and **reply** for arrays.

- **-r** — Raw mode: a **"\n"** at the end of a line does not signify line continuation and backslashes in the line don't quote the following character and are not removed.

- **-s** — Don't echo back characters if reading from the terminal.

- **-q** — Read only one character from the terminal and set *name* to **"y"** if this character was **"y"** or **"Y"** and to **"n"** otherwise. With this flag set the return status is zero only if the character was **"y"** or **"Y"**. This option may be used with a timeout (see **-t**); if the read times out, or encounters end of file, status 2 is returned. Input is read from the terminal unless one of **-u** or **-p** is present. This option may also be used within zle widgets.

- **-k [num]** — Read only one (or *num*) characters. All are assigned to the first *name*, without word splitting. This flag is ignored when **-q** is present. Input is read from the terminal unless one of **-u** or **-p** is present. This option may also be used within **zle** widgets.

Note that despite the mnemonic "key" this option does read full characters, which may consist of multiple bytes if the option **MULTIBYTE** is set.

- **-z** — Read one entry from the editor buffer stack and assign it to the first *name*, without word splitting. Text is pushed onto the stack with "print -z" or with **push-line** from the line editor (see **nmref(Zsh Line Editor)(zshzle)**). This flag is ignored when the **-k** or **-q** flags are present.

- **-e**

- **-E** — The input read is printed (echoed) to the standard output. If the **-e** flag is used, no input is assigned to the parameters.

- **-A** — The first *name* is taken as the name of an array and all words are assigned to it.

- **-c**

- **-l** — These flags are allowed only if called inside a function used for completion (specified with the **-K** flag to **compctl**). If the **-c** flag is given, the words of the current command are read. If the **-l** flag is given, the whole line is assigned as a scalar. If both flags are present, **-l** is used and **-c** is ignored.

- **-n** — Together with **-c**, the number of the word the cursor is on is read. With **-l**, the index of the character the cursor is on is read. Note that the command name is word number 1, not word 0, and that when the cursor is at the end of the line, its character index is the length of the line plus one.

- **-u n** — Input is read from file descriptor *n*.

- **-p** — Input is read from the coprocess.

- **-d delim** — Input is terminated by the first character of *delim* instead of by newline. For compatibility with other shells, if *delim* is an empty string, input is terminated at the first NUL.

- **-t [num]** — Test if input is available before attempting to read. If *num* is present, it must begin with a digit and will be evaluated to give a number of seconds, which may be a floating point number; in this case the read times out if input is not available within this time. If *num* is not present, it is taken to be zero, so that **read** returns immediately if no input is available. If no input is available, return status 1 and do not set any variables. **ifzman()** This option is not available when reading from the editor buffer with **-z**, when called from within completion with **-c** or **-l**, with **-q** which clears the input queue before reading, or within **zle** where other mechanisms should be used to test for input. **ifzman()** Note that **read** does not attempt to alter the input processing mode. The default mode is canonical input, in which an entire line is read at a time, so usually "read -t" will not read anything until an entire line has been typed. However, when reading from the terminal with **-k** input is processed one key at a time; in this case, only availability of the first character is tested, so that e.g. "read -t -k 2" can still block on the second character. Use two instances of "read -t -k" if this is not what is wanted.

If the first argument contains a "?", the remainder of this word is used as a *prompt* on standard error when the shell is interactive.

The value (exit status) of **read** is 1 when an end-of-file is encountered, or when **-c** or **-l** is present and the command is not called from a **compctl** function, or as described for **-q**. Otherwise the value is 0.

The behavior of some combinations of the **-k**, **-p**, **-q**, **-u** and **-z** flags is undefined. Presently **-q** cancels all the others, **-p** cancels **-u**, **-k** cancels **-z**, and otherwise **-z** cancels both **-p** and **-u**.

The **-c** or **-l** flags cancel any and all of **-kpquz**.

readonly ¶

Same as **typeset -r**. With the **POSIX_BUILTINS** option set, same as **typeset -gr**.

rehash ¶

If this is set when completing external commands, the internal list (hash) of commands will be updated for each search by issuing the **rehash** command. There is a speed penalty for this which is only likely to be noticeable when directories in the path have slow file access.

return ¶

Causes a shell function or "." script to return to the invoking script with the return status specified by an arithmetic expression *n*. Also causes a non-interactive shell to exit, allowing files containing shell code to be used both as scripts and as autoloadable shell functions. For example, the following prints "42":

```
example() { integer foo=40; return "foo + 2" } echo $?
```

If *n* is omitted, the return status is that of the last command executed.

If **return** was executed from a trap in a **TRAPNAL** function, the effect is different for zero and non-zero return status. With zero status (or after an implicit return at the end of the trap), the shell will return to whatever it was previously processing; with a non-zero status, the shell will behave as interrupted except that the return status of the trap is retained. Note that the numeric value of the signal which caused the trap is passed as the first argument, so the statement "return "128+\$1"" will return the same status as if the signal had not been trapped.

rm ¶

Removes files and directories specified.

Normally, **rm** will not remove directories (except with the **-R** or **-r** options). The **-d** option causes **rm** to try removing directories with **unlink** (see `manref(unlink)(2)`), the same method used for files. Typically only the super-user can actually succeed in unlinking directories in this way. **-d** takes precedence over **-R** and **-r**.

By default, the user will be queried before removing any file that the user cannot write to, but writable files will be silently removed. The **-i** option causes the user to be queried about removing any files. The **-f** option causes files to be silently deleted, without querying, and suppresses all error indications. **-f** takes precedence.

The **-R** and **-r** options cause **rm** to recursively descend into directories, deleting all files in the directory before removing the directory with the **rmdir** system call (see `manref(rmdir)(2)`).

The **-s** option is a zsh extension to **rm** functionality. It enables paranoid behaviour, intended to avoid common security problems involving a root-run **rm** being tricked into removing files other than the ones intended. It will refuse to follow symbolic links, so that (for example) `rm /tmp/foo/passwd` can't accidentally remove `/etc/passwd` if `/tmp/foo` happens to be a link to `/etc`. It will also check where it is after leaving directories, so that a recursive removal of a deep directory tree can't end up recursively removing `/usr` as a result of directories being moved up the tree.

rmdir ¶

Removes empty directories specified.

sched ¶

Make an entry in the scheduled list of commands to execute. The time may be specified in either absolute or relative time, and either as hours, minutes and (optionally) seconds separated by a colon, or seconds alone. An absolute number of seconds indicates the time since the epoch (1970/01/01 00:00); this is useful in combination with the features in the **zsh/datetime** module, see *The zsh/datetime Module* (`zshmodules`).

With no arguments, prints the list of scheduled commands. If the scheduled command has the **-o** flag set, this is shown at the start of the command.

With the argument `-"item"`, removes the given item from the list. The numbering of the list is continuous and entries are in time order, so the numbering can change when entries are added or deleted.

Commands are executed either immediately before a prompt, or while the shell's line editor is waiting for input. In the latter case it is useful to be able to produce output that does not interfere with the line being edited. Providing the option **-o** causes the shell to clear the command line before the event and redraw it afterwards. This should be used with any scheduled event that produces visible output to the terminal; it is not needed, for example, with output that updates a terminal emulator's title bar.

To effect changes to the editor buffer when an event executes, use the `zle` command with no arguments to test whether the editor is active, and if it is, then use `zle "widget"` to access the editor via the named *widget*.

The **sched** builtin is not made available by default when the shell starts in a mode emulating another shell. It can be made available with the command `zmodload -F zsh/sched b:sched`.

set ¶

Set the options for the shell and/or set the positional parameters, or declare and set an array. If the `-s` option is given, it causes the specified arguments to be sorted before assigning them to the positional parameters (or to the array *name* if `-A` is used). With `+s` sort arguments in descending order. For the meaning of the other flags, see `nmref(Options)(zshoptions)`. Flags may be specified by name using the `-o` option. If no option name is supplied with `-o`, the current option states are printed: see the description of `setopt` below for more information on the format. With `+o` they are printed in a form that can be used as input to the shell.

If the `-A` flag is specified, *name* is set to an array containing the given `_arg_s`; if no *name* is specified, all arrays are printed together with their values.

If `+A` is used and *name* is an array, the given arguments will replace the initial elements of that array; if no *name* is specified, all arrays are printed without their values.

The behaviour of arguments after `-A name` or `+A name` depends on whether the option `KSH_ARRAYS` is set. If it is not set, all arguments following *name* are treated as values for the array, regardless of their form. If the option is set, normal option processing continues at that point; only regular arguments are treated as values for the array. This means that

```
set -A array -x -- foo
```

sets `array` to `-x -- foo` if `KSH_ARRAYS` is not set, but sets the array to `foo` and turns on the option `-x` if it is set.

If the `-A` flag is not present, but there are arguments beyond the options, the positional parameters are set. If the option list (if any) is terminated by `--`, and there are no further arguments, the positional parameters will be unset.

If no arguments and no `--` are given, then the names and values of all parameters are printed on the standard output. If the only argument is `+`, the names of all parameters are printed.

For historical reasons, `set -` is treated as `set +xv` and `set - args` as `set +xv -- args` when in any other emulation mode than `zsh`'s native mode.

setcap ¶

This is a built-in implementation of the POSIX standard utility. It sets the capability sets on each specified *filename* to the specified *capabilities*.

setopt ¶

Set the options for the shell. All options specified either with flags or by name are set.

If no arguments are supplied, the names of all options currently set are printed. The form is chosen so as to minimize the differences from the default options for the current emulation (the default emulation being native `zsh`, shown as `<Z>` in *Description of Options* (`zshoptions`)). Options that are on by default for the emulation are shown with the prefix `no` only if they are off, while other options are shown without the prefix `no` and only if they are on. In addition to options changed from the default state by the user, any options activated automatically by the shell (for example, `SHIN_STDIN` or `INTERACTIVE`) will be shown in the list. The format is further modified by the option `KSH_OPTION_PRINT`, however the rationale for choosing options with or without the `no` prefix remains the same in this case.

If the `-m` flag is given the arguments are taken as patterns (which should be quoted to protect them from filename expansion), and all options with names matching these patterns are set.

Note that a bad option name does not cause execution of subsequent shell code to be aborted; this is behaviour is different from that of `set -o`. This is because `set` is regarded as a special builtin by the POSIX standard, but `setopt` is not.

shift ¶

The positional parameters `${n+1t()}` ... are renamed to `$1` ..., where *n* is an arithmetic expression that defaults to 1. If any `_name_s` are given then the arrays with these names are shifted instead of the positional parameters.

If the option `-p` is given arguments are instead removed (popped) from the end rather than the start of the array.

source ¶

Same as `.`, except that the current directory is always searched and is always searched first, before directories in `$path`.

stat ¶

The command acts as a front end to the `stat` system call (see `manref(stat)(2)`). The same command is provided with two names; as the name `stat` is often used by an external command it is recommended that only the `zstat` form of the command is used. This can be arranged by loading the module with the command `zmodload -F zsh/stat b:zstat`.

If the `stat` call fails, the appropriate system error message is printed and status 1 is returned. The fields of `struct stat` give information about the files provided as arguments to the command. In addition to those available from the `stat` call, an extra element "link" is provided. These elements are:

- `device` — The number of the device on which the file resides.
- `inode` — The unique number of the file on this device ("inode" number).
- `mode` — The mode of the file; that is, the file's type and access permissions. With the `-s` option, this will be returned as a string corresponding to the first column in the display of the `ls -l` command.
- `nlink` — The number of hard links to the file.
- `uid` — The user ID of the owner of the file. With the `-s` option, this is displayed as a user name.
- `gid` — The group ID of the file. With the `-s` option, this is displayed as a group name.
- `rdev` — The raw device number. This is only useful for special devices.
- `size` — The size of the file in bytes.
- `atime`
- `mtime`
- `ctime` — The last access, modification and inode change times of the file, respectively, as the number of seconds since midnight GMT on 1st January, 1970. With the `-s` option, these are printed as strings for the local time zone; the format can be altered with the `-F` option, and with the `-g` option the times are in GMT.
- `blksize` — The number of bytes in one allocation block on the device on which the file resides.
- `block` — The number of disk blocks used by the file.
- `link` — If the file is a link and the `-L` option is in effect, this contains the name of the file linked to, otherwise it is empty. Note that if this element is selected (""zstat +link"") then the `-L` option is automatically used.

A particular element may be selected by including its name preceded by a "+" in the option list; only one element is allowed. The element may be shortened to any unique set of leading characters. Otherwise, all elements will be shown for all files.

Options:

- `-A array` — Instead of displaying the results on standard output, assign them to an *array*, one `struct stat` element per array element for each file in order. In this case neither the name of the element nor the name of the files appears in *array* unless the `-t` or `-n` options were given, respectively. If `-t` is given, the element name appears as a prefix to the appropriate array element; if `-n` is given, the file name appears as a separate array element preceding all the others. Other formatting options are respected.
- `-H hash` — Similar to `-A`, but instead assign the values to *hash*. The keys are the elements listed above. If the `-n` option is provided then the name of the file is included in the hash with key *name*.
- `-f fd` — Use the file on file descriptor *fd* instead of named files; no list of file names is allowed in this case.
- `-F fmt` — Supplies a `strftime` (see `manref(strftime)(3)`) string for the formatting of the time elements. The format string supports all of the zsh extensions (see the description of `%D{string}` prompt escape in *Simple Prompt Escapes* (zshmisc)). In particular, `-F %5.%N` can be used to show timestamps with nanosecond precision if supported by the system. The `-s` option is implied.
- `-g` — Show the time elements in the GMT time zone. The `-s` option is implied.
- `-l` — List the names of the type elements (to standard output or an array as appropriate) and return immediately; arguments, and options other than `-A`, are ignored.
- `-L` — Perform an `lstat` (see `manref(lstat)(2)`) rather than a `stat` system call. In this case, if the file is a link, information about the link itself rather than the target file is returned. This option is required to make the `link` element useful. It's important to note that this is the exact opposite from `manref(ls)(1)`, etc.
- `-n` — Always show the names of files. Usually these are only shown when output is to standard output and there is more than one file in the list.

- **-N** — Never show the names of files.
- **-o** — If a raw file mode is printed, show it in octal, which is more useful for human consumption than the default of decimal. A leading zero will be printed in this case. Note that this does not affect whether a raw or formatted file mode is shown, which is controlled by the **-r** and **-s** options, nor whether a mode is shown at all.
- **-r** — Print raw data (the default format) alongside string data (the **-s** format); the string data appears in parentheses after the raw data.
- **-s** — Print **mode**, **uid**, **gid** and the three time elements as strings instead of numbers. In each case the format is like that of **ls -l**.
- **-t** — Always show the type names for the elements of **struct stat**. Usually these are only shown when output is to standard output and no individual element has been selected.
- **-T** — Never show the type names of the **struct stat** elements.

strftime ¶

Output the date in the *format* specified. With no *epochtime*, the current system date/time is used; optionally, *epochtime* may be used to specify the number of seconds since the epoch, and *nanoseconds* may additionally be used to specify the number of nanoseconds past the second (otherwise that number is assumed to be 0). See `manref(strftime)(3)` for details. The zsh extensions described under `%D{string}` prompt escape in *Simple Prompt Escapes* (`zshmisc`) are also available.

- **-n** — Suppress printing a newline after the formatted string.
- **-q** — Run quietly; suppress printing of all error messages described below. Errors for invalid *epochtime* values are always printed.
- **-r** — With the option **-r** (reverse), use *format* to parse the input string *timestring* and output the number of seconds since the epoch at which the time occurred. The parsing is implemented by the system function **strptime**; see `manref(strptime)(3)`. This means that zsh format extensions are not available, but for reverse lookup they are not required.

In most implementations of **strptime** any timezone in the *timestring* is ignored and the local timezone declared by the **TZ** environment variable is used; other parameters are set to zero if not present.

If *timestring* does not match *format* the command returns status 1 and prints an error message. If *timestring* matches *format* but not all characters in *timestring* were used, the conversion succeeds but also prints an error message.

If either of the system functions **strptime** or **mktime** is not available, status 2 is returned and an error message is printed.

- **-s scalar** — Assign the date string (or epoch time in seconds if **-r** is given) to *scalar* instead of printing it.

Note that depending on the system's declared integral time type, **strftime** may produce incorrect results for epoch times greater than 2147483647 which corresponds to 2038-01-19 03:14:07 +0000.

suspend ¶

Suspend the execution of the shell (send it a **SIGTSTP**) until it receives a **SIGCONT**. Unless the **-f** option is given, this will refuse to suspend a login shell.

sync ¶

Calls the system call of the same name (see `manref(sync)(2)`), which flushes dirty buffers to disk. It might return before the I/O has actually been completed.

syserror ¶

This command prints out the error message associated with *errno*, a system error number, followed by a newline to standard error.

Instead of the error number, a name *errname*, for example **ENOENT**, may be used. The set of names is the same as the contents of the array **errno**s, see below.

If the string *prefix* is given, it is printed in front of the error message, with no intervening space.

If *errvar* is supplied, the entire message, without a newline, is assigned to the parameter names *errvar* and nothing is output.

A return status of 0 indicates the message was successfully printed (although it may not be useful if the error number was out of the system's range), a return status of 1 indicates an error in the parameters, and a return status of 2 indicates the error name was not recognised (no message is printed for this).

sysopen ¶

This command opens a file. The `-r`, `-w` and `-a` flags indicate whether the file should be opened for reading, writing and appending, respectively. The `-m` option allows the initial permissions to use when creating a file to be specified in octal form. The file descriptor is specified with `-u`. Either an explicit file descriptor in the range 0 to 9 can be specified or a variable name can be given to which the file descriptor number will be assigned.

The `-o` option allows various system specific options to be specified as a comma-separated list. The following is a list of possible options. Note that, depending on the system, some may not be available.

- `cloexec` — mark file to be closed when other programs are executed (else the file descriptor remains open in subshells and forked external executables)
- `create`
- `creat` — create file if it does not exist
- `excl` — create file, error if it already exists
- `noatime` — suppress updating of the file atime
- `nofollow` — fail if *file* is a symbolic link
- `nonblock` — the file is opened in nonblocking mode
- `sync` — request that writes wait until data has been physically written
- `truncate`
- `trunc` — truncate file to size 0

A return status of 0 indicates the descriptor was successfully opened, otherwise an error message is printed, and 1 is returned for an error in the parameters to the command, or 2 is returned for a system error. The parameter `ERRNO` is nonzero for system errors.

To close the file, use one of the following:

```
exec {fd}<&- exec {fd}>&-
```

sysread ¶

Perform a single system read from file descriptor *infd*, or zero if that is not given. The result of the read is stored in *param* or `REPLY` if that is not given. If *countvar* is given, the number of bytes read is assigned to the parameter named by *countvar*.

The maximum number of bytes read is *bufsize* or 8192 if that is not given, however the command returns as soon as any number of bytes was successfully read.

If *timeout* is given, it specifies a timeout in seconds, which may be zero to poll the file descriptor. This is handled by the `poll` system call if available, otherwise the `select` system call if available.

If *outfd* is given, an attempt is made to write all the bytes just read to the file descriptor *outfd*. If this fails, because of a system error other than `EINTR` or because of an internal zsh error during an interrupt, the bytes read but not written are stored in the parameter named by *param* if supplied (no default is used in this case), and the number of bytes read but not written is stored in the parameter named by *countvar* if that is supplied. If it was successful, *countvar* contains the full number of bytes transferred, as usual, and *param* is not set.

The error `EINTR` (interrupted system call) is handled internally so that shell interrupts are transparent to the caller. Any other error causes a return.

The possible return statuses are

- **0** — At least one byte of data was successfully read and, if appropriate, written.
- **1** — There was an error in the parameters to the command. This is the only error for which a message is printed to standard error.

- **2** — There was an error on the read, or on polling the input file descriptor for a timeout. The parameter **ERRNO** identifies the error.
- **3** — Data were successfully read, but there was an error writing them to *outfd*. The parameter **ERRNO** identifies the error.
- **4** — The attempt to read timed out. Note this does not set **ERRNO** as this is not a system error.
- **5** — No system error occurred, but zero bytes were read. This usually indicates end of file. The parameters are set according to the usual rules; no write to *outfd* is attempted.

sysseek ¶

The current file position at which future reads and writes will take place is adjusted to the specified byte offset. The *offset* is evaluated as a math expression. The **-u** option allows the file descriptor to be specified. By default the offset is specified relative to the start of the file but, with the **-w** option, it is possible to specify that the offset should be relative to the current position or the end of the file.

The return status may be 0 for success, 1 for an error in the parameters to the command, or 2 for an error on the seek; no error message is printed in the last case, but the parameter **ERRNO** reflects the error that occurred.

syswrite ¶

The data (a single string of bytes) are written to the file descriptor *outfd*, or 1 if that is not given, using the **write** system call. Multiple write operations may be used if the first does not write all the data.

If *countvar* is given, the number of byte written is stored in the parameter named by *countvar*; this may not be the full length of *data* if an error occurred.

The error **EINTR** (interrupted system call) is handled internally by retrying; otherwise an error causes the command to return. For example, if the file descriptor is set to non-blocking output, an error **EAGAIN** (on some systems, **EWouldBlock**) may result in the command returning early.

The return status may be 0 for success, 1 for an error in the parameters to the command, or 2 for an error on the write; no error message is printed in the last case, but the parameter **ERRNO** reflects the error that occurred.

test ¶

Like the system version of **test**. Added for compatibility; use conditional expressions instead (see *Conditional Expressions* (zshmisc)). The main differences between the conditional expression syntax and the **test** and **[]** builtins are: these commands are not handled syntactically, so for example an empty variable expansion may cause an argument to be omitted; syntax errors cause status 2 to be returned instead of a shell error; and arithmetic operators expect integer arguments rather than arithmetic expressions.

The command attempts to implement POSIX and its extensions where these are specified. Unfortunately there are intrinsic ambiguities in the syntax; in particular there is no distinction between test operators and strings that resemble them. The standard attempts to resolve these for small numbers of arguments (up to four); for five or more arguments compatibility cannot be relied on. Users are urged wherever possible to use the **[[** test syntax which does not have these ambiguities.

times ¶

Print the accumulated user and system times for the shell and for processes run from the shell.

trap ¶

arg is a series of commands (usually quoted to protect it from immediate evaluation by the shell) to be read and executed when the shell receives any of the signals specified by one or more *sig* args. Each *sig* can be given as a number, or as the name of a signal either with or without the string **SIG** in front (e.g. 1, HUP, and SIGHUP are all the same signal).

If *arg* is "-", then the specified signals are reset to their defaults, or, if no *sig* args are present, all traps are reset.

If *arg* is an empty string, then the specified signals are ignored by the shell (and by the commands it invokes).

If *arg* is omitted but one or more *sig* args are provided (i.e. the first argument is a valid signal number or name), the effect is the same as if *arg* had been specified as "-".

The **trap** command with no arguments prints a list of commands associated with each signal.

If *sig* is **ZERR** then *arg* will be executed after each command with a nonzero exit status. **ERR** is an alias for **ZERR** on systems that have no **SIGERR** signal (this is the usual case).

If *sig* is **DEBUG** then *arg* will be executed before each command if the option **DEBUG_BEFORE_CMD** is set (as it is by default), else after each command. Here, a "command" is what is described as a "sublist" in the shell grammar, see *Simple Commands & Pipelines* (zshmisc). If **DEBUG_BEFORE_CMD** is set various additional features are available. First, it is possible to skip the next command by setting the option **ERR_EXIT**; see the description of the **ERR_EXIT** option in [nmref\(Description of Options\)](#)(zshoptions). Also, the shell parameter **ZSH_DEBUG_CMD** is set to the string corresponding to the command to be executed following the trap. Note that this string is reconstructed from the internal format and may not be formatted the same way as the original text. The parameter is unset after the trap is executed.

If *sig* is **0** or **EXIT** and the **trap** statement is executed inside the body of a function, then the command *arg* is executed after the function completes. The value of **\$?** at the start of execution is the exit status of the shell or the return status of the function exiting. If *sig* is **0** or **EXIT** and the **trap** statement is not executed inside the body of a function, then the command *arg* is executed when the shell terminates; the trap runs before any **zshexit** hook functions.

ZERR, **DEBUG**, and **EXIT** traps are not executed inside other traps. **ZERR** and **DEBUG** traps are kept within subshells, while other traps are reset.

Note that traps defined with the **trap** builtin are slightly different from those defined as `TRAP="NAL () { ... }"`, as the latter have their own function environment (line numbers, local variables, etc.) while the former use the environment of the command in which they were called. For example,

```
trap 'print $LINENO' DEBUG
```

will print the line number of a command executed after it has run, while

```
example(TRAPDEBUG+) { print $LINENO; }
```

will always print the number zero.

Alternative signal names are allowed as described under **kill** above. Defining a trap under either name causes any trap under an alternative name to be removed. However, it is recommended that for consistency users stick exclusively to one name or another.

true ¶

Do nothing and return an exit status of 0.

ttctl ¶

The **-f** option freezes the tty (i.e. terminal or terminal emulator), and **-u** unfreezes it. When the tty is frozen, no changes made to the tty settings by external programs will be honored by the shell, except for changes in the size of the screen; the shell will simply reset the settings to their previous values as soon as each command exits or is suspended. Thus, **stty** and similar programs have no effect when the tty is frozen. Freezing the tty does not cause the current state to be remembered; instead, it causes future changes to the state to be blocked.

Without options it reports whether the terminal is frozen or not.

Note that, regardless of whether the tty is frozen or not, the shell needs to change the settings when the line editor starts, so unfreezing the tty does not guarantee settings made on the command line are preserved. Strings of commands run between editing the command line will see a consistent tty state. See also the shell variable **STTY** for a means of initialising the tty before running external commands and/or freezing the tty around a single command.

type ¶

Equivalent to **whence -v**.

typeset ¶

Set or display attributes and values for shell parameters.

Except as noted below for control flags that change the behavior, a parameter is created for each *name* that does not already refer to one. When inside a function, a new parameter is created for every *name* (even those that already exist), and is unset again when the function completes. See *Local Parameters* (zshparam). The same rules apply to special shell parameters, which retain their special attributes when made local.

For each *name=value* assignment, the parameter *name* is set to *value*. If the assignment is omitted and *name* does *not* refer to an existing parameter, a new parameter is initialized to empty string, zero, or empty array (as appropriate), *unless* the shell option `TYPESET_TO_UNSET` is set. When that option is set, the parameter attributes are recorded but the parameter remains unset.

If the shell option `TYPESET_SILENT` is not set, for each remaining *name* that refers to a parameter that is already set, the name and value of the parameter are printed in the form of an assignment. Nothing is printed for newly-created parameters, or when any attribute flags listed below are given along with the *name*. Using "+" instead of minus to introduce an attribute turns it off.

If no *name* is present, the names and values of all parameters are printed. In this case the attribute flags restrict the display to only those parameters that have the specified attributes, and using "+" rather than "-" to introduce the flag suppresses printing of the values of parameters when there is no parameter name.

All forms of the command handle scalar assignment. Array assignment is possible if any of the reserved words `declare`, `export`, `float`, `integer`, `local`, `readonly` or `typeset` is matched when the line is parsed (N.B. not when it is executed). In this case the arguments are parsed as assignments, except that the "+=" syntax and the `GLOB_ASSIGN` option are not supported, and scalar values after = are *not* split further into words, even if expanded (regardless of the setting of the `KSH_TYPESET` option; this option is obsolete).

Examples of the differences between command and reserved word parsing:

```
example(# Reserved word parsing typeset svar=$(echo one word) avar=(several words))
```

The above creates a scalar parameter `svar` and an array parameter `avar` as if the assignments had been

```
example(svar="one word" avar=(several words))
```

On the other hand:

```
example(# Normal builtin interface builtin typeset svar=$(echo two words))
```

The `builtin` keyword causes the above to use the standard builtin interface to `typeset` in which argument parsing is performed in the same way as for other commands. This example creates a scalar `svar` containing the value `two` and another scalar parameter `words` with no value. An array value in this case would either cause an error or be treated as an obscure set of glob qualifiers.

Arbitrary arguments are allowed if they take the form of assignments after command line expansion; however, these only perform scalar assignment:

```
var='svar=val' typeset $var
```

The above sets the scalar parameter `svar` to the value `val`. Parentheses around the value within `var` would not cause array assignment as they will be treated as ordinary characters when `$var` is substituted. Any non-trivial expansion in the name part of the assignment causes the argument to be treated in this fashion:

```
typeset {var1,var2,var3}=name
```

The above syntax is valid, and has the expected effect of setting the three parameters to the same value, but the command line is parsed as a set of three normal command line arguments to `typeset` after expansion. Hence it is not possible to assign to multiple arrays by this means.

Note that each interface to any of the commands may be disabled separately. For example, "disable -r typeset" disables the reserved word interface to `typeset`, exposing the builtin interface, while "disable typeset" disables the builtin. Note that disabling the reserved word interface for `typeset` may cause problems with the output of "typeset -p", which assumes the reserved word interface is available in order to restore array and associative array values.

Unlike parameter assignment statements, `typeset`'s exit status on an assignment that involves a command substitution does not reflect the exit status of the command substitution. Therefore, to test for an error in a command substitution, separate the declaration of the parameter from its initialization:

```
example(# WRONG typeset var1=$(exit 1) || echo "Trouble with var1"
```

```
# RIGHT typeset var1 && var1=$(exit 1) || echo "Trouble with var1" )
```

To initialize a parameter *param* to a command output and mark it readonly, use `typeset -r param` or `readonly param` after the parameter assignment statement.

`cindex(named reference) cindex(reference, named)` The flag `-n` creates a *named reference* to another parameter. The second parameter need not exist at the time the reference is created. Only the `-H`, `-g`, and `-r` flags may be used in conjunction with `-n`, having their usual meanings. The `-u` flag is special and may be applied to alter the scope of the reference. The *name* so created may not be an array element nor use a subscript, but the *value* assigned may be any valid parameter name syntax, even a subscripted array element (including an associative array element) or an array slice, which is evaluated when

the named reference is expanded. It is an error for a named reference to refer to itself, even indirectly through a chain of references. When `-u` is applied to a named reference, the parameter identified by *value* is always found in the calling function scope rather than the current local scope. In this case, if there is no such parameter in the calling scope, assignments to the named reference may fail, setting `$?` to 1. See *Parameter Expansion* (zshexpn) and *Named References* (zshparam) for details of the behavior of named references.

Local function scoping rules for "typeset" do apply with `-n`, so a declaration within a function persists only until the end of the function unless `-g -n` is specified, and any local parameter (of any type) with the same *name* supplants a named reference from a surrounding scope.

A scalar parameter, including an existing named reference, may be converted to a new named reference by `typeset -n "name"`, so the `-p` option must be included to display the value of a specific named reference *name*.

If no attribute flags are given, and either no *name* arguments are present or the flag `+m` is used, then each parameter name printed is preceded by a list of the attributes of that parameter (`array`, `association`, `exported`, `float`, `integer`, `readonly`, or `undefined` for autoloaded parameters not yet loaded). If `+m` is used with attribute flags, and all those flags are introduced with `+`, the matching parameter names are printed but their values are not.

The following control flags change the behavior of `typeset`:

- `+` — If `+` appears by itself in a separate word as the last option, then the names of all parameters (functions with `-f`) are printed, but the values (function bodies) are not. If *name* arguments appear, both those names and their values are printed in the form of assignments. It is an error for any other options to follow `+`, but the effect of `+` is as if all attribute flags which precede it were given with a `+` prefix. For example, `"typeset -U +"` is equivalent to `"typeset +U"` and displays the names of all arrays having the uniqueness attribute, whereas `"typeset -f -U +"` displays the names of all autoloadable functions. If `+` is the only option, then type information (`array`, `readonly`, etc.) is also printed for each parameter, in the same manner as `"typeset +m"`.

- `-g` — The `-g` (global) means that any resulting parameter will not be restricted to local scope. Note that this does not necessarily mean that the parameter will be global, as the flag will apply to any existing parameter (even if unset) from an enclosing function. This flag does not affect the parameter after creation, hence it has no effect when listing existing parameters, nor does the flag `+g` have any effect except in combination with `-m` (see below).

- `-m` — If the `-m` flag is given the *name* arguments are taken as patterns (use quoting to prevent these from being interpreted as file patterns). With no attribute flags, all parameters (or functions with the `-f` flag) with matching names are printed (the shell option `TYPESET_SILENT` is not used in this case).

If the `+g` flag is combined with `-m`, a new local parameter is created for every matching parameter that is not already local. Otherwise `-m` applies all other flags or assignments to the existing parameters, except that the `-n` option cannot create named references in this way.

Except when assignments are made with *name=value*, using `+m` forces the matching parameters and their attributes to be printed, even inside a function. Note that `-m` is ignored if no patterns are given, so `"typeset -m"` displays attributes but `"typeset -a +m"` does not. Ordinary scalar string parameters have no attributes, so for those `+m` prints only the names.

- `-p [ n ]` — If the `-p` option is given, parameters and values are printed in the form of a `typeset` command with an assignment, regardless of other flags and options. Note that the `-H` flag on parameters is respected; no value will be shown for these parameters.

`-p` may be followed by an optional integer argument. Currently only the value `1` is supported. In this case arrays and associative arrays are printed with newlines between indented elements for readability.

The names and values of `readonly` special parameters (most of the parameters marked "<S>" in *Parameters Set by the Shell* (zshparam), except those documented as settable) are not printed with `-p` because to execute those `typeset` commands would cause errors. However, these parameters are printed when they have been made local to the scope where `"typeset -p"` is run.

- `-T [ scalar[=value] array[=( )_value_ ...] [ sep ] ]` — This flag has a different meaning when used with `-f`; see below. Otherwise the `-T` option requires zero, two, or three arguments to be present. With no arguments, the list of parameters created in this fashion is shown. With two or three arguments, the first two are the name of a scalar and of an array parameter (in that order) that will be tied together in the manner of `$PATH` and `$path`. The optional third argument is a single-character separator which will be used to join the elements of the array to form the scalar; if absent, a colon is used, as with `$PATH`. Only the first character of the separator is significant; any remaining characters are ignored. Multibyte characters are not yet supported.

Only one of the scalar and array parameters may be assigned an initial value (the restrictions on assignment forms described above also apply).

Both the scalar and the array may be manipulated as normal. If one is unset, the other will automatically be unset too. There is no way of untying the variables without unsetting them, nor of converting the type of one of them with another `typeset` command; `+T` does not work, assigning an array to *scalar* is an error, and assigning a scalar to *array* sets it to be a single-element array.

Note that both "`typeset -xT ...`" and "`export -T ...`" work, but only the scalar will be marked for export. Setting the value using the scalar version causes a split on all separators (which cannot be quoted). It is possible to apply `-T` to two previously tied variables but with a different separator character, in which case the variables remain joined as before but the separator is changed.

When an existing scalar is tied to a new array, the value of the scalar is preserved but no attribute other than export will be preserved.

Attribute flags that transform the final value (`-L`, `-R`, `-Z`, `-l`, `-u`) are only applied to the expanded value at the point of a parameter expansion expression using "\$". They are not applied when a parameter is retrieved internally by the shell for any purpose.

The following attribute flags may be specified:

- `-A` — The names refer to associative array parameters; see *Array Parameters* (zshparam).
- `-L [ n ]` — Left justify and remove leading blanks from the value when the parameter is expanded. If *n* is nonzero, it defines the width of the field. If *n* is zero, the width is determined by the width of the value of the first assignment. In the case of numeric parameters, the length of the complete value assigned to the parameter is used to determine the width, not the value that would be output.

The width is the count of characters, which may be multibyte characters if the `MULTIBYTE` option is in effect. Note that the screen width of the character is not taken into account; if this is required, use padding with parameter expansion flags `tt${+(ml)}...` as described in `subref(Parameter Expansion Flags)(zshexpn)`.
- When the parameter is expanded, it is filled on the right with blanks or truncated if necessary to fit the field. Note truncation can lead to unexpected results with numeric parameters. Leading zeros are removed if the `-Z` flag is also set.
- `-R [ n ]` — Similar to `-L`, except that right justification is used; when the parameter is expanded, the field is left filled with blanks or truncated from the end. May not be combined with the `-Z` flag.
- `-U` — For arrays (but not for associative arrays), keep only the first occurrence of each duplicated value. This may also be set for tied parameters (see `-T`) or colon-separated special parameters like `PATH` or `IGNORE`, etc. Note the flag takes effect on assignment, and the type of the variable being assigned to is determinative; for variables with shared values it is therefore recommended to set the flag for all interfaces, e.g. "`typeset -U PATH path`".

This flag has a different meaning when used with `-f`; see below.

- `-Z [ n ]` — Specially handled if set along with the `-L` flag. Otherwise, similar to `-R`, except that leading zeros are used for padding instead of blanks if the first non-blank character is a digit. Numeric parameters are specially handled: they are always eligible for padding with zeroes, and the zeroes are inserted at an appropriate place in the output.
- `-a` — The names refer to array parameters. An array parameter may be created this way, but it may be assigned to in the `typeset` statement only if the reserved word form of `typeset` is enabled (as it is by default). When displaying, both normal and associative arrays are shown.
- `-f` — The names refer to functions rather than parameters. No assignments can be made, and the only other valid flags are `-t`, `-T`, `-k`, `-u`, `-U` and `-z`. The flag `-t` turns on execution tracing for this function; the flag `-T` does the same, but turns off tracing for any named (not anonymous) function called from the present one, unless that function also has the `-t` or `-T` flag. The `-u` and `-U` flags cause the function to be marked for autoloading; `-U` also causes alias expansion to be suppressed when the function is loaded. See the description of the "autoload" builtin for details.

Note that the builtin `functions` provides the same basic capabilities as `typeset -f` but gives access to a few extra options; `autoload` gives further additional options for the case `typeset -fu` and `typeset -fU`.

- `-h` — Hide: only useful for special parameters (those marked "<S>" in the table in *Parameters Set by the Shell* (zsh-param)), and for local parameters with the same name as a special parameter, though harmless for others. A special parameter with this attribute will not retain its special effect when made local. Thus after `typeset -h PATH`, a function containing "`typeset PATH`" will create an ordinary local parameter without the usual behaviour of `PATH`. Alternatively, the local parameter may itself be given this attribute; hence inside a function "`typeset -h PATH`" creates an ordinary local parameter and the special `PATH` parameter is not altered in any way. It is also possible to create a local parameter using `typeset +h "_special_"`, where the local copy of `_special_` will retain its special properties regardless of

having the `-h` attribute. Global special parameters loaded from shell modules (for example, those in `zsh/mapfile` and `zsh/parameter`) are automatically given the `-h` attribute to avoid name clashes.

- `-H` — Hide value: specifies that `typeset` will not display the value of the parameter when listing parameters; the display for such parameters is always as if the `+"` flag were given, but use of the parameter is in other respects normal. This effect does not apply when the parameter is specified by name or by pattern with the `-m` option. This is on by default for the parameters in the `zsh/parameter` and `zsh/mapfile` modules. Note, however, that unlike the `-h` flag this is also useful for non-special parameters.

- `-i [n]` — Use an internal integer representation. If *n* is nonzero it defines the output arithmetic base, otherwise it is determined by the first assignment. Bases from 2 to 36 inclusive are allowed.

- `-E [n]` — Use an internal double-precision floating point representation. On output the variable will be converted to scientific notation. If *n* is nonzero it defines the number of significant figures to display; the default is ten.

- `-F [n]` — Use an internal double-precision floating point representation. On output the variable will be converted to fixed-point decimal notation. If *n* is nonzero it defines the number of digits to display after the decimal point; the default is ten.

- `-l` — Convert the result to lower case whenever the parameter is expanded. The value is *not* converted when assigned.

- `-r` — The given `_name_s` are marked readonly. Note that if *name* is a special parameter, the readonly attribute can be turned on, but cannot then be turned off.

If the `POSIX_BUILTINS` option is set, the readonly attribute is more restrictive: unset variables can be marked readonly and cannot then be set; furthermore, the readonly attribute cannot be removed from any variable.

It is still possible to change other attributes of the variable though, some of which like `-U` or `-Z` would affect the value. More generally, the readonly attribute should not be relied on as a security mechanism.

Note that in `zsh` (like in `pdsh` but unlike most other shells) it is still possible to create a local variable of the same name as this is considered a different variable (though this variable, too, can be marked readonly). Special variables that have been made readonly retain their value and readonly attribute when made local.

- `-t` — Tags the named parameters. Tags only exist to flag the parameter for the user's own purposes --- the list of tagged parameters can be queried using `"typeset -t"`. Tags have no other use. Note that the `-t` flag has a different meaning when used with `-f`; see above.

- `-u` — Convert the result to upper case whenever the parameter is expanded. The value is *not* converted when assigned. This flag has different meanings when used with `-f` or `-n`; see above.

- `-x` — Mark for automatic export to the environment of subsequently executed commands. If the option `GLOBAL_EXPORT` is set, this implies the option `-g`, unless `+g` is also explicitly given; in other words the parameter is not made local to the enclosing function. This is for compatibility with previous versions of `zsh`.

ulimit ¶

Set or display resource limits of the shell and the processes started by the shell. The value of *limit* can be a number in the unit specified below or one of the values "unlimited", which removes the limit on the resource, or "hard", which uses the current value of the hard limit on the resource.

By default, only soft limits are manipulated. If the `-H` flag is given use hard limits instead of soft limits. If the `-S` flag is given together with the `-H` flag set both hard and soft limits.

If no options are used, the file size limit (`-f`) is assumed.

If *limit* is omitted the current value of the specified resources are printed. When more than one resource value is printed, the limit name and unit is printed before each value.

When looping over multiple resources, the shell will abort immediately if it detects a badly formed argument. However, if it fails to set a limit for some other reason it will continue trying to set the remaining limits.

Not all the following resources are supported on all systems. Running `ulimit -a` will show which are supported.

- `-a` — List all of the current resource limits.

- `-b` — Socket buffer size in bytes (N.B. not kilobytes+)

- `-c` — 512-byte blocks on the size of core dumps. - `-d` — Kilobytes on the size of the data segment. - `-e` — Maximum nice value. - `-f` — 512-byte blocks on the size of files written. - `-i` — The number of pending signals. - `-k` — The number of kqueues allocated. - `-l` — Kilobytes on the size of locked-in memory. - `-m` — Kilobytes on the size of physical memory. - `-n` — Open file descriptors. - `-o` — Maximum number of POSIX thread library objects. - `-p` — The number of pseudo-terminals. -

`-q` — Bytes in POSIX message queues. `-r` — Maximum real time priority. On some systems where this is not available, such as NetBSD, this has the same effect as `-T` for compatibility with `sh`. `-s` — Kilobytes on the size of the stack. `-T` — The number of simultaneous threads available to the user. `-t` — CPU seconds to be used. `-u` — The number of processes available to the user. `-v` — Kilobytes on the size of virtual memory. On some systems this refers to the limit called "address space". `-w` — Kilobytes on the size of swapped out memory. `-x` — The number of locks on files. `-y` — Maximum size of buffers for pipes/fifos.

A resource may also be specified by integer in the form `-N _resource_`, where `_resource_` corresponds to the integer defined for the resource by the operating system. This may be used to set the limits for resources known to the shell which do not correspond to option letters. Such limits will be shown by number in the output of `"ulimit -a"`.

The number may alternatively be out of the range of limits compiled into the shell. The shell will try to read or write the limit anyway, and will report an error if this fails.

umask ¶

The umask is set to *mask*. *mask* can be either an octal number or a symbolic value as described in the `manref(chmod)(1)` man page. If *mask* is omitted, the current value is printed. The `-S` option causes the mask to be printed as a symbolic value. Otherwise, the mask is printed as an octal number. Note that in the symbolic form the permissions you specify are those which are to be allowed (not denied) to the users specified.

unalias ¶

Removes aliases. This command works the same as `unhash -a`, except that the `-a` option removes all regular or global aliases, or with `-s` all suffix aliases: in this case no *name* arguments may appear. The options `-m` (remove by pattern) and `-s` without `-a` (remove listed suffix aliases) behave as for `unhash -a`. Note that the meaning of `-a` is different between `unalias` and `unhash`.

unfunction ¶

Remove a function definition. Equivalent to `unhash -f / unset -f name`.

unhash ¶

Remove the element named *name* from an internal hash table. The default is remove elements from the command hash table. The `-a` option causes `unhash` to remove regular or global aliases; note when removing a global aliases that the argument must be quoted to prevent it from being expanded before being passed to the command. The `-s` option causes `unhash` to remove suffix aliases. The `-f` option causes `unhash` to remove shell functions. The `-d` options causes `unhash` to remove named directories. If the `-m` flag is given the arguments are taken as patterns (should be quoted) and all elements of the corresponding hash table with matching names will be removed.

unlimit ¶

The resource limit for each *resource* is set to the hard limit. If the `-h` flag is given and the shell has appropriate privileges, the hard resource limit for each *resource* is removed. The resources of the shell process are only changed if the `-s` flag is given.

The `unlimit` command is not made available by default when the shell starts in a mode emulating another shell. It can be made available with the command `"zmodload -F zsh/rlimits b:unlimit"`.

unset ¶

Each named parameter is unset. Local parameters remain local even if unset; they appear unset within scope, but the previous value will still reappear when the scope ends.

Individual elements of associative array parameters may be unset by using subscript syntax on *name*, which should be quoted (or the entire command prefixed with `noglob`) to protect the subscript from filename generation.

If the `-m` flag is specified the arguments are taken as patterns (should be quoted) and all parameters with matching names are unset. Note that this cannot be used when unsetting associative array elements, as the subscript will be treated as part of the pattern.

The `-v` flag specifies that *name* refers to parameters. This is the default behaviour. If the `-n` option is supplied, and *name* is a named reference, *name* will be unset rather than the variable it references.

`unset -f` is equivalent to `unfunction`. The `-n` option has no effect with `-f`.

unsetopt ¶

Unset the options for the shell. All options specified either with flags or by name are unset. If no arguments are supplied, the names of all options currently unset are printed. If the `-m` flag is given the arguments are taken as patterns (which should be quoted to preserve them from being interpreted as glob patterns), and all options with names matching these patterns are unset.

vared ¶

The value of the parameter *name* is loaded into the edit buffer, and the line editor is invoked. When the editor exits, *name* is set to the string value returned by the editor. When the `-c` flag is given, the parameter is created if it doesn't already exist. The `-a` flag may be given with `-c` to create an array parameter, or the `-A` flag to create an associative array. If the type of an existing parameter does not match the type to be created, the parameter is unset and recreated. The `-g` flag may be given to suppress warnings from the `WARN_CREATE_GLOBAL` and `WARN_NESTED_VAR` options.

If an array or array slice is being edited, separator characters as defined in `$IFS` will be shown quoted with a backslash, as will backslashes themselves. Conversely, when the edited text is split into an array, a backslash quotes an immediately following separator character or backslash; no other special handling of backslashes, or any handling of quotes, is performed.

Individual elements of existing array or associative array parameters may be edited by using subscript syntax on *name*. New elements are created automatically, even without `-c`.

If the `-p` flag is given, the following string will be taken as the prompt to display at the left. If the `-r` flag is given, the following string gives the prompt to display at the right. If the `-h` flag is specified, the history can be accessed from ZLE. If the `-e` flag is given, typing `^D` (Control-D) in an empty buffer causes `vared` to exit immediately with a non-zero return value.

The `-M` option gives a keymap to link to the `main` keymap during editing, and the `-m` option gives a keymap to link to the `vicmd` keymap during editing. For vi-style editing, this allows a pair of keymaps to override `viins` and `vicmd`. For emacs-style editing, only `-M` is normally needed but the `-m` option may still be used. On exit, the previous keymaps will be restored.

`Vared` calls the usual "zle-line-init" and "zle-line-finish" hooks before and after it takes control. Using the `-i` and `-f` options, it is possible to replace these with other custom widgets.

If `-t` *tty* is given, *tty* is the name of a terminal device to be used instead of the default `/dev/tty`. If *tty* does not refer to a terminal an error is reported.

wait ¶

Wait for the specified jobs or processes. If *job* is not given then all currently active child processes are waited for. Each *job* can be either a job specification or the process ID of a job in the job table. The exit status from this command is that of the job waited for. If *job* represents an unknown job or process ID, a warning is printed (unless the `POSIX_BUILTINS` option is set) and the exit status is 127.

It is possible to wait for recent processes (specified by process ID, not by job) that were running in the background even if the process has exited. Typically the process ID will be recorded by capturing the value of the variable `$_` immediately after the process has been started. There is a limit on the number of process IDs remembered by the shell; this is given by the value of the system configuration parameter `CHILD_MAX`. When this limit is reached, older process IDs are discarded, least recently started processes first.

Note there is no protection against the process ID wrapping, i.e. if the wait is not executed soon enough there is a chance the process waited for is the wrong one. A conflict implies both process IDs have been generated by the shell, as other processes are not recorded, and that the user is potentially interested in both, so this problem is intrinsic to process IDs.

whence ¶

For each *name*, indicate how it would be interpreted if used as a command name.

If *name* is not an alias, built-in command, external command, shell function, hashed command, or a reserved word, the exit status shall be non-zero, and DASH()- if `-v`, `-c`, or `-w` was passed DASH()- a message will be written to standard output. (This is different from other shells that write that message to standard error.)

`whence` is most useful when *name* is only the last path component of a command, i.e. does not include a `"/`; in particular, pattern matching only succeeds if just the non-directory component of the command is passed.

- `-v` — Produce a more verbose report.
- `-c` — Print the results in a `cs`h-like format. This takes precedence over `-v`.
- `-w` — For each *name*, print `_name_:" word"` where *word* is one of `alias`, `builtin`, `command`, `function`, `hashed`, `reserved` or `none`, according as *name* corresponds to an alias, a built-in command, an external command, a shell function, a command defined with the `hash` builtin, a reserved word, or is not recognised. This takes precedence over `-v` and `-c`.
- `-f` — Causes the contents of a shell function to be displayed, which would otherwise not happen unless the `-c` flag were used.
- `-p` — Do a path search for *name* even if it is an alias, reserved word, shell function or builtin.
- `-a` — Do a search for all occurrences of *name* throughout the command path. Normally only the first occurrence is printed. When combined with `-m`, only names appearing in the command hash table are searched, but all occurrences of those names are printed.
- `-m` — The arguments are taken as patterns (pattern characters should be quoted), and the information is displayed for each entry in the command hash table matching one of these patterns. The hash table is first refilled, in case of changes to `PATH`.
- `-s` — If a pathname contains symlinks, print the symlink-free pathname as well.
- `-S` — As `-s`, but if the pathname had to be resolved by following multiple symlinks, the intermediate steps are printed, too. The symlink resolved at each step might be anywhere in the path.
- `-x num` — Expand tabs when outputting shell functions using the `-c` option. This has the same effect as the `-x` option to the `functions` builtin.

where ¶

Equivalent to `whence -ca`.

which ¶

Equivalent to `whence -c`.

zcompile ¶

This builtin command can be used to compile functions or scripts, storing the compiled form in a file, and to examine files containing the compiled form. This allows faster autoloading of functions and sourcing of scripts by avoiding parsing of the text when the files are read.

The first form (without the `-c`, `-a` or `-t` options) creates a compiled file. If only the *file* argument is given, the output file has the name `_file_.zwc` and will be placed in the same directory as the `_file_`. The shell will load the compiled file instead of the normal function file when the function is autoloaded; see `_Autoloading Functions_` (`zshmisc`) for a description of how autoloaded functions are searched. The extension `.zwc` stands for "zsh word code".

`vindex(fpath, with zcompile)` If there is at least one *name* argument, all the named files are compiled into the output *file* given as the first argument. If *file* does not end in `.zwc`, this extension is automatically appended. Files containing multiple compiled functions are called "digest" files, and are intended to be used as elements of the `FPATH/fpath` special array.

The second form, with the `-c` or `-a` options, writes the compiled definitions for all the named functions into *file*. For `-c`, the names must be functions currently defined in the shell, not those marked for autoloading. Undefined functions that are marked for autoloading may be written by using the `-a` option, in which case the `fpath` is searched and the contents of the definition files for those functions, if found, are compiled into *file*. If both `-c` and `-a` are given, names of both defined functions and functions marked for autoloading may be given. In either case, the functions in files written with the `-c` or `-a` option will be autoloaded as if the `KSH_AUTOLOAD` option were unset.

The reason for handling loaded and not-yet-loaded functions with different options is that some definition files for autoloading define multiple functions, including the function with the same name as the file, and, at the end, call that function. In such

cases the output of "zcompile -c" does not include the additional functions defined in the file, and any other initialization code in the file is lost. Using "zcompile -a" captures all this extra information.

If the **-m** option is combined with **-c** or **-a**, the `_name_s` are used as patterns and all functions whose names match one of these patterns will be written. If no *name* is given, the definitions of all functions currently defined or marked as autoloaded will be written.

Note the second form cannot be used for compiling functions that include redirections as part of the definition rather than within the body of the function; for example

```
example(fn1() { { ... } >~/logfile })
```

can be compiled but

```
example(fn1() { ... } >~/logfile)
```

cannot. It is possible to use the first form of **zcompile** to compile autoloadable functions that include the full function definition instead of just the body of the function.

The third form, with the **-t** option, examines an existing compiled file. Without further arguments, the names of the original files compiled into it are listed. The first line of output shows the version of the shell which compiled the file and how the file will be used (i.e. by reading it directly or by mapping it into memory). With arguments, nothing is output and the return status is set to zero if definitions for *all* `_name_s` were found in the compiled file, and non-zero if the definition for at least one *name* was not found.

Other options:

- **-U** — Aliases are not expanded when compiling the `_name_d` files.

- **-R** — When the compiled file is read, its contents are copied into the shell's memory, rather than memory-mapped (see **-M**). This happens automatically on systems that do not support memory mapping.

When compiling scripts instead of autoloadable functions, it is often desirable to use this option; otherwise the whole file, including the code to define functions which have already been defined, will remain mapped, consequently wasting memory.

- **-M** — The compiled file is mapped into the shell's memory when read. This is done in such a way that multiple instances of the shell running on the same host will share this mapped file. If neither **-R** nor **-M** is given, the **zcompile** builtin decides what to do based on the size of the compiled file.

- **-k**

- **-z** — These options are used when the compiled file contains functions which are to be autoloaded. If **-z** is given, the function will be autoloaded as if the `KSH_AUTOLOAD` option is *not* set, even if it is set at the time the compiled file is read, while if the **-k** is given, the function will be loaded as if `KSH_AUTOLOAD` is set. These options also take precedence over any **-k** or **-z** options specified to the **autoload** builtin. If neither of these options is given, the function will be loaded as determined by the setting of the `KSH_AUTOLOAD` option at the time the compiled file is read. `ifzman()` These options may also appear as many times as necessary between the listed `_name_s` to specify the loading style of all following functions, up to the next **-k** or **-z**.

`ifnzman()` The created file always contains two versions of the compiled format, one for big-endian machines and one for small-endian machines. The upshot of this is that the compiled file is machine independent and if it is read or mapped, only one half of the file is actually used (and mapped). `ifzman()`

zcurSES ¶

Manipulate curses windows. All uses of this command should be bracketed by "zcurSES init" to initialise use of curses, and "zcurSES end" to end it; omitting "zcurSES end" can cause the terminal to be in an unwanted state.

The subcommand **addwin** creates a window with *nlines* lines and *ncols* columns. Its upper left corner will be placed at row `_begin_y_` and column `_begin_x_` of the screen. *targetwin* is a string and refers to the name of a window that is not currently assigned. Note in particular the curses convention that vertical values appear before horizontal values.

If **addwin** is given an existing window as the final argument, the new window is created as a subwindow of *parentwin*. This differs from an ordinary new window in that the memory of the window contents is shared with the parent's memory. Subwindows must be deleted before their parent. Note that the coordinates of subwindows are relative to the screen, not the parent, as with other windows.

Use the subcommand **delwin** to delete a window created with **addwin**. Note that **end** does *not* implicitly delete windows, and that **delwin** does not erase the screen image of the window.

The window corresponding to the full visible screen is called `stdscr`; it always exists after "zurses init" and cannot be deleted with `delwin`.

The subcommand `refresh` will refresh window *targetwin*; this is necessary to make any pending changes (such as characters you have prepared for output with `char`) visible on the screen. `refresh` without an argument causes the screen to be cleared and redrawn. If multiple windows are given, the screen is updated once at the end.

The subcommand `touch` marks the `_targetwin_s` listed as changed. This is necessary before `refreshing` windows if a window that was in front of another window (which may be `stdscr`) is deleted.

The subcommand `move` moves the cursor position in *targetwin* to new coordinates `_new_y_` and `_new_x_`. Note that the subcommand `string` (but not the subcommand `char`) advances the cursor position over the characters added.

The subcommand `clear` erases the contents of *targetwin*. One (and no more than one) of three options may be specified. With the option `redraw`, in addition the next `refresh` of *targetwin* will cause the screen to be cleared and repainted. With the option `eol`, *targetwin* is only cleared to the end of the current cursor line. With the option `bot`, *targetwin* is cleared to the end of the window, i.e. everything to the right and below the cursor is cleared.

The subcommand `position` writes various positions associated with *targetwin* into the array named *array*. These are, in order:

- - — The y and x coordinates of the cursor relative to the top left of *targetwin* - - — The y and x coordinates of the top left of *targetwin* on the screen - - — The size of *targetwin* in y and x dimensions.

Outputting characters and strings are achieved by `char` and `string` respectively.

To draw a border around window *targetwin*, use `border`. Note that the border is not subsequently handled specially: in other words, the border is simply a set of characters output at the edge of the window. Hence it can be overwritten, can scroll off the window, etc.

The subcommand `attr` will set *_targetwin_'s attributes or foreground/background color pair for any successive character output. Each _attribute_ given on the line may be prepended by a + to set or a - to unset that attribute; + is assumed if absent. The attributes supported are blink, bold, dim, reverse, standout, and underline.*

Each `_fg_col_/_bg_col_` attribute (to be read as "`_fg_col_` on `_bg_col_`") sets the foreground and background color for character output. The color `default` is sometimes available (in particular if the library is ncurses), specifying the foreground or background color with which the terminal started. The color pair `default/default` is always available. To use more than the 8 named colors (red, green, etc.) construct the `_fg_col_/_bg_col_` pairs where `_fg_col_` and `_bg_col_` are decimal integers, e.g. `128/200`. The maximum color value is 254 if the terminal supports 256 colors.

`bg` overrides the color and other attributes of all characters in the window. Its usual use is to set the background initially, but it will overwrite the attributes of any characters at the time when it is called. In addition to the arguments allowed with `attr`, an argument `@char` specifies a character to be shown in otherwise blank areas of the window. Owing to limitations of curses this cannot be a multibyte character (use of ASCII characters only is recommended). As the specified set of attributes override the existing background, turning attributes off in the arguments is not useful, though this does not cause an error.

The subcommand `scroll` can be used with `on` or `off` to enable or disable scrolling of a window when the cursor would otherwise move below the window due to typing or output. It can also be used with a positive or negative integer to scroll the window up or down the given number of lines without changing the current cursor position (which therefore appears to move in the opposite direction relative to the window). In the second case, if scrolling is `off` it is temporarily turned `on` to allow the window to be scrolled.

The subcommand `input` reads a single character from the window without echoing it back. If *param* is supplied the character is assigned to the parameter *param*, else it is assigned to the parameter `REPLY`.

If both *param* and *kparam* are supplied, the key is read in "keypad" mode. In this mode special keys such as function keys and arrow keys return the name of the key in the parameter *kparam*. The key names are the macros defined in the `curses.h` or `ncurses.h` with the prefix "KEY_" removed; see also the description of the parameter `zurses_keycodes` below. Other keys cause a value to be set in *param* as before. On a successful return only one of *param* or *kparam* contains a non-empty string; the other is set to an empty string.

If *mparam* is also supplied, `input` attempts to handle mouse input. This is only available with the ncurses library; mouse handling can be detected by checking for the exit status of "zurses mouse" with no arguments. If a mouse button is clicked (or double- or triple-clicked, or pressed or released with a configurable delay from being clicked) then *kparam* is set to the string `MOUSE`, and *mparam* is set to an array consisting of the following elements:

- - — An identifier to discriminate different input devices; this is only rarely useful.

- - — The x, y and z coordinates of the mouse click relative to the full screen, as three elements in that order (i.e. the y coordinate is, unusually, after the x coordinate). The z coordinate is only available for a few unusual input devices and is otherwise set to zero.

- - — Any events that occurred as separate items; usually there will be just one. An event consists of **PRESSED**, **RELEASED**, **CLICKED**, **DOUBLE_CLICKED** or **TRIPLE_CLICKED** followed immediately (in the same element) by the number of the button.

- - — If the shift key was pressed, the string **SHIFT**. - - — If the control key was pressed, the string **CTRL**. - - — If the alt key was pressed, the string **ALT**.

Not all mouse events may be passed through to the terminal window; most terminal emulators handle some mouse events themselves. Note that the ncurses manual implies that using input both with and without mouse handling may cause the mouse cursor to appear and disappear.

The subcommand **mouse** can be used to configure the use of the mouse. There is no window argument; mouse options are global. "zurses mouse" with no arguments returns status 0 if mouse handling is possible, else status 1. Otherwise, the possible arguments (which may be combined on the same command line) are as follows. **delay num** sets the maximum delay in milliseconds between press and release events to be considered as a click; the value 0 disables click resolution, and the default is one sixth of a second. **motion** preceded by an optional "+" (the default) or **-** turns on or off reporting of mouse motion in addition to clicks, presses and releases, which are always reported. However, it appears reports for mouse motion are not currently implemented.

The subcommand **timeout** specifies a timeout value for input from *targetwin*. If *intval* is negative, "zurses input" waits indefinitely for a character to be typed; this is the default. If *intval* is zero, "zurses input" returns immediately; if there is typeahead it is returned, else no input is done and status 1 is returned. If *intval* is positive, "zurses input" waits *intval* milliseconds for input and if there is none at the end of that period returns status 1.

The subcommand **querychar** queries the character at the current cursor position. The return values are stored in the array named *param* if supplied, else in the array **reply**. The first value is the character (which may be a multibyte character if the system supports them); the second is the color pair in the usual **_fg_col_/_bg_col_** notation, or **0** if color is not supported. Any attributes other than color that apply to the character, as set with the subcommand **attr**, appear as additional elements.

The subcommand **resize** resizes **stdscr** and all windows to given dimensions (windows that stick out from the new dimensions are resized down). The underlying curses extension (**resize_term call**) can be unavailable. To verify, zeroes can be used for *height* and *width*. If the result of the subcommand is **0**, **resize_term** is available (**2** otherwise). Tests show that resizing can be normally accomplished by calling **zurses end** and **zurses refresh**. The **resize** subcommand is provided for versatility. Multiple system configurations have been checked and **zurses end** and **zurses refresh** are still needed for correct terminal state after resize. To invoke them with **resize**, use *endwin* argument. Using *nosave* argument will cause new terminal state to not be saved internally by **zurses**. This is also provided for versatility and should normally be not needed.

zdelattr ¶

Remove the extended attribute *attribute* from the specified *filename*.

zf_chgrp ¶

zftp: change group of remote files. Mirrors **chgrp(1)**.

zf_chmod ¶

zftp: change mode of remote files. Mirrors **chmod(1)**.

zf_chown ¶

zftp: change owner of remote files. Mirrors **chown(1)**.

zf_ln ¶

zftp: link / rename remote files. Mirrors **ln(1)**.

zf_mkdir ㉿

zftp: create remote directories. Mirrors `mkdir(1)`.

zf_mv ㉿

zftp: move / rename remote files. Mirrors `mv(1)`.

zf_rm ㉿

zftp: remove remote files. Mirrors `rm(1)`.

zf_rmdir ㉿

zftp: remove remote directories. Mirrors `rmdir(1)`.

zf_sync ㉿

zftp: flush pending writes on the FTP control channel.

zformat ㉿

This builtin provides different forms of formatting. The first form is selected with the `-f` option. In this case the *format* string will be modified by replacing sequences starting with a percent sign in it with strings from the `_spec_s`. Each *spec* should be of the form `_char_:"string"` which will cause every appearance of the sequence `%char` in *format* to be replaced by the *string*. The `"%"` sequence may also contain optional minimum and maximum field width specifications between the `"%"` and the *char* in the form `_%min_._max_c'`, i.e. the minimum field width is given first and if the maximum field width is used, it has to be preceded by a dot. Specifying a minimum field width makes the result be padded with spaces to the right if the `_string_` is shorter than the requested width. Padding to the left can be achieved by giving a negative minimum field width. If a maximum field width is specified, the `_string_` will be truncated after that many characters. After all `"%"` sequences for the given `_spec_s` have been processed, the resulting string is stored in the parameter `_param_`.

The `%`-escapes also understand ternary expressions in the form used by prompts. The `%` is followed by a `tt()` and then an ordinary format specifier character as described above. There may be a set of digits either before or after the `tt()`; these specify a test number, which defaults to zero. Negative numbers are also allowed. An arbitrary delimiter character follows the format specifier, which is followed by a piece of "true" text, the delimiter character again, a piece of "false" text, and a closing parenthesis. The complete expression (without the digits) thus looks like `tt(%)_X_.text1_.text2_)`, except that the `"."` character is arbitrary. The value given for the format specifier in the *char:string* expressions is evaluated as a mathematical expression, and compared with the test number. If they are the same, *text1* is output, else *text2* is output. A parenthesis may be escaped in *text2* as `%)`. Either of *text1* or *text2* may contain nested `%`-escapes.

For example:

```
example(zformat -f REPLY "The answer is '%3(c.yes.no)'." c:3)
```

outputs "The answer is 'yes'." to `REPLY` since the value for the format specifier `c` is 3, agreeing with the digit argument to the ternary expression.

With `-F` instead of `-f`, ternary expressions choose between the "true" or "false" text on the basis of whether the format specifier is present and non-empty. A test number indicates a minimum width for the value given in the format specifier. Negative numbers reverse this, so the test is for whether the value exceeds a maximum width.

The form, using the `-a` option, can be used for aligning strings. Here, the `_spec_s` are of the form `_left_:"right"` where *left* and *right* are arbitrary strings. These strings are modified by replacing the colons by the *sep* string and padding the *left* strings with spaces to the right so that the *sep* strings in the result (and hence the *right* strings after them) are all aligned if the strings are printed below each other. All strings without a colon are left unchanged and all strings with an empty *right* string have the trailing colon removed. In both cases the lengths of the strings are not used to determine how the other strings are to be aligned. A colon in the *left* string can be escaped with a backslash. The resulting strings are stored in the *array*.

zftp ¶

The **zsh/zftp** module is a client for FTP (file transfer protocol). It is implemented as a builtin to allow full use of shell command line editing, file I/O, and job control mechanisms. Often, users will access it via shell functions providing a more powerful interface; a set is provided with the **zsh** distribution and is described in `nmref(Zftp Function System)(zshzftpsys)`. However, the **zftp** command is entirely usable in its own right.

All commands consist of the command name **zftp** followed by the name of a subcommand. These are listed below. The return status of each subcommand is supposed to reflect the success or failure of the remote operation. See a description of the variable **ZFTP_VERBOSE** for more information on how responses from the server may be printed.

zgdbmpath ¶

Put path to database file assigned to *parametername* into **REPLY** scalar.

zgetattr ¶

Get the extended attribute *attribute* from the specified *filename*. If the optional argument *parameter* is given, the attribute is set on that parameter instead of being printed to stdout.

zle ¶

The **zle** builtin performs a number of different actions concerning ZLE.

With no options and no arguments, only the return status will be set. It is zero if ZLE is currently active and widgets could be invoked using this builtin command and non-zero otherwise. Note that even if non-zero status is returned, **zle** may still be active as part of the completion system; this does not allow direct calls to ZLE widgets.

Otherwise, which operation it performs depends on its options:

- **-l [-L | -a] [*string*]** — List all existing user-defined widgets. If the **-L** option is used, list in the form of **zle** commands to create the widgets.

When combined with the **-a** option, all widget names are listed, including the builtin ones. In this case the **-L** option is ignored.

If at least one *string* is given, and **-a** is present or **-L** is not used, nothing will be printed. The return status will be zero if all *_string_s* are names of existing widgets and non-zero if at least one *string* is not a name of a defined widget. If **-a** is also present, all widget names are used for the comparison including builtin widgets, else only user-defined widgets are used.

If at least one *string* is present and the **-L** option is used, user-defined widgets matching any *string* are listed in the form of **zle** commands to create the widgets.

- **-D *widget* ...** — Delete the named *_widget_s*.

- **-A *old-widget new-widget*** — Make the *new-widget* name an alias for *old-widget*, so that both names refer to the same widget. The names have equal standing; if either is deleted, the other remains. If there is already a widget with the *new-widget* name, it is deleted.

- **-N *widget* [*function*]** — Create a user-defined widget. If there is already a widget with the specified name, it is overwritten. When the new widget is invoked from within the editor, the specified shell *function* is called. If no function name is specified, it defaults to the same name as the widget. For further information, see *Zle Widgets* (below).

- **-f *flag* [*flag*...]** — Set various flags on the running widget. Possible values for *flag* are:

yank for indicating that the widget has yanked text into the buffer. If the widget is wrapping an existing internal widget, no further action is necessary, but if it has inserted the text manually, then it should also take care to set **YANK_START** and **YANK_END** correctly. **yankbefore** does the same but is used when the yanked text appears after the cursor.

kill for indicating that text has been killed into the cutbuffer. When repeatedly invoking a kill widget, text is appended to the cutbuffer instead of replacing it, but when wrapping such widgets, it is necessary to call "**zle -f kill**" to retain this effect.

vichange for indicating that the widget represents a vi change that can be repeated as a whole with "vi-repeat-change". The flag should be set early in the function before inspecting the value of **NUMERIC** or invoking other widgets. This has no effect for a widget invoked from insert mode. If insert mode is active when the widget finishes, the change extends until next returning to command mode.

`cindex(completion widgets, creating)`

- **-C *widget completion-widget function*** — Create a user-defined completion widget named *widget*. The completion widget will behave like the built-in completion-widget whose name is given as *completion-widget*. To generate the completions, the shell function *function* will be called. For further information, see `nmref(Completion Widgets)(zshcompwid)`.

- **-R [-c] [*display-string*] [*string ...*]** — Redisplay the command line. If a *display-string* is given and not empty, this is shown in the status line (immediately below the line being edited).

If the optional *_string_s* are given they are listed below the prompt in the same way as completion lists are printed. If no *_string_s* are given but the **-c** option is used such a list is cleared.

Note that immediately after returning from running widgets, the command line will be redisplayed and the strings displayed will be erased. Therefore, this option is only useful for widgets that do not exit immediately after using it.

This command can safely be called outside user defined widgets; if zle is active, the display will be refreshed, while if zle is not active, the command has no effect. In this case there will usually be no other arguments.

The status is zero if zle was active, else one.

- **-M *string*** — As with the **-R** option, the *string* will be displayed below the command line; unlike the **-R** option, the string will not be put into the status line but will instead be printed normally below the prompt. This means that the *string* will still be displayed after the widget returns (until it is overwritten by subsequent commands).

- **-U *string*** — This pushes the characters in the *string* onto the input stack of ZLE. After the widget currently executed finishes ZLE will behave as if the characters in the *string* were typed by the user.

As ZLE uses a stack, if this option is used repeatedly the last string pushed onto the stack will be processed first. However, the characters in each *string* will be processed in the order in which they appear in the string.

- **-K *keymap*** — Selects the keymap named *keymap*. An error message will be displayed if there is no such keymap.

This keymap selection affects the interpretation of following keystrokes within this invocation of ZLE. Any following invocation (e.g., the next command line) will start as usual with the "main" keymap selected.

- **-F [-L | -w] [*fd* [*handler*]]** — Only available if your system supports one of the "poll" or "select" system calls; most modern systems do.

Installs *handler* (the name of a shell function) to handle input from file descriptor *fd*. Installing a handler for an *fd* which is already handled causes the existing handler to be replaced. Any number of handlers for any number of readable file descriptors may be installed. Note that zle makes no attempt to check whether this *fd* is actually readable when installing the handler. The user must make their own arrangements for handling the file descriptor when zle is not active.

When zle is attempting to read data, it will examine both the terminal and the list of handled *fd*'s. If data becomes available on a handled *fd*, zle calls *handler* with the *fd* which is ready for reading as the first argument. Under normal circumstances this is the only argument, but if an error was detected, a second argument provides details: "hup" for a disconnect, "nval" for a closed or otherwise invalid descriptor, or "err" for any other condition. Systems that support only the "select" system call always use "err".

If the option **-w** is also given, the *handler* is instead a line editor widget, typically a shell function made into a widget using "zle -N". In that case *handler* can use all the facilities of zle to update the current editing line. Note, however, that as handling *fd* takes place at a low level changes to the display will not automatically appear; the widget should call "zle -R" to force redisplay. As of this writing, widget handlers only support a single argument and thus are never passed a string for error state, so widgets must be prepared to test the descriptor themselves.

If either type of handler produces output to the terminal, it should call "zle -l" before doing so (see below). Handlers should not attempt to read from the terminal.

If no *handler* is given, but an *fd* is present, any handler for that *fd* is removed. If there is none, an error message is printed and status 1 is returned.

If no arguments are given, or the **-L** option is supplied, a list of handlers is printed in a form which can be stored for later execution.

An *fd* (but not a *handler*) may optionally be given with the **-L** option; in this case, the function will list the handler if any, else silently return status 1.

Note that this feature should be used with care. Activity on one of the *fd*'s which is not properly handled can cause the terminal to become unusable. Removing an *fd* handler from within a signal trap may cause unpredictable behavior.

Here is a simple example of using this feature. A connection to a remote TCP port is created using the `ztcp` command; see *The zsh/net/tcp Module* (`zshmodules`). Then a handler is installed which simply prints out any data which arrives on this connection.

Note that "select" will indicate that the file descriptor needs handling if the remote side has closed the connection; we handle that by testing for a failed read.

```
example(if ztcp pwspc 2811; then tcpfd=$REPLY handler+) { zle -I local line if ! read -r line <&$1; then # select marks this fd if
we reach EOF, # so handle this specially. print "[Read on fd $1 failed, removing.]" >&2 zle -F $1 return 1 fi print -r - $line } zle
-F $tcpfd handler fi)
```

- **-I** — Unusually, this option is most useful outside ordinary widget functions, though it may be used within if normal output to the terminal is required. It invalidates the current zle display in preparation for output; typically this will be from a trap function. It has no effect if zle is not active. When a trap exits, the shell checks to see if the display needs restoring, hence the following will print output in such a way as not to disturb the line being edited:

```
example(TRAPUSR1) { # Invalidate zle display [[ -o zle ]] && zle -I # Show output print Hello }
```

In general, the trap function may need to test whether zle is active before using this method (as shown in the example), since the **zsh/zle** module may not even be loaded; if it is not, the command can be skipped.

It is possible to call "zle -I" several times before control is returned to the editor; the display will only be invalidated the first time to minimise disruption.

Note that there are normally better ways of manipulating the display from within zle widgets; see, for example, "zle -R" above.

The returned status is zero if zle was invalidated, even though this may have been by a previous call to "zle -I" or by a system notification. To test if a zle widget may be called at this point, execute **zle** with no arguments and examine the return status.

- **-T** — This is used to add, list or remove internal transformations on the processing performed by the line editor. It is typically used only for debugging or testing and is therefore of little interest to the general user.

zle -T" *transformation func*" specifies that the given *transformation* (see below) is effected by shell function *func*.

zle -Tr" *transformation*" removes the given *transformation* if it was present (it is not an error if none was).

"zle -TL" can be used to list all transformations currently in operation.

Currently the only transformation is **tc**. This is used instead of outputting termcap codes to the terminal. When the transformation is in operation the shell function is passed the termcap code that would be output as its first argument; if the operation required a numeric argument, that is passed as a second argument. The function should set the shell variable **REPLY** to the transformed termcap code. Typically this is used to produce some simply formatted version of the code and optional argument for debugging or testing. Note that this transformation is not applied to other non-printing characters such as carriage returns and newlines.

- **widget [-n num] [-f flag] [-Nw] [-K keymap] args ...** — Invoke the specified *widget*. This can only be done when ZLE is active; normally this will be within a user-defined widget.

With the options **-n** and **-N**, the current numeric argument will be saved and then restored after the call to *widget*; **-n num** sets the numeric argument temporarily to *num*, while **-N** sets it to the default, i.e. as if there were none.

With the option **-K**, *keymap* will be used as the current keymap during the execution of the widget. The previous keymap will be restored when the widget exits.

Normally, calling a widget in this way does not set the special parameter **WIDGET** and related parameters, so that the environment appears as if the top-level widget called by the user were still active. With the option **-w**, **WIDGET** and related parameters are set to reflect the widget being executed by the **zle** call.

Normally, when *widget* returns the special parameter **LASTWIDGET** will point to it. This can be inhibited by passing the option **-f no last**.

Any further arguments will be passed to the widget; note that as standard argument handling is performed, any general argument list should be preceded by **--**. If it is a shell function, these are passed down as positional parameters; for builtin widgets it is up to the widget in question what it does with them. Currently arguments are only handled by the incremental-search commands, the **history-search-forward** and **-backward** and the corresponding functions prefixed by **vi-**, and by **universal-argument**. No error is flagged if the command does not use the arguments, or only uses some of them.

The return status reflects the success or failure of the operation carried out by the widget, or if it is a user-defined widget the return status of the shell function.

A non-zero return status causes the shell to beep when the widget exits, unless the **BEEP** options was unset or the widget was called via the **zle** command. Thus if a user defined widget requires an immediate beep, it should call the **beep** widget directly.

zlistattr ¶

List the extended attributes currently set on the specified *filename*. If the optional argument *parameter* is given, the list of attributes is set on that parameter instead of being printed to stdout.

zmodload ¶

Performs operations relating to zsh's loadable modules. Loading of modules while the shell is running ("dynamical loading") is not available on all operating systems, or on all installations on a particular operating system, although the **zmodload** command itself is always available and can be used to manipulate modules built into versions of the shell executable without dynamical loading.

Without arguments the names of all currently loaded binary modules are printed. The **-L** option causes this list to be in the form of a series of **zmodload** commands. Forms with arguments are:

- zmodload [-is] name ...

- zmodload -u [-i] name ... — In the simplest case, **zmodload** loads a binary module. The module must be in a file with a name consisting of the specified *name* followed by a standard suffix, usually ".so" (".sl" on HP/UX). If the module to be loaded is already loaded the duplicate module is ignored. If **zmodload** detects an inconsistency, such as an invalid module name or circular dependency list, the current code block is aborted. If it is available, the module is loaded if necessary, while if it is not available, non-zero status is silently returned. The option **-i** is accepted for compatibility but has no effect.

The `_name_d` module is searched for in the same way a command is, using `$module_path` instead of `$path`. However, the path search is performed even when the module name contains a "/", which it usually does. There is no way to prevent the path search.

If the module supports features (see below), **zmodload** tries to enable all features when loading a module. If the module was successfully loaded but not all features could be enabled, **zmodload** returns status 2.

If the option **-s** is given, no error is printed if the module was not available (though other errors indicating a problem with the module are printed). The return status indicates if the module was loaded. This is appropriate if the caller considers the module optional.

With **-u**, **zmodload** unloads modules. The same *name* must be given that was given when the module was loaded, but it is not necessary for the module to exist in the file system. The **-i** option suppresses the error if the module is already unloaded (or was never loaded).

Each module has a boot and a cleanup function. The module will not be loaded if its boot function fails. Similarly a module can only be unloaded if its cleanup function runs successfully.

- zmodload -F [-almLe -P param] module [+/-]feature ... — **zmodload -F** allows more selective control over the features provided by modules. With no options apart from **-F**, the module named *module* is loaded, if it was not already loaded, and the list of `_feature_s` is set to the required state. If no `_feature_s` are specified, the module is loaded, if it was not already loaded, but the state of features is unchanged. Each feature may be preceded by a **+** to turn the feature on, or **-** to turn it off; the **+** is assumed if neither character is present. Any feature not explicitly mentioned is left in its current state; if the module was not previously loaded this means any such features will remain disabled. The return status is zero if all features were set, 1 if the module failed to load, and 2 if some features could not be set (for example, a parameter couldn't be added because there was a different parameter of the same name) but the module was loaded.

The standard features are builtins, conditions, parameters and math functions; these are indicated by the prefix "b:", "c:" ("C:" for an infix condition), "p:" and "f:", respectively, followed by the name that the corresponding feature would have in the shell. For example, "b:strftime" indicates a builtin named `strftime` and `p:EPOCHSECONDS` indicates a parameter named `EPOCHSECONDS`. The module may provide other ("abstract") features of its own as indicated by its documentation; these have no prefix.

With **-l** or **-L**, features provided by the module are listed. With **-l** alone, a list of features together with their states is shown, one feature per line. With **-L** alone, a **zmodload -F** command that would cause enabled features of the module to be turned on is shown. With **-ll**, a **zmodload -F** command that would cause all the features to be set to their current state is shown. If one of these combinations is given with the option **-P param** then the parameter *param* is set to an array of features, either features together with their state or (if **-L** alone is given) enabled features.

With the option **-L** the module name may be omitted; then a list of all enabled features for all modules providing features is printed in the form of **zmodload -F** commands. If **-l** is also given, the state of both enabled and disabled features is output in that form.

A set of features may be provided together with `-l` or `-L` and a module name; in that case only the state of those features is considered. Each feature may be preceded by `+` or `-` but the character has no effect. If no set of features is provided, all features are considered.

With `-e`, the command first tests that the module is loaded; if it is not, status 1 is returned. If the module is loaded, the list of features given as an argument is examined. Any feature given with no prefix is simply tested to see if the module provides it; any feature given with a prefix `+` or `-` is tested to see if it is provided and in the given state. If the tests on all features in the list succeed, status 0 is returned, else status 1.

With `-m`, each entry in the given list of features is taken as a pattern to be matched against the list of features provided by the module. An initial `+` or `-` must be given explicitly. This may not be combined with the `-a` option as autoloading must be specified explicitly.

With `-a`, the given list of features is marked for autoload from the specified module, which may not yet be loaded. An optional `+` may appear before the feature name. If the feature is prefixed with `-`, any existing autoload is removed. The options `-l` and `-L` may be used to list autoloading. Autoloading is specific to individual features; when the module is loaded only the requested feature is enabled. Autoload requests are preserved if the module is subsequently unloaded until an explicit `zmodload -Fa _module_ -"feature"` is issued. It is not an error to request an autoload for a feature of a module that is already loaded.

When the module is loaded each autoload is checked against the features actually provided by the module; if the feature is not provided the autoload request is deleted. A warning message is output; if the module is being loaded to provide a different feature, and that autoload is successful, there is no effect on the status of the current command. If the module is already loaded at the time when `zmodload -Fa` is run, an error message is printed and status 1 returned.

`zmodload -Fa` can be used with the `-l`, `-L`, `-e` and `-P` options for listing and testing the existence of autoloadable features. In this case `-l` is ignored if `-L` is specified. `zmodload -FaL` with no module name lists autoloading for all modules.

Note that only standard features as described above can be autoloaded; other features require the module to be loaded before enabling.

- `zmodload -d [-L] [name]`

- `zmodload -d name dep ...`

- `zmodload -ud name [dep ...]` — The `-d` option can be used to specify module dependencies. The modules named in the second and subsequent arguments will be loaded before the module named in the first argument.

With `-d` and one argument, all dependencies for that module are listed. With `-d` and no arguments, all module dependencies are listed. This listing is by default in a Makefile-like format. The `-L` option changes this format to a list of `zmodload -d` commands.

If `-d` and `-u` are both used, dependencies are removed. If only one argument is given, all dependencies for that module are removed.

- `zmodload -ab [-L]`

- `zmodload -ab [-i] name [builtin ...]`

- `zmodload -ub [-i] builtin ...` — The `-ab` option defines autoloaded builtins. It defines the specified `_builtin_s`. When any of those builtins is called, the module specified in the first argument is loaded and all its features are enabled (for selective control of features use `"zmodload -F -a"` as described above). If only the `name` is given, one builtin is defined, with the same name as the module. `-i` suppresses the error if the builtin is already defined or autoloaded, but not if another builtin of the same name is already defined.

With `-ab` and no arguments, all autoloaded builtins are listed, with the module name (if different) shown in parentheses after the builtin name. The `-L` option changes this format to a list of `zmodload -a` commands.

If `-b` is used together with the `-u` option, it removes builtins previously defined with `-ab`. This is only possible if the builtin is not yet loaded. `-i` suppresses the error if the builtin is already removed (or never existed).

Autoload requests are retained if the module is subsequently unloaded until an explicit `zmodload -ub "builtin"` is issued.

- `zmodload -ac [-IL]`

- `zmodload -ac [-iI] name [cond ...]`

- `zmodload -uc [-iI] cond ...` — The `-ac` option is used to define autoloaded condition codes. The `cond` strings give the names of the conditions defined by the module. The optional `-I` option is used to define infix condition names. Without this option prefix condition names are defined.

If given no condition names, all defined names are listed (as a series of `zmodload` commands if the `-L` option is given).

The `-uc` option removes definitions for autoloaded conditions.

- `zmodload -ap [ -L ]`
- `zmodload -ap [ -i ] name [ parameter ... ]`
- `zmodload -up [ -i ] parameter ...` — The `-p` option is like the `-b` and `-c` options, but makes `zmodload` work on autoloaded parameters instead.
- `zmodload -af [ -L ]`
- `zmodload -af [ -i ] name [ function ... ]`
- `zmodload -uf [ -i ] function ...` — The `-f` option is like the `-b`, `-p`, and `-c` options, but makes `zmodload` work on autoloaded math functions instead.
- `zmodload -a [ -L ]`
- `zmodload -a [ -i ] name [ builtin ... ]`
- `zmodload -ua [ -i ] builtin ...` — Equivalent to `-ab` and `-ub`.
- `zmodload -e [ -A ] [ string ... ]` — The `-e` option without arguments lists all loaded modules; if the `-A` option is also given, module aliases corresponding to loaded modules are also shown. If arguments are provided, nothing is printed; the return status is set to zero if all `_string_s` given as arguments are names of loaded modules and to one if at least on `string` is not the name of a loaded module. This can be used to test for the availability of things implemented by modules. In this case, any aliases are automatically resolved and the `-A` flag is not used.
- `zmodload -A [ -L ] [ modalias[=module] ... ]` — For each argument, if both `modalias` and `module` are given, define `modalias` to be an alias for the module `module`. If the module `modalias` is ever subsequently requested, either via a call to `zmodload` or implicitly, the shell will attempt to load `module` instead. If `module` is not given, show the definition of `modalias`. If no arguments are given, list all defined module aliases. When listing, if the `-L` flag was also given, list the definition as a `zmodload` command to recreate the alias.

The existence of aliases for modules is completely independent of whether the name resolved is actually loaded as a module: while the alias exists, loading and unloading the module under any alias has exactly the same effect as using the resolved name, and does not affect the connection between the alias and the resolved name which can be removed either by `zmodload -R` or by redefining the alias. Chains of aliases (i.e. where the first resolved name is itself an alias) are valid so long as these are not circular. As the aliases take the same format as module names, they may include path separators: in this case, there is no requirement for any part of the path named to exist as the alias will be resolved first. For example, "any/old/alias" is always a valid alias.

Dependencies added to aliased modules are actually added to the resolved module; these remain if the alias is removed. It is valid to create an alias whose name is one of the standard shell modules and which resolves to a different module. However, if a module has dependencies, it will not be possible to use the module name as an alias as the module will already be marked as a loadable module in its own right.

Apart from the above, aliases can be used in the `zmodload` command anywhere module names are required. However, aliases will not be shown in lists of loaded modules with a bare "zmodload".

- `zmodload -R modalias ...` — For each `modalias` argument that was previously defined as a module alias via `zmodload -A`, delete the alias. If any was not defined, an error is caused and the remainder of the line is ignored.

Note that `zsh` makes no distinction between modules that were linked into the shell and modules that are loaded dynamically. In both cases this builtin command has to be used to make available the builtins and other things defined by modules (unless the module is autoloaded on these definitions). This is true even for systems that don't support dynamic loading of modules.

zparseopts ¶

This builtin simplifies the parsing of options in positional parameters, i.e. the set of arguments given by `$*`. Each `spec` describes one option and must be of the form `_opt_[="array"]`. If an option described by `opt` is found in the positional parameters it is copied into the `array` specified with the `-a` option; if the optional `"array"` is given, it is instead copied into that array, which should be declared as a normal array and never as an associative array.

Note that it is an error to give any `spec` without an `"array"` unless one of the `-a` or `-A` options is used.

Unless the `-E` option is given, parsing stops at the first string that isn't described by one of the `_spec_s`. Even with `-E`, parsing always stops at a positional parameter equal to `-"` or (without `-G`) `"-`". See also `-F`.

The *opt* description must be one of the following. Any of the special characters can appear in the option name provided it is preceded by a backslash.

- **name**

- **name+** — The *name* is the name of the option without the leading "-". To specify a GNU-style long option, one of the usual two leading "-" must be included in *name*; for example, a "-file" option is represented by a *name* of "-file".

If a "+" appears after *name*, the option is appended to *array* each time it is found in the positional parameters; without the "+" only the *last* occurrence of the option is preserved.

If one of these forms is used, the option takes no argument, so parsing stops if the next positional parameter does not also begin with "-" (unless the **-E** option is used).

- **name:**

- **name:-**

- **name:** — If one or two colons are given, the option takes an argument; with one colon, the argument is mandatory and with two colons it is optional. The argument is appended to the *array* after the option itself.

An optional argument is put into the same array element as the option name (note that this makes empty strings as arguments indistinguishable). A mandatory argument is added as a separate element unless the ":-" form is used, in which case the argument is put into the same element.

A "+" as described above may appear between the *name* and the first colon.

By default, option-arguments may appear either immediately following the option in the same positional parameter or in the next one. Even an optional argument may appear in the next parameter, unless it begins with a "-". Additionally, there is no special consideration given to an "=" between an option and its argument. To make parsing use the more common GNU-style conventions, use the **-G** option.

When the names of two options that take no arguments overlap, the longest one wins, so that parsing for the `_spec_s` "-foo-foobar" (for example) is unambiguous. However, due to the aforementioned handling of option-arguments, ambiguities may arise when at least one overlapping *spec* takes an argument, as in "-foo: -foobar". In that case, the last matching *spec* wins. (This is not an issue with GNU-style parsing.)

The options of `zparseopts` itself cannot be stacked because, for example, the stack "-DEK" is indistinguishable from a *spec* for the GNU-style long option "-DEK". The options of `zparseopts` itself are:

- **-a array** — As described above, this names the default array in which to store the recognised options.

- **-A assoc** — If this is given, the options and their values are also put into an associative array with the option names as keys and the arguments (if any) as the values.

- **-D** — If this option is given, all options found are removed from the positional parameters of the calling shell or shell function, up to but not including any not described by the `_spec_s`. If the first such parameter is "-" or (without **-G**) "-", it is removed as well. This is similar to using the `shift` builtin.

- **-E** — This changes the parsing rules to *not* stop at the first string that isn't described by one of the `_spec_s`. It can be used to test for or (if used together with **-D**) extract options and their arguments, ignoring all other options and arguments that may be in the positional parameters. As indicated above, parsing still stops at the first "-" or "-" not described by a *spec*, but it is not removed when used with **-D**.)

- **-F** — If this option is given, `zparseopts` immediately stops at the first option-like parameter not described by one of the `_spec_s`, prints an error message, and returns status 1. Removal (**-D**) and extraction (**-E**) are not performed, and option arrays are not updated. This provides basic validation for the given options.

Note that the appearance in the positional parameters of an option without its required argument always aborts parsing and returns an error as described above regardless of whether this option is used.

- **-G** — This option specifies that options are parsed in the GNU style, similar to `tt(getopt_long+(3+))`. In particular:

Given the *spec* "-foo:", the positional parameter "-foo=bar" is interpreted as option "-foo" with argument "bar", rather than argument "=bar" as is the default, whilst the positional parameter "-foobar" is considered an unknown option (unless another *spec* describes it). This applies to both singly and doubly hyphenated long options.

An empty option-argument can be given to its long option in the same parameter using a trailing "=".

An optional argument to a long option must be given with the equals syntax, and an optional argument to a short option must follow it immediately in the same parameter; in both cases the following parameter is considered unrelated.

Whenever a long option and its argument are stored in the same array element, an "=" is used to separate them.

A mandatory option-argument given in a separate parameter from its option (as in `--foo bar'` , or any option-argument given to a short option in the same parameter, is always treated the same regardless of whether this option is in effect.

Lastly, when this option is active, only `-"` is treated as an explicit option-parsing terminator in the parsed arguments; a single `-"` is considered a normal operand.

Note: Unlike most `tt(getopt_long+(3+))` implementations, `zparseopts` does not support abbreviating long options.)

- **-K** — With this option, the arrays specified with the **-a** option and with the `=array` forms are kept unchanged when none of the `_spec_s` for them is used. Otherwise the entire array is replaced when any of the `_spec_s` is used. Individual elements of associative arrays specified with the **-A** option are preserved by **-K**. This allows assignment of default values to arrays before calling `zparseopts`.

- **-M** — This changes the assignment rules to implement a map among equivalent option names. If any `spec` uses the `=array` form, the string `array` is interpreted as the name of another `spec`, which is used to choose where to store the values. If no other `spec` is found, the values are stored as usual. This changes only the way the values are stored, not the way `$*` is parsed, so results may be unpredictable if the `_name_+` specifier is used inconsistently.

- **-v argv** — With this option, the elements of the named array `argv` are used as the positional parameters to parse, rather than those given by `$*`. The array may be modified according to the **-D** option.

For example,

```
set -- -a -bx -c y -cz baz -cend zparseopts a=foo b:=bar c+::=bar
```

will have the effect of

```
example(foo=(-a) bar=(-b x -c y -c z))
```

The arguments from `"baz"` on will not be used.

As an example for the **-E** option, consider:

```
set -- -a x -b y -c z arg1 arg2 zparseopts -E -D b:=bar
```

will have the effect of

```
example(bar=(-b y) set -- -a x -c z arg1 arg2)
```

I.e., the option **-b** and its arguments are taken from the positional parameters and put into the array `bar`.

The **-M** option can be used like this:

```
set -- -a -bx -c y -cz baz -cend zparseopts -A bar -M a=foo b+: c:=b
```

to have the effect of

```
example(foo=(-a) bar=(-a " -b xyz))
```

`zprof` 𐀀

Without the **-c** option, `zprof` lists profiling results to standard output. The format is comparable to that of commands like `gprof`.

At the top there is a summary listing all functions that were called at least once. This summary is sorted in decreasing order of the amount of time spent in each. The lines contain the number of the function in order, which is used in other parts of the list in suffixes of the form `[_num_]`, then the number of calls made to the function. The next three columns list the time in milliseconds spent in the function and its descendants, the average time in milliseconds spent in the function and its descendants per call and the percentage of time spent in all shell functions used in this function and its descendants. The following three columns give the same information, but counting only the time spent in the function itself. The final column shows the name of the function.

After the summary, detailed information about every function that was invoked is listed, sorted in decreasing order of the amount of time spent in each function and its descendants. Each of these entries consists of descriptions for the functions that called the function described, the function itself, and the functions that were called from it. The description for the function itself has the same format as in the summary (and shows the same information). The other lines don't show the number of the function at the beginning and have their function named indented to make it easier to distinguish the line showing the function described in the section from the surrounding lines.

The information shown in this case is almost the same as in the summary, but only refers to the call hierarchy being displayed. For example, for a calling function the column showing the total running time lists the time spent in the described function and its descendants only for the times when it was called from that particular calling function. Likewise, for a called function, this column lists the total time spent in the called function and its descendants only for the times when it was called from the function described.

Also in this case, the column showing the number of calls to a function also shows a slash and then the total number of invocations made to the called function.

As long as the `zsh/zprof` module is loaded, profiling will be done and multiple invocations of the `zprof` builtin command will show the times and numbers of calls since the module was loaded. With the `-c` option, the `zprof` builtin command will reset its internal counters and will not show the listing.

`zpty` ¶

The arguments following *name* are concatenated with spaces between, then executed as a command, as if passed to the `eval` builtin. The command runs under a newly assigned pseudo-terminal; this is useful for running commands non-interactively which expect an interactive environment. The *name* is not part of the command, but is used to refer to this command in later calls to `zpty`.

With the `-e` option, the pseudo-terminal is set up so that input characters are echoed.

With the `-b` option, input to and output from the pseudo-terminal are made non-blocking.

The shell parameter `REPLY` is set to the file descriptor assigned to the master side of the pseudo-terminal. This allows the terminal to be monitored with ZLE descriptor handlers (see the description of option `-F` for `zle` builtin in *Zle Builtins* (zshzle)) or manipulated with `sysread` and `syswrite` builtins (see *The zsh/system Module* (zshmodules)). *Warning:* Use of `sysread` and `syswrite` is *not* recommended; use `zpty -r` and `zpty -w` unless you know exactly what you are doing.

`zregexparse` ¶

This implements some internals of the `_regex_arguments` function.

`zselect` ¶

The `zselect` builtin is a front-end to the "select" system call, which blocks until a file descriptor is ready for reading or writing, or has an error condition, with an optional timeout. If this is not available on your system, the command prints an error message and returns status 2 (normal errors return status 1). For more information, see your system's documentation for `manref(select)(2)`. Note there is no connection with the shell builtin of the same name.

Arguments and options may be intermingled in any order. Non-option arguments are file descriptors, which must be decimal integers. By default, file descriptors are to be tested for reading, i.e. `zselect` will return when data is available to be read from the file descriptor, or more precisely, when a read operation from the file descriptor will not block. After a `-r`, `-w` and `-e`, the given file descriptors are to be tested for reading, writing, or error conditions. These options and an arbitrary list of file descriptors may be given in any order.

(The presence of an "error condition" is not well defined in the documentation for many implementations of the select system call. According to recent versions of the POSIX specification, it is really an *exception* condition, of which the only standard example is out-of-band data received on a socket. So zsh users are unlikely to find the `-e` option useful.)

The option `-t` "timeout" specifies a timeout in hundredths of a second. This may be zero, in which case the file descriptors will simply be polled and `zselect` will return immediately. It is possible to call `zselect` with no file descriptors and a non-zero timeout for use as a finer-grained replacement for "sleep"; note, however, the return status is always 1 for a timeout.

The option `-a` "array" indicates that *array* should be set to indicate the file descriptor+(s+) which are ready. If the option is not given, the array `reply` will be used for this purpose. The array will contain a string similar to the arguments for `zselect`. For example,

```
zselect -t 0 -r 0 -w 1
```

might return immediately with status 0 and `$reply` containing `-r 0 -w 1` to show that both file descriptors are ready for the requested operations.

The option `-A` "assoc" indicates that the associative array *assoc* should be set to indicate the file descriptor+(s+) which are ready. This option overrides the option `-a`, nor will `reply` be modified. The keys of `assoc` are the file descriptors, and the corresponding values are any of the characters "rwe" to indicate the condition.

The command returns status 0 if some file descriptors are ready for reading. If the operation timed out, or a timeout of 0 was given and no file descriptors were ready, or there was an error, it returns status 1 and the array will not be set (nor modified in any way). If there was an error in the select operation the appropriate error message is printed.

zsetattr ¶

Set the extended attribute *attribute* on the specified *filename* to *value*.

zsocket ¶

zsocket is implemented as a builtin to allow full use of shell command line editing, file I/O, and job control mechanisms.

zstat ¶

The command acts as a front end to the **stat** system call (see `manref(stat)(2)`). The same command is provided with two names; as the name **stat** is often used by an external command it is recommended that only the **zstat** form of the command is used. This can be arranged by loading the module with the command `zmodload -F zsh/stat b:zstat`.

If the **stat** call fails, the appropriate system error message printed and status 1 is returned. The fields of **struct stat** give information about the files provided as arguments to the command. In addition to those available from the **stat** call, an extra element "link" is provided. These elements are:

- **device** — The number of the device on which the file resides.
- **inode** — The unique number of the file on this device ("inode" number).
- **mode** — The mode of the file; that is, the file's type and access permissions. With the **-s** option, this will be returned as a string corresponding to the first column in the display of the **ls -l** command.
- **nlink** — The number of hard links to the file.
- **uid** — The user ID of the owner of the file. With the **-s** option, this is displayed as a user name.
- **gid** — The group ID of the file. With the **-s** option, this is displayed as a group name.
- **rdev** — The raw device number. This is only useful for special devices.
- **size** — The size of the file in bytes.
- **atime**
- **mtime**
- **ctime** — The last access, modification and inode change times of the file, respectively, as the number of seconds since midnight GMT on 1st January, 1970. With the **-s** option, these are printed as strings for the local time zone; the format can be altered with the **-F** option, and with the **-g** option the times are in GMT.
- **blksize** — The number of bytes in one allocation block on the device on which the file resides.
- **block** — The number of disk blocks used by the file.
- **link** — If the file is a link and the **-L** option is in effect, this contains the name of the file linked to, otherwise it is empty. Note that if this element is selected ("`zstat +link`") then the **-L** option is automatically used.

A particular element may be selected by including its name preceded by a "+" in the option list; only one element is allowed. The element may be shortened to any unique set of leading characters. Otherwise, all elements will be shown for all files.

Options:

- **-A array** — Instead of displaying the results on standard output, assign them to an *array*, one **struct stat** element per array element for each file in order. In this case neither the name of the element nor the name of the files appears in *array* unless the **-t** or **-n** options were given, respectively. If **-t** is given, the element name appears as a prefix to the appropriate array element; if **-n** is given, the file name appears as a separate array element preceding all the others. Other formatting options are respected.
- **-H hash** — Similar to **-A**, but instead assign the values to *hash*. The keys are the elements listed above. If the **-n** option is provided then the name of the file is included in the hash with key **name**.
- **-f fd** — Use the file on file descriptor *fd* instead of named files; no list of file names is allowed in this case.

- **-F *fmt*** — Supplies a **strftime** (see `manref(strftime)(3)`) string for the formatting of the time elements. The format string supports all of the zsh extensions (see the description of `%D{string}` prompt escape in *Simple Prompt Escapes* (zshmisc)). In particular, **-F %s.%N** can be used to show timestamps with nanosecond precision if supported by the system. The **-s** option is implied.
- **-g** — Show the time elements in the GMT time zone. The **-s** option is implied.
- **-l** — List the names of the type elements (to standard output or an array as appropriate) and return immediately; arguments, and options other than **-A**, are ignored.
- **-L** — Perform an **lstat** (see `manref(lstat)(2)`) rather than a **stat** system call. In this case, if the file is a link, information about the link itself rather than the target file is returned. This option is required to make the **link** element useful. It's important to note that this is the exact opposite from `manref(ls)(1)`, etc.
- **-n** — Always show the names of files. Usually these are only shown when output is to standard output and there is more than one file in the list.
- **-N** — Never show the names of files.
- **-o** — If a raw file mode is printed, show it in octal, which is more useful for human consumption than the default of decimal. A leading zero will be printed in this case. Note that this does not affect whether a raw or formatted file mode is shown, which is controlled by the **-r** and **-s** options, nor whether a mode is shown at all.
- **-r** — Print raw data (the default format) alongside string data (the **-s** format); the string data appears in parentheses after the raw data.
- **-s** — Print **mode**, **uid**, **gid** and the three time elements as strings instead of numbers. In each case the format is like that of `ls -l`.
- **-t** — Always show the type names for the elements of **struct stat**. Usually these are only shown when output is to standard output and no individual element has been selected.
- **-T** — Never show the type names of the **struct stat** elements.

zstyle ¶

This makes defining styles a bit simpler by using a single "+" as a special token that allows you to append a context name to the previously used context name. Like this:

```
zstyle+ 'foo:bar' style1 value1 \ + 'baz' style2 value2 \ + 'frob' style3 value3
```

This defines *style1* with *value1* for the context **foo:bar** as usual, but it also defines *style2* with *value2* for the context **foo:bar:baz** and *style3* with *value3* for **foo:bar:frob**. Any *subcontext* may be the empty string to re-use the first context unchanged.

zsystem ¶

The builtin **zsystem**'s subcommand **flock** performs advisory file locking (via the `manref(fcntl)(2)` system call) over the entire contents of the given file. This form of locking requires the processes accessing the file to cooperate; its most obvious use is between two instances of the shell itself.

In the first form the named *file*, which must already exist, is locked by opening a file descriptor to the file and applying a lock to the file descriptor. The lock terminates when the shell process that created the lock exits; it is therefore often convenient to create file locks within subshells, since the lock is automatically released when the subshell exits. Note that use of the **print** builtin with the **-u** option will, as a side effect, release the lock, as will redirection to the file in the shell holding the lock. To work around this use a subshell, e.g. `"tt((print message) » )file"`. Status 0 is returned if the lock succeeds, else status 1.

In the second form the file descriptor given by the arithmetic expression `_fd_expr_` is closed, releasing a lock. The file descriptor can be queried by using the `-f var` form during the lock; on a successful lock, the shell variable *var* is set to the file descriptor used for locking. The lock will be released if the file descriptor is closed by any other means, for example using `exec {_var_}>&-'`; however, the form described here performs a safety check that the file descriptor is in use for file locking.

By default the shell waits indefinitely for the lock to succeed. The option **-t timeout** specifies a timeout for the lock in seconds; fractional seconds are allowed. During this period, the shell will attempt to lock the file every *interval* seconds if the **-i interval** option is given, otherwise once a second. (This *interval* is shortened before the last attempt if needed, so that the shell waits only until the *timeout* and not longer.) If the attempt times out, status 2 is returned.

(Note: *timeout* is limited to $2^{30}-1$ seconds (about 34 years), and *interval* to $0.999 * \text{LONG_MAX}$ microseconds (only about 35 minutes on 32-bit systems).)

If the option **-e** is given, the file descriptor for the lock is preserved when the shell uses **exec** to start a new process; otherwise it is closed at that point and the lock released.

If the option **-r** is given, the lock is only for reading, otherwise it is for reading and writing. The file descriptor is opened accordingly.

ztcp ¶

ztcp is implemented as a builtin to allow full use of shell command line editing, file I/O, and job control mechanisms.

If **ztcp** is run with no options, it will output the contents of its session table.

If it is run with only the option **-L**, it will output the contents of the session table in a format suitable for automatic parsing. The option is ignored if given with a command to open or close a session. The output consists of a set of lines, one per session, each containing the following elements separated by spaces:

- **File descriptor** — The file descriptor in use for the connection. For normal inbound (**I**) and outbound (**O**) connections this may be read and written by the usual shell mechanisms. However, it should only be close with "ztcp -c".
- **Connection type** — A letter indicating how the session was created:
 - **Z** — A session created with the **zftp** command.
 - **L** — A connection opened for listening with "ztcp -l".
 - **I** — An inbound connection accepted with "ztcp -a".
 - **O** — An outbound connection created with ztcp "host ...".
- **The local host** — This is usually set to an all-zero IP address as the address of the localhost is irrelevant.
- **The local port** — This is likely to be zero unless the connection is for listening.
- **The remote host** — This is the fully qualified domain name of the peer, if available, else an IP address. It is an all-zero IP address for a session opened for listening.
- **The remote port** — This is zero for a connection opened for listening.

ztie ¶

Open the GDBM database identified by *filename* and, if successful, create the associative array *arrayname* linked to the file. To create a local tied array, the parameter must first be declared, so commands similar to the following would be executed inside a function scope:

```
local -A sampled_b ztie -d db/gdbm -f sample.gdbm sampled_b
```

The **-r** option opens the database file for reading only, creating a parameter with the readonly attribute. Without this option, using "ztie" on a file for which the user does not have write permission is an error. If writable, the database is opened synchronously so fields changed in *arrayname* are immediately written to *filename*.

Changes to the file modes *filename* after it has been opened do not alter the state of *arrayname*, but `typeset -r "arrayname"` works as expected.

zuntie ¶

Close the GDBM database associated with each *arrayname* and then unset the parameter. The **-u** option forces an unset of parameters made readonly with "ztie -r".

This happens automatically if the parameter is explicitly unset or its local scope (function) ends. Note that a readonly parameter may not be explicitly unset, so the only way to unset a global parameter created with "ztie -r" is to use "zuntie -u".

Keyword Index

31 entries · zsh reserved words from `Src/hashtable.c reswds[]`. Mirrors the `man zshmisc` "Reserved Words" section. Declarers (`declare`, `export`, `float`, `integer`, `local`, `readonly`, `typeset`) are reserved AND also appear in the Builtin Index — they're both.

! 𐀀

Pipeline negation (also a reserved word). `! cmd` inverts `cmd`'s exit status — zero becomes 1, non-zero becomes 0. Distinct from `!` history expansion (lexer-stage).

[[𐀀

Open zsh conditional expression. `[ [ EXPR ]]` evaluates a boolean. No word splitting / glob inside; supports `&&`, `||`, `!`, `==`, `!=`, `=~`, `-e`, `-f`, `-d`, `-z`, `-n`, etc. Prefer this over `[ ]` in zsh.

{ 𐀀

Command-group open brace. `{ cmd1; cmd2; }` runs the commands in the current shell (no subshell), grouping them as one syntactic unit. Reserved word — must be followed by whitespace or a newline.

} 𐀀

Command-group close brace. Pairs with `{ ... }`. Reserved word — preceded by `;` or newline.

case 𐀀

Execute the *list* associated with the first *pattern* that matches *word*, if any. The form of the patterns is the same as that used for filename generation. See *Filename Generation* (zshexpn).

Note further that, unless the `SH_GLOB` option is set, the whole pattern with alternatives is treated by the shell as equivalent to a group of patterns within parentheses, although white space may appear about the parentheses and the vertical bar and will be stripped from the pattern at those points. White space may appear elsewhere in the pattern; this is not stripped. If the `SH_GLOB` option is set, so that an opening parenthesis can be unambiguously treated as part of the case syntax, the expression is parsed into separate words and these are treated as strict alternatives (as in other shells).

If the *list* that is executed is terminated with `;&` rather than `;;`, the following list is also executed. The rule for the terminator of the following list `;;`, `;&` or `;` is applied unless the `esac` is reached.

If the *list* that is executed is terminated with `;` the shell continues to scan the `_pattern_s` looking for the next match, executing the corresponding *list*, and applying the rule for the corresponding terminator `;;`, `;&` or `;`. Note that *word* is not re-expanded; all applicable `_pattern_s` are tested with the same *word*.

coproc 𐀀

Run a command as a coprocess (background, attached I/O).

declare 𐀀

Alias for `typeset`. Set variable attributes. `-a` array, `-A` assoc, `-i` integer, `-r` readonly.

do 𐀀

Body-introducer for `for/while/until/select/repeat`. `for v in ...; do body; done`

done ¶

Closes a `for` / `foreach` / `while` / `until` / `select` / `repeat` loop body. `for v in a b c; do echo $v; done`. Required terminator.

elif ¶

Alternative test in an `if` chain. `if a; then ...; elif b; then ...; fi`

else ¶

Alternative branch for `if`. `if cmd; then a; else b; fi`

end ¶

Closes the alternate-form compound statement (`foreach NAME (WORDS) ... end, if COND ... end, while COND ... end`). Csh-style syntactic mirror of `fi` / `done` / `esac` for users coming from csh / tcsh.

esac ¶

Closes a `case` statement. `case word in pat) ...; esac`

export ¶

The specified `_name_s` are marked for automatic export to the environment of subsequently executed commands. Equivalent to `typeset -gx`. If a parameter specified does not already exist, it is created in the global scope.

fi ¶

Closes an `if` block. `if cmd; then body; fi`. Required terminator — without it the parser keeps reading until EOF.

float ¶

Equivalent to `typeset -E`, except that options irrelevant to floating point numbers are not permitted.

for ¶

Expand the list of `_word_s`, and set the parameter `name` to each of them in turn, executing `list` each time. If the in" `word`" is omitted, use the positional parameters instead of the `_word_s`. If any `name` has been declared as a named reference, the corresponding `word` is treated as the name of a parameter and `name` is made a reference to that. Note that for the positional parameters, this treats the value of each positional as parameter name rather than creating a reference to the position.

The *term* consists of one or more newline or `;` which terminate the `_word_s`, and are optional when the in" `word`" is omitted.

More than one parameter `name` can appear before the list of `_word_s`. If `N _name_s` are given, then on each execution of the loop the next `N _word_s` are assigned to the corresponding parameters. If there are more `_name_s` than remaining `_word_s`, the remaining parameters are each set to the empty string. Execution of the loop ends when there is no remaining `word` to assign to the first `name`. It is only possible for `in` to appear as the first `name` in the list, else it will be treated as marking the end of the list.

foreach ¶

Another form of `for`.

function 𐄂

where *term* is one or more newline or `;`. Define a function which is referenced by any one of *word*. Normally, only one *word* is provided; multiple `_word_s` are usually only useful for setting traps. The body of the function is the *list* between the `{` and `}`. See *Functions* (blow).

The options of `function` have the following meanings:

- `-T` — Enable tracing for this function, as though with `functions -T`. See the documentation of the `-f` option to the `typeset` builtin, in *Shell Builtin Commands* (zshbuiltins).

If the option `SH_GLOB` is set for compatibility with other shells, then whitespace may appear between the left and right parentheses when there is a single *word*; otherwise, the parentheses will be treated as forming a globbing pattern in that case.

In any of the forms above, a redirection may appear outside the function body, for example

```
example(func+() { ... } 2>&1)
```

The redirection is stored with the function and applied whenever the function is executed. Any variables in the redirection are expanded at the point the function is executed, but outside the function scope.

if 𐄂

The `if` *list* is executed, and if it returns a zero exit status, the `then` *list* is executed. Otherwise, the `elif` *list* is executed and if its status is zero, the `then` *list* is executed. If each `elif` *list* returns nonzero status, the `else` *list* is executed.

integer 𐄂

Equivalent to `typeset -i`, except that options irrelevant to integers are not permitted.

local 𐄂

Same as `typeset`, except that the options `-g`, and `-f` are not permitted. In this case the `-x` option does not force the use of `-g`, i.e. exported variables will be local to functions.

nocorrect 𐄂

Spelling correction is not done on any of the words. This must appear before any other precommand modifier, as it is interpreted immediately, before any parsing is done. It has no effect in non-interactive shells.

readonly 𐄂

Same as `typeset -r`. With the `POSIX_BUILTINS` option set, same as `typeset -gr`.

repeat 𐄂

word is expanded and treated as an arithmetic expression, which must evaluate to a number *n*. *list* is then executed *n* times.

The `repeat` syntax is disabled by default when the shell starts in a mode emulating another shell. It can be enabled with the command "enable -r repeat"

select 𐄂

where *term* is one or more newline or `;` to terminate the `_word_s`. `vindex(REPLY, use of)` Print the set of `_word_s`, each preceded by a number. If the `in word` is omitted, use the positional parameters. The `PROMPT3` prompt is printed and a line is read from the line editor if the shell is interactive and that is active, or else standard input. If this line consists of the number of one of the listed `_word_s`, then the parameter *name* is set to the *word* corresponding to this number. If this line is empty, the selection list is printed again. Otherwise, the value of the parameter *name* is set to null. The contents of the line read from standard input is saved in the parameter `REPLY`. *list* is executed for each selection until a break or end-of-file is encountered.

then 𐄂

Body separator for `if/elif`. `if cmd; then body; fi`

time ¶

The *pipeline* is executed, and timing statistics are reported on the standard error in the form specified by the `TIMEFMT` parameter. If *pipeline* is omitted, print statistics about the shell process and its children.

typeset ¶

Set or display attributes and values for shell parameters.

Except as noted below for control flags that change the behavior, a parameter is created for each *name* that does not already refer to one. When inside a function, a new parameter is created for every *name* (even those that already exist), and is unset again when the function completes. See *Local Parameters* (zshparam). The same rules apply to special shell parameters, which retain their special attributes when made local.

For each *name=value* assignment, the parameter *name* is set to *value*. If the assignment is omitted and *name* does *not* refer to an existing parameter, a new parameter is initialized to empty string, zero, or empty array (as appropriate), *unless* the shell option `TYPESET_TO_UNSET` is set. When that option is set, the parameter attributes are recorded but the parameter remains unset.

If the shell option `TYPESET_SILENT` is not set, for each remaining *name* that refers to a parameter that is already set, the name and value of the parameter are printed in the form of an assignment. Nothing is printed for newly-created parameters, or when any attribute flags listed below are given along with the *name*. Using "+" instead of minus to introduce an attribute turns it off.

If no *name* is present, the names and values of all parameters are printed. In this case the attribute flags restrict the display to only those parameters that have the specified attributes, and using "+" rather than "-" to introduce the flag suppresses printing of the values of parameters when there is no parameter name.

All forms of the command handle scalar assignment. Array assignment is possible if any of the reserved words `declare`, `export`, `float`, `integer`, `local`, `readonly` or `typeset` is matched when the line is parsed (N.B. not when it is executed). In this case the arguments are parsed as assignments, except that the "+=" syntax and the `GLOB_ASSIGN` option are not supported, and scalar values after = are *not* split further into words, even if expanded (regardless of the setting of the `KSH_TYPESET` option; this option is obsolete).

Examples of the differences between command and reserved word parsing:

```
example(# Reserved word parsing typeset svar=$(echo one word) avar=(several words))
```

The above creates a scalar parameter `svar` and an array parameter `avar` as if the assignments had been

```
example(svar="one word" avar=(several words))
```

On the other hand:

```
example(# Normal builtin interface builtin typeset svar=$(echo two words))
```

The `builtin` keyword causes the above to use the standard builtin interface to `typeset` in which argument parsing is performed in the same way as for other commands. This example creates a scalar `svar` containing the value `two` and another scalar parameter `words` with no value. An array value in this case would either cause an error or be treated as an obscure set of glob qualifiers.

Arbitrary arguments are allowed if they take the form of assignments after command line expansion; however, these only perform scalar assignment:

```
var='svar=val' typeset $var
```

The above sets the scalar parameter `svar` to the value `val`. Parentheses around the value within `var` would not cause array assignment as they will be treated as ordinary characters when `$var` is substituted. Any non-trivial expansion in the name part of the assignment causes the argument to be treated in this fashion:

```
typeset {var1,var2,var3}=name
```

The above syntax is valid, and has the expected effect of setting the three parameters to the same value, but the command line is parsed as a set of three normal command line arguments to `typeset` after expansion. Hence it is not possible to assign to multiple arrays by this means.

Note that each interface to any of the commands may be disabled separately. For example, "disable -r typeset" disables the reserved word interface to `typeset`, exposing the builtin interface, while "disable typeset" disables the builtin. Note that disabling the reserved word interface for `typeset` may cause problems with the output of "typeset -p", which assumes the reserved word interface is available in order to restore array and associative array values.

Unlike parameter assignment statements, `typeset`'s exit status on an assignment that involves a command substitution does not reflect the exit status of the command substitution. Therefore, to test for an error in a command substitution, separate the declaration of the parameter from its initialization:

```
example(# WRONG typeset var1=$(exit 1) || echo "Trouble with var1"
# RIGHT typeset var1 && var1=$(exit 1) || echo "Trouble with var1" )
```

To initialize a parameter *param* to a command output and mark it readonly, use `typeset -r param` or `readonly param` after the parameter assignment statement.

`cindex(named reference) cindex(reference, named)` The flag `-n` creates a *named reference* to another parameter. The second parameter need not exist at the time the reference is created. Only the `-H`, `-g`, and `-r` flags may be used in conjunction with `-n`, having their usual meanings. The `-u` flag is special and may be applied to alter the scope of the reference. The *name* so created may not be an array element nor use a subscript, but the *value* assigned may be any valid parameter name syntax, even a subscripted array element (including an associative array element) or an array slice, which is evaluated when the named reference is expanded. It is an error for a named reference to refer to itself, even indirectly through a chain of references. When `-u` is applied to a named reference, the parameter identified by *value* is always found in the calling function scope rather than the current local scope. In this case, if there is no such parameter in the calling scope, assignments to the named reference may fail, setting `$?` to 1. See *Parameter Expansion* (zshexpn) and *Named References* (zshparam) for details of the behavior of named references.

Local function scoping rules for "typeset" do apply with `-n`, so a declaration within a function persists only until the end of the function unless `-g -n` is specified, and any local parameter (of any type) with the same *name* supplants a named reference from a surrounding scope.

A scalar parameter, including an existing named reference, may be converted to a new named reference by `typeset -n "name"`, so the `-p` option must be included to display the value of a specific named reference *name*.

If no attribute flags are given, and either no *name* arguments are present or the flag `+m` is used, then each parameter name printed is preceded by a list of the attributes of that parameter (`array`, `association`, `exported`, `float`, `integer`, `readonly`, or `undefined` for autoloaded parameters not yet loaded). If `+m` is used with attribute flags, and all those flags are introduced with `+`, the matching parameter names are printed but their values are not.

The following control flags change the behavior of `typeset`:

- `+` — If `+` appears by itself in a separate word as the last option, then the names of all parameters (functions with `-f`) are printed, but the values (function bodies) are not. If *name* arguments appear, both those names and their values are printed in the form of assignments. It is an error for any other options to follow `+`, but the effect of `+` is as if all attribute flags which precede it were given with a `+` prefix. For example, `"typeset -U +"` is equivalent to `"typeset +U"` and displays the names of all arrays having the uniqueness attribute, whereas `"typeset -f -U +"` displays the names of all autoloadable functions. If `+` is the only option, then type information (array, readonly, etc.) is also printed for each parameter, in the same manner as `"typeset +m"`.

- `-g` — The `-g` (global) means that any resulting parameter will not be restricted to local scope. Note that this does not necessarily mean that the parameter will be global, as the flag will apply to any existing parameter (even if unset) from an enclosing function. This flag does not affect the parameter after creation, hence it has no effect when listing existing parameters, nor does the flag `+g` have any effect except in combination with `-m` (see below).

- `-m` — If the `-m` flag is given the *name* arguments are taken as patterns (use quoting to prevent these from being interpreted as file patterns). With no attribute flags, all parameters (or functions with the `-f` flag) with matching names are printed (the shell option `TYPESET_SILENT` is not used in this case).

If the `+g` flag is combined with `-m`, a new local parameter is created for every matching parameter that is not already local. Otherwise `-m` applies all other flags or assignments to the existing parameters, except that the `-n` option cannot create named references in this way.

Except when assignments are made with *name=value*, using `+m` forces the matching parameters and their attributes to be printed, even inside a function. Note that `-m` is ignored if no patterns are given, so `"typeset -m"` displays attributes but `"typeset -a +m"` does not. Ordinary scalar string parameters have no attributes, so for those `+m` prints only the names.

- `-p [ n ]` — If the `-p` option is given, parameters and values are printed in the form of a `typeset` command with an assignment, regardless of other flags and options. Note that the `-H` flag on parameters is respected; no value will be shown for these parameters.

`-p` may be followed by an optional integer argument. Currently only the value `1` is supported. In this case arrays and associative arrays are printed with newlines between indented elements for readability.

The names and values of readonly special parameters (most of the parameters marked "<S>" in *Parameters Set by the Shell* (zshparam), except those documented as settable) are not printed with "-p" because to execute those typeset commands would cause errors. However, these parameters are printed when they have been made local to the scope where "typeset -p" is run.

-T [*scalar*[=*value*] *array*[=()_value_ . . .] [*sep*]] — This flag has a different meaning when used with -f; see below. Otherwise the -T option requires zero, two, or three arguments to be present. With no arguments, the list of parameters created in this fashion is shown. With two or three arguments, the first two are the name of a scalar and of an array parameter (in that order) that will be tied together in the manner of \$PATH and \$path. The optional third argument is a single-character separator which will be used to join the elements of the array to form the scalar; if absent, a colon is used, as with \$PATH. Only the first character of the separator is significant; any remaining characters are ignored. Multibyte characters are not yet supported.

Only one of the scalar and array parameters may be assigned an initial value (the restrictions on assignment forms described above also apply).

Both the scalar and the array may be manipulated as normal. If one is unset, the other will automatically be unset too. There is no way of untying the variables without unsetting them, nor of converting the type of one of them with another typeset command; +T does not work, assigning an array to scalar is an error, and assigning a scalar to array sets it to be a single-element array.

Note that both "typeset -xT ..." and "export -T ..." work, but only the scalar will be marked for export. Setting the value using the scalar version causes a split on all separators (which cannot be quoted). It is possible to apply -T to two previously tied variables but with a different separator character, in which case the variables remain joined as before but the separator is changed.

When an existing scalar is tied to a new array, the value of the scalar is preserved but no attribute other than export will be preserved.

Attribute flags that transform the final value (-L, -R, -Z, -l, -u) are only applied to the expanded value at the point of a parameter expansion expression using "\$". They are not applied when a parameter is retrieved internally by the shell for any purpose.

The following attribute flags may be specified:

-A — The names refer to associative array parameters; see *Array Parameters* (zshparam).

-L [*n*] — Left justify and remove leading blanks from the value when the parameter is expanded. If *n* is nonzero, it defines the width of the field. If *n* is zero, the width is determined by the width of the value of the first assignment. In the case of numeric parameters, the length of the complete value assigned to the parameter is used to determine the width, not the value that would be output.

The width is the count of characters, which may be multibyte characters if the MULTIBYTE option is in effect. Note that the screen width of the character is not taken into account; if this is required, use padding with parameter expansion flags tt(\$+(ml)......) as described in subref(Parameter Expansion Flags)(zshexpn).

When the parameter is expanded, it is filled on the right with blanks or truncated if necessary to fit the field. Note truncation can lead to unexpected results with numeric parameters. Leading zeros are removed if the -Z flag is also set.

-R [*n*] — Similar to -L, except that right justification is used; when the parameter is expanded, the field is left filled with blanks or truncated from the end. May not be combined with the -Z flag.

-U — For arrays (but not for associative arrays), keep only the first occurrence of each duplicated value. This may also be set for tied parameters (see -T) or colon-separated special parameters like PATH or IGNORE, etc. Note the flag takes effect on assignment, and the type of the variable being assigned to is determinative; for variables with shared values it is therefore recommended to set the flag for all interfaces, e.g. "typeset -U PATH path".

This flag has a different meaning when used with -f; see below.

-Z [*n*] — Specially handled if set along with the -L flag. Otherwise, similar to -R, except that leading zeros are used for padding instead of blanks if the first non-blank character is a digit. Numeric parameters are specially handled: they are always eligible for padding with zeroes, and the zeroes are inserted at an appropriate place in the output.

-a — The names refer to array parameters. An array parameter may be created this way, but it may be assigned to in the typeset statement only if the reserved word form of typeset is enabled (as it is by default). When displaying, both normal and associative arrays are shown.

-f — The names refer to functions rather than parameters. No assignments can be made, and the only other valid flags are -t, -T, -k, -u, -U and -z. The flag -t turns on execution tracing for this function; the flag -T does the same, but turns off tracing for any named (not anonymous) function called from the present one, unless that function also has the -t or -T flag.

The `-u` and `-U` flags cause the function to be marked for autoloading; `-U` also causes alias expansion to be suppressed when the function is loaded. See the description of the "autoload" builtin for details.

Note that the builtin `functions` provides the same basic capabilities as `typeset -f` but gives access to a few extra options; `autoload` gives further additional options for the case `typeset -fu` and `typeset -fU`.

- `-h` — Hide: only useful for special parameters (those marked "<S>" in the table in *Parameters Set by the Shell* (zsh-param)), and for local parameters with the same name as a special parameter, though harmless for others. A special parameter with this attribute will not retain its special effect when made local. Thus after `typeset -h PATH`, a function containing `"typeset PATH"` will create an ordinary local parameter without the usual behaviour of `PATH`. Alternatively, the local parameter may itself be given this attribute; hence inside a function `"typeset -h PATH"` creates an ordinary local parameter and the special `PATH` parameter is not altered in any way. It is also possible to create a local parameter using `typeset +h "_special_"`, where the local copy of `_special_` will retain its special properties regardless of having the `-h` attribute. Global special parameters loaded from shell modules (for example, those in `zsh/mapfile` and `zsh/parameter`) are automatically given the `-h` attribute to avoid name clashes.

- `-H` — Hide value: specifies that `typeset` will not display the value of the parameter when listing parameters; the display for such parameters is always as if the `+` flag were given, but use of the parameter is in other respects normal. This effect does not apply when the parameter is specified by name or by pattern with the `-m` option. This is on by default for the parameters in the `zsh/parameter` and `zsh/mapfile` modules. Note, however, that unlike the `-h` flag this is also useful for non-special parameters.

- `-i [n]` — Use an internal integer representation. If `n` is nonzero it defines the output arithmetic base, otherwise it is determined by the first assignment. Bases from 2 to 36 inclusive are allowed.

- `-E [n]` — Use an internal double-precision floating point representation. On output the variable will be converted to scientific notation. If `n` is nonzero it defines the number of significant figures to display; the default is ten.

- `-F [n]` — Use an internal double-precision floating point representation. On output the variable will be converted to fixed-point decimal notation. If `n` is nonzero it defines the number of digits to display after the decimal point; the default is ten.

- `-l` — Convert the result to lower case whenever the parameter is expanded. The value is *not* converted when assigned.

- `-r` — The given `_name_s` are marked readonly. Note that if `name` is a special parameter, the readonly attribute can be turned on, but cannot then be turned off.

If the `POSIX_BUILTINS` option is set, the readonly attribute is more restrictive: unset variables can be marked readonly and cannot then be set; furthermore, the readonly attribute cannot be removed from any variable.

It is still possible to change other attributes of the variable though, some of which like `-U` or `-Z` would affect the value. More generally, the readonly attribute should not be relied on as a security mechanism.

Note that in zsh (like in `pdksh` but unlike most other shells) it is still possible to create a local variable of the same name as this is considered a different variable (though this variable, too, can be marked readonly). Special variables that have been made readonly retain their value and readonly attribute when made local.

- `-t` — Tags the named parameters. Tags only exist to flag the parameter for the user's own purposes --- the list of tagged parameters can be queried using `"typeset -t"`. Tags have no other use. Note that the `-t` flag has a different meaning when used with `-f`; see above.

- `-u` — Convert the result to upper case whenever the parameter is expanded. The value is *not* converted when assigned. This flag has different meanings when used with `-f` or `-n`; see above.

- `-x` — Mark for automatic export to the environment of subsequently executed commands. If the option `GLOBAL_EXPORT` is set, this implies the option `-g`, unless `+g` is also explicitly given; in other words the parameter is not made local to the enclosing function. This is for compatibility with previous versions of zsh.

until ¶

Execute the `do list` as long as `until list` returns a nonzero exit status.

while ¶

Execute the `do list` as long as the `while list` returns a zero exit status.

Option Index

197 entries · the canonical zsh option registry. Set / clear via `setopt NAME` / `unsetopt NAME`.

aliases ¶

This maps the names of the regular aliases currently enabled to their expansions.

aliasfuncdef ¶

By default, zsh does not allow the definition of functions using the "*name* tt()" syntax if *name* was expanded as an alias: this causes an error. This is usually the desired behaviour, as otherwise the combination of an alias and a function based on the same definition can easily cause problems.

When this option is set, aliases can be used for defining functions.

For example, consider the following definitions as they might occur in a startup file.

```
example(alias foo=bar foo+() { print This probably does not do what you expect. })
```

Here, `foo` is expanded as an alias to `bar` before the `tt()` is encountered, so the function defined would be named `bar`. By default this is instead an error in native mode. Note that quoting any part of the function name, or using the keyword `function`, avoids the problem, so is recommended when the function name can also be an alias.

allexport ¶

All parameters subsequently defined are automatically exported.

alwayslastprompt ¶

If unset, key functions that list completions try to return to the last prompt if given a numeric argument. If set these functions try to return to the last prompt if given *no* numeric argument.

alwaystoend ¶

If a completion is performed with the cursor within a word, and a full completion is inserted, the cursor is moved to the end of the word. That is, the cursor is moved to the end of the word if either a single match is inserted or menu completion is performed.

appendcreate ¶

This option only applies when `NO_CLOBBER` (`-C`) is in effect.

If this option is not set, the shell will report an error when a `append` redirection (`>>`) is used on a file that does not already exist (the traditional zsh behaviour of `NO_CLOBBER`). If the option is set, no error is reported (POSIX behaviour).

appendhistory ¶

If this is set, zsh sessions will append their history list to the history file, rather than replace it. Thus, multiple parallel zsh sessions will all have the new entries from their history lists added to the history file, in the order that they exit. The file will still be periodically re-written to trim it when the number of lines grows 20% beyond the value specified by `$SAVEHIST` (see also the `HIST_SAVE_BY_COPY` option).

autocd ¶

If a command is issued that can't be executed as a normal command, and the command is the name of a directory, perform the `cd` command to that directory. This option is only applicable if the option `SHIN_STOIN` is set, i.e. if commands are being read from standard input. The option is designed for interactive use; it is recommended that `cd` be used explicitly in scripts to avoid ambiguity.

autocontinue ¶

With this option set, stopped jobs that are removed from the job table with the `disown` builtin command are automatically sent a `CONT` signal to make them running.

autolist ¶

Automatically list choices on an ambiguous completion.

automenu ¶

Automatically use menu completion after the second consecutive request for completion, for example by pressing the tab key repeatedly. This option is overridden by `MENU_COMPLETE`.

autonamedirs ¶

Any parameter that is set to the absolute name of a directory immediately becomes a name for that directory, that will be used by the `%~` and related prompt sequences, and will be available when completion is performed on a word starting with `~`. (Otherwise, the parameter must be used in the form `~"param"` first.)

autoparamkeys ¶

If a parameter name was completed and a following character (normally a space) automatically inserted, and the next character typed is one of those that have to come directly after the name (like `"}"`, `".:"`, etc.), the automatically added character is deleted, so that the character typed comes immediately after the parameter name. Completion in a brace expansion is affected similarly: the added character is a `,`, which will be removed if `}` is typed next.

autoparamslash ¶

If a parameter is completed whose content is the name of a directory, then add a trailing slash instead of a space.

autopushd ¶

Make `cd` push the old directory onto the directory stack.

autoremoveslash ¶

When the last character resulting from a completion is a slash and the next character typed is a word delimiter, a slash, or a character that ends a command (such as a semicolon or an ampersand), remove the slash.

autoresume ¶

Treat single word simple commands without redirection as candidates for resumption of an existing job.

badpattern ¶

If a pattern for filename generation is badly formed, print an error message. (If this option is unset, the pattern will be left unchanged.)

banghist 𐀀

Perform textual history expansion, **cs**h-style, treating the character "!" specially.

bareglobqual 𐀀

In a glob pattern, treat a trailing set of parentheses as a qualifier list, if it contains no unquoted "|", "tt()" or (if special) "~" characters. See *Filename Generation* (zshexpn).

bashautolist 𐀀

On an ambiguous completion, automatically list choices when the completion function is called twice in succession. This takes precedence over **AUTO_LIST**. The setting of **LIST_AMBIGUOUS** is respected. If **AUTO_MENU** is set, the menu behaviour will then start with the third press. Note that this will not work with **MENU_COMPLETE**, since repeated completion calls immediately cycle through the list in that case.

bashrematch 𐀀

When set, matches performed with the **=~** operator will set the **BASH_REMATCH** array variable, instead of the default **MATCH** and **match** variables. The first element of the **BASH_REMATCH** array will contain the entire matched text and subsequent elements will contain extracted substrings. This option makes more sense when **KSH_ARRAYS** is also set, so that the entire matched portion is stored at index 0 and the first substring is at index 1. Without this option, the **MATCH** variable contains the entire matched text and the **match** array variable contains substrings.

beep 𐀀

Beep, unless the **BEEP** option is unset.

bgnice 𐀀

Run all background jobs at a lower priority. This option is set by default.

braceccl 𐀀

Expand expressions in braces which would not otherwise undergo brace expansion to a lexically ordered list of all the characters. See *Brace Expansion* (zshexpn).

braceexpand 𐀀

NO_IGNORE_BRACES (ksh and bash compatibility)

bsdecho 𐀀

Make the **echo** builtin compatible with the BSD **manref(echo)(1)** command. This disables backslashed escape sequences in echo strings unless the **-e** option is specified.

caseglob 𐀀

Make globbing (filename generation) sensitive to case. Note that other uses of patterns are always sensitive to case. If the option is unset, the presence of any character which is special to filename generation will cause case-insensitive matching. For example, **tt(cvs+(/+))** can match the directory **CVS** owing to the presence of the globbing flag (unless the option **BARE_GLOB_QUAL** is unset).

casematch 𐀀

Make regular expressions using the **zsh/regex** module (including matches with **=~**) sensitive to case.

casepaths ¶

If `CASE_PATHS` is not set (the default), `CASE_GLOB` affects the interpretation of every path component, whenever a special character appears in *any* component. When `CASE_PATHS` is set, file path components that do *not* contain special filename generation characters are always sensitive to case, thus restricting `NO_CASE_GLOB` to components that contain globbing characters.

Note that if the filesystem itself is not sensitive to case, then `CASE_PATHS` has no effect.

cbases ¶

Output hexadecimal numbers in the standard C format, for example "0xFF" instead of the usual "16#FF". If the option `OCTAL_ZEROES` is also set (it is not by default), octal numbers will be treated similarly and hence appear as "077" instead of "8#77". This option has no effect on the choice of the output base, nor on the output of bases other than hexadecimal and octal. Note that these formats will be understood on input irrespective of the setting of `C_BASES`.

cdablevars ¶

If the argument to a `cd` command (or an implied `cd` with the `AUTO_CD` option set) is not a directory, and does not begin with a slash, try to expand the expression as if it were preceded by a "~" (see *Filename Expansion* (zshexpn)).

cdsilent ¶

Never print the working directory after a `cd` (whether explicit or implied with the `AUTO_CD` option set). `cd` normally prints the working directory when the argument given to it was `-`, a stack entry, or the name of a directory found under `CDPATH`. Note that this is distinct from `pushd`'s stack-printing behaviour, which is controlled by `PUSHD_SILENT`. This option overrides the printing-related effects of `POSIX_CD`.

chasedots ¶

When changing to a directory containing a path segment `..` which would otherwise be treated as canceling the previous segment in the path (in other words, `"foo/.."` would be removed from the path, or if `.."` is the first part of the path, the last part of the current working directory would be removed), instead resolve the path to the physical directory. This option is overridden by `CHASE_LINKS`.

For example, suppose `/foo/bar` is a link to the directory `/alt/rod`. Without this option set, `"cd /foo/bar/.."` changes to `/foo`; with it set, it changes to `/alt`. The same applies if the current directory is `/foo/bar` and `"cd .."` is used. Note that all other symbolic links in the path will also be resolved.

chaselinks ¶

Resolve symbolic links to their true values when changing directory. This also has the effect of `CHASE_DOTS`, i.e. a `.."` path segment will be treated as referring to the physical parent, even if the preceding path segment is a symbolic link.

checkjobs ¶

Report the status of background and suspended jobs before exiting a shell with job control; a second attempt to exit the shell will succeed. `NO_CHECK_JOBS` is best used only in combination with `NO_HUP`, else such jobs will be killed automatically.

The check is omitted if the commands run from the previous command line included a `"jobs"` command, since it is assumed the user is aware that there are background or suspended jobs. A `"jobs"` command run from one of the hook functions defined in *Special Functions* (zshmisc) is not counted for this purpose.

checkrunningjobs ¶

Check for both running and suspended jobs when `CHECK_JOBS` is enabled. When this option is disabled, zsh checks only for suspended jobs, which matches the default behavior of `bash`.

This option has no effect unless `CHECK_JOBS` is set.

clobber 𐀀

Allows ">" redirection to truncate existing files. Otherwise ">!" or ">|" must be used to truncate a file.

If the option is not set, and the option `APPEND_CREATE` is also not set, ">!" or ">|" must be used to create a file. If either option is set, ">" may be used.

clobberempty 𐀀

This option is only used if the option `CLOBBER` is not set: note that it is set by default.

If this option is set, then regular files of zero length may be overwritten ("clobbered"). Note that it is possible another process has written to the file between this test and use of the file by the current process. This option should therefore not be used in cases where files to be clobbered may be written to asynchronously.

combiningchars 𐀀

Assume that the terminal displays combining characters correctly. Specifically, if a base alphanumeric character is followed by one or more zero-width punctuation characters, assume that the zero-width characters will be displayed as modifications to the base character within the same width. Not all terminals handle this. If this option is not set, zero-width characters are displayed separately with special mark-up.

If this option is set, the pattern test `[[:WORD:]]` matches a zero-width punctuation character on the assumption that it will be used as part of a word in combination with a word character. Otherwise the base shell does not handle combining characters specially.

completealiases 𐀀

Prevents aliases on the command line from being internally substituted before completion is attempted. The effect is to make the alias a distinct command for completion purposes.

completeinword 𐀀

If unset, the cursor is set to the end of the word if completion is started. Otherwise it stays there and completion is done from both ends.

continueonerror 𐀀

If a fatal error is encountered (see *Errors* (zshmisc)), and the code is running in a script, the shell will resume execution at the next statement in the script at the top level, in other words outside all functions or shell constructs such as loops and conditions. This mimics the behaviour of interactive shells, where the shell returns to the line editor to read a new command; it was the normal behaviour in versions of zsh before 5.0.1.

correct 𐀀

Try to correct the spelling of commands. Note that, when the `HASH_LIST_ALL` option is not set or when some directories in the path are not readable, this may falsely report spelling errors the first time some commands are used.

Refer to the shell variable `SPROMPT` for an explanation of the "[nyae]" (no/yes/abort/edit) prompt that is offered.

The shell variable `CORRECT_IGNORE` may be set to a pattern to match words that will never be offered as corrections.

correctall 𐀀

Try to correct the spelling of all arguments in a line, in order from left to right, treating each as a file name. Answering "a" or "e" at any prompt stops all corrections, otherwise every correction is prompted for.

The shell variable `CORRECT_IGNORE_FILE` may be set to a pattern to match file names that will never be offered as corrections.

cprecedences ¶

This alters the precedence of arithmetic operators to be more like C and other programming languages; *Arithmetic Evaluation* (zshmisc) has an explicit list.

cshjunkiehistory ¶

A history reference without an event specifier will always refer to the previous command. Without this option, such a history reference refers to the same event as the previous history reference on the current command line, defaulting to the previous command.

cshjunkieloops ¶

Allow loop bodies to take the form `_list_`; end' instead of `do list; done`'.

cshjunkiequotes ¶

Changes the rules for single- and double-quoted text to match that of `csh`. These require that embedded newlines be preceded by a backslash; unescaped newlines will cause an error message. In double-quoted strings, it is made impossible to escape "\$", ' or "" (and "\" itself no longer needs escaping). Command substitutions are only expanded once, and cannot be nested.

cshnullcmd ¶

Do not use the values of `NULLCMD` and `READNULLCMD` when running redirections with no command. This make such redirections fail (see *Redirection* (zshmisc)).

cshnullglob ¶

If a pattern for filename generation has no matches, delete the pattern from the argument list; do not report an error unless all the patterns in a command have no matches. Overrides `NOMATCH`.

debugbeforecmd ¶

Run the `DEBUG` trap before each command; otherwise it is run after each command. Setting this option mimics the behaviour of ksh 93; with the option unset the behaviour is that of ksh 88.

dotglob ¶

`GLOB_DOTS` (bash compatibility)

dvorak ¶

Use the Dvorak keyboard instead of the standard qwerty keyboard as a basis for examining spelling mistakes for the `CORRECT` and `CORRECT_ALL` options and the `spell-word` editor command.

emacs ¶

Move to the beginning of the previous word.

equals ¶

Perform `=` filename expansion. (See *Filename Expansion* (zshexpn).)

errexit ¶

If a command has a non-zero exit status, execute the `ZERR` trap, if set, and exit.

The option is ignored when executing the commands following `while`, `until`, `if`, or `elif`, a pipeline beginning with `!`, or any command other than the last in command list containing `&&` or `||`. Hence neither "if foo; then true; fi", nor `foo && true` trigger exit when foo returns with a non-zero exit status. Note that if foo is a function, the option is also ignored during its whole execution.

The option is also ignored when executing a complex command (`if`, `for`, `while`, `until`, `repeat`, `case`, `select`, `always`, or a list in braces) if its exit status comes from a command executed while the option is ignored. Hence, the `if` command in "if true; then false && true; fi" does not trigger exit.

Finally, the option is also ignored while running initialization scripts and inside `DEBUG` traps. In the latter case, the option is handled specially: it is unset on entry to the trap. If the option `DEBUG_BEFORE_CMD` is set, as it is by default, and the option `ERR_EXIT` is found to have been set on exit, then the command for which the `DEBUG` trap is being executed is skipped. The option is restored after the trap exits.

Exiting due to `ERR_EXIT` has certain interactions with asynchronous jobs noted in *Signals* (zshmisc).

Note this behaviour is not disabled in interactive shells --- a non-zero status on the command line causes the shell to exit.

`errreturn` ¶

If a command has a non-zero exit status, return immediately from the enclosing function. Except as explained below, an implicit `return` statement is executed following the same logic described for `ERR_EXIT`. This will trigger an exit at the outermost level of a non-interactive script. At the top level of an interactive shell, it will trigger a return to the command prompt; in other words, the sequence of commands typed by the user may be thought of as a function for this purpose.

Unlike for `ERR_EXIT`, when a function is called while the option is being ignored, the option is NOT ignored during the execution of the function. Hence, if `foo` in "foo && true" is a function, code inside it is considered separately: it may force a return from `foo` (assuming the option remains set within `foo`).

Like for `ERR_EXIT`, the option is ignored inside `DEBUG` traps but it's not unset on entry to the trap and setting or unsetting it inside the trap has no special effect.

If `ERR_RETURN` and `ERR_EXIT` are both set, it may happen that both exit and return should be triggered. In that case only exit is triggered.

`evallineno` ¶

If set, line numbers of expressions evaluated using the builtin `eval` are tracked separately of the enclosing environment. This applies both to the parameter `LINENO` and the line number output by the prompt escape `%i`. If the option is set, the prompt escape `%N` will output the string "tt((eval))" instead of the script or function name as an indication. (The two prompt escapes are typically used in the parameter `PS4` to be output when the option `XTRACE` is set.) If `EVAL_LINENO` is unset, the line number of the surrounding script or function is retained during the evaluation.

`exec` ¶

The following command together with any arguments is run in place of the current process, rather than as a sub-process. The shell does not fork and is replaced. The shell does not invoke `TRAPEXIT`, nor does it source `zlogout` files. The options are provided for compatibility with other shells.

The `-c` option clears the environment.

The `-l` option is equivalent to the `-` precommand modifier, to treat the replacement command as a login shell; the command is executed with a `-` prepended to its `argv[0]` string. This flag has no effect if used together with the `-a` option.

The `-a` option is used to specify explicitly the `argv[0]` string (the name of the command as seen by the process itself) to be used by the replacement command and is directly equivalent to setting a value for the `ARGV0` environment variable.

`extendedglob` ¶

Treat the `"#"`, `"~"` and `"^"` characters as part of patterns for filename generation, etc. (An initial unquoted `"~"` always produces named directory expansion.)

extendedhistory ¶

Save each command's beginning timestamp (in seconds since the epoch) and the duration (in seconds) to the history file. The format of this prefixed data is:

```
:_<beginning time>_:_<elapsed seconds>_;"<command>".
```

flowcontrol ¶

If this option is unset, output flow control via start/stop characters (usually assigned to ^S/^Q) is disabled in the shell's editor.

forcefloat ¶

Constants in arithmetic evaluation will be treated as floating point even without the use of a decimal point; the values of integer variables will be converted to floating point when used in arithmetic expressions. Integers in any base will be converted.

functionargzero ¶

When executing a shell function or sourcing a script, set `$0` temporarily to the name of the function/script. Note that toggling `FUNCTION_ARGZERO` from on to off (or off to on) does not change the current value of `$0`. Only the state upon entry to the function or script has an effect. Compare `POSIX_ARGZERO`.

glob ¶

This is used by the `_expand` completer. If it is set to "true" (the default), globbing will be attempted on the words resulting from a previous substitution (see the `substitute` style) or else the original string from the line.

globalexport ¶

If this option is set, passing the `-x` flag to the builtins `declare`, `float`, `integer`, `readonly` and `typeset` (but not `local`) will also set the `-g` flag; hence parameters exported to the environment will not be made local to the enclosing function, unless they were already or the flag `+g` is given explicitly. If the option is unset, exported parameters will be made local in just the same way as any other parameter.

This option is set by default for backward compatibility; it is not recommended that its behaviour be relied upon. Note that the builtin `export` always sets both the `-x` and `-g` flags, and hence its effect extends beyond the scope of the enclosing function; this is the most portable way to achieve this behaviour.

globalrcs ¶

If this option is unset, the startup files `tt(zprofile())`, `tt(zshrc())`, `tt(zlogin())` and `tt(zlogout())` will not be run. It can be disabled and re-enabled at any time, including inside local startup files (`.zshrc`, etc.).

globassign ¶

If this option is set, filename generation (globbing) is performed on the right hand side of scalar parameter assignments of the form `_name=_pattern_` (e.g. `"foo=*"`). If the result has more than one word the parameter will become an array with those words as arguments. This option is provided for backwards compatibility only: globbing is always performed on the right hand side of array assignments of the form `_name_tt(=()value)"` (e.g. `"tt(foo=(*))"`) and this form is recommended for clarity; with this option set, it is not possible to predict whether the result will be an array or a scalar.

globcomplete ¶

When the current word has a glob pattern, do not insert all the words resulting from the expansion but generate matches as for completion and cycle through them like `MENU_COMPLETE`. The matches are generated as if a "*" was added to the end of the word, or inserted at the cursor when `COMPLETE_IN_WORD` is set. This actually uses pattern matching, not globbing, so it works not only for files but for any completion, such as options, user names, etc.

Note that when the pattern matcher is used, matching control (for example, case-insensitive or anchored matching) cannot be used. This limitation only applies when the current word contains a pattern; simply turning on the `GLOB_COMPLETE` option does not have this effect.

`globdots` ¶

Do not require a leading `"."` in a filename to be matched explicitly.

`globstarshort` ¶

When this option is set and the default zsh-style globbing is in effect, the pattern `"/**"` **can be abbreviated to `"`** and the pattern `"/**/*"` **can be abbreviated to `*`**. Hence `"**.*"` finds a file ending in `.c` in any subdirectory, and `"***.*"` does the same while also following symbolic links. A `/` immediately after the `"**"` or `"***"` forces the pattern to be treated as the unabbreviated form.

`globsubst` ¶

Treat any characters resulting from parameter expansion as being eligible for filename expansion and filename generation, and any characters resulting from command substitution as being eligible for filename generation. Braces (and commas in between) do not become eligible for expansion.

`hashall` ¶

`HASH_CMDS` (bash compatibility)

`hashcmds` ¶

Note the location of each command the first time it is executed. Subsequent invocations of the same command will use the saved location, avoiding a path search. If this option is unset, no path hashing is done at all. However, when `CORRECT` is set, commands whose names do not appear in the functions or aliases hash tables are hashed in order to avoid reporting them as spelling errors.

`hashdirs` ¶

Whenever a command name is hashed, hash the directory containing it, as well as all directories that occur earlier in the path. Has no effect if neither `HASH_CMDS` nor `CORRECT` is set.

`hashexecutablesonly` ¶

When hashing commands because of `HASH_CMDS`, check that the file to be hashed is actually an executable. This option is unset by default as if the path contains a large number of commands, or consists of many remote files, the additional tests can take a long time. Trial and error is needed to show if this option is beneficial.

`hashlistall` ¶

Whenever a command completion or spelling correction is attempted, make sure the entire command path is hashed first. This makes the first completion slower but avoids false reports of spelling errors.

`histallowclobber` ¶

Add `"|"` to output redirections in the history. This allows history references to clobber files even when `CLOBBER` is unset.

`histappend` ¶

`APPEND_HISTORY` (bash compatibility)

histbeep ¶

Beep in ZLE when a widget attempts to access a history entry which isn't there.

histexpand ¶

`BANG_HIST` (bash compatibility)

histexpiredupsfirst ¶

If the internal history needs to be trimmed to add the current command line, setting this option will cause the oldest history event that has a duplicate to be lost before losing a unique event from the list. You should be sure to set the value of `HISTSIZE` to a larger number than `SAVEHIST` in order to give you some room for the duplicated events, otherwise this option will behave just like `HIST_IGNORE_ALL_DUPS` once the history fills up with unique events.

histfcntllock ¶

When writing out the history file, by default zsh uses ad-hoc file locking to avoid known problems with locking on some operating systems. With this option locking is done by means of the system's `fcntl` call, where this method is available. On recent operating systems this may provide better performance, in particular avoiding history corruption when files are stored on NFS.

histfindnodups ¶

When searching for history entries in the line editor, do not display duplicates of a line previously found, even if the duplicates are not contiguous.

histignorealldups ¶

If a new command line being added to the history list duplicates an older one, the older command is removed from the list (even if it is not the previous event).

histignoredups ¶

Do not enter command lines into the history list if they are duplicates of the previous event.

histignoreospace ¶

Remove command lines from the history list when the first character on the line is a space, or when one of the expanded aliases contains a leading space. Only normal aliases (not global or suffix aliases) have this behaviour. Note that the command lingers in the internal history until the next command is entered before it vanishes, allowing you to briefly reuse or edit the line. If you want to make it vanish right away without entering another command, type a space and press return.

histlexwords ¶

By default, shell history that is read in from files is split into words on all white space. This means that arguments with quoted whitespace are not correctly handled, with the consequence that references to words in history lines that have been read from a file may be inaccurate. When this option is set, words read in from a history file are divided up in a similar fashion to normal shell command line handling. Although this produces more accurately delimited words, if the size of the history file is large this can be slow. Trial and error is necessary to decide.

histnofunctions ¶

Remove function definitions from the history list. Note that the function lingers in the internal history until the next command is entered before it vanishes, allowing you to briefly reuse or edit the definition.

histnostore ¶

Remove the `history` (`fc -l`) command from the history list when invoked. Note that the command lingers in the internal history until the next command is entered before it vanishes, allowing you to briefly reuse or edit the line.

histreduceblanks ¶

Remove superfluous blanks from each command line being added to the history list.

histsavebycopy ¶

When the history file is re-written, we normally write out a copy of the file named `$HISTFILE.new` and then rename it over the old one. However, if this option is unset, we instead truncate the old history file and write out the new version in-place. If one of the history-appending options is enabled, this option only has an effect when the enlarged history file needs to be re-written to trim it down to size. Disable this only if you have special needs, as doing so makes it possible to lose history entries if zsh gets interrupted during the save.

When writing out a copy of the history file, zsh preserves the old file's permissions and group information, but will refuse to write out a new file if it would change the history file's owner.

histsavenodups ¶

When writing out the history file, older commands that duplicate newer ones are omitted.

histsubstpattern ¶

Substitutions using the `:s` and `:&` history modifiers are performed with pattern matching instead of string matching. This occurs wherever history modifiers are valid, including glob qualifiers and parameters. See `subref(Modifiers)(zshexpn)`.

histverify ¶

Whenever the user enters a line with history expansion, don't execute the line directly; instead, perform history expansion and reload the line into the editing buffer.

hup ¶

Send the `HUP` signal to running jobs when the shell exits.

ignorebraces ¶

Do not perform brace expansion. For historical reasons this also includes the effect of the `IGNORE_CLOSE_BRACES` option.

ignoreclosebraces ¶

When neither this option nor `IGNORE_BRACES` is set, a sole close brace character `"}"` is syntactically significant at any point on a command line. This has the effect that no semicolon or newline is necessary before the brace terminating a function or current shell construct. When either option is set, a closing brace is syntactically significant only in command position. Unlike `IGNORE_BRACES`, this option does not disable brace expansion.

For example, with both options unset a function may be defined in the following fashion:

```
example(args+) { echo $# }
```

while if either option is set, this does not work and something equivalent to the following is required:

```
example(args+) { echo $#; }
```

ignoreeof ¶

Do not exit on end-of-file. Require the use of `exit` or `logout` instead. However, ten consecutive EOFs will cause the shell to exit anyway, to avoid the shell hanging if its tty goes away.

Also, if this option is set and the Zsh Line Editor is used, widgets implemented by shell functions can be bound to EOF (normally Control-D) without printing the normal warning message. This works only for normal widgets, not for completion widgets.

incappendhistory 𐀀

This option works like `APPEND_HISTORY` except that new history lines are added to the `$HISTFILE` incrementally (as soon as they are entered), rather than waiting until the shell exits. The file will still be periodically re-written to trim it when the number of lines grows 20% beyond the value specified by `$SAVEHIST` (see also the `HIST_SAVE_BY_COPY` option).

incappendhistorytime 𐀀

This option is a variant of `INC_APPEND_HISTORY` in which, where possible, the history entry is written out to the file after the command is finished, so that the time taken by the command is recorded correctly in the history file in `EXTENDED_HISTORY` format. This means that the history entry will not be available immediately from other instances of the shell that are using the same history file.

This option is only useful if `INC_APPEND_HISTORY` and `SHARE_HISTORY` are turned off. The three options should be considered mutually exclusive.

interactive 𐀀

This is an interactive shell. This option is set upon initialisation if the standard input is a tty and commands are being read from standard input. (See the discussion of `SHIN_STDIN`.) This heuristic may be overridden by specifying a state for this option on the command line. The value of this option can only be changed via flags supplied at invocation of the shell. It cannot be changed once zsh is running.

interactivecomments 𐀀

Allow comments even in interactive shells.

ksharrays 𐀀

Emulate `ksh` array handling as closely as possible. If this option is set, array elements are numbered from zero, an array parameter without subscript refers to the first element instead of the whole array, and braces are required to delimit a subscript ("`${path[2]}`") rather than just "`$path[2]`") or to apply modifiers to any parameter ("`${PWD:h}`") rather than "`$PWD:h`").

kshautoload 𐀀

Emulate `ksh` function autoloading. This means that when a function is autoloaded, the corresponding file is merely executed, and must define the function itself. (By default, the function is defined to the contents of the file. However, the most common `ksh`-style case - of the file containing only a simple definition of the function - is always handled in the `ksh`-compatible manner.)

kshglob 𐀀

In pattern matching, the interpretation of parentheses is affected by a preceding "`@`", "`*`", "`+`", "`?`" or "`!`". See *Filename Generation* (zshexpn).

kshoptionprint 𐀀

Alters the way options settings are printed: instead of separate lists of set and unset options, all options are shown, marked "on" if they are in the non-default state, "off" otherwise.

kshtypeset 𐀀

This option is now obsolete: a better approximation to the behaviour of other shells is obtained with the reserved word interface to `declare`, `export`, `float`, `integer`, `local`, `readonly` and `typeset`. Note that the option is only applied when the reserved word interface is *not* in use.

Alters the way arguments to the `typeset` family of commands, including `declare`, `export`, `float`, `integer`, `local` and `readonly`, are processed. Without this option, zsh will perform normal word splitting after command and parameter expansion in arguments of an assignment; with it, word splitting does not take place in those cases.

kshzerosubscript ¶

Treat use of a subscript of value zero in array or string expressions as a reference to the first element, i.e. the element that usually has the subscript 1. Ignored if `KSH_ARRAYS` is also set.

If neither this option nor `KSH_ARRAYS` is set, accesses to an element of an array or string with subscript zero return an empty element or string, while attempts to set element zero of an array or string are treated as an error. However, attempts to set an otherwise valid subscript range that includes zero will succeed. For example, if `KSH_ZERO_SUBSCRIPT` is not set,

```
example(array[0]=(element))
```

is an error, while

```
example(array[0,1]=(element))
```

is not and will replace the first element of the array.

This option is for compatibility with older versions of the shell and is not recommended in new code.

listambiguous ¶

This option works when `AUTO_LIST` or `BASH_AUTO_LIST` is also set. If there is an unambiguous prefix to insert on the command line, that is done without a completion list being displayed; in other words, auto-listing behaviour only takes place when nothing would be inserted. In the case of `BASH_AUTO_LIST`, this means that the list will be delayed to the third call of the function.

listbeep ¶

Beep on an ambiguous completion. More accurately, this forces the completion widgets to return status 1 on an ambiguous completion, which causes the shell to beep if the option `BEEP` is also set; this may be modified if completion is called from a user-defined widget.

listpacked ¶

Try to make the completion list smaller (occupying less lines) by printing the matches in columns with different widths.

listrowsfirst ¶

Lay out the matches in completion lists sorted horizontally, that is, the second match is to the right of the first one, not under it as usual.

listtypes ¶

When listing files that are possible completions, show the type of each file with a trailing identifying mark.

localloops ¶

When this option is not set, the effect of `break` and `continue` commands may propagate outside function scope, affecting loops in calling functions. When the option is set in a calling function, a `break` or a `continue` that is not caught within a called function (regardless of the setting of the option within that function) produces a warning and the effect is cancelled.

localoptions ¶

If this option is set at the point of return from a shell function, most options (including this one) which were in force upon entry to the function are restored; the `PRIVILEGED` option is not restored. Otherwise, only this option, and the `LOCAL_LOOPS`, `XTRACE` and `PRINT_EXIT_VALUE` options are restored. Hence if this is explicitly unset by a shell function the other options in force at the point of return will remain so. A shell function can also guarantee itself a known shell configuration with a formulation like "emulate -L zsh"; the `-L` activates `LOCAL_OPTIONS`.

localpatterns ¶

If this option is set at the point of return from a shell function, the state of pattern disables, as set with the builtin command "disable -p", is restored to what it was when the function was entered. The behaviour of this option is similar to the effect of `LOCAL_OPTIONS` on options; hence "emulate -L sh" (or indeed any other emulation with the `-L` option) activates `LOCAL_PATTERNS`.

localtraps ¶

If this option is set when a signal trap is set inside a function, then the previous status of the trap for that signal will be restored when the function exits. Note that this option must be set *prior* to altering the trap behaviour in a function; unlike `LOCAL_OPTIONS`, the value on exit from the function is irrelevant. However, it does not need to be set before any global trap for that to be correctly restored by a function. For example,

```
example(unsetopt localtraps trap - INT fn+() { setopt localtraps; trap " INT; sleep 3; })
```

will restore normal handling of `SIGINT` after the function exits.

log ¶

List all users currently logged in who are affected by the current setting of the `watch` parameter.

login ¶

This is a login shell. If this option is not explicitly set, the shell becomes a login shell if the first character of the `argv[0]` passed to the shell is a "-".

longlistjobs ¶

Print job notifications in the long format by default.

magicequalsubst ¶

All unquoted arguments of the form `_anything_="expression"` appearing after the command name have filename expansion (that is, where *expression* has a leading "~" or "=") performed on *expression* as if it were a parameter assignment. The argument is not otherwise treated specially; it is passed to the command as a single argument, and not used as an actual parameter assignment. For example, in `echo foo=~/bar:~/rod`, both occurrences of ~ would be replaced. Note that this happens anyway with `typeset` and similar statements.

This option respects the setting of the `KSH_TYPESET` option. In other words, if both options are in effect, arguments looking like assignments will not undergo word splitting.

mailwarn ¶

`MAIL_WARNING` (bash compatibility)

mailwarning ¶

Print a warning message if a mail file has been accessed since the shell last checked.

markdirs ¶

Append a trailing "/" to all directory names resulting from filename generation (globbing).

menucomplete ¶

On an ambiguous completion, instead of listing possibilities or beeping, insert the first match immediately. Then when completion is requested again, remove the first match and insert the second match, etc. When there are no more matches, go back to the first one again. `reverse-menu-complete` may be used to loop through the list in the other direction. This option overrides `AUTO_MENU`.

monitor 𐀀

Allow job control. Set by default in interactive shells.

multibyte 𐀀

Respect multibyte characters when found in strings. When this option is set, strings are examined using the system library to determine how many bytes form a character, depending on the current locale. This affects the way characters are counted in pattern matching, parameter values and various delimiters.

The option is on by default if the shell was compiled with `MULTIBYTE_SUPPORT`; otherwise it is off by default and has no effect if turned on.

If the option is off a single byte is always treated as a single character. This setting is designed purely for examining strings known to contain raw bytes or other values that may not be characters in the current locale. It is not necessary to unset the option merely because the character set for the current locale does not contain multibyte characters.

The option does not affect the shell's editor, which always uses the locale to determine multibyte characters. This is because the character set displayed by the terminal emulator is independent of shell settings.

multifuncdef 𐀀

Allow definitions of multiple functions at once in the form `fn1 fn2 "..._tt()"`; if the option is not set, this causes a parse error. Definition of multiple functions with the `function` keyword is always allowed. Multiple function definitions are not often used and can cause obscure errors.

Note that no error is raised if multiple functions are defined as a result of a set of names that were originally read as a single word on the command line, for example `"TRAP{INT,QUIT}"`. Although there are no plans to change this behaviour at present, it is not guaranteed.

multios 𐀀

Perform implicit `tees` or `cats` when multiple redirections are attempted (see *Redirection* (zshmisc)).

nomatch 𐀀

If a pattern for filename generation has no matches, print an error, instead of leaving it unchanged in the argument list. This also applies to file expansion of an initial `"~"` or `"="`.

notify 𐀀

Report the status of background jobs immediately, rather than waiting until just before printing a prompt.

nullglob 𐀀

If a pattern for filename generation has no matches, delete the pattern from the argument list instead of reporting an error. Overrides `NOMATCH`.

numericglob sort 𐀀

If numeric filenames are matched by a filename generation pattern, sort the filenames numerically rather than lexicographically.

octalzeroes 𐀀

Interpret any integer constant beginning with a 0 as octal, per IEEE Std 1003.2-1992 (ISO 9945-2:1993). This is not enabled by default as it causes problems with parsing of, for example, date and time strings with leading zeroes.

Sequences of digits indicating a numeric base such as the `"08"` component in `"08#77"` are always interpreted as decimal, regardless of leading zeroes.

onecmd ¶

`SINGLE_COMMAND` (bash compatibility)

overstrike ¶

Start up the line editor in overstrike mode.

pathdirs ¶

Perform a path search even on command names with slashes in them. Thus if `/usr/local/bin` is in the user's path, and he or she types `"X11/xinit"`, the command `/usr/local/bin/X11/xinit` will be executed (assuming it exists). Commands explicitly beginning with `"/"`, `"/."` or `"./"` are not subject to the path search. This also applies to the `"."` and `source` builtins.

Note that subdirectories of the current directory are always searched for executables specified in this form. This takes place before any search indicated by this option, and regardless of whether `"."` or the current directory appear in the command search path.

pathscript ¶

If this option is not set, a script passed as the first non-option argument to the shell must contain the name of the file to open. If this option is set, and the script does not specify a directory path, the script is looked for first in the current directory, then in the command path. See *Invocation* (zsh).

physical ¶

`CHASE_LINKS` (ksh and bash compatibility)

pipefail ¶

By default, when a pipeline exits the exit status recorded by the shell and returned by the shell variable `$?` reflects that of the rightmost element of a pipeline. If this option is set, the exit status instead reflects the status of the rightmost element of the pipeline that was non-zero, or zero if all elements exited with zero status.

posixaliases ¶

When this option is set, reserved words are not candidates for alias expansion: it is still possible to declare any of them as an alias, but the alias will never be expanded. Reserved words are described in *Reserved Words* (zshmisc).

Alias expansion takes place while text is being read; hence when this option is set it does not take effect until the end of any function or other piece of shell code parsed as one unit. Note this may cause differences from other shells even when the option is in effect. For example, when running a command with `"zsh -c"`, or even `"zsh -o posixaliases -c"`, the entire command argument is parsed as one unit, so aliases defined within the argument are not available even in later lines. If in doubt, avoid use of aliases in non-interactive code.

posixargzero ¶

This option may be used to temporarily disable `FUNCTION_ARGZERO` and thereby restore the value of `$0` to the name used to invoke the shell (or as set by the `-c` command line option). For compatibility with previous versions of the shell, emulations use `NO_FUNCTION_ARGZERO` instead of `POSIX_ARGZERO`, which may result in unexpected scoping of `$0` if the emulation mode is changed inside a function or script. To avoid this, explicitly enable `POSIX_ARGZERO` in the `emulate` command:

```
emulate sh -o POSIX_ARGZERO
```

Note that `NO_POSIX_ARGZERO` has no effect unless `FUNCTION_ARGZERO` was already enabled upon entry to the function or script.

posixbuiltins ¶

When this option is set the `command` builtin can be used to execute shell builtin commands. Parameter assignments specified before shell functions and special builtins are kept after the command completes unless the special builtin is prefixed with the `command` builtin. Special builtins are `.`, `:`, `break`, `continue`, `declare`, `eval`, `exit`, `export`, `integer`, `local`, `readonly`, `return`, `set`, `shift`, `source`, `times`, `trap` and `unset`.

In addition, various error conditions associated with the above builtins or `exec` cause a non-interactive shell to exit and an interactive shell to return to its top-level processing.

Furthermore, functions and shell builtins are not executed after an `exec` prefix; the command to be executed must be an external command found in the path.

Furthermore, the `getopts` builtin behaves in a POSIX-compatible fashion in that the associated variable `OPTARG` is not made local to functions, and its value is calculated differently to match other shells.

Moreover, the warning and special exit code from `[[ -o _non_existent_option_ ]]` are suppressed.

posixcd ¶

Modifies the behaviour of `cd`, `chdir` and `pushd` commands to make them more compatible with the POSIX standard. The behaviour with the option unset is described in the documentation for the `cd` builtin in `nmref(Shell Builtin Commands)(zshbuiltins)`. If the option is set, the shell does not test for directories beneath the local directory (`.`) until after all directories in `cdpath` have been tested, and the `cd` and `chdir` commands do not recognise arguments of the form `{+|-}n` as directory stack entries.

Also, if the option is set, the conditions under which the shell prints the new directory after changing to it are modified. It is no longer restricted to interactive shells (although printing of the directory stack with `pushd` is still limited to interactive shells); and any use of a component of `CDPATH`, including a `.` but excluding an empty component that is otherwise treated as `.`, causes the directory to be printed.

posixidentifiers ¶

When this option is set, only the ASCII characters `a` to `z`, `A` to `Z`, `0` to `9` and `_` may be used in identifiers (names of shell parameters and modules).

In addition, setting this option limits the effect of parameter substitution with no braces, so that the expression `$#` is treated as the parameter `$#` even if followed by a valid parameter name. When it is unset, zsh allows expressions of the form `$#name` to refer to the length of `$name`, even for special variables, for example in expressions such as `$#-` and `$#*`.

Another difference is that with the option set assignment to an unset variable in arithmetic context causes the variable to be created as a scalar rather than a numeric type. So after `tt(unset t; (( t = 3 )))`, without `POSIX_IDENTIFIERS` set `t` has integer type, while with it set it has scalar type.

When the option is unset and multibyte character support is enabled (i.e. it is compiled in and the option `MULTIBYTE` is set), then additionally any alphanumeric characters in the local character set may be used in identifiers. Note that scripts and functions written with this feature are not portable, and also that both options must be set before the script or function is parsed; setting them during execution is not sufficient as the syntax `variable=value` has already been parsed as a command rather than an assignment.

If multibyte character support is not compiled into the shell this option is ignored; all octets with the top bit set may be used in identifiers. This is non-standard but is the traditional zsh behaviour.

posixjobs ¶

This option makes job control more compliant with the POSIX standard.

When the option is not set, the `MONITOR` option is unset on entry to subshells, so that job control is no longer active. When the option is set, the `MONITOR` option and job control remain active in the subshell, but note that the subshell has no access to jobs in the parent shell.

When the option is not set, jobs put in the background or foreground with `bg` or `fg` are displayed with the same information that would be reported by `jobs`. When the option is set, only the text is printed. The output from `jobs` itself is not affected by the option.

When the option is not set, job information from the parent shell is saved for output within a subshell (for example, within a pipeline). When the option is set, the output of `jobs` is empty until a job is started within the subshell.

In previous versions of the shell, it was necessary to enable `POSIX_JOBS` in order for the builtin command `wait` to return the status of background jobs that had already exited. This is no longer the case.

`posixstrings` ¶

This option affects processing of quoted strings. Currently it only affects the behaviour of null characters, i.e. character 0 in the portable character set corresponding to US ASCII.

When this option is not set, null characters embedded within strings of the form `"$_..._"` are treated as ordinary characters. The entire string is maintained within the shell and output to files where necessary, although owing to restrictions of the library interface the string is truncated at the null character in file names, environment variables, or in arguments to external programs.

When this option is set, the `"$_..._"` expression is truncated at the null character. Note that remaining parts of the same string beyond the termination of the quotes are not truncated.

For example, the command line argument `"a$b0c'd` is treated with the option off as the characters `a`, `b`, `null`, `c`, `d`, and with the option on as the characters `a`, `b`, `d`.

`posixtraps` ¶

When this option is set, the usual zsh behaviour of executing traps for `EXIT` on exit from shell functions is suppressed. In that case, manipulating `EXIT` traps always alters the global trap for exiting the shell; the `LOCAL_TRAPS` option is ignored for the `EXIT` trap.

Also, a `return` statement executed in a trap with no argument passes back from the function the value from the surrounding context, not from code executed within the trap.

Furthermore, if a trap is set to be ignored, this state persists when a subshell is entered. Without the option, the trap would be reset to its default state at this point.

`printeightbit` ¶

Print eight bit characters literally in completion lists, etc. This option is not necessary if your system correctly returns the printability of eight bit characters (see `manref(ctype)(3)`).

`printexitvalue` ¶

Print the exit value of programs with non-zero exit status. This is only available at the command line in interactive shells.

`privileged` ¶

Turn on privileged mode. Typically this is used when script is to be run with elevated privileges. This should be done as follows directly with the `-p` option to zsh so that it takes effect during startup.

```
#!/bin/zsh -p
```

The option is enabled automatically on startup if the effective user (group) ID is not equal to the real user (group) ID. In this case, turning the option off causes the effective user and group IDs to be set to the real user and group IDs. Be aware that if that fails the shell may be running with different IDs than was intended so a script should check for failure and act accordingly, for example:

```
unsetopt privileged || exit
```

The `PRIVILEGED` option disables sourcing user startup files. If zsh is invoked as `"sh"` or `"ksh"` with this option set, `/etc/suid_profile` is sourced (after `/etc/profile` on interactive shells). Sourcing `~/.profile` is disabled and the contents of the `ENV` variable is ignored. This option cannot be changed using the `-m` option of `setopt` and `unsetopt`, and changing it inside a function always changes it globally regardless of the `LOCAL_OPTIONS` option.

`promptbang` ¶

If set, `"!"` is treated specially in prompt expansion. See *Expansion of Prompt Sequences* (zshmisc).

promptcr ¶

Print a carriage return just before printing a prompt in the line editor. This is on by default as multi-line editing is only possible if the editor knows where the start of the line appears.

promptpercent ¶

If set, "%" is treated specially in prompt expansion. See *Expansion of Prompt Sequences* (zshmisc).

promptsp ¶

Attempt to preserve a partial line (i.e. a line that did not end with a newline) that would otherwise be covered up by the command prompt due to the `PROMPT_CR` option. This works by outputting some cursor-control characters, including a series of spaces, that should make the terminal wrap to the next line when a partial line is present (note that this is only successful if your terminal has automatic margins, which is typical).

When a partial line is preserved, by default you will see an inverse+bold character at the end of the partial line: a "%" for a normal user or a "#" for root. If set, the shell parameter `PROMPT_EOL_MARK` can be used to customize how the end of partial lines are shown.

NOTE: if the `PROMPT_CR` option is not set, enabling this option will have no effect. This option is on by default.

promptsubst ¶

If set, *parameter expansion*, *command substitution* and *arithmetic expansion* are performed in prompts. Substitutions within prompts do not affect the command status.

promptvars ¶

`PROMPT_SUBST` (bash compatibility)

pushdignoredups ¶

Don't push multiple copies of the same directory onto the directory stack.

pushdminus ¶

Exchanges the meanings of "+" and "-" when used with a number to specify a directory in the stack.

pushdsilent ¶

Do not print the directory stack after `pushd` or `popd`.

pushdthome ¶

Have `pushd` with no arguments act like "pushd \$HOME".

rcexpandparam ¶

Array expansions of the form `_foo_${_xx_}"bar"`, where the parameter `xx` is set to `tt()a b c`, are substituted with `"fooabar foobbar fooobar"` instead of the default `"fooa b cbar"`. Note that an empty array will therefore cause all arguments to be removed.

rcquotes ¶

Allow the character sequence `"'` to signify a single quote within singly quoted strings. Note this does not apply in quoted strings using the format `"$"..."`, where a backslashed single quote can be used.

ttcs ¶

After `tt(zshenv())` is sourced on startup, source the `.zshenv`, `tt(zprofile())`, `.zprofile`, `tt(zshrc())`, `.zshrc`, `tt(zlogin())`, `.zlogin`, and `.zlogout` files, as described in *Startup/Shutdown Files* (zsh). If this option is unset, the `tt(zshenv())` file is still sourced, but any of the others will not be; it can be set at any time to prevent the remaining startup files after the currently executing one from being sourced.

recexact ¶

If the string on the command line exactly matches one of the possible completions, it is accepted, even if there is another completion (i.e. that string with something else added) that also matches.

rematchpcre ¶

If set, regular expression matching with the `=~` operator will use Perl-Compatible Regular Expressions from the PCRE library. (The `zsh/pcre` module must be available.) If not set, regular expressions will use the extended regexp syntax provided by the system libraries.

restricted ¶

Restricted-shell mode (equivalent to invoking zsh as `rzsh` or with `-r`).

Disables: `cd`, modifying `$PATH` / `$ENV` / `$SHELL`, `>` / `>>` redirects, creating functions with the `function` keyword, `exec`-ing commands containing `/`, `kill`-ing by pid, and several `setopt` toggles. Designed for sandboxed login shells where the user must stay inside a curated command set. Once set, cannot be cleared within the running shell.

rmstarsilent ¶

Do not query the user before executing `"rm *"` or `"rm path/*"`.

rmstarwait ¶

If querying the user before executing `"rm *"` or `"rm path/*"`, first wait ten seconds and ignore anything typed in that time. This avoids the problem of reflexively answering "yes" to the query when one didn't really mean it. The wait and query can always be avoided by expanding the `"*"` in ZLE (with `tab`).

sharehistory ¶

This option both imports new commands from the history file, and also causes your typed commands to be appended to the history file (the latter is like specifying `INC_APPEND_HISTORY`, which should be turned off if this option is in effect). The history lines are also output with timestamps ala `EXTENDED_HISTORY` (which makes it easier to find the spot where we left off reading the file after it gets re-written).

By default, history movement commands visit the imported lines as well as the local lines, but you can toggle this on and off with the `set-local-history` zle binding. It is also possible to create a zle widget that will make some commands ignore imported commands, and some include them.

If you find that you want more control over when commands get imported, you may wish to turn `SHARE_HISTORY` off, `INC_APPEND_HISTORY` or `INC_APPEND_HISTORY_TIME` (see above) on, and then manually import commands whenever you need them using `fc -RI`.

shfileexpansion ¶

Perform filename expansion (e.g., `~` expansion) *before* parameter expansion, command substitution, arithmetic expansion and brace expansion. If this option is unset, it is performed *after* brace expansion, so things like `"~$USERNAME"` and `"~{pfalstad,rc}"` will work.

shglob ¶

Disables the special meaning of "tt()", "|", ")" and '<' for globbing the result of parameter and command substitutions, and in some other places where the shell accepts patterns. If `SH_GLOB` is set but `KSH_GLOB` is not, the shell allows the interpretation of subshell expressions enclosed in parentheses in some cases where there is no space before the opening parenthesis, e.g. `tt(!(true+))` is interpreted as if there were a space after the `!`. This option is set by default if `zsh` is invoked as `sh` or `ksh`.

shstdin ¶

Commands are being read from the standard input. Commands are read from standard input if no command is specified with `-c` and no file of commands is specified. If `SHIN_STDIN` is set explicitly on the command line, any argument that would otherwise have been taken as a file to run will instead be treated as a normal positional parameter. Note that setting or unsetting this option on the command line does not necessarily affect the state the option will have while the shell is running - that is purely an indicator of whether or not commands are *actually* being read from standard input. The value of this option can only be changed via flags supplied at invocation of the shell. It cannot be changed once `zsh` is running.

shnullcmd ¶

Do not use the values of `NULLCMD` and `READNULLCMD` when doing redirections, use ":" instead (see *Redirection* (`zshmisc`)).

shoptionletters ¶

If this option is set the shell tries to interpret single letter options (which are used with `set` and `setopt`) like `ksh` does. This also affects the value of the `-` special parameter.

shortloops ¶

Allow the short forms of `for`, `repeat`, `select`, `if`, and `function` constructs.

shortrepeat ¶

Allow the short form `repeat` as `SHORT_LOOPS` but without enabling it for the other constructs.

shwordsplit ¶

Causes field splitting to be performed on unquoted parameter expansions. Note that this option has nothing to do with word splitting. (See *Parameter Expansion* (`zshexpn`).)

singlecommand ¶

If the shell is reading from standard input, it exits after a single command has been executed. This also makes the shell non-interactive, unless the `INTERACTIVE` option is explicitly set on the command line. The value of this option can only be changed via flags supplied at invocation of the shell. It cannot be changed once `zsh` is running.

singlelinezle ¶

Use single-line command line editing instead of multi-line.

Note that although this is on by default in `ksh` emulation it only provides superficial compatibility with the `ksh` line editor and reduces the effectiveness of the `zsh` line editor. As it has no effect on shell syntax, many users may wish to disable this option when using `ksh` emulation interactively.

sourcetrace ¶

If set, `zsh` will print an informational message announcing the name of each file it loads. The format of the output is similar to that for the `XTRACE` option, with the message `<sourcetrace>`. A file may be loaded by the shell itself when it starts up and shuts down (*Startup/Shutdown Files*) or by the use of the `"source"` and `"dot"` builtin commands.

stdin 𐀀

SHIN_STDIN (ksh compatibility)

sunkeyboardhack 𐀀

If a line ends with a backquote, and there are an odd number of backquotes on the line, ignore the trailing backquote. This is useful on some keyboards where the return key is too small, and the backquote key lies annoyingly close to it. As an alternative the variable `KEYBOARD_HACK` lets you choose the character to be removed.

trackall 𐀀

HASH_CMDS (ksh compatibility)

transientprompt 𐀀

Remove any right prompt from display when accepting a command line. This may be useful with terminals with other cut/paste methods.

trapsasync 𐀀

While waiting for a program to exit, handle signals and run traps immediately. Otherwise the trap is run after a child process has exited. Note this does not affect the point at which traps are run for any case other than when the shell is waiting for a child process.

typesetsilent 𐀀

If this is unset, executing any of the "typeset" family of commands with no options and a list of parameters that have no values to be assigned but already exist will display the value of the parameter. If the option is set, they will only be shown when parameters are selected with the "-m" option. The option "-p" is available whether or not the option is set.

typesettounset 𐀀

When declaring a new parameter with any of the "typeset" family of related commands, the parameter remains unset unless and until a value is explicitly assigned to it, either in the "typeset" command itself or as a later assignment statement.

unset 𐀀

Each named parameter is unset. Local parameters remain local even if unset; they appear unset within scope, but the previous value will still reappear when the scope ends.

Individual elements of associative array parameters may be unset by using subscript syntax on *name*, which should be quoted (or the entire command prefixed with `noglob`) to protect the subscript from filename generation.

If the `-m` flag is specified the arguments are taken as patterns (should be quoted) and all parameters with matching names are unset. Note that this cannot be used when unsetting associative array elements, as the subscript will be treated as part of the pattern.

The `-v` flag specifies that *name* refers to parameters. This is the default behaviour. If the `-n` option is supplied, and *name* is a named reference, *name* will be unset rather than the variable it references.

`unset -f` is equivalent to `unfunction`. The `-n` option has no effect with `-f`.

verbose 𐀀

If set, as it is by default, the completion listing is more verbose. In particular many commands show descriptions for options if this style is "true".

vi 𐀀

Move backward one word, where a word is defined as a series of non-blank characters.

warncreateglobal ¶

Print a warning message when a global parameter is created in a function by an assignment or in math context. This often indicates that a parameter has not been declared local when it should have been. Parameters explicitly declared global from within a function using `typeset -g` do not cause a warning. Note that there is no warning when a local parameter is assigned to in a nested function, which may also indicate an error.

warnnestedvar ¶

Print a warning message when an existing parameter from an enclosing function scope, or global, is set in a function by an assignment or in math context. Assignment to shell special parameters does not cause a warning. This is the companion to `WARN_CREATE_GLOBAL` as in this case the warning is only printed when a parameter is *not* created. Where possible, use of `typeset -g` to set the parameter suppresses the error, but note that this needs to be used every time the parameter is set. To restrict the effect of this option to a single function scope, use "functions -W".

For example, the following code produces a warning for the assignment inside the function `nested` as that overrides the value within `toplevel`

```
example(toplevel+()) { local foo="in fn" nested } nested+() { foo="in nested" } setopt warn_nested_var toplevel)
```

xtrace ¶

Print commands and their arguments as they are executed. The output is preceded by the value of `$PS4`, formatted as described in *Expansion of Prompt Sequences* (zshmisc).

zle ¶

The `zle` builtin performs a number of different actions concerning ZLE.

With no options and no arguments, only the return status will be set. It is zero if ZLE is currently active and widgets could be invoked using this builtin command and non-zero otherwise. Note that even if non-zero status is returned, `zle` may still be active as part of the completion system; this does not allow direct calls to ZLE widgets.

Otherwise, which operation it performs depends on its options:

- `-l [ -L | -a ] [ string ]` — List all existing user-defined widgets. If the `-L` option is used, list in the form of `zle` commands to create the widgets.

When combined with the `-a` option, all widget names are listed, including the builtin ones. In this case the `-L` option is ignored.

If at least one *string* is given, and `-a` is present or `-L` is not used, nothing will be printed. The return status will be zero if all `_string_s` are names of existing widgets and non-zero if at least one *string* is not a name of a defined widget. If `-a` is also present, all widget names are used for the comparison including builtin widgets, else only user-defined widgets are used.

If at least one *string* is present and the `-L` option is used, user-defined widgets matching any *string* are listed in the form of `zle` commands to create the widgets.

- `-D widget ...` — Delete the named `_widget_s`.

- `-A old-widget new-widget` — Make the *new-widget* name an alias for *old-widget*, so that both names refer to the same widget. The names have equal standing; if either is deleted, the other remains. If there is already a widget with the *new-widget* name, it is deleted.

- `-N widget [ function ]` — Create a user-defined widget. If there is already a widget with the specified name, it is overwritten. When the new widget is invoked from within the editor, the specified shell *function* is called. If no function name is specified, it defaults to the same name as the widget. For further information, see *Zle Widgets* (below).

- `-f flag [ flag... ]` — Set various flags on the running widget. Possible values for *flag* are:

yank for indicating that the widget has yanked text into the buffer. If the widget is wrapping an existing internal widget, no further action is necessary, but if it has inserted the text manually, then it should also take care to set `YANK_START` and `YANK_END` correctly. **yankbefore** does the same but is used when the yanked text appears after the cursor.

kill for indicating that text has been killed into the cutbuffer. When repeatedly invoking a kill widget, text is appended to the cutbuffer instead of replacing it, but when wrapping such widgets, it is necessary to call "zle -f kill" to retain this effect.

vichange for indicating that the widget represents a vi change that can be repeated as a whole with "vi-repeat-change". The flag should be set early in the function before inspecting the value of `NUMERIC` or invoking other widgets. This has no effect for

a widget invoked from insert mode. If insert mode is active when the widget finishes, the change extends until next returning to command mode.

cindex(completion widgets, creating)

- **-C *widget completion-widget function*** — Create a user-defined completion widget named *widget*. The completion widget will behave like the built-in completion-widget whose name is given as *completion-widget*. To generate the completions, the shell function *function* will be called. For further information, see nmref(Completion Widgets)(zshcompwid).

- **-R [-c] [*display-string*] [*string* ...]** — Redisplay the command line. If a *display-string* is given and not empty, this is shown in the status line (immediately below the line being edited).

If the optional *_string_s* are given they are listed below the prompt in the same way as completion lists are printed. If no *_string_s* are given but the **-c** option is used such a list is cleared.

Note that immediately after returning from running widgets, the command line will be redisplayed and the strings displayed will be erased. Therefore, this option is only useful for widgets that do not exit immediately after using it.

This command can safely be called outside user defined widgets; if zle is active, the display will be refreshed, while if zle is not active, the command has no effect. In this case there will usually be no other arguments.

The status is zero if zle was active, else one.

- **-M *string*** — As with the **-R** option, the *string* will be displayed below the command line; unlike the **-R** option, the string will not be put into the status line but will instead be printed normally below the prompt. This means that the *string* will still be displayed after the widget returns (until it is overwritten by subsequent commands).

- **-U *string*** — This pushes the characters in the *string* onto the input stack of ZLE. After the widget currently executed finishes ZLE will behave as if the characters in the *string* were typed by the user.

As ZLE uses a stack, if this option is used repeatedly the last string pushed onto the stack will be processed first. However, the characters in each *string* will be processed in the order in which they appear in the string.

- **-K *keymap*** — Selects the keymap named *keymap*. An error message will be displayed if there is no such keymap.

This keymap selection affects the interpretation of following keystrokes within this invocation of ZLE. Any following invocation (e.g., the next command line) will start as usual with the "main" keymap selected.

- **-F [-L | -w] [*fd* [*handler*]]** — Only available if your system supports one of the "poll" or "select" system calls; most modern systems do.

Installs *handler* (the name of a shell function) to handle input from file descriptor *fd*. Installing a handler for an *fd* which is already handled causes the existing handler to be replaced. Any number of handlers for any number of readable file descriptors may be installed. Note that zle makes no attempt to check whether this *fd* is actually readable when installing the handler. The user must make their own arrangements for handling the file descriptor when zle is not active.

When zle is attempting to read data, it will examine both the terminal and the list of handled *fd*'s. If data becomes available on a handled *fd*, zle calls *handler* with the *fd* which is ready for reading as the first argument. Under normal circumstances this is the only argument, but if an error was detected, a second argument provides details: "hup" for a disconnect, "nval" for a closed or otherwise invalid descriptor, or "err" for any other condition. Systems that support only the "select" system call always use "err".

If the option **-w** is also given, the *handler* is instead a line editor widget, typically a shell function made into a widget using "zle -N". In that case *handler* can use all the facilities of zle to update the current editing line. Note, however, that as handling *fd* takes place at a low level changes to the display will not automatically appear; the widget should call "zle -R" to force redisplay. As of this writing, widget handlers only support a single argument and thus are never passed a string for error state, so widgets must be prepared to test the descriptor themselves.

If either type of handler produces output to the terminal, it should call "zle -l" before doing so (see below). Handlers should not attempt to read from the terminal.

If no *handler* is given, but an *fd* is present, any handler for that *fd* is removed. If there is none, an error message is printed and status 1 is returned.

If no arguments are given, or the **-L** option is supplied, a list of handlers is printed in a form which can be stored for later execution.

An *fd* (but not a *handler*) may optionally be given with the **-L** option; in this case, the function will list the handler if any, else silently return status 1.

Note that this feature should be used with care. Activity on one of the *fd*'s which is not properly handled can cause the terminal to become unusable. Removing an *fd* handler from within a signal trap may cause unpredictable behavior.

Here is a simple example of using this feature. A connection to a remote TCP port is created using the `ztcp` command; see *The zsh/net/tcp Module* (zshmodules). Then a handler is installed which simply prints out any data which arrives on this connection. Note that "select" will indicate that the file descriptor needs handling if the remote side has closed the connection; we handle that by testing for a failed read.

```
example(if ztcp pwspc 2811; then tcpfd=$REPLY handler+() { zle -I local line if ! read -r line <&$1; then # select marks this fd if
we reach EOF, # so handle this specially. print "[Read on fd $1 failed, removing.]" >&2 zle -F $1 return 1 fi print -r - $line } zle
-F $tcpfd handler fi)
```

- **-I** — Unusually, this option is most useful outside ordinary widget functions, though it may be used within if normal output to the terminal is required. It invalidates the current `zle` display in preparation for output; typically this will be from a trap function. It has no effect if `zle` is not active. When a trap exits, the shell checks to see if the display needs restoring, hence the following will print output in such a way as not to disturb the line being edited:

```
example(TRAPUSR1) { # Invalidate zle display [[ -o zle ]] && zle -I # Show output print Hello }
```

In general, the trap function may need to test whether `zle` is active before using this method (as shown in the example), since the `zsh/zle` module may not even be loaded; if it is not, the command can be skipped.

It is possible to call "zle -I" several times before control is returned to the editor; the display will only be invalidated the first time to minimise disruption.

Note that there are normally better ways of manipulating the display from within `zle` widgets; see, for example, "zle -R" above.

The returned status is zero if `zle` was invalidated, even though this may have been by a previous call to "zle -I" or by a system notification. To test if a `zle` widget may be called at this point, execute `zle` with no arguments and examine the return status.

- **-T** — This is used to add, list or remove internal transformations on the processing performed by the line editor. It is typically used only for debugging or testing and is therefore of little interest to the general user.

`zle -T" transformation func"` specifies that the given *transformation* (see below) is effected by shell function *func*.

`zle -Tr" transformation"` removes the given *transformation* if it was present (it is not an error if none was).

"zle -TL" can be used to list all transformations currently in operation.

Currently the only transformation is `tc`. This is used instead of outputting termcap codes to the terminal. When the transformation is in operation the shell function is passed the termcap code that would be output as its first argument; if the operation required a numeric argument, that is passed as a second argument. The function should set the shell variable `REPLY` to the transformed termcap code. Typically this is used to produce some simply formatted version of the code and optional argument for debugging or testing. Note that this transformation is not applied to other non-printing characters such as carriage returns and newlines.

- **widget** [**-n num**] [**-f flag**] [**-Nw**] [**-K keymap**] *args* ... — Invoke the specified *widget*. This can only be done when `ZLE` is active; normally this will be within a user-defined widget.

With the options **-n** and **-N**, the current numeric argument will be saved and then restored after the call to *widget*; **-n num** sets the numeric argument temporarily to *num*, while **-N** sets it to the default, i.e. as if there were none.

With the option **-K**, *keymap* will be used as the current keymap during the execution of the widget. The previous keymap will be restored when the widget exits.

Normally, calling a widget in this way does not set the special parameter `WIDGET` and related parameters, so that the environment appears as if the top-level widget called by the user were still active. With the option **-w**, `WIDGET` and related parameters are set to reflect the widget being executed by the `zle` call.

Normally, when *widget* returns the special parameter `LASTWIDGET` will point to it. This can be inhibited by passing the option **-f noLast**.

Any further arguments will be passed to the widget; note that as standard argument handling is performed, any general argument list should be preceded by `--`. If it is a shell function, these are passed down as positional parameters; for builtin widgets it is up to the widget in question what it does with them. Currently arguments are only handled by the incremental-search commands, the `history-search-forward` and `-backward` and the corresponding functions prefixed by `vi-`, and by `universal-argument`. No error is flagged if the command does not use the arguments, or only uses some of them.

The return status reflects the success or failure of the operation carried out by the widget, or if it is a user-defined widget the return status of the shell function.

A non-zero return status causes the shell to beep when the widget exits, unless the `BEEP` options was unset or the widget was called via the `zle` command. Thus if a user defined widget requires an immediate beep, it should call the `beep` widget directly.

Special Variable Index

256 entries · zsh-defined parameters and well-known env vars. Includes both scalar (`$?`) and array (`$path`) forms.

! ¶

Pipeline negation (also a reserved word). `! cmd` inverts `cmd`'s exit status — zero becomes 1, non-zero becomes 0. Distinct from `!` history expansion (lexer-stage).

¶

The number of positional parameters in decimal. Note that some confusion may occur with the syntax `$#param` which substitutes the length of `param`. Use `${#}` to resolve ambiguities. In particular, the sequence `$#-..."` in an arithmetic expression is interpreted as the length of the parameter `-`, q.v.

\$ ¶

The process ID of this shell, set when the shell initializes. Processes forked from the shell without executing a new program, such as command substitutions and commands grouped with `tt(...)`, are subshells that duplicate the current shell, and thus substitute the same value for `$$` as their parent shell.

\$ARGC ¶

Same as `#`.

\$ARGV0 ¶

If exported, its value is used as the `argv[0]` of external commands. Usually used in constructs like `"ARGV0=emacs nethack"`.

\$BAUD ¶

The rate in bits per second at which data reaches the terminal. The line editor will use this value in order to compensate for a slow terminal by delaying updates to the display until necessary. If the parameter is unset or the value is zero the compensation mechanism is turned off. The parameter is not set by default.

This parameter may be profitably set in some circumstances, e.g. for slow modems dialing into a communications server, or on a slow wide area network. It should be set to the baud rate of the slowest part of the link for best performance.

\$BUFFER ¶

The entire contents of the edit buffer. If it is written to, the cursor remains at the same offset, unless that would put it outside the buffer.

\$BUFFERLINES ¶

The number of screen lines needed for the edit buffer currently displayed on screen (i.e. without any changes to the preceding parameters done after the last redisplay); read-only.

\$COLUMNS ¶

The number of columns for this terminal session. Used for printing select lists and for the line editor.

\$CONTEXT ¶

The context in which zle was called to read a line; read-only. One of the values:

- **start** — The start of a command line (at prompt **P51**).
- **cont** — A continuation to a command line (at prompt **P52**).
- **select** — In a **select** loop (at prompt **P53**).
- **vared** — Editing a variable in **vared**.

\$CORRECT_IGNORE ¶

If set, is treated as a pattern during spelling correction. Any potential correction that matches the pattern is ignored. For example, if the value is `"*"` *then completion functions (which, by convention, have names beginning with `"_"`)* will never be offered as spelling corrections. The pattern does not apply to the correction of file names, as applied by the **CORRECT_ALL** option (so with the example just given files beginning with `"_"` in the current directory would still be completed).

\$CORRECT_IGNORE_FILE ¶

If set, is treated as a pattern during spelling correction of file names. Any file name that matches the pattern is never offered as a correction. For example, if the value is `".*"` then dot file names will never be offered as spelling corrections. This is useful with the **CORRECT_ALL** option.

\$CPUTYPE ¶

The machine type (microprocessor class or machine model), as determined at run time.

\$CURSOR ¶

The offset of the cursor, within the edit buffer. This is in the range 0 to **\$#BUFFER**, and is by definition equal to **\$#LBUFFER**. Attempts to move the cursor outside the buffer will result in the cursor being moved to the appropriate end of the buffer.

\$CUTBUFFER ¶

The last item cut using one of the "kill-" commands; the string which the next yank would insert in the line. Later entries in the kill ring are in the array **killring**. Note that the command `zle copy-region-as-kill` *string* can be used to set the text of the cut buffer from a shell function and cycle the kill ring in the same way as interactively killing text.

\$DIRSTACKSIZE ¶

The maximum size of the directory stack, by default there is no limit. If the stack gets larger than this, it will be truncated automatically. This is useful with the **AUTO_PUSHD** option. `pindeX(AUTO_PUSHD, use of)`

\$EGID ¶

The effective group ID of the shell process. If you have sufficient privileges, you may change the effective group ID of the shell process by assigning to this parameter. Also (assuming sufficient privileges), you may start a single command with a different effective group ID by `tt((EGID=)_gid_; command+)"`

If this is made local, it is not implicitly set to 0, but may be explicitly set locally.

\$ENV ¶

If the **ENV** environment variable is set when zsh is invoked as **sh** or **ksh**, **\$ENV** is sourced after the profile scripts. The value of **ENV** is subjected to parameter expansion, command substitution, and arithmetic expansion before being interpreted as a pathname. Note that **ENV** is *not* used unless the shell is interactive and zsh is emulating **sh** or **ksh**.

\$EPOCHREALTIME ¶

A floating point value representing the number of seconds since the epoch. The notional accuracy is to nanoseconds if the `clock_gettime` call is available and to microseconds otherwise, but in practice the range of double precision floating point and shell scheduling latencies may be significant effects.

\$EPOCHSECONDS ¶

An integer value representing the number of seconds since the epoch.

\$ERRNO ¶

The value of `errno` (see `manref(errno)(3)`) as set by the most recently failed system call. This value is system dependent and is intended for debugging purposes. It is also useful with the `zsh/system` module which allows the number to be turned into a name or message.

To use this parameter, it must first be assigned a value (typically 0 (zero)). It is initially unset for scripting compatibility.

\$EUID ¶

The effective user ID of the shell process. If you have sufficient privileges, you may change the effective user ID of the shell process by assigning to this parameter. Also (assuming sufficient privileges), you may start a single command with a different effective user ID by `tt((EUID=)_uid_; command+)"`

If this is made local, it is not implicitly set to 0, but may be explicitly set locally.

\$FCEDIT ¶

The default editor for the `fc` builtin. If `FCEDIT` is not set, the parameter `EDITOR` is used; if that is not set either, a builtin default, usually `vi`, is used.

\$FUNCNEST ¶

Integer. If greater than or equal to zero, the maximum nesting depth of shell functions. When it is exceeded, an error is raised at the point where a function is called. The default value is determined when the shell is configured, but is typically 500. Increasing the value increases the danger of a runaway function recursion causing the shell to crash. Setting a negative value turns off the check.

\$GID ¶

The real group ID of the shell process. If you have sufficient privileges, you may change the group ID of the shell process by assigning to this parameter. Also (assuming sufficient privileges), you may start a single command under a different group ID by `tt((GID=)_gid_; command+)"`

If this is made local, it is not implicitly set to 0, but may be explicitly set locally.

\$HISTCHARS ¶

Same as `histchars`. (Deprecated.)

\$HISTCMD ¶

The current history event number in an interactive shell, in other words the event number for the command that caused `$HISTCMD` to be read. If the current history event modifies the history, `HISTCMD` changes to the new maximum history event number.

\$HISTFILE ¶

The file to save the history in when an interactive shell exits. If unset, the history is not saved.

\$HISTNO ¶

The current history number. Setting this has the same effect as moving up or down in the history to the corresponding history line. An attempt to set it is ignored if the line is not stored in the history. Note this is not the same as the parameter `HISTCMD`, which always gives the number of the history line being added to the main shell's history. `HISTNO` refers to the line being retrieved within `zle`.

\$HISTORY_IGNORE ¶

If set, is treated as a pattern at the time history files are written. Any potential history entry that matches the pattern is skipped. For example, if the value is `"fc *"` then commands that invoke the interactive history editor are never written to the history file.

Note that `HISTORY_IGNORE` defines a single pattern: to specify alternatives use the `tt(())_first_|_second_|_..._|_"` syntax.

Compare the `HIST_NO_STORE` option or the `zshaddhistory` hook, either of which would prevent such commands from being added to the interactive history at all. If you wish to use `HISTORY_IGNORE` to stop history being added in the first place, you can define the following hook:

```
example(zshaddhistory+) { emulate -L zsh ## uncomment if HISTORY_IGNORE ## should use EXTENDED_GLOB syntax #
setopt extendedglob [[ $1 != ${~HISTORY_IGNORE} ]] }
```

\$HISTSIZE ¶

The maximum number of events stored in the internal history list. If you use the `HIST_EXPIRE_DUPS_FIRST` option, setting this value larger than the `SAVEHIST` size will give you the difference as a cushion for saving duplicated history events.

If this is made local, it is not implicitly set to 0, but may be explicitly set locally.

\$HOME ¶

The default argument for the `cd` command. This is not set automatically by the shell in `sh`, `ksh` or `csh` emulation, but it is typically present in the environment anyway, and if it becomes set it has its usual special behaviour.

\$HOST ¶

The current hostname.

\$IFS ¶

Internal field separators (by default space, tab, newline and NUL), that are used to separate words which result from command or parameter expansion and words read by the `read` builtin. Any characters from the set space, tab and newline that appear in the `IFS` are called *IFS white space*. One or more IFS white space characters or one non-IFS white space character together with any adjacent IFS white space character delimit a field. If an IFS white space character appears twice consecutively in the `IFS`, this character is treated as if it were not an IFS white space character.

If the parameter is unset, the default is used. Note this has a different effect from setting the parameter to an empty string.

If `MULTIBYTE` option is on and `IFS` contains invalid characters in the current locale, it is reset to the default.

\$ISEARCHMATCH_END ¶

`ISEARCHMATCH_ACTIVE` indicates whether a part of the `BUFFER` is currently matched by an incremental search pattern. `ISEARCHMATCH_START` and `ISEARCHMATCH_END` give the location of the matched part and are in the same units as `CURSOR`. They are only valid for reading when `ISEARCHMATCH_ACTIVE` is non-zero.

All parameters are read-only.

\$KEYBOARD_HACK ¶

This variable defines a character to be removed from the end of the command line before interpreting it (interactive shells only). It is intended to fix the problem with keys placed annoyingly close to return and replaces the `SUNKEYBOARDHACK` option

which did this for backquotes only. Should the chosen character be one of singlequote, doublequote or backquote, there must also be an odd number of them on the command line for the last one to be removed.

For backward compatibility, if the `SUNKEYBOARDHACK` option is explicitly set, the value of `KEYBOARD_HACK` reverts to backquote. If the option is explicitly unset, this variable is set to empty.

`$KEYMAP` 𐀀

The name of the currently selected keymap; read-only.

`$KEYS` 𐀀

The keys typed to invoke this widget, as a literal string; read-only.

`$KEYS_QUEUED_COUNT` 𐀀

The number of bytes pushed back to the input queue and therefore available for reading immediately before any I/O is done; read-only. See also `PENDING`; the two values are distinct.

`$KEYTIMEOUT` 𐀀

The time the shell waits, in hundredths of seconds, for another key to be pressed when reading bound multi-character sequences.

`$LANG` 𐀀

This variable determines the locale category for any category not specifically selected via a variable starting with "LC_".

`$LASTABORTEDSEARCH` 𐀀

The last search string used by an interactive search that was aborted by the user (status 3 returned by the search widget).

`$LASTSEARCH` 𐀀

The last search string used by an interactive search; read-only. This is set even if the search failed (status 0, 1 or 2 returned by the search widget), but not if it was aborted by the user.

`$LASTWIDGET` 𐀀

The name of the last widget that was executed; read-only.

`$LBUFFER` 𐀀

The part of the buffer that lies to the left of the cursor position. If it is assigned to, only that part of the buffer is replaced, and the cursor remains between the new `$LBUFFER` and the old `$RBUFFER`.

`$LC_ALL` 𐀀

This variable overrides the value of the "LANG" variable and the value of any of the other variables starting with "LC_".

`$LC_COLLATE` 𐀀

This variable determines the locale category for character collation information within ranges in glob brackets and for sorting.

\$LC_CTYPE ¶

This variable determines the locale category for character handling functions. If the **MULTIBYTE** option is in effect this variable or **LANG** should contain a value that reflects the character set in use, even if it is a single-byte character set, unless only the 7-bit subset (ASCII) is used. For example, if the character set is ISO-8859-1, a suitable value might be **en_US.iso88591** (certain Linux distributions) or **en_US.ISO8859-1** (MacOS).

\$LC_MESSAGES ¶

This variable determines the language in which messages should be written. Note that zsh does not use message catalogs.

\$LC_NUMERIC ¶

This variable affects the decimal point character and thousands separator character for the formatted input/output functions and string conversion functions. Note that zsh ignores this setting when parsing floating point mathematical expressions.

\$LC_TIME ¶

This variable determines the locale category for date and time formatting in prompt escape sequences.

\$LINENO ¶

The line number of the current line within the current script, sourced file, or shell function being executed, whichever was started most recently. Note that in the case of shell functions the line number refers to the function as it appeared in the original definition, not necessarily as displayed by the **functions** builtin.

\$LINES ¶

The number of lines for this terminal session. Used for printing select lists and for the line editor.

\$LISTMAX ¶

In the line editor, the number of matches to list without asking first. If the value is negative, the list will be shown if it spans at most as many lines as given by the absolute value. If set to zero, the shell asks only if the top of the listing would scroll off the screen.

\$LOGCHECK ¶

The interval in seconds between checks for login/logout activity using the **watch** parameter.

\$LOGNAME ¶

If the corresponding variable is not set in the environment of the shell, it is initialized to the login name corresponding to the current login session. This parameter is exported by default but this can be disabled using the **typeset** builtin. The value is set to the string returned by the **manref(getlogin)(3)** system call if that is available.

\$MACHTYPE ¶

The machine type (microprocessor class or machine model), as determined at compile time.

\$MAIL ¶

If this parameter is set and **mailpath** is not set, the shell looks for mail in the specified file.

\$MAILCHECK ¶

The interval in seconds between checks for new mail.

\$MARK ¶

Like `CURSOR`, but for the mark. With vi-mode operators that wait for a movement command to select a region of text, setting `MARK` allows the selection to extend in both directions from the initial cursor position.

\$NULLCMD ¶

The command name to assume if a redirection is specified with no command. Defaults to `cat`. For `sh/ksh` behavior, change this to `:`. For `cs`h-like behavior, unset this parameter; the shell will print an error message if null commands are entered.

\$NUMERIC ¶

The numeric argument. If no numeric argument was given, this parameter is unset. When this is set inside a widget function, builtin widgets called with the `zle` builtin command will use the value assigned. If it is unset inside a widget function, builtin widgets called behave as if no numeric argument was given.

\$OLDPWD ¶

The previous working directory. This is set when the shell initializes and whenever the directory changes.

\$OPTARG ¶

The value of the last option argument processed by the `getopts` command.

\$OPTIND ¶

The index of the last option argument processed by the `getopts` command.

\$OSTYPE ¶

The operating system, as determined at compile time.

\$PENDING ¶

The number of bytes pending for input, i.e. the number of bytes which have already been typed and can immediately be read. On systems where the shell is not able to get this information, this parameter will always have a value of zero. Read-only. See also `KEYS_QUEUED_COUNT`; the two values are distinct.

\$POSTDISPLAY ¶

Text to be displayed after the end of the editable text buffer. This does not have to be a complete line; to display a complete line, a newline must be prepended explicitly. The text is reset on each new invocation (but not recursive invocation) of `zle`.

\$POSTEDIT ¶

This string is output whenever the line editor exits. It usually contains termcap strings to reset the terminal.

\$PPID ¶

The process ID of the parent of the shell, set when the shell initializes. As with `$$`, the value does not change in subshells created as a duplicate of the current shell.

\$PREBUFFER ¶

In a multi-line input at the secondary prompt, this read-only parameter contains the contents of the lines before the one the cursor is currently in.

\$PREDISPLAY ¶

Text to be displayed before the start of the editable text buffer. This does not have to be a complete line; to display a complete line, a newline must be appended explicitly. The text is reset on each new invocation (but not recursive invocation) of `zle`.

\$PROMPT4 ¶

Same as `P51`, `P52`, `P53` and `P54`, respectively.

\$PROMPT_EOL_MARK ¶

When the `PROMPT_CR` and `PROMPT_SP` options are set, the `PROMPT_EOL_MARK` parameter can be used to customize how the end of partial lines are shown. This parameter undergoes prompt expansion, with the `PROMPT_PERCENT` option set. If not set, the default behavior is equivalent to the value `"%B%S%#%s%b"`.

\$P51 ¶

The primary prompt string, printed before a command is read. It undergoes a special form of expansion before being displayed; see *Expansion of Prompt Sequences* (`zshmisc`). The default is `"%m%# "`.

\$P52 ¶

The secondary prompt, printed when the shell needs more information to complete a command. It is expanded in the same way as `P51`. The default is `"%_> "`, which displays any shell constructs or quotation marks which are currently being processed.

\$P53 ¶

Selection prompt used within a `select` loop. It is expanded in the same way as `P51`. The default is `"?# "`.

\$P54 ¶

The execution trace prompt. Default is `"+%N:%i> "`, which displays the name of the current shell structure and the line number within it. In `sh` or `ksh` emulation, the default is `"+ "`.

\$PWD ¶

The present working directory. This is set when the shell initializes and whenever the directory changes.

\$RANDOM ¶

A pseudo-random integer from 0 to 32767, newly generated each time this parameter is referenced. The random number generator can be seeded by assigning a numeric value to `RANDOM`.

The values of `RANDOM` form an intentionally-repeatable pseudo-random sequence; subshells that reference `RANDOM` will result in identical pseudo-random values unless the value of `RANDOM` is referenced or seeded in the parent shell in between subshell invocations.

`RANDOM` uses the system's library function `manref(rand)(3)`. If higher degree of randomness is required, please consider using the autoloadable parameter `SRANDOM` from `ifzman`(the `zsh/random` module (see `zmanref(zshmodules)`))\ `ifnzman`((The `zsh/random` Module)).

\$RBUFFER ¶

The part of the buffer that lies to the right of the cursor position. If it is assigned to, only that part of the buffer is replaced, and the cursor remains between the old `$LBUFFER` and the new `$RBUFFER`.

\$READNULLCMD ¶

The command name to assume if a single input redirection is specified with no command. Defaults to `more`.

\$REGION_ACTIVE ¶

Indicates if the region is currently active. It can be assigned 0 or 1 to deactivate and activate the region respectively. A value of 2 activates the region in line-wise mode with the highlighted text extending for whole lines only; see *Character Highlighting* (below).

\$REPLY ¶

This parameter is reserved by convention to pass string values between shell scripts and shell builtins in situations where a function call or redirection are impossible or undesirable. The `read` builtin and the `select` complex command may set `REPLY`, and filename generation both sets and examines its value when evaluating certain expressions. Some modules also employ `REPLY` for similar purposes.

\$REPORTMEMORY ¶

If nonnegative, commands whose maximum resident set size (roughly speaking, main memory usage) in kilobytes is greater than this value have timing statistics reported. The format used to output statistics is the value of the `TIMEFMT` parameter, which is the same as for the `REPORTTIME` variable and the `time` builtin; note that by default this does not output memory usage. Appending " `max RSS %M`" to the value of `TIMEFMT` causes it to output the value that triggered the report. If `REPORTTIME` is also in use, at most a single report is printed for both triggers. This feature requires the `tt(getrusage+())` system call, commonly supported by modern Unix-like systems.

\$REPORTTIME ¶

If nonnegative, commands whose combined user and system execution times (measured in seconds) are greater than this value have timing statistics printed for them. Output is suppressed for commands executed within the line editor, including completion; commands explicitly marked with the `time` keyword still cause the summary to be printed in this case.

\$RPS1 ¶

This prompt is displayed on the right-hand side of the screen when the primary prompt is being displayed on the left. This does not work if the `SINGLE_LINE_ZLE` option is set. It is expanded in the same way as `P51`.

\$RPS2 ¶

This prompt is displayed on the right-hand side of the screen when the secondary prompt is being displayed on the left. This does not work if the `SINGLE_LINE_ZLE` option is set. It is expanded in the same way as `P52`.

\$SAVEHIST ¶

The maximum number of history events to save in the history file.

If this is made local, it is not implicitly set to 0, but may be explicitly set locally.

\$SECONDS ¶

The number of seconds since shell invocation. On most platforms, this is a monotonic value, so it is not affected by NTP time jumps or other clock changes (though it may be affected by slewing). If this parameter is assigned a value, then the value returned upon reference will be the value that was assigned plus the number of seconds since the assignment.

Unlike other special parameters, the type of the `SECONDS` parameter can be changed using the `typeset` command. The type may be changed only to one of the floating point types or back to integer. For example, "typeset -F SECONDS" causes the value to be reported as a floating point number. The value is nominally available to nanosecond precision, although this varies by platform (and probably isn't accurate to 1 ns regardless), and the shell may show more or fewer digits depending on the use of `typeset`. See the documentation for the builtin `typeset` in `nmref(Shell Builtin Commands)(zshbuiltins)` for more details.

\$SHLVL ¶

Incremented by one each time a new shell is started.

\$SPROMPT ¶

The prompt used for spelling correction. The sequence "%R" expands to the string which presumably needs spelling correction, and "%r" expands to the proposed correction. All other prompt escapes are also allowed.

The actions available at the prompt are [nyae]:

- **n** + ("no"+) + (default+) — Discard the correction. If there are no more corrections, accept the command line, else (with `CORRECT_ALL`) prompt for the next.
- **y** + ("yes"+) — Make the correction. If there are no more corrections, accept the command line.
- **a** + ("abort"+) — Place the entire command line in the history for later edit, but without accepting it.
- **e** + ("edit"+) — Resume editing the command line.

\$SRANDOM ¶

A random positive 32-bit integer between 0 and 4,294,967,295. This parameter is read-only. The name was chosen for compatibility with Bash and to distinguish it from `RANDOM` which has a documented repeatable behavior.

\$STTY ¶

If this parameter is set in a command's environment, the shell runs the `stty` command with the value of this parameter as arguments in order to set up the terminal before executing the command. The modes apply only to the command, and are reset when it finishes or is suspended. If the command is suspended and continued later with the `fg` or `wait` builtins it will see the modes specified by `STTY`, as if it were not suspended. This (intentionally) does not apply if the command is continued via `kill -CONT`. `STTY` is ignored if the command is run in the background, or if it is in the environment of the shell but not explicitly assigned to in the input line. This avoids running `stty` at every external command by accidentally exporting it. Also note that `STTY` should not be used for window size specifications; these will not be local to the command.

If the parameter is set and empty, all of the above applies except that `stty` is not run. This can be useful as a way to freeze the tty around a single command, blocking its changes to tty settings, similar to the `ttctl` builtin.

\$SUFFIX_END ¶

`SUFFIX_ACTIVE` indicates whether an auto-removable completion suffix is currently active. `SUFFIX_START` and `SUFFIX_END` give the location of the suffix and are in the same units as `CURSOR`. They are only valid for reading when `SUFFIX_ACTIVE` is non-zero.

All parameters are read-only.

\$TERM ¶

The type of terminal in use. This is used when looking up termcap sequences. An assignment to `TERM` causes zsh to re-initialize the terminal, even if the value does not change (e.g., "`TERM=$TERM`"). It is necessary to make such an assignment upon any change to the terminal definition database or terminal type in order for the new settings to take effect.

\$TERMINFO ¶

A reference to your terminfo database, used by the "terminfo" library when the system has it; see `manref(terminfo)(5)`. If set, this causes the shell to reinitialise the terminal, making the workaround "`TERM=$TERM`" unnecessary.

\$TERMINFO_DIRS ¶

A colon-separated list of terminfo databases, used by the "terminfo" library when the system has it; see `manref(terminfo)(5)`. This variable is only used by certain terminal libraries, in particular ncurses; see `manref(terminfo)(5)` to check support on your system. If set, this causes the shell to reinitialise the terminal, making the workaround "`TERM=$TERM`" unnecessary. Note that unlike other colon-separated arrays this is not tied to a zsh array.

\$TIMEFMT ¶

The format of process time reports with the `time` keyword. The default is "%J %U user %S system %P cpu %*E total". Recognizes the following escape sequences, although not all may be available on all systems, and some that are available may not be useful:

- %a — A "%". - %U — CPU seconds spent in user mode. - %S — CPU seconds spent in kernel mode. - %E — Elapsed time in seconds.
- %P — The CPU percentage, computed as 100*(%U+%S)/%E.
- %W — Number of times the process was swapped.
- %X — The average amount in (shared) text space used in kilobytes.
- %D — The average amount in (unshared) data/stack space used in kilobytes.
- %K — The total space used (%X+%D) in kilobytes.
- %M — The maximum memory the process had in use at any time in kilobytes.
- %F — The number of major page faults (page needed to be brought from disk).
- %R — The number of minor page faults. - %I — The number of input operations. - %O — The number of output operations. - %r — The number of socket messages received. - %s — The number of socket messages sent. - %k — The number of signals received.
- %w — Number of voluntary context switches (waits).
- %c — Number of involuntary context switches. - %J — The name of this job.

A star may be inserted between the percent sign and flags printing time (e.g., "%*E"); this causes the time to be printed in `_hh_:_mm_:_ss_.ttt` format (hours and minutes are only printed if they are not zero). Alternatively, "m", "u", or "n" may be used (e.g., "%mE") to produce time output in milliseconds, microseconds, or nanoseconds, respectively. Note that some timings on some platforms are not actually nanosecond-precise (nor accurate to 1 ns when they are); in fact on many systems user and kernel times are not even microsecond-precise.

\$TMOUT ¶

If this parameter is nonzero, the shell will receive an `ALRM` signal if a command is not entered within the specified number of seconds after issuing a prompt. If there is a trap on `SIGALRM`, it will be executed and a new alarm is scheduled using the value of the `TMOUT` parameter after executing the trap. If no trap is set, and the idle time of the terminal is not less than the value of the `TMOUT` parameter, zsh terminates. Otherwise a new alarm is scheduled to `TMOUT` seconds after the last keypress.

\$TMPPREFIX ¶

A pathname prefix which the shell will use for all temporary files. Note that this should include an initial part for the file name as well as any directory names. The default is `/tmp/zsh`.

\$T MPSUFFIX ¶

A filename suffix which the shell will use for temporary files created by process substitutions (e.g., `tt=(*)_list_`). Note that the value should include a leading dot "." if intended to be interpreted as a file extension. The default is not to append any suffix, thus this parameter should be assigned only when needed and then unset again.

\$TRY_BLOCK_ERROR ¶

In an `always` block, indicates whether the preceding list of code caused an error. The value is 1 to indicate an error, 0 otherwise. It may be reset, clearing the error condition. See *Complex Commands* (zshmisc)

\$TRY_BLOCK_INTERRUPT ¶

This variable works in a similar way to `TRY_BLOCK_ERROR`, but represents the status of an interrupt from the signal `SIGINT`, which typically comes from the keyboard when the user types `^C`. If set to 0, any such interrupt will be reset; otherwise, the interrupt is propagated after the `always` block.

Note that it is possible that an interrupt arrives during the execution of the `always` block; this interrupt is also propagated.

`$TTY` ¶

The name of the tty associated with the shell, if any.

`$TTYIDLE` ¶

The idle time of the tty associated with the shell in seconds or -1 if there is no such tty.

`$UID` ¶

The real user ID of the shell process. If you have sufficient privileges, you may change the user ID of the shell by assigning to this parameter. Also (assuming sufficient privileges), you may start a single command under a different user ID by `tt((UID=)_uid_; command+')`

If this is made local, it is not implicitly set to 0, but may be explicitly set locally.

`$UNDO_CHANGE_NO` ¶

A number representing the state of the undo history. The only use of this is passing as an argument to the `undo` widget in order to undo back to the recorded point. Read-only.

`$UNDO_LIMIT_NO` ¶

A number corresponding to an existing change in the undo history; compare `UNDO_CHANGE_NO`. If this is set to a value greater than zero, the `undo` command will not allow the line to be undone beyond the given change number. It is still possible to use `zle undo "change"` in a widget to undo beyond that point; in that case, it will not be possible to undo at all until `UNDO_LIMIT_NO` is reduced. Set to 0 to disable the limit.

A typical use of this variable in a widget function is as follows (note the additional function scope is required):

```
example() { local UNDO_LIMIT_NO=$UNDO_CHANGE_NO # Perform some form of recursive edit. }
```

`$USERNAME` ¶

The username corresponding to the real user ID of the shell process. If you have sufficient privileges, you may change the username (and also the user ID and group ID) of the shell by assigning to this parameter. Also (assuming sufficient privileges), you may start a single command under a different username (and user ID and group ID) by `tt((USERNAME=)_username_; command+')`

`$VENDOR` ¶

The vendor, as determined at compile time.

`$WATCHFMT` ¶

The format of login/logout reports if the `watch` parameter is set. Default is `"%n has %a %l from %m"`. Recognizes the following escape sequences:

- `%n` — The name of the user that logged in/out.
- `%a` — The observed action, i.e. "logged on" or "logged off".
- `%l` — The line (tty) the user is logged in on.
- `%M` — The full hostname of the remote host.
- `%m` — The hostname up to the first ".". If only the IP address is available or the utmp field contains the name of an X-windows display, the whole name is printed.

NOTE: The `"%m"` and `"%M"` escapes will work only if there is a host name field in the utmp on your machine. Otherwise they are treated as ordinary strings.

- `%F{color} (%f)` — Start (stop) using a different foreground color.
- `%K{color} (%k)` — Start (stop) using a different background color.
- `%S (%s)` — Start (stop) standout mode.
- `%U (%u)` — Start (stop) underline mode.
- `%B (%b)` — Start (stop) boldface mode.
- `%t`
- `%@` — The time, in 12-hour, am/pm format.
- `%T` — The time, in 24-hour format.
- `%w` — The date in `_day_-"dd"` format.
- `%W` — The date in `_mm_/_dd_/"yy"` format.
- `%D` — The date in `_yy_-"_mm_-"dd"` format.
- `%D{string}` — The date formatted as *string* using the `strftime` function, with zsh extensions as described for `%D{string}` escape sequence in *Simple Prompt Escapes* (zshmisc).
- `%( )_x_:true-text:_false-text_` — Specifies a ternary expression. The character following the *x* is arbitrary; the same character is used to separate the text for the "true" result from that for the "false" result. Both the separator and the right parenthesis may be escaped with a backslash. Ternary expressions may be nested.

The test character *x* may be any one of "l", "n", "m" or "M", which indicate a "true" result if the corresponding escape sequence would return a non-empty value; or it may be "a", which indicates a "true" result if the watched user has logged in, or "false" if he has logged out. Other characters evaluate to neither true nor false; the entire expression is omitted in this case.

If the result is "true", then the *true-text* is formatted according to the rules above and printed, and the *false-text* is skipped. If "false", the *true-text* is skipped and the *false-text* is formatted and printed. Either or both of the branches may be empty, but both separators must be present in any case.

\$WIDGET ¶

The name of the widget currently being executed; read-only.

\$WIDGETFUNC ¶

The name of the shell function that implements a widget defined with either `zle -N` or `zle -C`. In the former case, this is the second argument to the `zle -N` command that defined the widget, or the first argument if there was no second argument. In the latter case this is the third argument to the `zle -C` command that defined the widget. Read-only.

\$WIDGETSTYLE ¶

Describes the implementation behind the completion widget currently being executed; the second argument that followed `zle -C` when the widget was defined. This is the name of a builtin completion widget. For widgets defined with `zle -N` this is set to the empty string. Read-only.

\$WORDCHARS ¶

A list of non-alphanumeric characters considered part of a word by the line editor.

\$YANK_END ¶

`YANK_ACTIVE` indicates whether text has just been yanked (pasted) into the buffer. `YANK_START` and `YANK_END` give the location of the pasted text and are in the same units as `CURSOR`. They are only valid for reading when `YANK_ACTIVE` is non-zero. They can also be assigned by widgets that insert text in a yank-like fashion, for example wrappers of `bracketed-paste`. See also `zle -f`.

`YANK_ACTIVE` is read-only.

`$ZBEEP` ¶

If set, this gives a string of characters, which can use all the same codes as the `bindkey` command as described in *The zsh/zle Module* (zshmodules), that will be output to the terminal instead of beeping. This may have a visible instead of an audible effect; for example, the string `"\e[?5h\e[?5l"` on a vt100 or xterm will have the effect of flashing reverse video on and off (if you usually use reverse video, you should use the string `"\e[?5l\e[?5h"` instead). This takes precedence over the `NOBEEP` option.

`$ZCURSES_COLORS` ¶

Readonly integer. The maximum number of colors the terminal supports. This value is initialised by the curses library and is not available until the first time `zcurSES init` is run.

`$ZCURSES_COLOR_PAIRS` ¶

Readonly integer. The maximum number of color pairs `_fg_col_/_bg_col_` that may be defined in "zcurSES attr" commands; note this limit applies to all color pairs that have been used whether or not they are currently active. This value is initialised by the curses library and is not available until the first time `zcurSES init` is run.

`$ZDOTDIR` ¶

The directory to search for shell startup files (`.zshrc`, etc), if not `$HOME`.

`$ZFTP_ACCOUNT` ¶

Readonly. The account name of the current user, if any. Most servers do not require an account name.

`$ZFTP_CODE` ¶

Readonly. The three digit code of the last FTP reply from the server as a string. This can still be read after the connection is closed, and is not changed when the current session changes.

`$ZFTP_COUNT` ¶

The amount of data so far transferred; a number between zero and `$ZFTP_SIZE`, if that is set. This number is always available.

`$ZFTP_FILE` ¶

The name of the remote file being transferred from or to.

`$ZFTP_HOST` ¶

Readonly. The hostname of the current remote server. If the host was opened as an IP number, `ZFTP_HOST` contains that instead; this saves the overhead for a name lookup, as IP numbers are most commonly used when a nameserver is unavailable.

`$ZFTP_IP` ¶

Readonly. The IP address of the current connection in dot notation.

`$ZFTP_PORT` ¶

Readonly. The number of the remote TCP port to which the connection is open (even if the port was originally specified as a named service). Usually this is the standard FTP port, 21.

In the unlikely event that your system does not have the appropriate conversion functions, this appears in network byte order. If your system is little-endian, the port then consists of two swapped bytes and the standard port will be reported as 5376. In that case, numeric ports passed to `zftp open` will also need to be in this format.

\$ZFTP_PREFS ¶

A string of preferences for altering aspects of `zftp`'s behaviour. Each preference is a single character. The following are defined:

- **P** — Passive: attempt to make the remote server initiate data transfers. This is slightly more efficient than sendport mode. If the letter **S** occurs later in the string, `zftp` will use sendport mode if passive mode is not available.
- **S** — Sendport: initiate transfers by the FTP **PORT** command. If this occurs before any **P** in the string, passive mode will never be attempted.
- **D** — Dumb: use only the bare minimum of FTP commands. This prevents the variables `ZFTP_SYSTEM` and `ZFTP_PWD` from being set, and will mean all connections default to ASCII type. It may prevent `ZFTP_SIZE` from being set during a transfer if the server does not send it anyway (many servers do).

If `ZFTP_PREFS` is not set when `zftp` is loaded, it will be set to a default of "PS", i.e. use passive mode if available, otherwise fall back to sendport mode.

\$ZFTP_PWD ¶

Readonly. The current directory on the server.

\$ZFTP_REPLY ¶

Readonly. The last line of the last reply sent by the server. This can still be read after the connection is closed, and is not changed when the current session changes.

\$ZFTP_SESSION ¶

Readonly. The name of the current FTP session; see the description of the `session` subcommand.

\$ZFTP_SIZE ¶

The total size of the complete file being transferred: the same as the first value provided by the `remote` and `local` subcommands for a particular file. If the server cannot supply this value for a remote file being retrieved, it will not be set. If input is from a pipe the value may be incorrect and correspond simply to a full pipe buffer.

\$ZFTP_SYSTEM ¶

Readonly. The system type string returned by the server in response to an FTP **SYST** request. The most interesting case is a string beginning "**UNIX Type: L8**", which ensures maximum compatibility with a local UNIX host.

\$ZFTP_TMOUT ¶

Integer. The time in seconds to wait for a network operation to complete before returning an error. If this is not set when the module is loaded, it will be given the default value 60. A value of zero turns off timeouts. If a timeout occurs on the control connection it will be closed. Use a larger value if this occurs too frequently.

\$ZFTP_TRANSFER ¶

A **G** for a `get` operation and a **P** for a `put` operation.

\$ZFTP_TYPE ¶

Readonly. The type to be used for data transfers, either "A" or "I". Use the `type` subcommand to change this.

\$ZFTP_USER ¶

Readonly. The username currently logged in, if any.

\$ZFTP_VERBOSE ¶

A string of digits between 0 and 5 inclusive, specifying which responses from the server should be printed. All responses go to standard error. If any of the numbers 1 to 5 appear in the string, raw responses from the server with reply codes beginning with that digit will be printed to standard error. The first digit of the three digit reply code is defined by RFC959 to correspond to:

- 1. — A positive preliminary reply.
- 2. — A positive completion reply.
- 3. — A positive intermediate reply.
- 4. — A transient negative completion reply.
- 5. — A permanent negative completion reply.

It should be noted that, for unknown reasons, the reply `Service not available`, which forces termination of a connection, is classified as 421, i.e. "transient negative", an interesting interpretation of the word "transient".

The code 0 is special: it indicates that all but the last line of multiline replies read from the server will be printed to standard error in a processed format. By convention, servers use this mechanism for sending information for the user to read. The appropriate reply code, if it matches the same response, takes priority.

If `ZFTP_VERBOSE` is not set when `zftp` is loaded, it will be set to the default value `450`, i.e., messages destined for the user and all errors will be printed. A null string is valid and specifies that no messages should be printed.

\$ZLE_LINE_ABORTED ¶

This parameter is set by the line editor when an error occurs. It contains the line that was being edited at the point of the error. "print -zr -- \$ZLE_LINE_ABORTED" can be used to recover the line. Only the most recent line of this kind is remembered.

\$ZLE_RECURSIVE ¶

Usually zero, but incremented inside any instance of `recursive-edit`. Hence indicates the current recursion level.

`ZLE_RECURSIVE` is read-only.

\$ZLE_RPROMPT_INDENT ¶

If set, used to give the indentation between the right hand side of the right prompt in the line editor as given by `RP51` or `RPROMPT` and the right hand side of the screen. If not set, the value 1 is used.

Typically this will be used to set the value to 0 so that the prompt appears flush with the right hand side of the screen. This is not the default as many terminals do not handle this correctly, in particular when the prompt appears at the extreme bottom right of the screen. Recent virtual terminals are more likely to handle this case correctly. Some experimentation is necessary.

\$ZLE_SPACE_SUFFIX_CHARS ¶

These parameters are used by the line editor. In certain circumstances suffixes (typically space or slash) added by the completion system will be removed automatically, either because the next editing command was not an insertable character, or because the character was marked as requiring the suffix to be removed.

These variables can contain the sets of characters that will cause the suffix to be removed. If `ZLE_REMOVE_SUFFIX_CHARS` is set, those characters will cause the suffix to be removed; if `ZLE_SPACE_SUFFIX_CHARS` is set, those characters will cause the suffix to be removed and replaced by a space.

If `ZLE_REMOVE_SUFFIX_CHARS` is not set, the default behaviour is equivalent to:

```
ZLE_REMOVE_SUFFIX_CHARS=$'\t\n:&|'
```

If `ZLE_REMOVE_SUFFIX_CHARS` is set but is empty, no characters have this behaviour. `ZLE_SPACE_SUFFIX_CHARS` takes precedence, so that the following:

```
ZLE_SPACE_SUFFIX_CHARS=$'&|'
```

causes the characters "&" and "|" to remove the suffix but to replace it with a space.

To illustrate the difference, suppose that the option `AUTO_REMOVE_SLASH` is in effect and the directory `DIR` has just been completed, with an appended `/`, following which the user types `"&".` The default result is `"DIR&".` With `ZLE_REMOVE_SUFFIX_CHARS` set but without including `"&"` the result is `"DIR/&".` With `ZLE_SPACE_SUFFIX_CHARS` set to include `"&"` the result is `"DIR &".`

Note that certain completions may provide their own suffix removal or replacement behaviour which overrides the values described here. See the completion system documentation in `nmref(Completion System)(zshcompsys).`

`$ZLE_STATE` ¶

Contains a set of space-separated words that describe the current `zle` state.

Currently, the states shown are the insert mode as set by the `overwrite-mode` or `vi-replace` widgets and whether history commands will visit imported entries as controlled by the `set-local-history` widget. The string contains `"insert"` if characters to be inserted on the command line move existing characters to the right or `"overwrite"` if characters to be inserted overwrite existing characters. It contains `"localhistory"` if only local history commands will be visited or `"globalhistory"` if imported history commands will also be visited.

The substrings are sorted in alphabetical order so that if you want to test for two specific substrings in a future-proof way, you can do match by doing:

```
if [[ $ZLE_STATE == *globalhistory*insert* ]]; then ...; fi
```

`$ZSH_ARGZERO` ¶

If `zsh` was invoked to run a script, this is the name of the script. Otherwise, it is the name used to invoke the current shell. This is the same as the value of `$0` when the `POSIX_ARGZERO` option is set, but is always available.

`$ZSH_EXECUTION_STRING` ¶

If the shell was started with the option `-c`, this contains the argument passed to the option. Otherwise it is not set.

`$ZSH_EXEPATH` ¶

Full pathname of the executable file of the current `zsh` process.

`$ZSH_NAME` ¶

Expands to the basename of the command used to invoke this instance of `zsh`.

`$ZSH_PATCHLEVEL` ¶

The output of `"git describe --tags --long"` for the `zsh` repository used to build the shell. This is most useful in order to keep track of versions of the shell during development between releases; hence most users should not use it and should instead rely on `$ZSH_VERSION`.

`$ZSH_SCRIPT` ¶

If `zsh` was invoked to run a script, this is the name of the script, otherwise it is unset.

`$ZSH_SUBSHELL` ¶

Readonly integer. Initially zero, incremented each time the shell forks to create a subshell for executing code. Hence `"tt((print $ZSH_SUBSHELL))"` and `"tt(print $(print $ZSH_SUBSHELL))"` output 1, while `"tt(( (print $ZSH_SUBSHELL) ))"` outputs 2.

`$ZSH_VERSION` ¶

The version number of the release of `zsh`.

`$_`

Initially, if `_` exists in the environment, then this parameter is set to its value. This value may be the full pathname of the current zsh executable or the script command file. Later, this parameter is set to the last argument of the previous command. Also, this parameter is set in the environment of every command executed to the full pathname of the command.

`$aliases`

This maps the names of the regular aliases currently enabled to their expansions.

`$argv`

Same as `*`. Assigning to `argv` changes the local positional parameters, but `argv` is *not* itself a local parameter. Deleting `argv` with `unset` in any function deletes it everywhere. This can be avoided by declaring "local +h argv" before unsetting. Even when not so declared, only the innermost positional parameter array is deleted (so `*` and `@` in other scopes are not affected).

A named reference to `argv` *does not* permit a called function to access the positional parameters of its caller.

`$builtins`

This associative array gives information about the builtin commands currently enabled. The keys are the names of the builtin commands and the values are either "undefined" for builtin commands that will automatically be loaded from a module if invoked or "defined" for builtin commands that are already loaded.

`$cdpath`

An array (colon-separated list) of directories specifying the search path for the `cd` command.

`$commands`

This array gives access to the command hash table. The keys are the names of external commands, the values are the pathnames of the files that would be executed when the command would be invoked. Setting a key in this array defines a new entry in this table in the same way as with the `hash` builtin. Unsetting a key as in `unset "commands[foo]"` **removes the entry for the given key from the command hash table.**

`$dirstack`

A normal array holding the elements of the directory stack. Note that the output of the `dirs` builtin command includes one more directory, the current working directory.

`$dis_aliases`

Like `aliases` but for disabled regular aliases.

`$dis_builtins`

Like `builtins` but for disabled builtin commands.

`$dis_functions`

Like `functions` but for disabled functions.

`$dis_functions_source`

Like `functions_source` but for disabled functions.

`$dis_galiases`

Like `galiases` but for disabled global aliases.

`$dis_patchars` ¶

Like `patchars` but for disabled pattern characters.

`$dis_reswords` ¶

Like `reswords` but for disabled reserved words.

`$dis_saliases` ¶

Like `saliases` but for disabled suffix aliases.

`$epochtime` ¶

An array value containing the number of seconds since the epoch in the first element and the remainder of the time since the epoch in nanoseconds in the second element. To ensure the two elements are consistent the array should be copied or otherwise referenced as a single substitution before the values are used. The following idiom may be used:

```
for secs nsecs in $epochtime; do ... done
```

`$errno` ¶

A readonly array of the names of errors defined on the system. These are typically macros defined in C by including the system header file `errno.h`. The index of each name (assuming the option `KSH_ARRAYS` is unset) corresponds to the error number. Error numbers *num* before the last known error which have no name are given the name `Enum` in the array.

Note that aliases for errors are not handled; only the canonical name is used.

`$fignore` ¶

An array (colon separated list) containing the suffixes of files to be ignored during filename completion. However, if completion only generates files with suffixes in this list, then these files are completed anyway.

`$fpath` ¶

An array (colon separated list) of directories specifying the search path for function definitions. This path is searched when a function with the `-u` attribute is referenced. If an executable file is found, then it is read and executed in the current environment.

`$funcfiletrace` ¶

This array contains the absolute line numbers and corresponding file names for the point where the current function, sourced file, or (if `EVAL_LINENO` is set) `eval` command was called. The array is of the same length as `funcsourcetrace` and `functrace`, but differs from `funcsourcetrace` in that the line and file are the point of call, not the point of definition, and differs from `functrace` in that all values are absolute line numbers in files, rather than relative to the start of a function, if any.

`$funcsourcetrace` ¶

This array contains the file names and line numbers of the points where the functions, sourced files, and (if `EVAL_LINENO` is set) `eval` commands currently being executed were defined. The line number is the line where the function "name" or "name tt()" started. In the case of an autoloaded function the line number is reported as zero. The format of each element is `filename:lineno`.

For functions autoloaded from a file in native zsh format, where only the body of the function occurs in the file, or for files that have been executed by the `source` or `."` builtins, the trace information is shown as `filename:0`, since the entire file is the definition. The source file name is resolved to an absolute path when the function is loaded or the path to it otherwise resolved.

Most users will be interested in the information in the `funcfiletrace` array instead.

`$funcstack` ¶

This array contains the names of the functions, sourced files, and (if `EVAL_LINENO` is set) `eval` commands. currently being executed. The first element is the name of the function using the parameter.

The standard shell array `zsh_eval_context` can be used to determine the type of shell construct being executed at each depth: note, however, that is in the opposite order, with the most recent item last, and it is more detailed, for example including an entry for `toplevel`, the main shell code being executed either interactively or from a script, which is not present in `$funcstack`.

`$functions` ¶

This associative array maps names of enabled functions to their definitions. Setting a key in it is like defining a function with the name given by the key and the body given by the value. Unsetting a key removes the definition for the function named by the key.

`$functions_source` ¶

This readonly associative array maps names of enabled functions to the name of the file containing the source of the function.

For an autoloaded function that has already been loaded, or marked for autoload with an absolute path, or that has had its path resolved with "functions -r", this is the file found for autoloading, resolved to an absolute path.

For a function defined within the body of a script or sourced file, this is the name of that file. In this case, this is the exact path originally used to that file, which may be a relative path.

For any other function, including any defined at an interactive prompt or an autoload function whose path has not yet been resolved, this is the empty string. However, the hash element is reported as defined just so long as the function is present: the keys to this hash are the same as those to `$functions`.

`$functrace` ¶

This array contains the names and line numbers of the callers corresponding to the functions currently being executed. The format of each element is *name:lineno*. Callers are also shown for sourced files; the caller is the point where the `source` or `."` command was executed.

`$galiases` ¶

Like `aliases`, but for global aliases.

`$histchars` ¶

Three characters used by the shell's history and lexical analysis mechanism. The first character signals the start of a history expansion (default `!`). The second character signals the start of a quick history substitution (default `^`). The third character is the comment character (default `#`).

The characters must be in the ASCII character set; any attempt to set `histchars` to characters with a locale-dependent meaning will be rejected with an error message.

`$history` ¶

This associative array maps history event numbers to the full history lines. Although it is presented as an associative array, the array of all values (`${history[@]}`) is guaranteed to be returned in order from most recent to oldest history event, that is, by decreasing history event number.

`$historywords` ¶

A special array containing the words stored in the history. These also appear in most to least recent order.

\$incarg ¶

This widget allows you to increment integers on the current line. In addition to decimals, it can handle hexadecimals prefixed with `0x`, binaries with `0b`, and octals with `0o`.

By default, the target integer will be incremented by one. With a numeric argument, the integer is incremented by the amount of the argument. The shell parameter `incarg` may be set to change the default increment to something other than one.

The behavior of this widget changes depending on the widget name.

When the widget is named `incarg`, the widget will increment an integer placed under the cursor placed or just to the left of it. `decarg`, on the other hand, decrements the integer. When the name is prefixed with `vi`, the cursor will jump to the nearest integer after the cursor before incrementing it. The `vi` prefix can also be combined with a `backward-` prefix to make the widget search backwards for numbers.

There's also a `sync-` prefix that can be added to the widget name. This variant is used for creating a sequence of numbers on split terminals with synchronized key input. The first pane won't increment the integer at all, but each pane after that will have the integer incremented once more than the previous pane. It currently supports tmux and iTerm2.

The prefixes `vi`, `backward-`, and `sync-` can be combined, for example, into `vim-sync-` or `vim-backward-sync-`. The `vi` prefix needs to be at the very beginning.

bindkey '^X+' incarg

\$jobdirs ¶

This associative array maps job numbers to the directories from which the job was started (which may not be the current directory of the job).

The keys of the associative arrays are usually valid job numbers, and these are the values output with, for example, `tt(${(k)jobdirs})`. Non-numeric job references may be used when looking up a value; for example, `${jobdirs[%+]}` refers to the current job.

See the `jobs` builtin for how job information is provided in a subshell.

\$jobstates ¶

This associative array gives information about the states of the jobs currently known. The keys are the job numbers and the values are strings of the form `__job-state__:__mark__:__pid__="state..."`. The *job-state* gives the state the whole job is currently in, one of "running", "suspended", or "done". The *mark* is "+" for the current job, "-" for the previous job and empty otherwise. This is followed by one `__pid__="state"` for every process in the job. The `__pid_s` are, of course, the process IDs and the *state* describes the state of that process.

Handling of the keys of the associative array is as described for `jobdirs` above.

See the `jobs` builtin for how job information is provided in a subshell.

\$jobtexts ¶

This associative array maps job numbers to the texts of the command lines that were used to start the jobs.

Handling of the keys of the associative array is as described for `jobdirs` above.

See the `jobs` builtin for how job information is provided in a subshell.

\$keymaps ¶

This array contains the names of the keymaps currently defined.

\$killring ¶

The array of previously killed items, with the most recently killed first. This gives the items that would be retrieved by a `yank-pop` in the same order. Note, however, that the most recently killed item is in `$CUTBUFFER`; `$killring` shows the array of previous entries.

The default size for the kill ring is eight, however the length may be changed by normal array operations. Any empty string in the kill ring is ignored by the `yank-pop` command, hence the size of the array effectively sets the maximum length of the kill ring, while the number of non-zero strings gives the current length, both as seen by the user at the command line.

`$langinfo` ¶

An associative array that maps langinfo elements to their values.

Your implementation may support a number of the following keys:

```
CODESET, D_T_FMT, D_FMT, T_FMT, RADIXCHAR, THOUSEP, YESEXPR, NOEXPR, CRNCYSTR, ABDAY_{1..7}, DAY_{1..7},  
ABMON_{1..12}, MON_{1..12}, T_FMT_AMPM, AM_STR, PM_STR, ERA, ERA_D_FMT, ERA_D_T_FMT, ERA_T_FMT, ALT_DIGITS
```

`$log` ¶

List all users currently logged in who are affected by the current setting of the `watch` parameter.

`$mailpath` ¶

An array (colon-separated list) of filenames to check for new mail. Each filename can be followed by a "?" and a message that will be printed. The message will undergo parameter expansion, command substitution and arithmetic expansion with the variable `$_` defined as the name of the file that has changed. The default message is "You have new mail". If an element is a directory instead of a file the shell will recursively check every file in every subdirectory of the element.

`$manpath` ¶

An array (colon-separated list) whose value is not used by the shell. The `manpath` array can be useful, however, since setting it also sets `MANPATH`, and vice versa.

`$mapfile` ¶

This associative array takes as keys the names of files; the resulting value is the content of the file. The value is treated identically to any other text coming from a parameter. The value may also be assigned to, in which case the file in question is written (whether or not it originally existed); or an element may be unset, which will delete the file in question. For example, `""varied "mapfile[myfile]" works as expected, editing the file "myfile".`

When the array is accessed as a whole, the keys are the names of files in the current directory, and the values are empty (to save a huge overhead in memory). Thus `tt(${(k)mapfile})` has the same effect as the glob operator `tt(*(D))`, since files beginning with a dot are not special. Care must be taken with expressions such as `tt(rm ${(k)mapfile})`, which will delete every file in the current directory without the usual `"rm *" test`.

The parameter `mapfile` may be made read-only; in that case, files referenced may not be written or deleted.

A file may conveniently be read into an array as one line per element with the form `_array_tt=( "${(f@)mapfile[_filename_]}")`. The double quotes and the "@" are necessary to prevent empty lines from being removed. Note that if the file ends with a newline, the shell will split on the final newline, generating an additional empty field; this can be suppressed by using `"_array_tt=( "${(f@)mapfile[_filename_]}%$'\n'" )"`.

`$module_path` ¶

An array (colon-separated list) of directories that `zmodload` searches for dynamically loadable modules. This is initialized to a standard pathname, usually `/usr/local/lib/zsh/$ZSH_VERSION`. (The `/usr/local/lib` part varies from installation to installation.) For security reasons, any value set in the environment when the shell is started will be ignored.

These parameters only exist if the installation supports dynamic module loading.

`$modules` ¶

An associative array giving information about modules. The keys are the names of the modules loaded, registered to be autoloaded, or aliased. The value says which state the named module is in and is one of the strings "loaded", "autoloaded", or alias:"name", where *name* is the name the module is aliased to.

Setting or unsetting keys in this array is not possible.

\$nameddirs ¶

This associative array maps the names of named directories to the pathnames they stand for.

\$options ¶

The keys for this associative array are the names of the options that can be set and unset using the `setopt` and `unsetopt` builtins. The value of each key is either the string `on` if the option is currently set, or the string `off` if the option is unset. Setting a key to one of these strings is like setting or unsetting the option, respectively. Unsetting a key in this array is like setting it to the value `off`.

\$parameters ¶

The keys in this associative array are the names of the parameters currently defined. The values are strings describing the type of the parameter, in the same format used by the `t` parameter flag, see `subref(Parameter Expansion Flags)(zshexpn)`. The value may also be "undefined" indicating a parameter that may be autoloaded from a module but has not yet been referenced. When the key is the name of a named reference, the value is "nameref-" prepended to the type of the referenced parameter, for example `ifzman()`

```
% typeset -n parms=parameters % print -r ${parameters[parms]} nameref-association-readonly-hide-hideval-special
```

Setting or unsetting keys in this array is not possible.

\$patchars ¶

This array contains the enabled pattern characters.

\$path ¶

An array (colon-separated list) of directories to search for commands. When this parameter is set, each directory is scanned and all files found are put in a hash table. Directories named by relative path are not scanned for hashing.

\$pipestatus ¶

An array containing the exit statuses returned by all commands in the last pipeline.

\$prompt ¶

Same as `PS1`.

\$psvar ¶

An array (colon-separated list) whose elements can be used in `PROMPT` strings. Setting `psvar` also sets `PSVAR`, and vice versa.

\$region_highlight ¶

Each element of this array may be set to a string that describes highlighting for an arbitrary region of the command line that will take effect the next time the command line is redisplayed. Highlighting of the non-editable parts of the command line in `PREDISPLAY` and `POSTDISPLAY` are possible, but note that the `P` flag is needed for character indexing to include `PREDISPLAY`.

Each string consists of the following whitespace-separated parts:

`startitemize()` `itemiz()` (Optionally, a "P" to signify that the start and end offset that follow include any string set by the `PREDISPLAY` special parameter; this is needed if the `predisplay` string itself is to be highlighted. Whitespace between the "P" and the start offset is optional.) `itemiz()` (A start offset in the same units as `CURSOR`.) `itemiz()` (An end offset in the same units as `CURSOR`.) `itemiz()` (A highlight specification in the same format as used for contexts in the parameter `zle_highlight`, see *Character Highlighting* (below); for example, `standout` or `fg=red,bold`.) `itemiz()` (Optionally, a string of the form `memo="token"`. The *token* consists of everything between the "=" and the next whitespace, comma, NUL, or the end of the string. The *token* is preserved verbatim but not parsed in any way.

Plugins may use this to identify array elements they have added: for example, a plugin might set *token* to its (the plugin's) name and then use `tt(region_highlight+=( ${region_highlight:#*memo=}_token_} +")` in order to remove array elements it added.

(This example uses the `${_name}_:_pattern_`' array-grepping syntax described in `_Parameter Expansion_ (zshexpn).)` `enditemize()`

For example,

```
example(region_highlight=("P0 20 bold memo=foobar"))
```

specifies that the first twenty characters of the text including any predisplay string should be highlighted in bold.

Note that the effect of `region_highlight` is not saved and disappears as soon as the line is accepted.

Note that zsh 5.8 and older do not support the `memo="token"` field and may misparse the third (highlight specification) field when a memo is given. COMMENT(The syntax "0 20 bold, memo=foobar" (with an auxiliary comma) happens to work on both zsh <=5.8 and zsh 5.9, but that seems to be more of an accident of implementation than something we should make a first-class-citizen API promise. It's mentioned in the "Incompatibilities" section of README.)

Where a particular region is covered by multiple entries in `region_highlight`, their effects are merged. In the case of conflicts, later entries take precedence over earlier ones. This precedence ordering can be overridden by specifying layers.

The final highlighting on the command line depends on both `region_highlight` and `zle_highlight`; see *Character Highlighting* (below) for details.

`$registers` ¶

The contents of each of the vi register buffers. These are typically set using `vi-set-buffer` followed by a delete, change or yank command.

`$reply` ¶

As `REPLY`, but for array values rather than strings.

`$reswords` ¶

This array contains the enabled reserved words.

`$aliases` ¶

Like `raliases`, but for suffix aliases.

`$signals` ¶

An array containing the names of the signals. Note that with the standard zsh numbering of array indices, where the first element has index 1, the signals are offset by 1 from the signal number used by the operating system. For example, on typical Unix-like systems `HUP` is signal number 1, but is referred to as `$signals[2]`. This is because of `EXIT` at position 1 in the array, which is used internally by zsh but is not known to the operating system. On many systems there is a block of reserved or unused signal numbers before the POSIX real-time signals so the array index can't be used as an accurate indicator of their signal number. Use, for example, `kill -l SIGRTMIN` instead.

`$status` ¶

Same as `?`.

`$sysparams` ¶

A readonly associative array. The keys are:

- `pid` — `vindex(pid, sysparams)` Returns the process ID of the current process, even in subshells. Compare `$$`, which returns the process ID of the main shell process.

- `ppid` — `vindex(ppid, sysparams)` Returns the current process ID of the parent of the current process, even in subshells. Compare `$PPID`, which returns the process ID of the initial parent of the main shell process.

- `procsubstpid` — Returns the process ID of the last process started for process substitution, i.e. the `tt(<()...)` and `tt(>()...)` expansions.

`$termcap` ¶

An associative array that maps termcap capability codes to their values.

`$terminfo` ¶

An associative array that maps terminfo capability names to their values.

`$userdirs` ¶

This associative array maps user names to the pathnames of their home directories.

`$usergroups` ¶

This associative array maps names of system groups of which the current user is a member to the corresponding group identifiers. The contents are the same as the groups output by the `id` command.

`$watch` ¶

An array (colon-separated list) of login/logout events to report.

If it contains the single word "all", then all login/logout events are reported. If it contains the single word "notme", then all events are reported as with "all" except `$USERNAME`.

An entry in this list may consist of a username, an "@" followed by a remote hostname, and a "%" followed by a line (tty). Any of these may be a pattern (be sure to quote this during the assignment to `watch` so that it does not immediately perform file generation); the setting of the `EXTENDED_GLOB` option is respected. Any or all of these components may be present in an entry; if a login/logout event matches all of them, it is reported.

For example, with the `EXTENDED_GLOB` option set, the following:

```
example(watch=('^(pws|barts)'))
```

causes reports for activity associated with any user other than `pws` or `barts`.

`$widgets` ¶

This associative array contains one entry per widget. The name of the widget is the key and the value gives information about the widget. It is either the string "builtin" for builtin widgets, a string of the form `user:"name"` for user-defined widgets, where `name` is the name of the shell function implementing the widget, a string of the form `completion:_type_:"name"` for completion widgets, or a null value if the widget is not yet fully defined. In the penultimate case, `type` is the name of the builtin widget the completion widget imitates in its behavior and `name` is the name of the shell function implementing the completion widget.

`$zcurses_attris` ¶

Readonly array. The attributes supported by `zsh/curses`; available as soon as the module is loaded.

`$zcurses_colors` ¶

Readonly array. The colors supported by `zsh/curses`; available as soon as the module is loaded.

`$zcurses_keycodes` ¶

Readonly array. The values that may be returned in the second parameter supplied to "zcur_{ses} input" in the order in which they are defined internally by curses. Not all function keys are listed, only `F0`; curses reserves space for `F0` up to `F63`.

`$zcurse$windows` ¶

Readonly array. The current list of windows, i.e. all windows that have been created with "zcurse\$addwin" and not removed with "zcurse\$delwin".

`$zle_$bracketed_$paste` ¶

This two-element array contains the terminal escape sequences for enabling and disabling the bracketed paste feature which allows ZLE to discern text that is pasted into the terminal. These escape sequences are used to enable bracketed paste when ZLE is active and disable it at other times. Unsetting the parameter has the effect of ensuring that bracketed paste remains disabled. However, see also the `.$term.$extensions` parameter which provides a single place to enable or disable terminal features.

`$zle_$cursorform` ¶

An array describing contexts in which ZLE should change the shape and color of the cursor. See `ifzman(Cursor Shape and Color in zmanref(zshzle))\ ifnzman((Cursor Shape and Color))`.

`$zle_$highlight` ¶

An array describing contexts in which ZLE should highlight the input text. See *Character Highlighting* (zshzle).

`$zsh_$eval_$context` ¶

An array (colon-separated list) indicating the context of shell code that is being run. Each time a piece of shell code that is stored within the shell is executed a string is temporarily appended to the array to indicate the type of operation that is being performed. Read in order the array gives an indication of the stack of operations being performed with the most immediate context last.

Note that the variable does not give information on syntactic context such as pipelines or subshells. Use `$ZSH_$SUBSHELL` to detect subshells.

The context is one of the following:

- `cmdarg` — Code specified by the `-c` option to the command line that invoked the shell.
- `cmdsubst` — Command substitution using of the `_..._`, `tt($+())`, `${{$name} ...}`, `${{|...}}`, or `${ { ... } }` constructs.
- `equalsubst` — The `tt(=+())` form of process substitution.
- `eval` — Code executed by the `eval` builtin.
- `evalautofunc` — Code executed with the `KSH_$AUTOLOAD` mechanism in order to define an autoloaded function.
- `fc` — Code from the shell history executed by the `-e` option to the `fc` builtin.
- `file` — Lines of code being read directly from a file, for example by the `source` builtin.
- `filecode` — Lines of code being read from a `.$zwc` file instead of directly from the source file.
- `globqual` — Code executed by the `e` or `+` glob qualifier.
- `globsort` — Code executed to order files by the `o` glob qualifier.
- `insubst` — The `tt(<())` form of process substitution.
- `loadautofunc` — Code read directly from a file to define an autoloaded function.
- `outsubst` — The `tt(>())` form of process substitution.
- `sched` — Code executed by the `sched` builtin.
- `shfunc` — A shell function.
- `stty` — Code passed to `stty` by the `STTY` environment variable. Normally this is passed directly to the system's `stty` command, so this value is unlikely to be seen in practice.
- `style` — Code executed as part of a style retrieved by the `zstyle` builtin from the `zsh/zutil` module.
- `toplevel` — The highest execution level of a script or interactive shell.

- `trap` — Code executed as a trap defined by the `trap` builtin. Traps defined as functions have the context `shfunc`. As traps are asynchronous they may have a different hierarchy from other code.
- `zpty` — Code executed by the `zpty` builtin from the `zsh/zpty` module.
- `zregexparse-guard` — Code executed as a guard by the `zregexparse` command from the `zsh/zutil` module.
- `zregexparse-action` — Code executed as an action by the `zregexparse` command from the `zsh/zutil` module.

`$zsh_scheduled_events` ¶

A readonly array corresponding to the events scheduled by the `sched` builtin. The indices of the array correspond to the numbers shown when `sched` is run with no arguments (provided that the `KSH_ARRAYS` option is not set). The value of the array consists of the scheduled time in seconds since the epoch (see *The zsh/datetime Module* (zshmodules) for facilities for using this number), followed by a colon, followed by any options (which may be empty but will be preceded by a "-" otherwise), followed by a colon, followed by the command to be executed.

The `sched` builtin should be used for manipulating the events. Note that this will have an immediate effect on the contents of the array, so that indices may become invalid.

`$zstyle` ¶

Delete style definitions. Without arguments all definitions are deleted, with a *pattern* all definitions for that pattern are deleted and if any `_style_s` are given, then only those styles are deleted for the *pattern*.

`*` ¶

Glob: match zero or more characters of any name (excluding leading `.` unless `GLOB_DOTS` is set). Also a multiplication operator inside `(( ... ))`.

`-` ¶

The command is executed with a "-" prepended to its `argv[0]` string.

`.sh.command` ¶

A named reference to "ZSH_DEBUG_CMD"

`.sh.edchar` ¶

In a ZLE widget, equivalent to the "KEYS" special parameter. In a future release, assignments to this parameter will affect the input stream seen by ZLE.

`.sh.edcol` ¶

A named reference to the ZLE special parameter "CURSOR".

`.sh.edmode` ¶

In a ZLE widget, this parameter has the value `ESC` ("{\$"\e") if the `VI` option is set and the "main" keymap is selected. This is intended for use with vi-mode key bindings ("bindkey -v"). In a future revision, assigning `$.sh.edchar=${.sh.edmode}` is expected to initiate "vicmd" mode when "viins" is active, and do nothing when "vicmd" is already active.

`.sh.edtext` ¶

A named reference to the "BUFFER" special ZLE parameter.

`.sh.file` ¶

A named reference to the "ZSH_SCRIPT" parameter.

`.sh.fun` ¶

In a shell function, the function's name. Usually the same as "\$0", but not affected by `POSIX_ARGZERO` or `FUNCTION_ARGZERO` options.

`.sh.level` ¶

In a shell function, initially set to the depth of the call stack. This may be assigned to change the apparent depth. However, such assignment does not affect the scope of local parameters.

`.sh.lineno` ¶

A named reference to the "LINENO" special parameter.

`.sh.match` ¶

An array equivalent to the "match" parameter as assigned by the `tt((#b))` extended globbing flag. When the `KSH_ARRAYS` option is set, "\${sh.match[0]}" gives the value of the "MATCH" parameter as set by the `tt((#m))` extended globbing flag. Currently, the `EXTENDED_GLOB` option must be enabled and those flags must be explicitly used in a pattern in order for these values of ".sh.match" to be set.

`.sh.name` ¶

When the "vared" command is used, this parameter may be used in user-defined ZLE widgets to get the name of the variable being edited. A future release is expected to use this parameter in the implementation of "discipline functions".

`.sh.subscript` ¶

When "vared" has been used on an array element, this parameter holds the array index (either a number, or an associative array key).

`.sh.subshell` ¶

A named reference to the "ZSH_SUBSHELL" parameter.

`.sh.value` ¶

In "vared" this is a named reference to the ZLE special "BUFFER". A future release is expected to use this parameter in the implementation of "discipline functions".

`.sh.version` ¶

A named reference to "ZSH_PATCHLEVEL".

`.term.bg` ¶

The background color of the terminal if the terminal supports the necessary interface for retrieving this information.

See also the `.term.extensions` parameter.

`.term.cursor` ¶

The default color of the cursor.

See also the `.term.extensions` parameter.

`.term.extensions` ¶

An array containing a list of extension features of the terminal. See *Terminal Extensions* (zshzle).

`.term.fg` ¶

Like `.term.bg` but for the foreground color.

`.term.id` ¶

Like the `TERM` parameter, this identifies the terminal but is obtained by querying the terminal. Many terminals name a different common terminal in `TERM` that they then emulate, often imperfectly. This allows them to work without the need to distribute updates to terminal capability databases. The more accurate identification in this parameter may be useful when configuring aspects of the shell differently for specific terminals.

See also the `.term.extensions` parameter.

`.term.mode` ¶

Either `light` or `dark`. In cases where the terminal background color is detected and set in `.term.bg`, this is set to provide a basic indication of how light that color is. This may help with configuring readable color combinations.

`.term.querywait` ¶

The time the shell waits, in hundredths of seconds, for the response to terminal queries. A value of `"0"` disables the timeout and the shell will wait indefinitely. If not set, a default of `"50"` or half a second is used.

`.term.version` ¶

The version number of the terminal application, set in conjunction with `.term.id`.

`.zle.esc` ¶

This associative array contains the literal escape sequences used to apply the highlighting for each group. An example use would be when setting the `LESS_TERMCAP_xx` environment variables for the `less` pager.

`.zle.sgr` ¶

Where highlighting makes use of CSI escape sequences, this parameter contains the "Select Graphic Rendition" number sequence. This is useful with, for example the `GREP_COLORS` and `LSCOLORS` environment variables and the `list-colors` style.

`0` ¶

The name used to invoke the current shell, or as set by the `-c` command line option upon invocation. If the `FUNCTION_ARGZERO` option is set, `$0` is set upon entry to a shell function to the name of the function, and upon entry to a sourced script to the name of the script, and reset to its previous value when the function or script returns.

`?` ¶

Glob: match exactly one character. Also the last-exit-status variable when used as `$?`.

`@` ¶

Same as `argv[@]`, even when `argv` is not set.

Compsys Function Index

52 entries · the `_arguments` / `_files` / `_describe` family of completion functions. Native Rust implementations in the `compsys` crate replace the upstream zsh shell-function versions for performance.

`_all_labels` ¶

This is a convenient interface to the `_next_label` function below, implementing the loop shown in the `_next_label` example. The `command` and its arguments are called to generate the matches. The options stored in the parameter `name` will automatically be inserted into the `_arg_s` passed to the `command`. Normally, they are put directly after the `command`, but if one of the `_arg_s` is a single hyphen, they are inserted directly before that. If the hyphen is the last argument, it will be removed from the argument list before the `command` is called. This allows `_all_labels` to be used in almost all cases where the matches can be generated by a single call to the `compadd` builtin command or by a call to one of the utility functions.

For example:

```
local expl ... if _requested foo; then ... _all_labels foo expl '...' compadd ... - $matches fi
```

Will complete the strings from the `matches` parameter, using `compadd` with additional options which will take precedence over those generated by `_all_labels`.

`_alternative` ¶

This function is useful in simple cases where multiple tags are available. Essentially it implements a loop like the one described for the `_tags` function below.

The tags to use and the action to perform if a tag is requested are described using the `_spec_s` which are of the form: `_tag_:_descr_:action`. The `_tag_s` are offered using `_tags` and if the tag is requested, the `action` is executed with the given description `descr`. The `_action_s` are those accepted by the `_arguments` function (described below), with the following exceptions: `startitemize()` `itemiz(\` The `->"state"` and `"..."` forms are not supported. `) itemiz(\` The `tt(((a\:bar b\:baz)))` form does not need the colon to be escaped, since the `_spec_s` have no colon-separated fields after the `action`. `) enditemize()`

For example, the `action` may be a simple function call:

```
_alternative \ 'users:user:_users' \ 'hosts:host:_hosts'
```

offers usernames and hostnames as possible matches, generated by the `_users` and `_hosts` functions respectively.

Like `_arguments`, this function uses `_all_labels` to execute the actions, which will loop over all sets of tags. Special handling is only required if there is an additional valid tag, for example inside a function called from `_alternative`.

The option `-O "name"` is used in the same way as by the `_arguments` function. In other words, the elements of the `name` array will be passed to `compadd` when executing an action.

Like `_tags` this function supports the `-C` option to give a different name for the argument context field.

`_approximate` ¶

This is similar to the basic `_complete` completer but allows the completions to undergo corrections. The maximum number of errors can be specified by the `max-errors` style; see *Approximate Matching* (zshexpn) for how errors are counted. Normally this completer will only be tried after the normal `_complete` completer:

```
zstyle ':completion:*' completer _complete _approximate
```

This will give correcting completion if and only if normal completion yields no possible completions. When corrected completions are found, the completer will normally start menu completion allowing you to cycle through these strings.

This completer uses the tags `corrections` and `original` when generating the possible corrections and the original string. The `format` style for the former may contain the additional sequences `%e` and `%o` which will be replaced by the number of errors accepted to generate the corrections and the original string, respectively.

The completer progressively increases the number of errors allowed up to the limit by the `max-errors` style, hence if a completion is found with one error, no completions with two errors will be shown, and so on. It modifies the completer name in the context to indicate the number of errors being tried: on the first try the completer field contains "approximate-1", on the second try "approximate-2", and so on.

When `_approximate` is called from another function, the number of errors to accept may be passed with the `-a` option. The argument is in the same format as the `max-errors` style, all in one string.

Note that this completer (and the `_correct` completer mentioned below) can be quite expensive to call, especially when a large number of errors are allowed. One way to avoid this is to set up the `completer` style using the `-e` option to `zstyle` so that some completers are only used when completion is attempted a second time on the same string, e.g.:

```
example(zstyle -e 'completion:*' completer ' if [[ $_last_try != "$HISTNO$BUFFER$CURSOR" ]]; then
_last_try="$HISTNO$BUFFER$CURSOR" reply=( _complete _match _prefix ) else reply=( _ignored _correct _approximate ) fi )
```

This uses the `HISTNO` parameter and the `BUFFER` and `CURSOR` special parameters that are available inside `zle` and completion widgets to find out if the command line hasn't changed since the last time completion was tried. Only then are the `_ignored`, `_correct` and `_approximate` completers called.

`_arguments` ¶

This function can be used to give a complete specification for completion for a command whose arguments follow standard UNIX option and argument conventions.

Options Overview

Options to `_arguments` itself must be in separate words, i.e. `-s -w`, not `-sw`. The options are followed by `_spec_s` that describe options and arguments of the analyzed command. To avoid ambiguity, all options to `_arguments` itself may be separated from the `spec` forms by a single colon.

The `-"` form is used to intuit `spec` forms from the help output of the command being analyzed, and is described in detail below. The `opts` for the `-"` form are otherwise the same options as the first form. Note that `-s` following `-"` has a distinct meaning from `-s` preceding `-"`, and both may appear.

The option switches `-s`, `-S`, `-A`, `-w`, and `-W` affect how `_arguments` parses the analyzed command line's options. These switches are useful for commands with standard argument parsing.

The options of `_arguments` have the following meanings:

- `-n` — With this option, `_arguments` sets the parameter `NORMARG` to the position of the first normal argument in the `$words` array, i.e. the position after the end of the options. If that argument has not been reached, `NORMARG` is set to `-1`. The caller should declare "integer NORMARG" if the `-n` option is passed; otherwise the parameter is not used.

- `-s` — Enable *option stacking* for single-letter options, whereby multiple single-letter options may be combined into a single word. For example, the two options `-x` and `-y` may be combined into a single word `-xy`. By default, every word corresponds to a single option name (`-xy` is a single option named `"xy"`).

Options beginning with a single hyphen or plus sign are eligible for stacking; words beginning with two hyphens are not.

Note that `-s` after `--` has a different meaning, which is documented in the segment entitled "Deriving `spec` forms from the help output".

- `-w` — In combination with `-s`, allow option stacking even if one or more of the options take arguments. For example, if `-x` takes an argument, with no `-s`, `-xy` is considered as a single (unhandled) option; with `-s`, `-xy` is an option with the argument `"y"`; with both `-s` and `-w`, `-xy` is the option `-x` and the option `-y` with arguments to `-x` (and to `-y`, if it takes arguments) still to come in subsequent words.

- `-W` — This option takes `-w` a stage further: it is possible to complete single-letter options even after an argument that occurs in the same word. However, it depends on the action performed whether options will really be completed at this point. For more control, use a utility function like `_guard` as part of the action.

- `-C` — Modify the `curcontext` parameter for an action of the form `->"state"`. This is discussed in detail below.

- `-R` — Return status 300 instead of zero when a `$state` is to be handled, in the `->"string"` syntax.

- `-S` — Do not complete options after a `-"` appearing on the line, and ignore the `-"`. For example, with `-S`, in the line

```
foobar -x -- -y
```

the `-x` is considered an option, the `-y` is considered an argument, and the `-"` is considered to be neither.

- **-A *pat*** — Do not complete options after the first non-option argument on the line. *pat* is a pattern matching all strings which are not to be taken as arguments. For example, to make `_arguments` stop completing options after the first normal argument, but ignoring all strings starting with a hyphen even if they are not described by one of the `_optspec_s`, the form is `"-A "-*"`.

- **-O *name*** — Pass the elements of the array *name* as arguments to functions called to execute `_action_s`. This is discussed in detail below.

- **-M *matchspec*** — Use the match specification *matchspec* for completing option names and values. The default *matchspec* allows partial word completion after `"_ "` and `"-"`, such as completing `"-f-b"` to `"-foo-bar"`. The default *matchspec* is:

```
r:[_]=* r:|=*
```

- **-Ø** — When populating values of the `"opt_args"` associative array, don't backslash-escape colons and backslashes and use NUL rather than colon for joining multiple values. This option is described in more detail below, under the heading `__spec_s: actions_`.

`__spec_s: overview_`

Each of the following forms is a *spec* describing individual sets of options or arguments on the command line being analyzed.

- ***n:message:action***

- ***n::message:"_action_"*** — This describes the *n*th normal argument. The *message* will be printed above the matches generated and the *action* indicates what can be completed in this position (see below). If there are two colons before the *message* the argument is optional. If the *message* contains only white space, nothing will be printed above the matches unless the action adds an explanation string itself.

- ***:message:action***

- ***::message:action*** — Similar, but describes the *next* argument, whatever number that happens to be. If all arguments are specified in this form in the correct order the numbers are unnecessary.

- ****:message:action***

- ****::message:action***

- ****:::message:action*** — This describes how arguments (usually non-option arguments, those not beginning with `-` or `+`) are to be completed when neither of the first two forms was provided. Any number of arguments can be completed in this fashion.

With two colons before the *message*, the `words` special array and the `CURRENT` special parameter are modified to refer only to the normal arguments when the *action* is executed or evaluated. With three colons before the *message* they are modified to refer only to the normal arguments covered by this description.

- ***optspec***

- ***optspec:...*** — This describes an option. The colon indicates handling for one or more arguments to the option; if it is not present, the option is assumed to take no arguments.

The following forms are available for the initial *optspec*, whether or not the option has arguments.

- ****optspec*** — Here *optspec* is one of the remaining forms below. This indicates the following *optspec* may be repeated. Otherwise if the corresponding option is already present on the command line to the left of the cursor it will not be offered again.

- ***-optname***

- ***+optname*** — In the simplest form the *optspec* is just the option name beginning with a minus or a plus sign, such as `"-foo"`. The first argument for the option (if any) must follow as a *separate* word directly after the option.

Either of `-"optname"` and `+"optname"` can be used to specify that `-optname` and `+optname` are both valid.

In all the remaining forms, the leading `"-"` may be replaced by or paired with `"+"` in this way.

- ***-optname-*** — The first argument of the option must come directly after the option name *in the same word*. For example, `-foo-:"..."` specifies that the completed option and argument will look like `-foo"arg"`.

- ***-optname+*** — The first argument may appear immediately after *optname* in the same word, or may appear as a separate word after the option. For example, `-foo+::"..."` specifies that the completed option and argument will look like either `-foo"arg"` or `-foo arg`.

- ***-optname=*** — The argument may appear as the next word, or in same word as the option name provided that it is separated from it by an equals sign, for example `-foo="arg"` or `-foo arg`.

- **`-optname=-`** — The argument to the option must appear after an equals sign in the same word, and may not be given in the next argument.

- **`optspec[explanation]`** — An explanation string may be appended to any of the preceding forms of *optspec* by enclosing it in brackets, as in `"-q[query operation]"`.

The **`verbose`** style is used to decide whether the explanation strings are displayed with the option in a completion listing.

If no bracketed explanation string is given but the **`auto-description`** style is set and only one argument is described for this *optspec*, the value of the style is displayed, with any appearance of the sequence `"%d"` in it replaced by the *message* of the first *optarg* that follows the *optspec*; see below.

It is possible for options with a literal `"+"` or `"="` to appear, but that character must be quoted, for example `"-\+"`.

Each *optarg* following an *optspec* must take one of the following forms:

- **`:message:action`**

- **`: :message:action`** — An argument to the option; *message* and *action* are treated as for ordinary arguments. In the first form, the argument is mandatory, and in the second form it is optional.

This group may be repeated for options which take multiple arguments. In other words, `:message1:action1:message2:action2` specifies that the option takes two arguments.

- **`:*pattern:message:action`**

- **`:*pattern:::message:action`**

- **`:*pattern:::message:action`** — This describes multiple arguments. Only the last *optarg* for an option taking multiple arguments may be given in this form. If the *pattern* is empty (i.e. `:*:`), all the remaining words on the line are to be completed as described by the *action*; otherwise, all the words up to and including a word matching the *pattern* are to be completed using the *action*.

Multiple colons are treated as for the `*:..."` forms for ordinary arguments: when the *message* is preceded by two colons, the **`words`** special array and the **`CURRENT`** special parameter are modified during the execution or evaluation of the *action* to refer only to the words after the option. When preceded by three colons, they are modified to refer only to the words covered by this description.

Any literal colon in an *optname*, *message*, or *action* must be preceded by a backslash, `"\"`.

Each of the forms above may be preceded by a list in parentheses of option names and argument numbers. If the given option is on the command line, the options and arguments indicated in parentheses will not be offered. For example, `"tt((-two -three 1)-one):..."` completes the option `"-one"`; if this appears on the command line, the options `-two` and `-three` and the first ordinary argument will not be completed after it. `"tt((-foo):)..."` specifies an ordinary argument completion; `-foo` will not be completed if that argument is already present.

Other items may appear in the list of excluded options to indicate various other items that should not be applied when the current specification is matched: a single star (`*`) for the rest arguments (i.e. a specification of the form `*:..."`); a colon (`:`) for all normal (non-option-) arguments; and a hyphen (`-`) for all options. For example, if `"tt((*)"` appears before an option and the option appears on the command line, the list of remaining arguments (those shown in the above table beginning with `*:..."`) will not be completed.

To aid in reuse of specifications, it is possible to precede any of the forms above with `"!"`; then the form will no longer be completed, although if the option or argument appears on the command line they will be skipped as normal. The main use for this is when the arguments are given by an array, and **`_arguments`** is called repeatedly for more specific contexts: on the first call `"_arguments $global_options"` is used, and on subsequent calls `"_arguments !${^global_options}"`.

`__spec_s:` actions_ COMMENT (If you change this section title, change the references to it in running text.)

In each of the forms above the *action* determines how completions should be generated. Except for the `->"string"` form below, the *action* will be executed by calling the **`_all_labels`** function to process all tag labels. No special handling of tags is needed unless a function call introduces a new one.

The functions called to execute **`_action_s`** will be called with the elements of the array named by the `-O" name"` option as arguments. This can be used, for example, to pass the same set of options for the **`compadd`** builtin to all **`_action_s`**.

The forms for *action* are as follows.

```
def(SP)(0)(@ )\
```

- **SP()**(single unquoted space) — This is useful where an argument is required but it is not possible or desirable to generate matches for it. The *message* will be displayed but no completions listed. Note that even in this case the colon at the end of the *message* is needed; it may only be omitted when neither a *message* nor an *action* is given.

- **()_item1_ _item2_ ..._** — One of a list of possible matches, for example:

```
example(tt(:foo:(foo bar baz)))
```

- **((_item1_\:_desc1_ ..._))** — Similar to the above, but with descriptions for each possible match. Note the backslash before the colon. For example,

```
example(tt(:foo:((a\:bar b\:baz))))
```

The matches will be listed together with their descriptions if the **description** style is set with the **values** tag in the context.

- **->string** — `vindex(context, use of)` `vindex(line, use of)` `vindex(opt_args, use of)` In this form, **_arguments** processes the arguments and options and then returns control to the calling function with parameters set to indicate the state of processing; the calling function then makes its own arrangements for generating completions. For example, functions that implement a state machine can use this type of action.

Where **_arguments** encounters *action* in the **->"string"** format, it will strip all leading and trailing whitespace from *string* and set the array **state** to the set of all **_string_s** for which an action is to be performed. The elements of the array **state_descr** are assigned the corresponding *message* field from each *optarg* containing such an *action*.

By default and in common with all other well behaved completion functions, **_arguments** returns status zero if it was able to add matches and non-zero otherwise. However, if the **-R** option is given, **_arguments** will instead return a status of 300 to indicate that **\$state** is to be handled.

In addition to **\$state** and **\$state_descr**, **_arguments** also sets the global parameters "context", "line" and "opt_args" as described below, and does not reset any changes made to the special parameters such as **PREFIX** and **words**. This gives the calling function the choice of resetting these parameters or propagating changes in them.

A function calling **_arguments** with at least one action containing a **->"string"** must therefore declare appropriate local parameters:

```
local context state state_descr line typeset -A opt_args
```

to prevent **_arguments** from altering the global environment.

- **{eval-string}** — `vindex(expl, use of)` A string in braces is evaluated as shell code to generate matches. If the *eval-string* itself does not begin with an opening parenthesis or brace it is split into separate words before execution.

- **= action** — If the *action* starts with "**=**" (an equals sign followed by a space), **_arguments** will insert the contents of the *argument* field of the current context as the new first element in the **words** special array and increment the value of the **CURRENT** special parameter. This has the effect of inserting a dummy word onto the completion command line while not changing the point at which completion is taking place.

This is most useful with one of the specifiers that restrict the words on the command line on which the *action* is to operate (the two- and three-colon forms above). One particular use is when an *action* itself causes **_arguments** on a restricted range; it is necessary to use this trick to insert an appropriate command name into the range for the second call to **_arguments** to be able to parse the line.

- **var(SP()word...)**

- **word...** — This covers all forms other than those above. If the *action* starts with a space, the remaining list of words will be invoked unchanged.

Otherwise it will be invoked with some extra strings placed after the first word; these are to be passed down as options to the **compadd** builtin. They ensure that the state specified by **_arguments**, in particular the descriptions of options and arguments, is correctly passed to the completion command. These additional arguments are taken from the array parameter "expl"; this will be set up before executing the *action* and hence may be referred to inside it, typically in an expansion of the form "\$expl[@]" which preserves empty elements of the array.

During the performance of the action the array "line" will be set to the normal arguments from the command line, i.e. the words from the command line after the command name excluding all options and their arguments. Options are stored in the associative array "opt_args" with option names as keys and their arguments as the values. By default, all colons and backslashes in the value are escaped with backslashes, and if an option has multiple arguments (for example, when using an *optspec* of the form ****optspec**), they are joined with (unescaped) colons. However, if the **-0** option was passed, no backslash escaping is performed, and multiple values are joined with NUL bytes. For example, after `zsh -o foo:foo -o bar:bar -o <TAB>`, the contents of "opt_args" would be

```
example(typeset -A opt_args=( [-o]='foo\foo\bar\bar:') )
```

by default, and

```
example(typeset -A opt_args=( [-o]=$foo:foo\x00bar:bar\x00' ) )
```

if `_arguments` had been called with the `-0` option.

The parameter "context" is set when returning to the calling function to perform an action of the form `->"string"`. It is set to an array of elements corresponding to the elements of `$state`. Each element is a suitable name for the argument field of the context: either a string of the form `option_-opt_"-n_"` for the `n`th argument of the option `_opt_`, or a string of the form `"argument-"n` for the `n`th argument. For "rest" arguments, that is those in the list at the end not handled by position, `n` is the string "rest". For example, when completing the argument of the `-o` option, the name is `"option-o-1"`, while for the second normal (non-option-) argument it is `"argument-2"`.

Furthermore, during the evaluation of the *action* the context name in the `curcontext` parameter is altered to append the same string that is stored in the `context` parameter.

The option `-C` tells `_arguments` to modify the `curcontext` parameter for an action of the form `->"state"`. This is the standard parameter used to keep track of the current context. Here it (and not the `context` array) should be made local to the calling function to avoid passing back the modified value and should be initialised to the current value at the start of the function:

```
local curcontext="$curcontext"
```

This is useful where it is not possible for multiple states to be valid together.

Grouping Options

Options can be grouped to simplify exclusion lists. A group is introduced with `+` followed by a name for the group in the subsequent word. Whole groups can then be referenced in an exclusion list or a group name can be used to disambiguate between two forms of the same option. For example:

```
example(_arguments \ '(group2--x)-a' \ + group1 \ -m \ '(group2)-n' \ + group2 \ -x -y)
```

If the name of a group is specified in the form `tt{()_name_}"` then only one value from that group will ever be completed; more formally, all specifications are mutually exclusive to all other specifications in that group. This is useful for defining options that are aliases for each other. For example:

```
example(_arguments \ -a -b \ + '(operation)' \ {-c,--compress}[compress] \ {-d,--decompress}[decompress] \ {-l,--list}[list])
```

If an option in a group appears on the command line, it is stored in the associative array `"opt_args"` with `'group-_"option_"` as a key. In the example above, a key `"operation--c"` is used if the option `"-c"` is present on the command line.

Specifying Multiple Sets of Arguments

It is possible to specify multiple sets of options and arguments with the sets separated by single hyphens. This differs from groups in that sets are considered to be mutually exclusive of each other.

Specifications before the first set and from any group are common to all sets. For example:

```
example(_arguments \ -a \ - set1 \ -c \ - set2 \ -d \ ':arg:(x2 y2)')
```

This defines two sets. When the command line contains the option `"-c"`, the `"-d"` option and the argument will not be considered possible completions. When it contains `"-d"` or an argument, the option `"-c"` will not be considered. However, after `"-a"` both sets will still be considered valid.

As for groups, the name of a set may appear in exclusion lists, either alone or preceding a normal option or argument specification.

The completion code has to parse the command line separately for each set. This can be slow so sets should only be used when necessary. A useful alternative is often an option specification with rest-arguments (as in `"-foo:*:..."`); here the option `-foo` swallows up all remaining arguments as described by the *optarg* definitions.

Deriving spec forms from the help output

The option `-h` allows `_arguments` to work out the names of long options that support the `-h` option which is standard in many GNU commands. The command word is called with the argument `-h` and the output examined for option names. Clearly, it can be dangerous to pass this to commands which may not support this option as the behaviour of the command is unspecified.

In addition to options, `_arguments -h` will try to deduce the types of arguments available for options when the form `--_opt_="val"` is valid. It is also possible to provide hints by examining the help text of the command and adding *helpspec* of the form

`_pattern_:_message_:"action"`; note that other `_arguments spec` forms are not used. The *pattern* is matched against the help text for an option, and if it matches the *message* and *action* are used as for other argument specifiers. The special case of `"*:"` means both *message* and *action* are empty, which has the effect of causing options having no description in the help output to be ordered in listings ahead of options that have a description.

For example:

```
example(_arguments -- '*:*:toggle:(yes no)' \
  '*=FILE*:file:_files' \
  '*=DIR*:directory:_files -/' \
  '*=PATH*:directory:_files -/')
```

Here, "yes" and "no" will be completed as the argument of options whose description ends in a star; file names will be completed for options that contain the substring "FILE" in the description; and directories will be completed for options whose description contains "DIR" or "PATH". The last three are in fact the default and so need not be given explicitly, although it is possible to override the use of these patterns. A typical help text which uses this feature is:

```
-C, --directory=DIR change to directory DIR
```

so that the above specifications will cause directories to be completed after `--directory`, though not after `-C`.

Note also that `_arguments` tries to find out automatically if the argument for an option is optional. This can be specified explicitly by doubling the colon before the *message*.

If the *pattern* ends in `"tt((-))"`, this will be removed from the pattern and the *action* will be used only directly after the `"="`, not in the next word. This is the behaviour of a normal specification defined with the form `"=-"`.

By default, the command (with the option `--help`) is run after resetting all the locale categories (except for `LC_CTYPE`) to `"C"`. If the localized help output is known to work, the option `-l` can be specified after the `_arguments -"` so that the command is run in the current locale.

The `_arguments -"` can be followed by the option `-i _patterns_'` to give patterns for options which are not to be completed. The patterns can be given as the name of an array parameter or as a literal list in parentheses. For example,

```
example(_arguments -- -i \ "(-(en|dis)able-FEATURE*)")
```

will cause completion to ignore the options `--enable-FEATURE` and `--disable-FEATURE` (this example is useful with GNU `configure`).

The `_arguments -"` form can also be followed by the option `-s _pair_'` to describe option aliases. The `_pair_` consists of a list of alternating patterns and corresponding replacements, enclosed in parens and quoted so that it forms a single argument word in the `_arguments` call.

For example, some `configure`-script help output describes options only as `--enable-foo`, but the script also accepts the negated form `--disable-foo`. To allow completion of the second form:

```
example(_arguments -- -s "((#s)--enable- --disable-)"
```

Miscellaneous notes

Finally, note that `_arguments` generally expects to be the primary function handling any completion for which it is used. It may have side effects which change the treatment of any matches added by other functions called after it. To combine `_arguments` with other functions, those functions should be called either before `_arguments`, as an *action* within a *spec*, or in handlers for `->"state"` actions.

Here is a more general example of the use of `_arguments`:

```
example(_arguments '-l+:left border:' \
  '-format:paper size:(letter A4)' \
  '*-copy:output file:_files::resolution:(300 600)' \
  ':postscript file:_files -g \*.\\(ps\\eps\\)' \
  '*:page number:')
```

This describes three options: `-l`, `-format`, and `-copy`. The first takes one argument described as `_left border_'` for which no completion will be offered because of the empty action. Its argument may come directly after the `-l` or it may be given as the next word on the line.

The `-format` option takes one argument in the next word, described as *paper size* for which only the strings "letter" and "A4" will be completed.

The `-copy` option may appear more than once on the command line and takes two arguments. The first is mandatory and will be completed as a filename. The second is optional (because of the second colon before the description *resolution*) and will be completed from the strings "300" and "600".

The last two descriptions say what should be completed as arguments. The first describes the first argument as a *postscript file* and makes files ending in "ps" or "eps" be completed. The last description gives all other arguments the description *page number* but does not offer completions.

`_call_program` ¶

This function provides a mechanism for the user to override the use of an external command. It looks up the `command` style with the supplied *tag*. If the style is set, its value is used as the command to execute. The `_string_s` from the call to `_call_program`, or from the style if set, are concatenated with spaces between them and the resulting string is evaluated. The return status is the return status of the command called.

By default, the command is run in an environment where all the locale categories (except for `LC_CTYPE`) are reset to "C" by calling the utility function `_comp_locale` (see below). If the option "-l" is given, the command is run with the current locale.

If the option "-p" is supplied it indicates that the command output is influenced by the permissions it is run with. If the `gain-privileges` style is set to true, `_call_program` will make use of commands such as `sudo`, if present on the command-line, to match the permissions to whatever the final command is likely to run under. When looking up the `gain-privileges` and `command` styles, the command component of the `zstyle` context will end with a slash ("/") followed by the command that would be used to gain privileges.

`_canonical_paths` ¶

This completion function completes all paths given to it, and also tries to offer completions which point to the same file as one of the paths given (relative path when an absolute path is given, and vice versa; when `..`'s are present in the word to be completed; and some paths got from symlinks).

`-A`, if specified, takes the paths from the array variable specified. Paths can also be specified on the command line as shown above. `-N`, if specified, prevents canonicalizing the paths given before using them for completion, in case they are already so. The options `-M`, `-J`, `-V`, `-1`, `-2`, `-n`, `-F`, `-X` are passed to `compadd`.

See `_description` for a description of *tag* and *descr*.

`_cargo` ¶

Completion for the Rust `cargo` command — subcommands, flags, target names, feature names, profile names. Synthesizes from `cargo --list` and the manifest. Rust-native; no shell-script fallback.

`_cat` ¶

Completion for `cat` — file paths only. No subcommands; flags pass through to `_files`.

`_cd` ¶

Completion for `cd` — directory paths from `$PWD`, `$cdpath`, and the `dirs` stack. Honors `AUTO_CD` and `CDABLE_VARS`.

`_combination` ¶

This function is used to complete combinations of values, for example pairs of hostnames and usernames. The *style* argument gives the style which defines the pairs; it is looked up in a context with the *tag* specified.

The style name consists of field names separated by hyphens, for example "users-hosts-ports". For each field for a value is already known, a *spec* of the form `_field_="pattern"` is given. For example, if the command line so far specifies a user "pws", the argument "users=pws" should appear.

The next argument with no equals sign is taken as the name of the field for which completions should be generated (presumably not one of the `_field_s` for which the value is known).

The matches generated will be taken from the value of the style. These should contain the possible values for the combinations in the appropriate order (users, hosts, ports in the example above). The values for the different fields are separated by colons. This can be altered with the option `-s` to `_combination` which specifies a pattern. Typically this is a character class, as for example `-s "[:@]"` in the case of the `users-hosts` style. Each `_field_="pattern"` specification restricts the completions which apply to elements of the style with appropriately matching fields.

If no style with the given name is defined for the given tag, or if none of the strings in style's value match, but a function name of the required field preceded by an underscore is defined, that function will be called to generate the matches. For example, if there is no "users-hosts-ports" or no matching hostname when a host is required, the function `"_hosts"` will automatically be called.

If the same name is used for more than one field, in both the `_field_="pattern"` and the argument that gives the name of the field to be completed, the number of the field (starting with one) may be given after the fieldname, separated from it by a colon.

All arguments after the required field name are passed to `compadd` when generating matches from the style value, or to the functions for the fields if they are called.

`_command_names` ¶

This function completes words that are valid at command position: names of aliases, builtins, hashed commands, functions, and so on. If the `-e` flag is given, or if the list of precommands contains a non-builtin command (e.g. because `_normal -p` has been used previously), only hashed commands are completed. The `-` flag is ignored.

`_complete` ¶

This completer generates all possible completions in a context-sensitive manner, i.e. using the settings defined with the `compdef` function explained above and the current settings of all special parameters. This gives the normal completion behaviour.

To complete arguments of commands, `_complete` uses the utility function `_normal`, which is in turn responsible for finding the particular function; it is described below. Various contexts of the form `-context-` are handled specifically. These are all mentioned above as possible arguments to the `#compdef` tag.

Before trying to find a function for a specific context, `_complete` checks if the parameter "compcontext" is set. Setting "compcontext" allows the usual completion dispatching to be overridden which is useful in places such as a function that uses `vared` for input. If it is set to an array, the elements are taken to be the possible matches which will be completed using the tag "values" and the description "value". If it is set to an associative array, the keys are used as the possible completions and the values (if non-empty) are used as descriptions for the matches. If "compcontext" is set to a string containing colons, it should be of the form `_tag_:_descr_:action`. In this case the *tag* and *descr* give the tag and description to use and the *action* indicates what should be completed in one of the forms accepted by the `_arguments` utility function described below.

Finally, if "compcontext" is set to a string without colons, the value is taken as the name of the context to use and the function defined for that context will be called. For this purpose, there is a special context named `-command-line-` that completes whole command lines (commands and their arguments). This is not used by the completion system itself but is nonetheless handled when explicitly called.

`_completers` ¶

This function completes names of completers.

`-p` — Include the leading underscore ("`_`") in the matches.

`_correct` ¶

Generate corrections, but not completions, for the current word; this is similar to `_approximate` but will not allow any number of extra characters at the cursor as that completer does. The effect is similar to spell-checking. It is based on `_approximate`, but the completer field in the context name is `correct`.

For example, with:

```
zstyle ':completion:::::' completer \_complete \_correct \_approximate
zstyle ':completion*:correct::::' max-errors 2 not-numeric
zstyle ':completion*:approximate::::' max-errors 3 numeric
```

correction will accept up to two errors. If a numeric argument is given, correction will not be performed, but correcting completion will be, and will accept as many errors as given by the numeric argument. Without a numeric argument, first correction and then correcting completion will be tried, with the first one accepting two errors and the second one accepting three errors.

When `_correct` is called as a function, the number of errors to accept may be given following the `-a` option. The argument is in the same form a values to the `accept` style, all in one string.

This completer function is intended to be used without the `_approximate` completer or, as in the example, just before it. Using it after the `_approximate` completer is useless since `_approximate` will at least generate the corrected strings generated by the `_correct` completer `DASH()`- and probably more.

`_cp` ¶

Completion for `cp` — flags + file paths. Source paths exclude the destination; destination directory is offered as the final candidate.

`_describe` ¶

This function associates completions with descriptions. Multiple groups separated by `--` can be supplied, potentially with different completion options `_opt_s`.

The `descr` is taken as a string to display above the matches if the `format` style for the `descriptions` tag is set. This is followed by one or two names of arrays followed by options to pass to `compadd`. The array `name1` contains the possible completions with their descriptions in the form `_completion_:"description"`. Any literal colons in `completion` must be quoted with a backslash. If a `name2` is given, it should have the same number of elements as `name1`; in this case the corresponding elements are added as possible completions instead of the `completion` strings from `name1`. The completion list will retain the descriptions from `name1`. Finally, a set of completion options can appear.

If the option `"-o"` appears before the first argument, the matches added will be treated as names of command options (N.B. not shell options), typically following a `"-"`, `"--"` or `"+"` on the command line. In this case `_describe` uses the `prefix-hidden`, `prefix-needed` and `verbose` styles to find out if the strings should be added as completions and if the descriptions should be shown. Without the `"-o"` option, only the `verbose` style is used to decide how descriptions are shown. If `"-O"` is used instead of `"-o"`, command options are completed as above but `_describe` will not handle the `prefix-needed` style.

With the `-t` option a tag can be specified. The default is `"values"` or, if the `-o` option is given, `"options"`.

The options `-1`, `-2`, `-J`, `-V`, `-x` are passed to `_next_label`.

If selected by the `list-grouped` style, strings with the same description will appear together in the list.

`_describe` uses the `_all_labels` function to generate the matches, so it does not need to appear inside a loop over tag labels.

`_description` ¶

This function is not to be confused with the previous one; it is used as a helper function for creating options to `compadd`. It is buried inside many of the higher level completion functions and so often does not need to be called directly.

The styles listed below are tested in the current context using the given `tag`. The resulting options for `compadd` are put into the array named `name` (this is traditionally `"expl"`, but this convention is not enforced). The description for the corresponding set of matches is passed to the function in `descr`.

The styles tested are: `format`, `hidden`, `matcher`, `ignore-line`, `ignored-patterns`, `group-name` and `sort`. The `format` style is first tested for the given `tag` and then for the `descriptions` tag if no value was found, while the remainder are only tested for the tag given as the first argument. The function also calls `_setup` which tests some more styles.

The string returned by the `format` style (if any) will be modified so that the sequence `"%d"` is replaced by the `descr` given as the third argument without any leading or trailing white space. If, after removing the white space, the `descr` is the empty string, the `format` style will not be used and the options put into the `name` array will not contain an explanation string to be displayed above the matches.

If `_description` is called with more than three arguments, the additional `_spec_s` should be of the form `_char_:"str"`. These supply escape sequence replacements for the `format` style: every appearance of `%"char"` will be replaced by `string`. If no additional `_spec_s` are given but the description in `descr` conforms to a common form then further escape sequences are set for elements of that description. These elements correspond to a default value (`"%o"`), the units (`"%m"`) range of acceptable values (`"%r"`) and the remaining initial part of the description (`"%h"`). The form the description takes consists of specifying the units and range in parentheses and the default value in square brackets, for example:

```
example(_description times expl 'timeout (seconds) (0-60) [20]')
```

It is possible to use `zformat` conditional expressions when styling these elements. So, for example, to add `"default:"` as a tag but only when there is a default value to show, the `format` style might include `"tt(%o.default: %o.)"`.

If the `-x` option is given, the description will be passed to `compadd` using the `-x` option instead of the default `-X`. This means that the description will be displayed even if there are no corresponding matches.

The options placed in the array `name` take account of the `group-name` style, so matches are placed in a separate group where necessary. The group normally has its elements sorted (by passing the option `-J` to `compadd`), but if an option starting

with "-V", "-J", "-1", or "-2" is passed to `_description`, that option will be included in the array. Hence it is possible for the completion group to be unsorted by giving the option "-V", "-1V", or "-2V".

In most cases, the function will be used like this:

```
local expl _description files expl file compadd "$expl[@]" - "$files[@]"
```

Note the use of the parameter `expl`, the hyphen, and the list of matches. Almost all calls to `compadd` within the completion system use a similar format; this ensures that user-specified styles are correctly passed down to the builtins which implement the internals of completion.

`_dir_list` ¶

Complete a list of directory names separated by colons (the same format as `$PATH`).

- `-s sep` — Use `sep` as separator between items. `sep` defaults to a colon (":").
- `-S` — Add `sep` instead of slash ("/") as an autoremoveable suffix.

`_directories` ¶

Complete directory names only. Equivalent to `_files -/`. Honors `path-files` zstyle and respects `GLOB_DOT5`. Rust impl in `crate::compsys::files::directories_execute`.

`_dispatch` ¶

This sets the current context to `context` and looks for completion functions to handle this context by hunting through the list of command names or special contexts (as described above for `compdef`) given as `_string_s`. The first completion function to be defined for one of the contexts in the list is used to generate matches. Typically, the last `string` is `-default-` to cause the function for default completion to be used as a fallback.

The function sets the parameter `$service` to the `string` being tried, and sets the `context/command` field (the fourth) of the `$curcontext` parameter to the `context` given as the first argument.

`_docker` ¶

Completion for the `docker` CLI — subcommands, image names, container names/IDs, network names, volume names. Queries the local daemon socket via the `docker` binary; falls back to static-only when the daemon is unavailable.

`_email_addresses` ¶

Complete email addresses. Addresses are provided by plugins.

- `-c` — Complete bare `localhost@domain.tld` addresses, without a name part or a comment. Without this option, RFC822 `_Firstname Lastname_ <_address_>` strings are completed.
- `-n plugin` — Complete aliases from `plugin`.

COMMENT(Intentionally leaving `-s` undocumented: new code should use `_sequence` instead.)\

The following plugins are available by default: `_email-ldap` (see the `filter` style), `_email-local` (completes `user@hostname` Unix addresses), `_email-mail` (completes aliases from `~/.mailrc`), `_email-mush`, `_email-mutt`, and `_email-pine`.

Addresses from the `_email-foo` plugin are added under the tag `email-"foo"`.

Writing plugins

Plugins are written as separate functions with names starting with `"_email-"`. They are invoked with the `-c` option and `compadd` options. They should either do their own completion or set the `$reply` array to a list of `_alias_:"address"` elements and return `300`. New plugins will be picked up and run automatically.

`_expand` ¶

This completer function does not really perform completion, but instead checks if the word on the command line is eligible for expansion and, if it is, gives detailed control over how this expansion is done. For this to happen, the completion system needs to be invoked with `complete-word`, not `expand-or-complete` (the default binding for `TAB`), as otherwise the string will be expanded by the shell's internal mechanism before the completion system is started. Note also this completer should be called before the `_complete` completer function.

The tags used when generating expansions are `all-expansions` for the string containing all possible expansions, `expansions` when adding the possible expansions as single matches and `original` when adding the original string from the line. The order in which these strings are generated, if at all, can be controlled by the `group-order` and `tag-order` styles, as usual.

The format string for `all-expansions` and for `expansions` may contain the sequence `"%o"` which will be replaced by the original string from the line.

The kind of expansion to be tried is controlled by the `substitute`, `glob` and `subst-globs-only` styles.

It is also possible to call `_expand` as a function, in which case the different modes may be selected with options: `-s` for `substitute`, `-g` for `glob` and `-o` for `subst-globs-only`.

`_files` ¶

The function `_files` is a wrapper around `_path_files`. It supports all of the same functionality, with some enhancements DASH()- notably, it respects the `list-dirs-first` style, and it allows users to override the behaviour of the `-g` and `-/` options with the `file-patterns` style. `_files` should therefore be preferred over `_path_files` in most cases.

This function accepts the full set of options allowed by `_path_files`, described below.

`_git` ¶

Completion for `git` — subcommands, branches, tags, refs, remotes, file paths sensitive to `git status`. The most heavily-used compsys function in practice; Rust-native rewrite is several hundred times faster than the upstream shell implementation.

`_grep` ¶

Completion for `grep` (GNU/BSD-flavor-aware) — flags then file paths. First positional argument is the pattern (no completion offered for free-text patterns).

`_history` ¶

Complete words from the shell's command history. This completer can be controlled by the `remove-all-dups`, and `sort` styles as for the `_history_complete_word` bindable command, see *Bindable Commands* (below) and *Completion System Configuration* (above).

`_ignored` ¶

The `ignored-patterns` style can be set to a list of patterns which are compared against possible completions; matching ones are removed. With this completer those matches can be reinstated, as if no `ignored-patterns` style were set. The completer actually generates its own list of matches; which completers are invoked is determined in the same way as for the `_prefix` completer. The `single-ignored` style is also available as described above.

`_kubectl` ¶

Completion for `kubectl` — subcommands, resource kinds, resource names (queried via `kubectl get`), context/namespace names from kubeconfig.

`_ls` ¶

Completion for `ls` — flags + file paths. Baseline stub that delegates path completion to `_files` and option completion to a static spec.

`_main_complete` ¶

Top-level entry the compsys dispatcher calls for every completion attempt. Walks the configured completer list (`_complete` / `_approximate` / `_match` / ...), invoking each until one returns matches. Sets `$compstate[insert]` based on the result. Rust impl in `crate::compsys::ported::_main_complete::_main_complete`.

`_match` ¶

This completer is intended to be used after the `_complete` completer. It behaves similarly but the string on the command line may be a pattern to match against trial completions. This gives the effect of the `GLOB_COMPLETE` option.

Normally completion will be performed by taking the pattern from the line, inserting a "*" at the cursor position and comparing the resulting pattern with the possible completions generated. This can be modified with the `match-original` style described above.

The generated matches will be offered in a menu completion unless the `insert-unambiguous` style is set to "true"; see the description above for other options for this style.

Note that matcher specifications defined globally or used by the completion functions (the styles `matcher-list` and `matcher`) will not be used.

`_menu` ¶

This completer was written as simple example function to show how menu completion can be enabled in shell code. However, it has the notable effect of disabling menu selection which can be useful with `_generic` based widgets. It should be used as the first completer in the list. Note that this is independent of the setting of the `MENU_COMPLETE` option and does not work with the other menu completion widgets such as `reverse-menu-complete`, or `accept-and-menu-complete`.

`_message` ¶

The `descr` is used in the same way as the third argument to the `_description` function, except that the resulting string will always be shown whether or not matches were generated. This is useful for displaying a help message in places where no completions can be generated.

The `format` style is examined with the `messages` tag to find a message; the usual tag, `descriptions`, is used only if the style is not set with the former.

If the `-r` option is given, no style is used; the `descr` is taken literally as the string to display. This is most useful when the `descr` comes from a pre-processed argument list which already contains an expanded description. Note that this option does not disable the "%" -sequence parsing done by `compadd`.

The `-12VJ` options and the `group` are passed to `compadd` and hence determine the group the message string is added to.

The second `-e` form gives a description for completions with the tag `tag` to be shown even if there are no matches for that tag. This form is called by `_arguments` in the event that there is no action for an option specification. The tag can be omitted and if so the tag is taken from the parameter `$curtag`; this is maintained by the completion system and so is usually correct. Note that if there are no matches at the time this function is called, `compstate[insert]` is cleared, so additional matches generated later are not inserted on the command line.

`_multi_parts` ¶

The argument `sep` is a separator character. The `array` may be either the name of an array parameter or a literal array in the form `tt((foo bar))`, a parenthesised list of words separated by whitespace. The possible completions are the strings from the array. However, each chunk delimited by `sep` will be completed separately. For example, the `_tar` function uses `_multi_parts /" patharray"` to complete partial file paths from the given array of complete file paths.

The `-i` option causes `_multi_parts` to insert a unique match even if that requires multiple separators to be inserted. This is not usually the expected behaviour with filenames, but certain other types of completion, for example those with a fixed set of possibilities, may be more suited to this form.

Like other utility functions, this function accepts the `-V`, `-J`, `-1`, `-2`, `-n`, `-f`, `-X`, `-M`, `-P`, `-S`, `-r`, `-R`, and `-q` options and passes them to the `compadd` builtin.

`_mv` ¶

Completion for `mv` — flags + file/directory paths. Source/destination split identical to `_cp`.

`_next_label` ¶

This function is used to implement the loop over different tag labels for a particular tag as described above for the `tag-order` style. On each call it checks to see if there are any more tag labels; if there is it returns status zero, otherwise non-zero. As this function requires a current tag to be set, it must always follow a call to `_tags` or `_requested`.

The `-x12VJ` options and the first three arguments are passed to the `_description` function. Where appropriate the *tag* will be replaced by a tag label in this call. Any description given in the `tag-order` style is preferred to the *descr* passed to `_next_label`.

The `_option_s` given after the *descr* are set in the parameter given by *name*, and hence are to be passed to `compadd` or whatever function is called to add the matches.

Here is a typical use of this function for the tag `foo`. The call to `_requested` determines if tag `foo` is required at all; the loop over `_next_label` handles any labels defined for the tag in the `tag-order` style.

```
local expl ret=1 ... if _requested foo; then ... while _next_label foo expl '...'; do compadd "$expl[@]" ... && ret=0 done ... fi
return ret
```

`_normal` ¶

This is the standard function called to handle completion outside any special `-context-`. It is called both to complete the command word and also the arguments for a command. In the second case, `_normal` looks for a special completion for that command, and if there is none it uses the completion for the `-default-` context.

A second use is to reexamine the command line specified by the `$words` array and the `$CURRENT` parameter after those have been modified. For example, the function `_precommand`, which completes after precommand specifiers such as `nohup`, removes the first word from the `words` array, decrements the `CURRENT` parameter, then calls `"_normal -p $service"`. The effect is that `nohup "cmd ..."` is treated in the same way as `"cmd ..."`.

- **-P** — Reset the list of precommands. This option should be used if completing a command line which allows internal commands (e.g. builtins and functions) regardless of prior precommands (e.g. `"zsh -c"`).

- **-p precommand** — Append *precommand* to the list of precommands. This option should be used in nearly all cases in which **-P** is not applicable. An advantage of using this option is that it can automatically signal to `_command_names` that subsequent completion should be limited to hashed commands, which is usually the desired behaviour following commands like `chroot` and `nohup`.

If the command name matches one of the patterns given by one of the options **-p** or **-P** to `compdef`, the corresponding completion function is called and then the parameter `_compskip` is checked. If it is set completion is terminated at that point even if no matches have been found. This is the same effect as in the `-first-` context.

`_numbers` ¶

This can be used where a number is followed by a suffix to indicate the units. The unit suffixes are completed and can also be included in the description used when completion is invoked for the preceding number.

In addition to common `compadd` options, `_numbers` accepts the following options:

- **-t tag** — Specify a tag to use instead of the default of `numbers`.
- **-u units** — Indicate the default units for the number, e.g. `bytes`.
- **-l min** — Specify the lowest possible value for the number.
- **-m max** — Specify the highest possible value for the number.
- **-d default** — Specify the default value.
- **-N** — Allow negative numbers. This is implied if the range includes a negative.
- **-f** — Allow decimal numbers.

Where a particular suffix represents the default units for a number, it should be prefixed with a colon. Additionally, suffixes can be followed by a colon and a description. So for example, the following allows the age of something to be specified, either in seconds or with an optional suffix with a longer unit of time:

```
_numbers -u seconds age :s:seconds m:minutes h:hours d:days
```

It is typically helpful for units to be presented in order of magnitude when completed. To facilitate this, the order in which they are given is preserved.

When the `format` style is looked up with the `descriptions` tag or the tag specified with `-t`, the list of suffixes is available as a `"%x"` escape sequence. This is in addition to the usual sequences documented under the `format` style. The form this list takes can also be configured. To this end, the `format` style is first looked up with the tag `unit-suffixes`. The retrieved format is applied to each suffix in turn and the results are then concatenated to form the completed list. For the `unit-suffixes` format, `"%x"` expands to the individual suffix and `"%X"` to its description. `%d` indicates a default suffix and can be used in a condition. The index and reverse index are set in `"%i"` and `"%r"` respectively and are useful for text included only with the first and last suffixes in the list. So for example, the following joins the suffixes together as a comma-separated list:

```
example(zstyle ':completion:*:unit-suffixes' format '%x%(r::,)' )
```

`_path_files` ¶

This function is used throughout the completion system to complete filenames. It allows completion of partial paths. For example, the string `"/u/i/s/sig"` may be completed to `"/usr/include/sys/signal.h"`.

The options accepted by both `_path_files` and `_files` are:

- `-f` — Complete all filenames. This is the default.
- `-/` — Specifies that only directories should be completed.
- `-g pattern` — Specifies that only files matching the *pattern* should be completed.
- `-W paths` — Specifies path prefixes that are to be prepended to the string from the command line to generate the filenames but that should not be inserted as completions nor shown in completion listings. Here, *paths* may be the name of an array parameter, a literal list of paths enclosed in parentheses or an absolute pathname.
- `-F ignored-files` — This behaves as for the corresponding option to the `compadd` builtin. It gives direct control over which filenames should be ignored. If the option is not present, the `ignored-patterns` style is used.

Both `_path_files` and `_files` also accept the following options which are passed to `compadd`: `"-J"`, `"-V"`, `"-1"`, `"-2"`, `"-n"`, `"-X"`, `"-M"`, `"-P"`, `"-S"`, `"-q"`, `"-r"`, and `"-R"`.

Finally, the `_path_files` function uses the styles `expand`, `ambiguous`, `special-dirs`, `list-suffixes` and `file-sort` described above.

`_pick_variant` ¶

This function is used to resolve situations where a single command name requires more than one type of handling, either because it has more than one variant or because there is a name clash between two different commands.

The command to run is taken from the first element of the array `words` unless this is overridden by the option `-c`. This command is run and its output is compared with a series of patterns. Arguments to be passed to the command can be specified at the end after all the other arguments. The patterns to try in order are given by the arguments *label=pattern*; if the output of `_command_ _arg_ ...` contains *_pattern_*, then *_label_* is selected as the label for the command variant. If none of the patterns match, the final command label is selected and status 1 is returned.

If the `-b "builtin-label"` is given, the command is tested to see if it is provided as a shell builtin, possibly autoloading; if so, the label *builtin-label* is selected as the label for the variant.

If the `-r "name"` is given, the *label* picked is stored in the parameter named *name*.

The results are also cached in the `_cmd_variant` associative array indexed by the name of the command run.

`_prefix` ¶

This completer can be used to try completion with the suffix (everything after the cursor) ignored. In other words, the suffix will not be considered to be part of the word to complete. The effect is similar to the `expand-or-complete-prefix` command.

The `completer` style is used to decide which other completers are to be called to generate matches. If this style is unset, the list of completers set for the current context is used DASH()- except, of course, the `_prefix` completer itself. Furthermore, if this completer appears more than once in the list of completers only those completers not already tried by the last invocation of `_prefix` will be called.

For example, consider this global `completer` style:

```
zstyle ':completion:*' completer \_complete \_prefix \_correct \_prefix:foo
```

Here, the `_prefix` completer tries normal completion but ignoring the suffix. If that doesn't generate any matches, and neither does the call to the `_correct` completer after it, `_prefix` will be called a second time and, now only trying correction with the suffix ignored. On the second invocation the completer part of the context appears as "foo".

To use `_prefix` as the last resort and try only normal completion when it is invoked:

```
zstyle ':completion:*' completer \_complete ... \_prefix zstyle ':completion::prefix:*' completer \_complete
```

The `add-space` style is also respected. If it is set to "true" then `_prefix` will insert a space between the matches generated (if any) and the suffix.

Note that this completer is only useful if the `COMPLETE_IN_WORD` option is set; otherwise, the cursor will be moved to the end of the current word before the completion code is called and hence there will be no suffix.

`_requested` ¶

This function is called to decide whether a tag already registered by a call to `_tags` (see below) has been requested by the user and hence completion should be performed for it. It returns status zero if the tag is requested and non-zero otherwise. The function is typically used as part of a loop over different tags as follows:

```
\_tags foo bar baz while \_tags; do if \_requested foo; then ... # perform completion for foo fi ... # test the tags bar and baz in the same way ... # exit loop if matches were generated done
```

Note that the test for whether matches were generated is not performed until the end of the `_tags` loop. This is so that the user can set the `tag-order` style to specify a set of tags to be completed at the same time.

If *name* and *descr* are given, `_requested` calls the `_description` function with these arguments together with the options passed to `_requested`.

If *command* is given, the `_all_labels` function will be called immediately with the same arguments. In simple cases this makes it possible to perform the test for the tag and the matching in one go. For example:

```
example(local expl ret=1 \_tags foo bar baz while \_tags; do \_requested foo expl 'description' \ compadd foobar foobaz && ret=0 ... (( ret )) || break done)
```

If the *command* is not `compadd`, it must nevertheless be prepared to handle the same options.

`_rm` ¶

Completion for `rm` — flags + file paths. `-r` enables directory completion; without it, directories are filtered out.

`_sep_parts` ¶

This function is passed alternating arrays and separators as arguments. The arrays specify completions for parts of strings to be separated by the separators. The arrays may be the names of array parameters or a quoted list of words in parentheses. For example, with the array `"tt(hosts=(ftp news))"` the call `"tt(_sep_parts "(foo bar)' @ hosts)"` will complete the string "f" to "foo" and the string "b@n" to "bar@news".

This function accepts the `compadd` options `-V`, `-J`, `-1`, `-2`, `-n`, `-X`, `-M`, `-P`, `-S`, `-r`, `-R`, and `-q` and passes them on to the `compadd` builtin used to add the matches.

`_sequence` ¶

This function is a wrapper to other functions for completing items in a separated list. The same function is used to complete each item in the list. The separator is specified with the `-s` option. If `-s` is omitted it will use ",". Duplicate values are not matched unless `-d` is specified. If there is a fixed or maximum number of items in the list, this can be specified with the `-n` option.

Common `compadd` options are passed on to the function. It is possible to use `compadd` directly with `_sequence`, though `_values` may be more appropriate in this situation.

`_tags` ¶

If called with arguments, these are taken to be the names of tags valid for completions in the current context. These tags are stored internally and sorted by using the `tag-order` style.

Next, `_tags` is called repeatedly without arguments from the same completion function. This successively selects the first, second, etc. set of tags requested by the user. The return status is zero if at least one of the tags is requested and non-zero otherwise. To test if a particular tag is to be tried, the `_requested` function should be called (see above).

If `-C "name"` is given, `name` is temporarily stored in the `argument` field (the fifth) of the context in the `curcontext` parameter during the call to `_tags`; the field is restored on exit. This allows `_tags` to use a more specific context without having to change and reset the `curcontext` parameter (which has the same effect).

`_terraform` ¶

Completion for `terraform` — subcommands, workspace names, state-file paths, providers, modules, variable names from the loaded HCL.

`_tilde_files` ¶

Like `_files`, but resolve leading tildes according to the rules of filename expansion, so the suggested completions don't start with a `~` even if the filename on the command-line does.

`_values` ¶

This is used to complete arbitrary keywords (values) and their arguments, or lists of such combinations.

If the first argument is the option `-O "name"`, it will be used in the same way as by the `_arguments` function. In other words, the elements of the `name` array will be passed to `compadd` when executing an action.

If the first argument (or the first argument after `-O "name"`) is `-s`, the next argument is used as the character that separates multiple values. This character is automatically added after each value in an auto-removable fashion (see below); all values completed by `_values -s` appear in the same word on the command line, unlike completion using `_arguments`. If this option is not present, only a single value will be completed per word.

Normally, `_values` will only use the current word to determine which values are already present on the command line and hence are not to be completed again. If the `-w` option is given, other arguments are examined as well.

The first non-option argument, `desc`, is used as a string to print as a description before listing the values.

All other arguments describe the possible values and their arguments in the same format used for the description of options by the `_arguments` function (see above). The only differences are that no minus or plus sign is required at the beginning, values can have only one argument, and the forms of action beginning with an equal sign are not supported.

The character separating a value from its argument can be set using the option `-S` (like `-s`, followed by the character to use as the separator in the next argument). By default the equals sign will be used as the separator between values and arguments.

Example:

```
example(_values -s , 'description' \ '*foo[bar]' \ '(two)*one[number]:first count:' \ 'two[another number]::second count:(1 2 3)')
```

This describes three possible values: `"foo"`, `"one"`, and `"two"`. The first is described as `"bar"`, takes no argument and may appear more than once. The second is described as `"number"`, may appear more than once, and takes one mandatory argument described as `"first count"`; no action is specified, so it will not be completed. The `"tt((two))"` at the beginning says that if the value `"one"` is on the line, the value `"two"` will no longer be considered a possible completion. Finally, the last value (`"two"`) is described as `"another number"` and takes an optional argument described as `"second count"` for which the completions (to appear after an `"="`) are `"1"`, `"2"`, and `"3"`. The `_values` function will complete lists of these values separated by commas.

Like `_arguments`, this function temporarily adds another context name component to the arguments element (the fifth) of the current context while executing the `action`. Here this name is just the name of the value for which the argument is completed.

The style `verbose` is used to decide if the descriptions for the values (but not those for the arguments) should be printed.

The associative array `val_args` is used to report values and their arguments; this works similarly to the `opt_args` associative array used by `_arguments`. Hence the function calling `_values` should declare the local parameters `state`, `state_descr`, `line`, `context` and `val_args`:

```
local context state state_descr line typeset -A val_args
```

when using an action of the form `->"string"`. With this function the `context` parameter will be set to the name of the value whose argument is to be completed. Note that for `_values`, the `state` and `state_descr` are scalars rather than arrays. Only a single matching state is returned.

Note also that `_values` normally adds the character used as the separator between values as an auto-removable suffix (similar to a `/` after a directory). However, this is not possible for a `->"string"` action as the matches for the argument are generated by the calling function. To get the usual behaviour, the calling function can add the separator `x` as a suffix by passing the options `-qS" x"` either directly or indirectly to `compadd`.

The option `-C` is treated in the same way as it is by `_arguments`. In that case the parameter `curcontext` should be made local instead of `context` (as described above).

`_wanted` ¶

In many contexts, completion can only generate one particular set of matches, usually corresponding to a single tag. However, it is still necessary to decide whether the user requires matches of this type. This function is useful in such a case.

The arguments to `_wanted` are the same as those to `_requested`, i.e. arguments to be passed to `_description`. However, in this case the *command* is not optional; all the processing of tags, including the loop over both tags and tag labels and the generation of matches, is carried out automatically by `_wanted`.

Hence to offer only one tag and immediately add the corresponding matches with the given description:

```
local expl _wanted tag expl 'description' \ compadd -- match1 match2...
```

Note that, as for `_requested`, the *command* must be able to accept options to be passed down to `compadd`.

Like `_tags` this function supports the `-C` option to give a different name for the argument context field. The `-x` option has the same meaning as for `_description`.

`_widgets` ¶

This function completes names of zle widgets (see *Zle Widgets* (zshzle)). The *pattern*, if present, is matched against values of the `$widgets` special parameter, documented in *The zsh/zleparameter Module* (zshmodules).

Extension Builtin Index

114 entries · zshrs-only builtins with NO upstream zsh counterpart. Split across in-process builtins (coreutils drop-ins, `async/await/barrier`, `doctor`, `intercept`, contrib autoloads) and daemon-backed `z*` builtins (`zd`, `zcache`, `zls`, `zlock`, `zpublish`, ...) that proxy to the local `zshrs-daemon` for cross-shell state.

`add_zsh_hook` 𐄂

`add-zsh-hook` builtin. Direct port of the shell function at `Src/Functions/Misc/add-zsh-hook`, which maintains a paramtab array per well-known hook (`chpwd_functions`, `precmd_functions`, `preexec_functions`, `periodic_functions`, `zshexit_functions`, `zshaddhistory_functions`). This is the SHELL-LEVEL mechanism; it is distinct from the C-module hookdef chain in `src/ported/module.rs` (`addhookfunc(name, Hookfn)` writes to `hooktab` and is consumed by `runhookdef` for C-module callbacks like `BEFORECOMPLETEHOOK` / `AFTERCOMPLETEHOOK`).

`arch` 𐄂

`arch` — print machine architecture name. Coreutils `arch(1)` (a synonym for `uname -m` on most systems). Useful in shell scripts that need a quick `[[ $(arch) == arm64 ]]` check.

`async` 𐄂

`async { cmd }` — run command on worker pool, return job ID immediately. Output captured in background, retrieve with `await $id`.

Usage: `id=$(async 'sleep 2; echo done') ... do other work ... result=$(await $id)`

`await` 𐄂

`await $id` — block until async job completes, print its stdout, return its status.

Usage: `id=$(async 'expensive_command') await $id # blocks until done, prints output echo $? # exit status of the async command`

`barrier` 𐄂

`barrier cmd1 ::: cmd2 ::: cmd3` — run commands in parallel, wait for ALL to complete. Returns the worst (highest) exit status.

Usage: `barrier 'make -C proj1' ::: 'make -C proj2' ::: 'make -C proj3' barrier 'npm test' ::: 'cargo test' ::: 'pytest'`

`base64` 𐄂

`base64 [-d] [FILE]` — encode/decode base64. coreutils `base64(1)` without `--wrap` (defaults to 76-char wrap on encode; 0 disables).

`basename` 𐄂

Strip leading directories and an optional suffix. `basename /a/b.txt .txt` → `b`. coreutils drop-in.

`caller` 𐄂

`caller` - display call stack (bash) `caller [N]` — bash builtin returning the location of the current frame N. With no arg or `N=0`: 'LINE FUNC' (or just 'LINE main' at top level). With `N>0`: 'LINE FUNC FILE' for frame N. Direct port of bash's `bin_caller` in `builtins.def`. Reads from the existing `$funcstack` array we now maintain (vm_helper:7828-7835).

cat 🍷

Concatenate files to stdout. **-n** numbers lines, **-A** shows tabs/EOLs. coreutils drop-in.

cdreplay 🍷

cdreplay - replay deferred compdef calls (zinit turbo mode) Usage: cdreplay [-q]

cksum 🍷

cksum [FILE...] — POSIX CRC-32 + byte-count + filename. Output: **<crc>** **<bytes>** **<name>**. With no files or **-** reads stdin (filename column omitted in that case, per coreutils). Polynomial: 0x04C11DB7, init 0, length appended (POSIX).

comm 🍷

comm [-123] FILE1 FILE2 — line-by-line comparison of two sorted files. Coreutils comm(1) / POSIX. Three columns: (1) unique to FILE1, (2) unique to FILE2, (3) common. Flags **-1/-2/-3** suppress the respective column. Either file may be **-** for stdin. Files MUST be sorted in the same collation; comm performs a streaming merge-compare and is undefined-behavior on unsorted input (matches coreutils).

compdef 🍷

compdef - register completion functions for commands Usage: compdef **_git** git compdef **_docker** docker docker-compose
compdef -d git # delete

compgen 🍷

Generate completion candidates

compinit 🍷

Initialize the completion system. Walks **\$fpath** in parallel via rayon, populates the rkyv-mmap'd autoload/completion shards, marks every **_*** as autoloaded. Default mode skips **.zcompdump** entirely.

complete 🍷

Define completion spec for a command

cp 🍷

cut 🍷

Extract fields or character ranges. **-d':'** **-f1,3** / **-c5-10**. coreutils drop-in.

date 🍷

Print or set the system date. **+%FORMAT** strftime; **-d 'rel'** parse relative; **-u** UTC. coreutils drop-in.

dbview 🍷

dbview — browse zshrs SQLite cache tables without SQL.

Usage: dbview — list all tables and row counts dbview autoloads — dump autoloads table (name, source, body len, ast len)
dbview autoloads **_git** — show single row by name dbview comps — dump comps table dbview history — recent history entries
dbview history **<pattern>** — search history dbview plugins — plugin cache entries dbview executables — PATH executables
cache dbview **<table>** --count — just the count

dircolors 🍌

dircolors [-bcp] [FILE] — emit shell commands to set LS_COLORS. Coreutils dircolors(1). Without args, emits the default ls color database. -b for Bourne (export VAR=val), -c for csh (setenv VAR val), -p prints the database. We hard-code coreutils' compiled-in default since shipping the full /etc/DIR_COLORS database file isn't feasible here.

dirname 🍌

Strip the last path component. `dirname /a/b/c → /a/b`. coreutils drop-in.

doctor 🍌

doctor - diagnostic report of shell health, caches, and performance

env 🍌

env [-i] [NAME=VALUE]... [COMMAND [ARG]...] — print env or run COMMAND with modified environment. Direct port of coreutils env(1) for the most-used invocations.

expand 🍌

expand [-t TAB] [FILE...] — convert tabs to spaces. coreutils expand(1).

expr 🍌

expr ARG... — evaluate expression. POSIX expr(1). Subset port: integer arithmetic (+ - * / %), string match ': REGEX', length STRING, substr STRING POS LEN, index STRING CHARS. Recognizes the closing arg list directly (single-pass shunting-yard would be heavier than needed for the common scripts).

factor 🍌

factor N... — print prime factorization of each integer arg. Coreutils factor(1). Format: `N: p1 p2 p3 ....`. Reads stdin if no args. Negative numbers and zero are rejected.

find 🍌

Walk the filesystem and print / act on matches. Supports `-name/-type/-mtime/-exec`. coreutils drop-in (subset).

fold 🍌

fold [-w WIDTH] [-s] [-b] [FILE...] — wrap input lines. coreutils fold(1).

groups 🍌

groups [USER...] — print group memberships. Coreutils groups(1) / POSIX. With no args, prints groups for the effective user; with args, prints "USER : group1 group2 ..." per user.

head 🍌

Print the first N lines (`-n N`) or bytes (`-c N`) of each file. coreutils drop-in.

help 🍌

help - display help for builtins (bash)

hostname 🍌

Print the system hostname. `-s` short, `-f` FQDN.

id ¶

Print user / group IDs. **-u** user only, **-g** group only, **-n** names. coreutils drop-in.

intercept ¶

intercept builtin — register AOP advice on commands.

Usage: intercept before <pattern> { code } intercept after <pattern> { code } intercept around <pattern> { code } intercept list — show all intercepts intercept remove <id> — remove by ID intercept clear — remove all

intercept_proceed ¶

intercept_proceed — called from around advice to execute the original command.

link ¶

link FILE1 FILE2 — call link(2) directly to create a hard link from FILE1 to FILE2. POSIX link(1) / coreutils link(1). Unlike **ln**, link takes EXACTLY two args and rejects flags (POSIX requirement).

logname ¶

logname — print login name. Coreutils logname(1) / POSIX. Calls getlogin(3) which reads from utmp. Falls back to `\$LOGNAME` if getlogin fails.

mkfifo ¶

mkfifo [-m MODE] FILE... — create named pipes (FIFOs). Coreutils mkfifo(1) / POSIX. -m sets the mode (default 0666 minus umask). Each FIFO is created independently; failures are reported per-file and the others continue.

mktemp ¶

Create a temp file or directory with a unique name. **-d** directory, **-p DIR** parent. coreutils drop-in.

nice ¶

nice [-n N] [-N N] [COMMAND...] — adjust niceness of the shell process (when no COMMAND given) or report current niceness. Coreutils nice(1) / POSIX nice(1). Without args, print the current niceness (like `\nice\` with no args). With just -n N, set the niceness for the SHELL ITSELF — not for a subsequently-exec'd command. Setting niceness for a child command requires fork-exec which the in-process model can't do without breaking subsequent commands; for that case callers should use `/usr/bin/nice` via PATH (zshrs's command dispatch will fall through to the external).

nl ¶

nl [-b STYLE] [FILE...] — number lines. Direct port of coreutils nl(1) for the most-used flag set.

nproc ¶

nproc — print number of online CPUs. coreutils nproc(1). --all uses CPU count from sysconf(_SC_NPROCESSORS_CONF); default uses _SC_NPROCESSORS_ONLN (the schedulable subset).

paste ¶

paste [-d LIST] [FILE...] — merge lines of files. coreutils paste(1). Default delim is TAB; -d cycles through the supplied delimiter chars.

peach 🍑

peach 'cmd {}' arg1 arg2 ... — parallel for-each, no output ordering. Like pmap but doesn't collect output — fire-and-forget, print as completed.

Usage: peach 'convert {} {} .png' *.svg peach 'rsync -a {} remote:{}' dir1 dir2 dir3

pgrep 🍑

pgrep 'pattern' arg1 arg2 ... — parallel grep/filter across worker pool. Runs the pattern command for each argument, prints args where command succeeds.

Usage: pgrep 'test -f {}' /path/a /path/b /path/c pgrep 'grep -q TODO {}' *.rs

pmap 🍑

pmap 'cmd {}' arg1 arg2 arg3 — parallel map across worker pool. Runs **cmd** for each argument, replacing {} with the argument. Output is collected in order. Returns 0 if all succeed.

Usage: pmap 'gzip {}' *.log pmap 'echo {}' a b c d ls *.rs | pmap 'wc -l {}'

printenv 🍑

printenv [VAR...] — print env. coreutils printenv(1). No args: print all env vars (sorted by key for stable output). Args: print VAR's value per arg, exit 1 if any unset.

profile 🍑

profile — in-process command profiling with nanosecond accuracy.

Unlike **time** (which measures one command) or **zprof** (which only profiles function calls), **profile** traces every execute_command, expansion, glob, and builtin dispatch inside the block.

Usage: profile { commands } — profile a block profile -s 'script' — profile a script string profile -f func — profile a function call profile --clear — clear accumulated profile data profile --dump — show accumulated profile data

realpath 🍑

Resolve symlinks and . / .. to a canonical absolute path. coreutils drop-in.

rev 🍑

Reverse each input line character-by-character. coreutils drop-in.

run_tests 🍑

Alias for **ztest_run** — print the per-block test summary and roll the per-block counters into the run-wide totals. Returns 0 on all-pass, 1 if any assertion failed. Port of strykelang's **test_run** / **run_tests** builtin pair.

seq 🍑

Print a sequence of numbers. **seq 1 10** / **seq 1 2 10** / **seq -w 1 10**. coreutils drop-in.

sha256sum 🍑

sha256sum [FILE...] — write SHA-256 of each file (or stdin when no FILE / '-'). coreutils-style 'HEX PATH' output.

shuf 🍑

shuf [-n N] [-i LO-HI] [-e [STR...]] [FILE] — random permutation. coreutils shuf(1).

sleep ㉿

Pause for the given duration. `sleep 1`, `sleep 0.5`, `sleep 1m`. coreutils drop-in.

sort ㉿

Sort lines. `-n` numeric, `-r` reverse, `-k N` by field, `-u` unique. coreutils drop-in.

sum ㉿

`sum [-rs] [FILE...]` — BSD or SysV checksum. `-r`: BSD 16-bit rotating checksum (default per POSIX). `-s`: SysV checksum (sum-of-bytes mod 65535, then folded). Output: `<sum> <512-byte-blocks> [name]` (BSD) or `<sum> <kbytes> [name]` (SysV).

tac ㉿

`tac [FILE...]` — concatenate files, reverse line order. coreutils tac(1).

tail ㉿

Print the last N lines (`-n N`) or follow appends (`-f`). coreutils drop-in.

tee ㉿

Copy stdin to stdout AND to each named file. `-a` append. coreutils drop-in.

touch ㉿

Create a file or update its mtime. `-d STR` set time, `-r REF` copy from REF. coreutils drop-in.

tput ㉿

`tput` — terminfo capability query (minimal subset). Common subset of ncurses `tput(1)`: `tput cols / lines` → terminal width / height `tput colors` → terminal color count `tput clear / cl` → clear screen `tput cup R C` → cursor to (row, col) (0-based) `tput sgr0 / op` → reset attributes / colors `tput bold / smso / rmso / smul / rmul / rev / blink` → text attributes `tput setaf N / setab N` → fg/bg color (8/16 colors) Many other terminfo capabilities aren't yet wired; unknown capabilities fall through to `echotc`'s two-letter mapping or silently exit 1 (`tput`'s standard error code).

tr ㉿

Translate / squeeze / delete characters. `tr a-z A-Z` uppercases. coreutils drop-in.

tsort ㉿

`tsort [FILE]` — topological sort. Coreutils `tsort(1)` / POSIX. Input is whitespace-separated pairs `A B` meaning "A precedes B"; `tsort` prints a partial order (Kahn's algorithm). Cycles are reported on stderr (one cycle node per line) and the program continues with that node treated as a leaf. Reads stdin when no file is given or `-`.

tty ㉿

`tty` — print the controlling-terminal device path. coreutils `tty(1)`. `-s` suppresses output (just sets the exit code).

uname ㉿

Print system info. `-a` all, `-s` kernel, `-m` machine, `-r` release. coreutils drop-in.

unexpand 𐀄

unexpand [-a] [-t TAB] [FILE...] — convert spaces to tabs. coreutils unexpand(1). Default tabstop 8; -a converts every run of spaces (not just leading); without -a only leading runs collapse.

uniq 𐀄

Filter adjacent matching lines. -c prefix count, -d only duplicates. coreutils drop-in.

unlink 𐀄

unlink FILE — call unlink(2) directly to remove a single file. POSIX unlink(1) / coreutils unlink(1). Strict: takes exactly ONE arg, no flags, no recursion. Cannot remove directories (errors if FILE is a directory).

users 𐀄

users — print logged-in usernames. Coreutils users(1) / POSIX. Fallback minimal impl: prints \$USER (or current effective user via getpwuid) since fully reading utmp is platform-specific. Multi-user output not yet implemented; shell scripts that just check `[[ $(users) ]]` still work.

wc 𐀄

Count newlines, words, bytes. -l lines, -w words, -c bytes. coreutils drop-in.

whoami 𐀄

Print the effective user name. coreutils drop-in.

yes 𐀄

yes [STRING] — print STRING (or 'y') forever. coreutils yes(1). Honors SIGPIPE: when stdout is piped and the consumer closes, the write fails and we exit 0 silently.

zask 𐀄

Send an ask-style request to the daemon and print the JSON response. Used by tools/agents that want a single synchronous query against the shared catalog.

zassert_contains 𐀄

zassert_contains HAYSTACK NEEDLE [MSG] — pass when NEEDLE is a substring of HAYSTACK. Bumps the per-block `ztest_pass_count` on success, `ztest_fail_count` on failure; the ✓/✗ glyph + label print on stderr. Port of strykelang's `builtin_assert_contains`.

zassert_dies 𐀄

zassert_dies "CMD" [MSG] — shell-native variant of strykelang's `assert_dies` (which takes a code ref). Runs CMD in a scratch `ShellExecutor` (inheriting the host's `zsh_compat` / `bash_compat` / `posix_mode`); passes iff the command exits non-zero or errors. Use to assert that a misuse correctly fails.

zassert_eq 𐀄

zassert_eq A B [MSG] — string equality. Like every `zassert_*` builtin, returns 0 on pass / 1 on fail and bumps the per-block counters that `ztest_run` reads.

`zassert_err`

`zassert_err V [MSG]` — pass when `V` is shell-falsy (empty `*or*` literal `"0"`). Convenient for asserting on `$?` after a command, or on a string that should be empty.

`zassert_false`

`zassert_false V [MSG]` — alias of `zassert_err`.

`zassert_ge`

`zassert_ge A B [MSG]` — pass iff numeric $A \geq B$. Inputs are `trim().parse::<f64>()`'d; non-numeric inputs default to 0.

`zassert_gt`

`zassert_gt A B [MSG]` — pass iff numeric $A > B$.

`zassert_le`

`zassert_le A B [MSG]` — pass iff numeric $A \leq B$.

`zassert_lt`

`zassert_lt A B [MSG]` — pass iff numeric $A < B$.

`zassert_match`

`zassert_match PATTERN STRING [MSG]` — regex match. `PATTERN` is compiled with the `regex` crate (Rust regex syntax, not POSIX BRE/ERE — so `\d/\w/\s` work, no backreferences). Fails (rather than panics) on a bad pattern.

`zassert_ne`

`zassert_ne A B [MSG]` — string inequality.

`zassert_near`

`zassert_near A B [EPS [MSG]]` — floating-point approximate equality: pass iff $|A - B| \leq EPS$. `EPS` defaults to `1e-9`. Right for asserting on math results without burning on the last few ULPs.

`zassert_ok`

`zassert_ok V [MSG]` — pass when `V` is shell-truthy (non-empty `*and*` not literal `"0"`). Use `zassert_eq $? 0 ...` for command-success assertions; reserve `zassert_ok` for assertions on populated strings or non-zero counts.

`zassert_true`

`zassert_true V [MSG]` — alias of `zassert_ok`.

`zbuild`

`zbuild --in PATHS... --out OUT` — bake one or more shell scripts into a copy of the running `zshrs` binary in input order, producing a self-contained AOT executable.

`zbuild --in *.zsh --out app # glob expansion`
`zbuild --in lib1.zsh lib2.zsh main.zsh --out app ./app # runs all three # sequentially under # one ShellExecutor`

zsh has no project concept, so there's no manifest, no entry point convention, no library directory walker. Order is exactly the order of paths given to `--in` (which honors shell glob expansion — `\*.zsh` expands sorted by default).

`--in` accepts ONE OR MORE paths until the next flag-style token (anything starting with `-`). This makes glob-driven invocations like `--in *.zsh` work naturally — every path the glob expanded to lands as another input file.

Flags: `--in PATHS...` / `-i PATHS...` script sources (1+, required) `--out PATH` / `-o PATH` output binary (required) `--help` / `-h` print usage

zcache 𐄂

Read / write / list the per-shell cache namespace. `zcache get K` / `zcache set K V [TTL]` / `zcache del K` / `zcache list [PREFIX]`. Backed by the daemon's in-memory KV with optional SQLite persistence.

zcmd-result 𐄂

Push the exit status + output of a just-completed command to the daemon's command-history catalog. Used by `precmd` hooks to populate the cross-shell `zhistory` index.

zcomplete 𐄂

Push a completion candidate to the daemon's shared completion cache. Other shells running compinit will see it without re-walking fpath.

zd 𐄂

`zd` — in-process builtin form of the standalone `zd` binary. Same arg surface (see `daemon::zd_dispatch::USAGE`); transport is the local Unix socket via the existing `Client`. **No fork:** `zshrs` calls `dispatch()` directly and gets the result back. ~5-15× faster than `fork+exec'ing` the binary for short ops.

`--url` / `--token` global flags are accepted (and ignored) for command-line shape compatibility with the binary form. The builtin always uses the local socket — there's no point talking to the daemon over HTTP from inside the same machine when a Unix socket is right there.

SSE ops (`zd events`, `zd watch`) return an error pointing at the binary form — long-lived stream pumps don't fit the request-response shape of a builtin.

zhistory 𐄂

Query the daemon's federated command-history catalog. Spans every shell that pushed via `zcmd-result`. SQLite FTS5-backed; `zhistory search 'pattern'`.

zid 𐄂

Print the current shell's federated ID — the stable `shell_id` (`bash` / `zsh` / `zshrs` / ...) and the per-process `bundle_id` the daemon uses to scope state.

zjob 𐄂

Manage background jobs through the daemon: `zjob submit -- cmd ...` queues, `zjob status ID`, `zjob output ID`, `zjob wait ID`, `zjob kill ID`. Jobs survive shell exit because the daemon owns them.

zlock 𐄂

Acquire / release / try a named cross-shell lock. `zlock acquire NAME [TIMEOUT]` / `zlock release NAME TOKEN` / `zlock try NAME` / `zlock do NAME -- cmd ....` PID-tagged so the daemon GCs stale entries.

zlog ㊦

Append a structured log entry to the daemon's log catalog. `zlog 'message' [key=val ...]`. Queryable later via `zhistory` / `dbview`.

zls ㊦

List entries in the daemon's federated catalog (aliases, functions, env vars, etc.). `zls --kind alias --shell-id bash`. The cross-shell mirror of `alias/functions/typeset`.

znotify ㊦

Send a desktop / system notification through the daemon. Routes to `osascript` (macOS), `notify-send` (Linux), or the in-shell UI when no platform notifier is available.

zping ㊦

Round-trip latency probe against the daemon. Prints the RTT in microseconds; non-zero exit if the daemon is unreachable.

zpublish ㊦

Publish a JSON event to a pubsub topic. `zpublish topic.name '{"key":"val"}'`. Subscribers receive via `zsubscribe`.

zsend ㊦

Send a one-shot message to another shell (by `shell_id` or `bundle_id`). Like `znotify` but targets a specific shell, not the user's desktop.

zsource ㊦

Push a sourced-file event to the daemon's federated catalog. Used by `source/`. hooks so the daemon knows which rc files have been loaded by which shells.

zsubscribe ㊦

Subscribe to a pubsub topic and stream incoming messages to stdout as SSE-style JSON lines. `zsubscribe 'shell:*.build_done'`.

zsuggest ㊦

Query the daemon's suggestion engine for the next command, given the current cwd + history. Used by ZLE's autosuggestion widget when the local history can't supply a candidate.

zsync ㊦

Force a flush of the daemon's in-memory state to the SQLite catalog. Normally happens in the background; `zsync` makes it synchronous so a snapshot is consistent.

ztag ㊦

Tag the current shell session with one or more labels. `ztag prod-deploy`. Other shells can filter by tag via `zls --tag prod-deploy`.

ztest_run 𐀄

Print the per-block test summary (green ✓ All N passed / red X M of N failed) and roll the per-block `ztest_pass_count` / `ztest_fail_count` / `ztest_skip_count` into the run-wide `_total` counters, then zero the per-block counters so a second test block in the same file starts fresh. A sticky `ztest_run_failed` flag latches when any block reports failures — the `--ztest` worker reads it to exit non-zero even if the script ends with `exit 0`. Alias: `run_tests`. Returns 0 on all-pass, 1 if any assertion failed.

ztest_skip 𐀄

`ztest_skip MSG` — bump the per-block skip counter and emit a yellow ~ marker on stderr. Typical use: gate an assertion behind a compat condition using shell short-circuit `(( cond )) || { ztest_skip "needs X"; continue; }`.

zunsubscribe 𐀄

Cancel a `zsubscribe` stream. `zunsubscribe TOPIC` or `zunsubscribe --all`.

zuntag 𐀄

Remove a tag from the current shell session. Inverse of `ztag`.

zwhere 𐀄

`zwhere` — query the daemon's federated definitions catalog from inside zshrs. Thin client over the `definitions_query` op (same thing `zd defs query` calls). Always available in default zshrs; not feature-gated on `recorder` because this is a read-only IPC path, not the AOP write path.

Usage: `zwhere KIND NAME # rows` for `KIND NAME` across all shells `zwhere KIND --prefix STR # all KIND rows` whose name starts with `STR` `zwhere --kinds # list every populated kind` `zwhere --shell-id ID ... # restrict to one shell_id` `zwhere --limit N ... # cap` (default 1000 for tty output)

Output format: one row per line, `kind\tname\tvalue\tshell_id\tfile:line`. Wide-column layout via padding is left to the caller piping through `column -t` if they want it; `zwhere` stays narrow so scripts can `awk` cleanly.

Operator / Punctuation Index

69 entries · pipelines (`|`, `|&`), list ops (`&&`, `||`, `;`, `&`, `;;`), redirects (`>`, `>>`, `<<`, `<<<`, `&>`, ...), conditional/arithmetic openers (`[[`, `]]`, `((`, `))`), substitution forms (`$()`, `${}`, `$((`, `<()`, `>()`), test ops (`-e`, `-eq`, `=~`, ...), pattern chars (`*`, `?`, `**`, `~`), brace expansion (`{a,b,c}`, `{1..10}`), and assignment (`=`, `+=`). Sourced from `man zshmisc` section prose — these have no per-name yodl `item` blocks so they're hand-curated.

| ¶

Pipeline. `cmd1 | cmd2` connects `cmd1`'s stdout to `cmd2`'s stdin. Each stage runs in a separate process; exit status is the last stage's (unless `PIPE_FAIL` is set, in which case the first non-zero in the chain wins).

|& ¶

Pipeline merging stderr. `cmd1 |& cmd2 = cmd1 2>&1 | cmd2`. Both stdout AND stderr of `cmd1` are piped to `cmd2`.

&& ¶

Logical AND list operator. `cmd1 && cmd2` runs `cmd2` only if `cmd1` succeeded (exit status 0). Short-circuits.

|| ¶

Logical OR list operator. `cmd1 || cmd2` runs `cmd2` only if `cmd1` failed (non-zero exit). Short-circuits.

; ¶

Sequential list separator. `cmd1; cmd2` runs `cmd2` after `cmd1` finishes, regardless of its exit status.

& ¶

Background list operator. `cmd &` runs `cmd` asynchronously in the background; the shell does not wait. Sets `$!` to the job's PID.

;; ¶

Case-branch terminator. Ends a `case` arm: `case x in pat) cmds ;; esac`. Stops case dispatch after this arm.

;;& ¶

Case-branch fall-through-and-test-next. Continues to the next `case` arm and tests its pattern.

;| ¶

Case-branch unconditional fall-through. Continues to the next `case` arm and runs it without testing its pattern.

! ¶

Pipeline negation (also a reserved word). `! cmd` inverts `cmd`'s exit status — zero becomes 1, non-zero becomes 0. Distinct from `!` history expansion (lexer-stage).

>

Stdout redirect. `cmd > file` writes `cmd`'s stdout to `file` (overwrite). With `NO_CLOBBER`, refuses to overwrite an existing file — use `>|` or `>!` to force.

>>

Stdout append. `cmd >> file` appends `cmd`'s stdout to `file` (creates if missing).

<

Stdin redirect. `cmd < file` makes `file` the source of `cmd`'s stdin.

<<

Heredoc start. `cmd <<MARKER` reads the following lines as `cmd`'s stdin until a line containing only `MARKER`. Variants: `<<-` strips leading tabs; `<<'MARKER'` disables expansion in the body.

<<-

Heredoc with tab-stripping. Like `<<` but every leading tab on body lines (and the terminator) is removed — lets you indent the heredoc for readability.

<<<

Here-string. `cmd <<< 'text'` makes the literal string `text` the source of `cmd`'s stdin. Adds a trailing newline.

&>

Redirect stdout + stderr together. `cmd &> file` = `cmd > file 2>&1`. Shorthand for the common combined redirect.

&>>

Append stdout + stderr together. `cmd &>> file` = `cmd >> file 2>&1`.

>&

Redirect a file descriptor. `2>&1` sends stderr to wherever stdout currently points. `>& file` is also accepted as `&> file`.

<&

Duplicate an input file descriptor. `cmd <&3` reads from fd 3. `<&` - closes stdin.

<>

Read+write redirect. `cmd <> file` opens `file` for both reading and writing on stdin.

>|

Force-overwrite redirect. Equivalent to `>` but ignores `NO_CLOBBER`.

>!

Same as `>|` — force-overwrite, bypass `NO_CLOBBER`.

`[[` ¶

Open zsh conditional expression. `[[ EXPR ]]` evaluates a boolean. No word splitting / glob inside; supports `&&`, `||`, `!`, `==`, `!=`, `=~`, `-e`, `-f`, `-d`, `-z`, `-n`, etc. Prefer this over `[ ]` in zsh.

`]]` ¶

Close zsh conditional expression. Pairs with `[[`. Must be a separate word — `[[ -n $x]]` is a syntax error; use `[[ -n $x ]]`.

`[` ¶

POSIX `test` command (also spelled `test`). Same conditional semantics as POSIX `test`. Prefer `[[ ... ]]` in zsh — it's safer (no word splitting) and supports more operators.

`]` ¶

Close POSIX `test`. Pairs with `[`.

`((` ¶

Open arithmetic command. `(( EXPR ))` evaluates `EXPR` as C-style integer arithmetic; exit 0 if the result is non-zero, 1 otherwise. Inside, `$` on var names is optional: `(( i++ ))`.

`)` ¶

Close arithmetic command. Pairs with `((`.

`$()` ¶

Command substitution open. `$(cmd)` runs `cmd` and substitutes its trimmed-trailing-newline stdout. Nestable: `$(echo $(date))`. Preferred over backticks.

`${` ¶

Parameter expansion open. `${VAR}` is the value of `VAR`. Rich modifier set: `${VAR:-default}`, `${VAR:=assign}`, `${VAR:+alt}`, `${#VAR}` length, `${VAR/p/r}` replace, `${VAR%suffix}` / `${VAR#prefix}` strip, `${(flags)VAR}` zsh flags.

`$((` ¶

Arithmetic expansion open. `$(( EXPR ))` evaluates `EXPR` as integer arithmetic and substitutes the result as a string. Distinct from `(( ... ))` which is a command, not an expansion.

`<()` ¶

Process substitution (input). `cmd <(producer)` exposes `producer`'s stdout as a filename (`/dev/fd/N`) to `cmd`. Lets commands that take filenames consume pipe output.

`>()` ¶

Process substitution (output). `cmd >(consumer)` exposes a filename to `cmd`; anything `cmd` writes there flows to `consumer`'s stdin.

``` ¶

Backtick command substitution. `cmd` runs `cmd` and substitutes its stdout. Legacy form — prefer `$(cmd)` for nestability and quoting clarity.

`-e` ¶

File-exists test. `[[ -e PATH ]]` is true if `PATH` exists (any type — file / dir / link / socket / ...).

`-f` ¶

Regular-file test. `[[ -f PATH ]]` is true if `PATH` exists AND is a regular file (not a directory / symlink / device).

`-d` ¶

Directory test. `[[ -d PATH ]]` is true if `PATH` exists AND is a directory.

`-r` ¶

Readable test. `[[ -r PATH ]]` is true if `PATH` exists AND is readable by the current process.

`-w` ¶

Writable test. `[[ -w PATH ]]` is true if `PATH` exists AND is writable by the current process.

`-x` ¶

Executable test. `[[ -x PATH ]]` is true if `PATH` exists AND has execute permission (or for directories, search permission).

`-s` ¶

Non-empty test. `[[ -s PATH ]]` is true if `PATH` exists AND has size > 0.

`-L` ¶

Symlink test. `[[ -L PATH ]]` is true if `PATH` is a symbolic link (does NOT dereference).

`-h` ¶

Same as `-L` — symlink test.

`-z` ¶

Empty-string test. `[[ -z $s ]]` is true if `$s` is the empty string.

`-n` ¶

Non-empty-string test. `[[ -n $s ]]` is true if `$s` has length > 0. Equivalent to `[[ $s ]]`.

`-eq` ¶

Numeric equality. `[[ a -eq b ]]` is true if integers `a` and `b` are equal. For strings use `==`.

`-ne` ¶

Numeric inequality. `[[ a -ne b ]]` is true if integers `a` and `b` differ.

`-lt` ¶

Numeric less-than. `[[ a -lt b ]]` is true if integer `a` < `b`.

`-le` ¶

Numeric less-or-equal. `[[ a -le b ]]` is true if integer `a` ≤ `b`.

`-gt` ¶

Numeric greater-than. `[[ a -gt b ]]` is true if integer `a` > `b`.

`-ge` ¶

Numeric greater-or-equal. `[[ a -ge b ]]` is true if integer `a` ≥ `b`.

`-ot` ¶

Older-than test. `[[ A -ot B ]]` is true if file `A` has an older mtime than `B`.

`-nt` ¶

Newer-than test. `[[ A -nt B ]]` is true if file `A` has a newer mtime than `B`.

`-ef` ¶

Same-file test. `[[ A -ef B ]]` is true if `A` and `B` are the same inode (hard-linked / same path).

`==` ¶

Pattern-match equality (inside `[[ ... ]]`). `[[ $s == pat* ]]` matches `$s` against the glob pattern `pat*`. RHS is a pattern unless quoted. For literal equality, quote: `[[ $s == "literal" ]]`.

`!=` ¶

Pattern-mismatch (inside `[[ ... ]]`). Inverse of `==`. Quote the RHS for literal comparison.

`=~` ¶

Regex match (inside `[[ ... ]]`). `[[ $s =~ pat ]]` matches `$s` against the regex `pat`. Capture groups land in `$match` / `$MATCH` / `$BASH_REMATCH`.

`*` ¶

Glob: match zero or more characters of any name (excluding leading `.` unless `GLOB_DOTS` is set). Also a multiplication operator inside `(( ... ))`.

`?` ¶

Glob: match exactly one character. Also the last-exit-status variable when used as `$?`.

****** ¶

Recursive glob (zsh extended). ****/*.*rs** matches ***.*rs** at any depth under the current directory. Requires **EXTENDED_GLOB** for additional pattern operators.

~ ¶

Pattern exclude (with **EXTENDED_GLOB**). ***~README** matches everything except **README**. Also tilde expansion: **~** = **\$HOME**, **~user** = user's home, **~+** = **\$PWD**, **~-** = **\$OLDPWD**.

^ ¶

Pattern negate first-match (with **EXTENDED_GLOB**). **^*.*rs** matches everything that's NOT ***.*rs**. Inside **[...]** ranges, negates: **[^abc]**.

{a,b,c} ¶

Brace expansion (literal list). Expands to multiple words: **cp file.{txt,bak}** becomes **cp file.txt file.bak**. No whitespace before commas.

{1..10} ¶

Brace range expansion. **{1..10}** expands to **1 2 3 4 5 6 7 8 9 10**. Supports step: **{1..10..2}** → **1 3 5 7 9**. Letters work too: **{a..z}**.

= ¶

Assignment. **VAR=value**. NO whitespace around **=**. With **local** / **typeset**: **local VAR=value** declares + assigns.

+= ¶

Append assignment. **VAR+=more** appends to a scalar; for arrays **arr+=(x y)** appends elements. Numeric for **integer**: **((count += 1))**.

:= ¶

Conditional-assign default (inside **\${...}**). **\${VAR:=fallback}** assigns **fallback** to **VAR** (and substitutes it) if **VAR** is unset or empty.

?= ¶

Error-if-unset (inside **\${...}**). **\${VAR:?msg}** substitutes **\$VAR** if set, else prints **msg** to stderr and exits.

50

Keywords

157

Builtins

196

Options

88

Special vars

38

Param flags

44

Glob quals

55

Operators

10

History exp.

16

Modifiers

Generated 2026-05-31 from `data/grammar/canonical.json`.

Reserved keywords (50)

Source: `src/zsh/src/hashtable.c::reswds[]` + zshrs lexer extensions.

Control flow (20)

`[]` always case do done elif else end esac fi for foreach if in repeat select then until while

Declaration (10)

`declare export float integer let local readonly set shift typeset`

Function (1)

`function`

Grouping (2)

`{ }`

I/O / source (5)

`. eval exec source trap`

Loop control (5)

`break continue exit logout return`

Modifier (precommand) (6)

`builtin command coproc nocorrect noglob time`

Operator-like (1)

`!`

Builtins (157)

Every command implemented inside the shell — no fork/exec required.

POSIX / zsh core (74)

Source: `src/zsh/src/builtin.c::builtins[]`.

`. : [alias autoload bg break bye cd chdir continue declare dirs disable disown echo emulate enable eval exit export false fc fg float functions getln getopts hash hashinfo history integer jobs kill let local logout mem patdebug popd print printf pushd pushln pwd r read readonly rehash return set setopt shift`

source suspend test times trap true ttyctl type typeset umask unalias unfunction unhash unset unsetopt
wait whence where which zcompile zmodload

zsh modules (59)

Source: `src/zsh/Src/Modules/*.c` (curses, datetime, pcre, files, stat, terminfo, zftp, zselect, zsystem, zutil, ...).

cap chgrp chmod chown clone echotc echoti example getcap ln local log mkdir mv nameref pcre_compile pcre_
match pcre_study private rm rmdir setcap stat strftime sync syserror sysopen sysread sysseek syswrite
zcurse zdelattr zf_chgrp zf_chmod zf_chown zf_ln zf_mkdir zf_mv zf_rm zf_rmdir zf_sync zformat zftp
zgdbmpath zgetattr zlistattr zparseopts zprof zpty zregexp zselect zsetattr zsocket zstat zstyle
zsystem ztcp ztie zuntie

zshrs extensions (24)

Source: `daemon/builtins.rs` + `src/extensions/ext_builtins.rs`.

zask zcache zcompdump zcomplete zd zhhistory zid zjob zlock zlog zls znotify zping zpublish zsend zsource
zsubscribe zsuggest zsync ztag zunsubscribe zuntag zwc zwhere

setopt options (196)

Source: `src/zsh/Src/options.c::optns[]`. All upstream zsh options recognized by name; use `setopt` / `unsetopt` to
toggle, or `NO_` prefix (e.g. `NO_GLOB`).

ALIASES ALIASFUNCDEF ALLEXPORT ALWAYSLASTPROMPT ALWAYSSTOEND APPENDCREATE APPENDHISTORY AUTOCD
AUTOCONTINUE AUTOLIST AUTOMENU AUTONAMEDIRS AUTOPARAMKEYS AUTOPARAMSLASH AUTOPUSHD AUTOREMOVESLASH
AUTORESUME BADPATTERN BANGHIST BAREGLOBQUAL BASHAUTOLIST BASHREMATCH BEEP BGNICE BRACECCL BRACEEXPAND
BSDECHO CASEGLOB CASEMATCH CASEPATHS CBASES CDABLEVARS CDSILENT CHASEDOTS CHASELINKS CHECKJOBS
CHECKRUNNINGJOBS CLOBBER CLOBBEREMPTY COMBININGCHARS COMPLETEALIASES COMPLETEINWORD CONTINUEONERROR
CORRECT CORRECTALL CPRECEDENCES CSHJUNKIEHISTORY CSHJUNKIELOOPS CSHJUNKIEQUOTES CSHNULLCMD
CSHNULLGLOB DEBUGBEFORECMD DOTGLOB DVORAK EMACS EQUALS ERREXIT ERRRETURN EVALLINENO EXEC EXTENDEDGLOB
EXTENDEDHISTORY FLOWCONTROL FORCEFLOAT FUNCTIONARGZERO GLOB GLOBALEXPORT GLOBALRCS GLOBASSIGN
GLOBCOMPLETE GLOBDOTS GLOBSTARSHORT GLOBSUBST HASHALL HASHCMDS HASHDIRS HASHEXECUTABLESONLY
HASHLISTALL HISTALLOWCLOBBER HISTAPPEND HISTBEEP HISTEXPAND HISTEXPIREDUPSFIRST HISTFCNTLOCK
HISTFINDNODUPS HISTIGNOREALLDUPS HISTIGNOREDUPS HISTIGNORESPACE HISTLEXWORDS HISTNOFUNCTIONS
HISTNOSTORE HISTREDUCEBLANKS HISTSAVEBYCOPY HISTSAVENODUPS HISTSUBSTPATTERN HISTVERIFY HUP
IGNOREBRACES IGNORECLOSEBRACES IGNOREEOF INCAPPENDHISTORY INCAPPENDHISTORYTIME INTERACTIVE
INTERACTIVECOMMENTS KSHARRAYS KSHAUTOLOAD KSHGLOB KSHOPTIONPRINT KSHTYPESET KSHZEROSUBSCRIPT
LISTAMBIGUOUS LISTBEEP LISTPACKED LISTROWSFIRST LISTTYPES LOCALLOOPS LOCALOPTIONS LOCALPATTERNS
LOCALTRAPS LOG LOGIN LONGLISTJOBS MAGICEQUALSUBST MAILWARN MAILWARNING MARKDIRS MENUCOMPLETE
MONITOR MULTIBYTE MULTIFUNCDEF MULTIOS NOMATCH NOTIFY NULLGLOB NUMERICGLOBSORT OCTALZEROS ONECMD
OVERSTRIKE PATHDIRS PATHSCRIPT PHYSICAL PIPEFAIL POSIXALIASES POSIXARGZERO POSIXBUILTINS POSIXCD
POSIXIDENTIFIERS POSIXJOBS POSIXSTRINGS POSIXTRAPS PRINTEIGHTBIT PRINTEXTITVALUE PRIVILEGED
PROMPTBANG PROMPTCR PROMPTPERCENT PROMPTSP PROMPTSUBST PROMPTVARS PUSHOIGNOREDUPS PUSHOMINUS
PUSHDSILENT PUSHDTOHOME RCEXPANDPARAM RCQUOTES RCS RECEXACT REMATCHPCRE RMSTARSILENT RMSTARWAIT
SHAREHISTORY SHFILEEXPANSION SHGLOB SHINSTDIR SHNULLCMD SHOPTIONLETTERS SHORTLOOPS SHORTREPEAT
SHWORDSPLIT SINGLECOMMAND SINGLELINEZLE SOURCETRACE STDIN SUNKEYBOARDHACK TRACKALL TRANSIENTRPROMPT
TRAPSASYNC TYPESETSILENT TYPESETTOUNSET UNSET VERBOSE VI WARNCREATEGLOBAL WARNNESTEDVAR XTRACE ZLE

Special variables (88)

Source: `src/zsh/Src/params.c::special_params[]`. Includes scalar (`?`, `$RANDOM`) and array forms (`$path`,
`$pipestatus`); upper-case and lower-case names are linked pairs where applicable.

#! \$# \$\$ \$* \$- \$0 \$? \$@ \$ARGC \$CDPATH \$COLUMNS \$EGID \$ERRNO \$EUID \$FIGNORE \$FPATH \$FUNCNEST \$GID
\$HISTCHARS \$HISTCMD \$HISTSIZE \$HOME \$IFS \$KEYBOARD_HACK \$LANG \$LC_ALL \$LC_COLLATE \$LC_CTYPE \$LC_
MESSAGES \$LC_NUMERIC \$LC_TIME \$LINENO \$LINES \$MAILPATH \$MANPATH \$MODULE_PATH \$NULLCMD \$OPTARG \$OPTIND
\$PATH \$POSTEDIT \$PPID \$PROMPT \$PROMPT2 \$PROMPT3 \$PROMPT4 \$PS1 \$PS2 \$PS3 \$PS4 \$PSVAR \$RANDOM \$READNULLCMD
\$RPROMPT \$RPROMPT2 \$RPS1 \$RPS2 \$SAVEHIST \$SECONDS \$SHLVL \$SPROMPT \$TERM \$TERMINFO \$TERMINFO_DIRS \$TRY_
BLOCK_ERROR \$TRY_BLOCK_INTERRUPT \$TTYIDLE \$UID \$USERNAME \$WORDCHARS \$ZLE_RPROMPT_INDENT \$ZSH_EVAL_

```
CONTEXT $ZSH_SUBSHELL $_ $argv $cdpath $fignore $fpath $histchars $mailpath $manpath $module_path $path
$pipestatus $prompt $psvar $status $zsh_eval_context
```

Parameter expansion flags (38)

Single-letter modifiers inside `${(X)var}`. Composable: `${(jL)arr}` = join lowercase.

Flag	Meaning
<code>\${(@)var}</code>	array-context retain <code>\$@</code> semantics; quoting-preserving even in scalar context
<code>\${(A)var}</code>	create as an array
<code>\${(a)var}</code>	sort by array index
<code>\${(c)var}</code>	count words in a parameter (e.g., scalar split count)
<code>\${(C)var}</code>	capitalize words
<code>\${(D)var}</code>	treat as DIRECTORY name (apply directory substitution like <code>~/...</code>)
<code>\${(e)var}</code>	perform parameter expansion / arithmetic / etc. on the result
<code>\${(f)var}</code>	split result at newlines
<code>\${(F)var}</code>	join array elements with newlines
<code>\${(g)var}</code>	process escape sequences like <code>print</code> does (<code>g:o:</code> process octals, <code>g:c:</code> process <code>\c</code>)
<code>\${(i)var}</code>	case-insensitive sort
<code>\${(j)var}</code>	join array with separator: <code>\${(j:sep:)arr}</code>
<code>\${(k)var}</code>	for assoc arrays: keys <code>\${(k)hash}</code>
<code>\${(K)var}</code>	subscript flags: use keys
<code>\${(L)var}</code>	lowercase
<code>\${(M)var}</code>	match: use longest match (also for case-insensitivity in sort)
<code>\${(n)var}</code>	numeric sort
<code>\${(o)var}</code>	sort ascending
<code>\${(O)var}</code>	sort descending
<code>\${(p)var}</code>	interpret embedded escape sequences in <code>j/s</code> separator
<code>\${(P)var}</code>	treat value as parameter name → indirect (P)
<code>\${(q)var}</code>	quote the result (<code>q-/q+/qq/qqq</code> variants for shell-quote levels)
<code>\${(Q)var}</code>	remove quoting
<code>\${(r)var}</code>	right-justify within field width: <code>\${(r:N::pad:)var}</code>
<code>\${(l)var}</code>	left-justify within field width: <code>\${(l:N::pad:)var}</code>
<code>\${(s)var}</code>	split at separator: <code>\${(s:sep:)var}</code>
<code>\${(S)var}</code>	subscript: search subscript ranges
<code>\${(t)var}</code>	test parameter type
<code>\${(u)var}</code>	unique (dedupe array)
<code>\${(U)var}</code>	uppercase
<code>\${(v)var}</code>	for assoc arrays: values <code>\${(v)hash}</code>
<code>\${(V)var}</code>	make invisible / control chars visible
<code>\${(w)var}</code>	split into words
<code>\${(W)var}</code>	split into words (alternate)
<code>\${(z)var}</code>	split as the shell would (z-tokens)
<code>\${(#)var}</code>	expand result as arithmetic; numeric value
<code>\${(%)var}</code>	expand prompt percent escapes in result
<code>\${(-)var}</code>	treat values as patterns (e.g., for <code>/pat/repl</code>)

Glob qualifiers (44)

Modifiers inside `*(X)`. Composable; use `^` to negate the group, `,` to OR-combine groups.

Qual	Meaning
<code>*(/)</code>	directories only
<code>*(.)</code>	regular files only
<code>*(@)</code>	symbolic links only
<code>*(=)</code>	sockets only
<code>*(p)</code>	named pipes (FIFOs) only

Qual	Meaning
()	executable plain files only
*(%)	device special files only
*(%b)	block special files
*(%c)	character special files
*(r)	owner-readable
*(w)	owner-writable
*(x)	owner-executable
*(A)	group-readable
*(I)	group-writable
*(E)	group-executable
*(R)	world-readable
*(W)	world-writable
*(X)	world-executable
*(s)	setuid (S_ISUID)
*(S)	setgid (S_ISGID)
*(t)	sticky bit
*(d N)	device number N
*(l[+ =]N)	exactly / less-than / greater-than N hard links
*(U)	owned by EUID
*(G)	owned by EGID
*(u N)	owned by uid N
*(g N)	group gid N
*(f spec)	permission mask: f:o+w: e.g.
*(L [+ =] N)	size: blocks / k / m / p suffix (Lk / Lm / Lp)
*(a [Mwhms] [+ =] N)	access time
*(m [Mwhms] [+ =] N)	modify time
*(c [Mwhms] [+ =] N)	ctime
*(o [name size links mtime atime ctime])	sort ascending
*(O [name size links mtime atime ctime])	sort descending
*([N,M])	select range of matches
*(e:str:)	external test: each match passed to expression
*(+func)	external test: each match passed to function
*(N)	nullglob: silently drop unmatched pattern
*(D)	include dotfiles
*(Y N)	limit to N results
*(M)	include directory names as if trailing slash
*(:mod)	apply history modifier (e.g., :t:h:r:e)
*(^)	negate the qualifier list
*(,)	OR-combine qualifier groups

Operators / redirections / substitution forms (55)

Lexer-recognized tokens. Grouped by **kind**: pipeline / list / case / redirect / procsb / subst / arith / cond / assign / compare / glob / tilde / brace / expansion / string / extension.

Symbol	Kind	Meaning
	pipeline	Pipeline. stdout of LHS → stdin of RHS.
&	pipeline	Pipeline with stderr merged (= 2>&1).
&&	list	Logical AND: run RHS only if LHS exit==0.
	list	Logical OR: run RHS only if LHS exit!=0.
;	list	Sequence: run RHS after LHS regardless of status.
&	list	Background: run LHS async; sets \$!.
::	case	End case arm.
::;&	case	Fall through and test next case pattern.
;;	case	Fall through without test.
!	neg	Negate exit status (reserved word).
>	redirect	Stdout redirect (overwrite).

Symbol	Kind	Meaning
>>	redirect	Stdout append.
<	redirect	Stdin redirect.
<<	redirect	Heredoc; body terminated by marker.
<<-	redirect	Heredoc, strip leading tabs from body.
<<<	redirect	Here-string; literal text as stdin.
>	redirect	Stdout force-overwrite (bypass NO_CLOBBER).
>!	redirect	Same as > ; force-overwrite.
&>	redirect	Redirect both stdout and stderr (bash-compat).
&>>	redirect	Append both stdout and stderr.
2>&1	redirect	Duplicate fd2 to fd1 (stderr → stdout).
>&-	redirect	Close fd.
<>	redirect	Open for read+write.
<(	procsb	Process substitution: <(cmd) is a path readable from cmd's stdout.
>(	procsb	Process substitution: >(cmd) is a path writable into cmd's stdin.
=(	procsb	Zsh-only =(cmd): tempfile capture.
\$(	subst	Command substitution: \$(cmd) captures cmd's stdout.
\${	subst	Parameter expansion: \${var}.
\$((	subst	Arithmetic expansion: \$((expr)).
((	arith	Arithmetic command. ((expr)) exits 0 iff expr != 0.
))	arith	Close arithmetic command.
[[	cond	Open conditional command: [[expr]].
]]	cond	Close conditional command.
=	assign	Assignment. Also: equality in [[]].
+=	assign	Append assignment (scalar concat / array push).
-=	assign	Numeric subtract-assign (in (())).
:=	assign	\${var:=default}: assign default if unset/empty.
?=	assign	(arith) ternary.
==	compare	Equality in (()) / [[]].
!=	compare	Inequality.
=~	compare	Regex match in [[]] (POSIX ERE / PCRE depending on opts).
*	glob	Glob: match any sequence (including empty).
**	glob	Recursive glob (matches dir/subdir/.../ levels).
?	glob	Glob: match one character.
~	tilde	Tilde expansion: ~ → \$HOME, ~user, ~+ / ~- / ~N for dirstack.
{a,b,c}	brace	Brace expansion: comma-separated list.
{1..10}	brace	Brace expansion: numeric range.
{a..z}	brace	Brace expansion: character range.
\${~var}	expansion	Treat result of var as pattern.
\${^var}	expansion	Array element rcexpansion.
\${=var}	expansion	Word-split on IFS.
\$'...'	string	ANSI-C quoted string: \n \t \xNN etc.
\$_"..."	string	Locale-translated string.
`...`	subst	Backtick command substitution (legacy form of \$()).
@{ }	extension	Zshrs @-prefix: dispatch to stryke embedded scripting.

History expansions (10)

Recall previous commands / words. Lexer-stage expansion before parsing.

Form	Meaning
!!	previous command
!N	command N in history
!-N	command N back
!?str	most recent containing str
!str	most recent starting with str
!\$	last word of previous command
!^	first arg of previous command

Form	Meaning
!*	all args of previous command
!:N	Nth word of previous command
^old^new	quick substitute in previous command

Word modifiers (16)

Trailing :X modifiers applied to history words, filename expansions, and parameter results.

Modifier	Meaning
:h	head: dirname of path
:t	tail: basename of path
:r	root: strip extension
:e	extension: keep only extension
:l	lowercase
:u	uppercase
:q	quote for shell re-input
:Q	remove quoting
:s/old/new/	substitute first match
:gs/old/new/	global substitute
:a	absolutize (resolve relative path)
:A	absolutize and resolve symlinks
:P	physical resolved path
:x	split into words on whitespace
:w	select words
:F	follow symlinks (in conjunction with above)

// generated against fusevm 0.12.2 dispatch · zshrs v0.11.46 · 15 chapters · 14,552 #[test] hooks · MenkeTechnologies